

TeacherSupporter

The Complete Stack

From Spring Boot to Kubernetes and CI/CD —
a self-contained field guide, taught from one real project

Vuong Dang

written with Claude

August 2026

How to Read This Book

This book teaches every technology inside the TeacherSupporter project — Spring Boot and Spring Cloud, Postgres and MongoDB, Kafka, Docker, Kubernetes, and the Jenkins pipeline that ties them together — using the project’s own files as the worked examples. Nothing is invented for the page: every configuration you will read is deployed, or about to be deployed, on a real two-core home server called `ts-server`.

The rules the book follows

Concepts before configs. Each chapter first teaches the idea in vendor-neutral terms, then reads the project’s real file line by line, then shows one or two examples from *outside* the project so you can tell what generalizes from what is local habit.

Decisions are first-class. Wherever the project chose between alternatives (Jib over docker build, Kustomize over Helm, a second Postgres server over a second database) the reasoning is in a `DECISION` box. These are the “why did you...” questions interviews are made of.

Failure is shown symptom-first. `PITFALL` boxes start with what you would actually see (CrashLoopBackOff, a TLS error, a silent 404) and work back to the cause, because that is the direction real debugging runs in.

Deep dives push a design until it breaks. Part VIII revisits one technology per chapter and asks the questions the main chapters postpone: why this choice, what it solves, what it costs, what breaks at 3 a.m., and what the concrete upgrade looks like — with complete code, not sketches. Read them after the corresponding main chapter, or before an interview.

The box legend

DECISION — EXAMPLE

Why the project chose one design over another, and what would have to change for the other choice to win.

PITFALL — EXAMPLE

A symptom you will eventually meet, followed by its cause and fix.

HANDS-ON — EXAMPLE

A command to run against the real stack, with the output you should expect. The book is meant to be read next to a terminal.

ON THIS HARDWARE — EXAMPLE

A choice that is only correct on this machine (2 cores, 19.4 GB RAM, SATA SSD, Wi-Fi). Do not copy these into a data center.

CHECK YOURSELF (answers: Appendix A)

Every chapter ends with a few questions in this box. Answers are in Appendix A. Write your answer down before checking — recognition is not recall.

Notation

Inline code and file names look like `application.yml`. Annotated listings carry circled markers — ①, ② — explained directly beneath the listing. Shell commands are shown with a `$` prompt for the server and `PS>` for Windows PowerShell; do not type the prompt.

What is assumed, and what is not

Assumed: you can read Java, and you have used Maven and git. Not assumed: any prior Kubernetes, Jenkins, Kafka, Docker internals, or frontend knowledge. Where the project's own documentation already covers one-time machine setup (`docs/RUNBOOK-SERVER-SETUP.md`), this book summarizes and moves on.

The Project Map

TeacherSupporter is a course-management platform built as Spring Cloud microservices: teachers manage courses, students, assignments and submissions; a dictionary service serves vocabulary lookups; registration and login (password, Google OAuth2, and TOTP two-factor) live in a dedicated auth service; e-mail notifications flow through Kafka. It is deliberately built with the “classic” Spring Cloud stack — Eureka, Config Server, Spring Cloud Gateway — because that stack still runs in a great many enterprises, and understanding it makes the cloud-native replacements (Chapter 27) meaningful rather than magical.

The cast

Service	Port	Backing store	Job
config-server	8888	— (classpath)	serves every service’s config
discovery-server	8761	—	Eureka registry
api-gateway	8080	—	single entry point, JWT check
auth-service	8081	Postgres ts_auth	accounts, JWT, OAuth2, 2FA
course-service	8082	Postgres ts_course, MinIO	courses, students, files
dictionary-service	8083	MongoDB	vocabulary lookups
notification-service	8084	— (Kafka in)	events → e-mail
frontend	3000	—	web UI (React)

Table 1: The eight deployables and what they own.

One request, end to end

Every chapter illuminates one link of this chain. Part I is the boxes on the right; Part II the data stores; Part III the Kafka path (not drawn — it carries events, not requests); Parts IV–VI are how all of it gets packaged, run, and shipped.

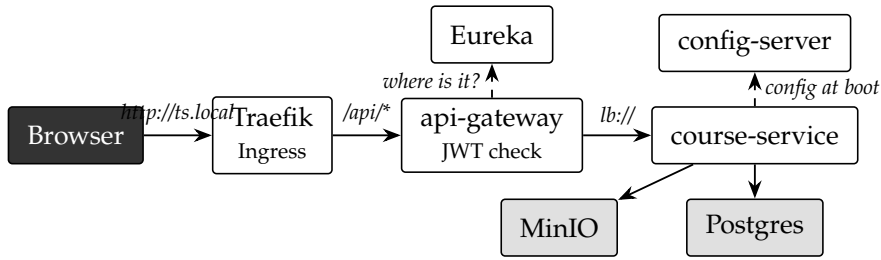


Figure 1: The request path this book keeps returning to.

Where things live in the repository

TeacherSupporter/	
pom.xml	parent: modules, shared plugin config (Jib!)
common/	shared DTOs and events
config-server/	+ src/main/resources/configurations/*.yaml
discovery-server/	api-gateway/ auth-service/
course-service/	dictionary-service/ notification-service/
frontend/	
docker-compose.yml	local dev stack
Jenkinsfile	the CI/CD pipeline (Chapter 23)
deploy/	
jenkins/	Jenkins controller image + run script
k8s/	
create-secrets.sh	secrets are made, not committed
jenkins-rbac.yaml	what Jenkins may touch in the cluster
base/	every Kubernetes manifest (Part V)
docs/	design docs, runbooks, and this book

The machine underneath

Everything deploys to *ts-server*: a repurposed laptop running Ubuntu 24.04 with 2 physical cores, 19.4 GB RAM and a SATA SSD, reachable from Windows over Tailscale. It runs *k3s* (a single-node Kubernetes), a Docker-hosted Jenkins, and a local image registry. The constraint that shapes almost every operational decision in this book is *CPU, not memory*: two cores means builds and demos must not overlap.

Contents

How to Read This Book	ii
The Project Map	iv
I The Application	1
1 Spring Boot: The Machine Under Every Service	2
1.1 The problem Spring Boot solves	2
1.1.1 Starters	2
1.1.2 Auto-configuration	3
1.2 Externalized configuration: the property resolution order	3
1.2.1 Relaxed binding	4
1.2.2 Placeholders with defaults	4
1.3 The project's file, annotated	4
1.4 Profiles	5
1.5 Actuator	5
1.6 Outside the project	5
2 Centralized Configuration: Config Server	7
2.1 The problem	7
2.2 How Spring Cloud Config works	7
2.3 The project's server, annotated	7
2.4 Startup order and the bootstrap dance	9
2.5 Refreshing config without restart	9
2.6 Outside the project	9
3 Service Discovery: Eureka	11
3.1 The problem	11
3.2 How Eureka works	11
3.3 The server, and the one k8s-critical setting	11
3.4 Outside the project	12

4	The API Gateway	14
4.1	The problem	14
4.2	Spring Cloud Gateway's model	14
4.3	The project's routes, annotated	14
4.4	Two routers: the gateway and the Ingress	16
4.5	Advanced corners	16
4.6	Outside the project	16
5	Security: JWT, OAuth2, and the Trusted Edge	18
5.1	Spring Security in one page	18
5.2	JWT: stateless identity	19
5.3	The trusted-edge pattern	19
5.4	OAuth2 login with Google	20
5.5	TOTP two-factor	20
5.6	Outside the project	20
6	The Business Services and the Common Module	22
6.1	course-service: the workhorse	22
6.2	auth-service: identity	22
6.3	dictionary-service: the small one	22
6.4	notification-service: the pure consumer	22
6.5	Synchronous vs. asynchronous, in this codebase	23
6.6	The common module	23
II	Data	25
7	Relational Data: Postgres, JPA, Flyway	26
7.1	Database-per-service	26
7.2	JPA and Hibernate	26
7.3	Why ddl-auto: none	27
7.4	Flyway: migrations as code	27
7.5	The connection pool	28
7.6	Outside the project	28
8	Document Data: MongoDB	30
8.1	The model	30
8.2	Spring Data MongoDB	30
8.3	The configuration — all of it	31
8.4	Schemaless cuts both ways	31
8.5	Operations in this cluster	31

8.6	Outside the project	32
9	Object Storage: MinIO and the S3 Model	33
9.1	The problem	33
9.2	The configuration, annotated	33
9.3	Presigned URLs: the pattern worth knowing	33
9.4	Outside the project	34
III	Messaging	36
10	Kafka: The Core Model	37
10.1	A log, not a queue	37
10.2	Consumer groups	38
10.3	Brokers, replication, KRaft	38
10.4	The advertised-listeners maze	38
10.5	Delivery semantics	39
10.6	Outside the project	39
11	Spring Kafka: Producers, Listeners, Poison Pills	41
11.1	Producing	41
11.2	Consuming	41
11.3	The poison pill	42
11.4	Advanced corners	43
IV	Containers	44
12	Docker: Images, Containers, Compose	45
12.1	What a container actually is	45
12.2	The project's Dockerfile, annotated	45
12.3	Networking and Compose	46
13	Images Without a Daemon: Jib and the Local Registry	48
13.1	Why not just docker build in CI?	48
13.2	The configuration, annotated	48
13.3	The registry, and why localhost:5000	49
13.4	Tags, digests, and the SNAPSHOT trap	50

V	Kubernetes	51
14	Kubernetes: Cluster Anatomy and the Reconciliation Loop	52
14.1	The one idea everything else hangs on	52
14.2	The moving parts	52
14.3	Objects, labels, namespaces	53
14.4	Getting kubectl to the cluster	53
15	Workloads: Pods, Deployments, StatefulSets	55
15.1	Pods	55
15.2	Deployments and the rollout	55
15.3	StatefulSets: when identity matters	56
15.4	Other workload kinds	57
16	Configuration and Secrets in the Cluster	58
16.1	Two injection mechanisms	58
16.2	Secrets: what they are and are not	58
16.3	create-secrets.sh, annotated	58
16.4	ConfigMaps	59
16.5	Outside the project	60
17	Health Probes and Resource Management	61
17.1	The three probes	61
17.2	Requests, limits, and the JVM	63
18	Networking: Services, DNS, Ingress	65
18.1	The problem pods create	65
18.2	Ingress and Traefik	65
18.3	The full path of one request	66
18.4	Outside the project	67
19	Storage: Volumes, Claims, and State on a Single Node	68
19.1	The abstraction stack	68
19.2	local-path on k3s	68
19.3	Two claim styles in the manifests	69
19.4	Outside the project	70
20	Kustomize: Manifests as a Set	71
20.1	The problem	71
20.2	The kustomization, annotated	71
20.3	Bases and overlays	72

VI	CI/CD	75
21	CI/CD: The Concepts	76
21.1	Two practices, one pipeline	76
21.2	The vocabulary	76
22	Jenkins: The CI Server	79
22.1	Architecture	79
22.2	The controller image, annotated	79
22.3	Credentials	80
22.4	The deploy identity: a ServiceAccount, not admin	80
23	The Pipeline, Line by Line	83
23.1	Preamble and environment	83
23.2	The three stages	84
23.3	Failure handling	86
23.4	The same pipeline, elsewhere	86
VII	Operations and the Road Ahead	88
24	Operating the Stack: A Debugging Field Manual	89
24.1	The four questions	89
24.2	A status taxonomy	89
24.3	Reading a service's world from inside	90
24.4	The routine that keeps weekends free	91
25	Kafka in the Cluster: Strimzi and the Mail Flow	93
25.1	Operators and custom resources	93
25.2	Installing the operator: an admin act	93
25.3	The cluster, declared	94
25.4	Topics as objects	95
25.5	The last service, and its mailbox	96
25.6	Outside the project	98
26	The Frontend and the Single Host	99
26.1	What a frontend is, to an operator	99
26.2	Two stages, one Dockerfile	99
26.3	nginx: the fallback that makes a SPA work	100
26.4	Claiming /: one origin	101
26.5	The manifest	102

26.6	The pipeline grows a stage	102
26.7	Verifying the milestone	104
26.8	Reaching it from anywhere: the tailnet name	105
26.9	Outside the project	105
27	The Road Ahead	107
27.1	Observability	107
27.2	GitOps: Argo CD	107
27.3	Retiring the classic stack	108
27.4	Jenkins into the cluster	108
27.5	OpenShift, briefly	108
27.6	Where this leaves you	109
VIII	Deep Dives	110
28	Deep Dive: JWT at Scale — Where to Verify, Fat Tokens, and Asymmetric Keys	111
28.1	Where to verify: gateway or every service?	111
28.1.1	Model 1: centralized verification (what the project does)	111
28.1.2	The “trusted internal network” fallacy	112
28.1.3	Model 2: zero trust — every service verifies	113
28.2	What a signature guarantees	114
28.3	HS256 versus RS256	115
28.3.1	HS256: one key, two roles	115
28.3.2	RS256: two keys, one role each	115
28.3.3	The costs	116
28.3.4	Converting the project	117
28.3.5	Rotation, and what kid is for	119
28.4	Payload bloat	119
28.5	The phantom token pattern	120
28.6	What breaks at 3 a.m.	121
28.7	When JWT is the wrong tool	122
28.8	Outside the project	122
28.9	Questions from real interviews	122
29	Deep Dive: Configuration — Config Server, Secrets, and Refresh at Scale	125
29.1	Why centralize, and what it solves	125
29.2	Reading the real path	126

29.2.1	Precedence, precisely	127
29.3	Pros and cons, honestly	129
29.4	What breaks	129
29.5	What breaks at 3 a.m.	130
29.6	Enhancing the resolution	131
29.6.1	Upgrade 1: git backend with encrypted values	131
29.6.2	Upgrade 2: retire the server	133
29.7	Outside the project	134
29.8	Questions from real interviews	135
30	Deep Dive: Service Discovery — Eureka Internals, Failure Modes, and the Kubernetes Alternative	138
30.1	Why a registry, and why on the client	138
30.2	Reading the real configuration	139
30.2.1	The timers	140
30.3	Pros and cons, honestly	142
30.4	What breaks	142
30.5	What breaks at 3 a.m.	143
30.6	Enhancing the resolution	144
30.6.1	Step 1: a profile document for the gateway	144
30.6.2	Step 2: Feign without the registry	145
30.6.3	Step 3: the manifests	146
30.7	Outside the project	147
30.8	Questions from real interviews	147
31	Deep Dive: The Gateway Under Load — Reactive Internals, Rate Limits, and the Edge’s Job	150
31.1	Why a gateway, and what it solves	150
31.2	The reactive stack, read from the filter	151
31.2.1	Threads: four, not four hundred	152
31.2.2	What blocking does to an event loop	152
31.2.3	Predicates, order, and the StripPrefix arithmetic	153
31.3	CORS at the edge	154
31.4	The edge’s job, and what is not	154
31.5	Pros and cons, honestly	155
31.6	What breaks	155
31.7	What breaks at 3 a.m.	156
31.8	Enhancing the resolution	156
31.9	Outside the project	160
31.10	Questions from real interviews	160

32 Deep Dive: Synchronous Calls — OpenFeign, Resilience4j, and Cascading Failure	163
32.1 Why Feign, and what it solves	163
32.2 The contract in common, and its versioning risk	164
32.3 The circuit breaker, with the project's numbers	165
32.4 Timeouts stack	166
32.5 Retry: only for calls you can repeat	166
32.6 Bulkhead: bounding the blast	167
32.7 What the fallback returns	167
32.8 Pros and cons, honestly	168
32.9 What breaks	168
32.10 What breaks at 3 a.m.	169
32.11 Enhancing the resolution	170
32.12 Outside the project	174
32.13 Questions from real interviews	174
33 Deep Dive: Data Across Services — Database-per-Service, Duplication, and the Outbox	178
33.1 Why this, and what it solves	178
33.2 How a student reaches course-service	179
33.2.1 What is not propagated at all	181
33.3 Pros and cons, candidly	181
33.4 The dual-write problem	181
33.5 Enhancing the resolution: the transactional outbox	182
33.5.1 Change data capture: the industrial relay	186
33.6 Sagas: when two services must both change	186
33.7 Reads across services	187
33.8 What breaks at 3 a.m.	187
33.9 Outside the project	188
33.10 Questions from real interviews	189
34 Deep Dive: Kafka Delivery Guarantees — At-Least-Once, Idempotent Consumers, and Dead Letters	192
34.1 Why this, and what it solves	192
34.2 The settings the project never wrote	193
34.2.1 Producer	193
34.2.2 Consumer	194
34.3 Exactly when a duplicate happens	195
34.4 Pros and cons, candidly	196
34.5 Enhancing the resolution	196

34.5.1	An idempotent consumer with a record of what it has seen .	196
34.5.2	Retry topics and a dead-letter topic	198
34.5.3	Exactly-once, and what it does not cover	199
34.6	What breaks at 3 a.m.	199
34.7	Outside the project	200
34.8	Questions from real interviews	201
35	Deep Dive: Files at Scale — Presigned URLs, Endpoints, and Why Bytes Don't Belong in Postgres	204
35.1	Why object storage, and what it solves	204
35.2	Reading the project's code	205
35.2.1	Two clients, two endpoints	205
35.2.2	The key, the upload, the download	206
35.3	Signature Version 4, and why the host is welded in	208
35.4	Who moves the bytes	209
35.5	Pros and cons, candidly	210
35.6	What breaks	211
35.7	What breaks at 3 a.m.	211
35.8	Enhancing the resolution: presigned PUT	212
35.9	Outside the project	215
35.10	Questions from real interviews	216
36	Deep Dive: The Dictionary on MongoDB — Document Modeling, Regex Search, and Honest Trade-offs	219
36.1	Why MongoDB here, and what it solves	219
36.2	Reading the search path	221
36.2.1	How the server executes it	222
36.2.2	The array predicate	222
36.3	The graph	223
36.4	Transactions and consistency, honestly	224
36.5	Pros and cons, candidly	224
36.6	What breaks	225
36.7	What breaks at 3 a.m.	225
36.8	Enhancing the resolution	226
36.8.1	Page in the database	226
36.8.2	Batch the links	227
36.8.3	Index for the sort, and anchored search	227
36.8.4	A text index for word-anywhere search	228
36.9	Outside the project	229
36.10	Questions from real interviews	229

37 Deep Dive: Observability — Traces Across Feign and Kafka, Sampling, and Alerts That Matter	233
37.1 Why this, and what it solves	233
37.2 How a trace travels through this project	234
37.2.1 The configuration, in every service	234
37.2.2 Hop by hop	235
37.2.3 Correlating logs	237
37.3 Sampling	237
37.4 Metrics: what exists and what does not	238
37.5 Pros and cons	238
37.6 What breaks	239
37.7 What breaks at 3 a.m.	239
37.8 Enhancing the resolution	240
37.9 Outside the project	244
37.10 Questions from real interviews	244
38 Deep Dive: The Browser Meets the Gateway — Token Storage, Refresh Flow, and CORS	248
38.1 Why this, and what it solves	248
38.2 The refresh flow, as written	249
38.2.1 The interceptor	249
38.2.2 The single-flight problem	251
38.2.3 Logout and the OAuth callback	251
38.3 CORS: the policy you are not using	252
38.4 Pros and cons	252
38.5 What breaks	253
38.6 What breaks at 3 a.m.	254
38.7 Enhancing the resolution	255
38.8 Outside the project	258
38.9 Questions from real interviews	259
A Quiz Answers	263
B Glossary	284
C Command Cheat Sheet	289
D The Configuration Files, Verbatim	292

Part I

The Application

Spring Boot: The Machine Under Every Service

Seven of the eight deployables in this project are Spring Boot applications. Before any YAML file can make sense, you need three ideas: what a *starter* pulls in, what *auto-configuration* decides on your behalf, and — most important for everything in Parts V and VI — exactly *how Spring resolves a configuration property* when the same key is defined in several places at once.

1.1 The problem Spring Boot solves

Classic Spring asked you to wire everything: a servlet container to deploy into, XML or Java config declaring every bean, and a long list of version-compatible dependencies. Spring Boot inverts this. An application is a plain `main()` method:

```
@SpringBootApplication
public class CourseServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(CourseServiceApplication.class, args);
    }
}
```

`@SpringBootApplication` is three annotations in a trench coat: `@Configuration` (this class may define beans), `@ComponentScan` (find `@Service`, `@RestController`, ... in this package and below), and `@EnableAutoConfiguration` — the interesting one.

1.1.1 Starters

A starter such as `spring-boot-starter-web` is an empty jar whose only job is a curated dependency list: Spring MVC, Jackson, and an *embedded* Tomcat. “Embedded” matters operationally: the server is inside the jar, so `java -jar course-service.jar` is a complete web server. That single fact is what makes the container images of Chapter 13 so simple — there is no Tomcat to install, only a JVM to provide.

1.1.2 Auto-configuration

At startup, Spring Boot walks a list of candidate configurations and applies each one whose conditions hold. The conditions read like:

- *is this class on the classpath?* (@ConditionalOnClass — you added the Postgres driver, so a DataSource is configured)
- *has the user defined their own bean already?* (@ConditionalOnMissingBean — your explicit bean always wins)
- *is a property set?* (@ConditionalOnProperty)

Auto-configuration is why `course-service` contains no code creating a connection pool, a Kafka producer, or an S3 client wiring — adding the dependency plus a few properties *is* the configuration. When behavior seems to appear from nowhere, run the app with `-debug` once: the “condition evaluation report” lists every auto-configuration applied and skipped, with reasons.

1.2 Externalized configuration: the property resolution order

This is the single most load-bearing mechanism in the whole book. Every trick the Kubernetes manifests play in Part V works because of it.

A Spring property such as `spring.datasource.url` can be defined in many *property sources*. Spring merges them all into one `Environment`, and when two sources define the same key, the more specific source wins. Simplified to the sources this project actually uses, from weakest to strongest:

1. defaults inside `application.yml` packaged in the jar
2. config fetched from `config-server` (Chapter 2)
3. OS environment variables
4. command-line arguments

DECISION — WHY THE CLUSTER OVERRIDES WITH ENV VARS

The same image must run unchanged on a laptop (Compose) and in k3s. Baking environment differences into the jar would mean one image per environment — exactly what containers are meant to avoid. Environment variables outrank both the packaged YAML and the `config-server`, so the Kubernetes Deployment can repoint a service at `postgres-course:5432` without touching a single file the developers own. One artifact, many environments, differences injected at the edge.

1.2.1 Relaxed binding

Environment variable names cannot contain dots. Spring therefore maps `SPRING_DATASOURCE_URL` → `spring.datasource.url`: uppercase, dots become underscores. Read any Deployment in `deploy/k8s/base/` and you can now decode every `env:` entry as “this overrides that YAML key.” For example `SPRING_DATA_MONGODB_URI` in `dictionary-service.yaml` overrides `spring.data.mongodb.uri` from the `config-server`.

1.2.2 Placeholders with defaults

The project’s config files use one more idiom:

```
app:
  jwt:
    secret: ${JWT_SECRET:mySecretKeyThatIs...}
```

`${NAME:default}` means “use the environment variable if present, else the default.” This is why a developer can run `auth-service` with zero setup, while production injects the real secret — and why in Chapter 16 the Kubernetes Secret only has to provide `JWT_SECRET` to take control of this value.

1.3 The project’s file, annotated

Every service carries only a tiny `application.yml`; the interesting config lives on the `config-server`. Here is `course-service`’s, complete:

```
spring:
  application:
    name: course-service      ①
  config:
    import: optional:configserver:http://localhost:8888 ②
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/ ③
```

- ① The service’s identity. The `config-server` uses it to pick which file to serve (`configurations/course-service.yml`); Eureka uses it as the registration name; the gateway routes to it as `lb://course-service`. One string ties all three systems together.
- ② Bootstrap-time import of remote config. `optional:` means “start anyway if the `config-server` is down” — kind in dev, and debatable in production (a

service that boots with half its config may fail in stranger ways than one that refuses to boot). The `localhost:8888` default is overridden by `SPRING_CONFIG_IMPORT` in Compose and Kubernetes alike.

- ③ Where Eureka lives — again a `localhost` default, again overridden per environment by `EUREKA_CLIENT_SERVICEURL_DEFAULTZONE`.

1.4 Profiles

A Spring *profile* is a named variant of configuration. `spring.profiles.active=native` on the config-server (Chapter 2) is the project’s only profile use so far, but the mechanism generalizes: `application-prod.yml` is loaded on top of `application.yml` when the `prod` profile is active, letting one jar carry several personalities. Kubernetes reduces the need — the environment injects differences instead — which is why this project leans on env vars, not profile files, for per-environment change.

1.5 Actuator

`spring-boot-starter-actuator` adds operational endpoints under `/actuator`: `/health` (aggregated readiness of DB, disk, custom checks), `/info`, `/metrics`, `/prometheus`. Two facts matter now:

- The starter must be on the classpath. Exposure configuration without `spring-boot-starter-actuator` is inert — the project shipped exactly that mistake in four services and found it through a failing Kubernetes probe (Chapter 17).
- Endpoints must be *exposed* (`management.endpoints.web.exposure.include`) — the project exposes `health,info,metrics,prometheus`.
- Exposure is not *authorization*: if Spring Security protects `/actuator/**`, the endpoint exists but answers 401. This exact interaction decides which Kubernetes probes each service can use in Chapter 17 — `dictionary-service` permits `actuator` and probes `/actuator/health`; `auth-service` does not, and must be probed via `/v3/api-docs` instead.

1.6 Outside the project

Two contrasting examples of the same property machinery:

A *Heroku-style twelve-factor app* configures everything by environment variable — `DATABASE_URL`, `PORT` — and carries an `application.yml` of nothing but `${...}` placeholders. Spring’s resolution order makes that style native.

A bank running one artifact through five environments (dev/test/uat/pre-prod/prod) typically pairs a config-server backed by a git repository with profiles per environment: `course-service-uat.yml` lives in an audited repo, and promotion is a git merge. The project's classpath-native config-server (Chapter 2) is the single-developer version of the same shape.

PITFALL — MY OVERRIDE IS BEING IGNORED

Symptom: you set an environment variable but the service still uses the old value. Usual causes, in order of frequency: the variable name does not map to the key you think (check the relaxed-binding spelling — `SERVICEURL` in the Eureka key is one word, an irregular name that must be copied exactly); the value is also set by a *stronger* source (a command-line arg); or the service reads the key at build time rather than from the Environment. Diagnose with the actuator `/actuator/env` endpoint (when exposed), which lists every property source and which one won.

CHECK YOURSELF (answers: Appendix A)

1. A key is defined in the jar's `application.yml`, in the config-server's file for that service, and as an env var in the Deployment. Which value wins, and why is that ordering the right one for containers?
2. What three annotations does `@SpringBootApplication` combine, and which of them explains why deleting an unused dependency can change runtime behavior?
3. `${GOOGLE_CLIENT_ID:placeholder}` — what does this resolve to when the variable is unset, and what operational property does that give auth-service at startup?
4. Why does exposing an actuator endpoint not guarantee that a Kubernetes probe can use it?

Centralized Configuration: Config Server

2.1 The problem

Seven services each need database URLs, Kafka addresses, JWT settings, mail hosts. Copy that into seven `application.yml` files and every change becomes seven edits, seven rebuilds, and the guarantee that two copies will eventually disagree. The classic Spring Cloud answer: one small HTTP service whose only job is handing each application its configuration at startup.

2.2 How Spring Cloud Config works

The server exposes configuration over a plain HTTP contract:

```
GET /{application}/{profile}
GET /course-service/default    -> the config for course-service
```

The client side is the `spring.config.import: configserver:...` line you saw in Chapter 1. During startup — *before* most beans are created — the client fetches its document and merges it into the Environment as a property source that outranks the packaged YAML but loses to environment variables. The resolution order of Section 1.2 is doing all the work; the config-server is just one more source in the stack.

2.3 The project's server, annotated

```
server:
  port: 8888                                ①
spring:
  application:
    name: config-server
  profiles:
    active: native                           ②
  cloud:
    config:
```

```
server:
  native:
    search-locations: classpath:/configurations ③
management:
  endpoints:
    web:
      exposure:
        include: health ④
```

- ① 8888 is the conventional config-server port, the way 8761 is Eureka's. Convention saves documentation.
- ② The native profile means “serve files from a filesystem/classpath location” instead of the default git backend.
- ③ The served files are *inside the config-server's own jar*: `configurations/course-service.yml`, `auth-service.yml`, and so on — selected by matching file name to the requesting service's `spring.application.name`.
- ④ Only health is exposed — and this is the endpoint the Compose healthcheck and the Kubernetes probes rely on.

DECISION — NATIVE CLASSPATH VS. GIT BACKEND

The enterprise-standard backend is a git repository: config changes are commits — audited, reviewed, revertible without rebuilding anything. The cost is a second repository and credentials management. For a single-developer project whose config changes ship with code changes anyway, classpath-native keeps everything in one repo and one pipeline. The trade traded away: changing a value now requires rebuilding the config-server image. Know which side of that trade your employer sits on.

PITFALL — A SERVICE STARTS WITH DEFAULTS AND FAILS ODDLY

Symptom: `auth-service` boots but every request fails with connection errors to `localhost:5432`. Cause: the config-server was unreachable at startup, and `optional:configserver:` let the service boot from its packaged defaults — which point at `localhost`. Because the failure appears minutes later and far from the cause, it reads as a database problem. Check: the startup log prints whether remote config was fetched. The Compose file prevents this ordering problem with `depends_on: condition: service_healthy`; Kubernetes has no startup ordering (Chapter 15), so services must tolerate a late config-server by crashing and restarting until it appears.

2.4 Startup order and the bootstrap dance

The config-server creates a hidden sequencing constraint across the whole system: every other service wants it up first. You will meet the consequences three times in this book:

- Compose encodes the order explicitly with `depends_on` and a curl-based healthcheck (Chapter 12).
- Kubernetes refuses to encode startup order at all; the system converges by retry (Chapter 15).
- The Jenkins pipeline deploys all manifests in one `kubectl apply -k` and lets rollouts settle concurrently (Chapter 23).

2.5 Refreshing config without restart

Out of the box, a client reads its config once at startup. Spring Cloud offers `/actuator/refresh` (re-fetch on demand, per instance, applies to `@RefreshScope` beans) and Spring Cloud Bus (broadcast refresh over a message broker). This project uses neither: on Kubernetes a config change ships as a new image or env change, and the Deployment's rolling update (Chapter 15) restarts pods anyway. That is the modern answer to a question the config-server was originally designed around.

2.6 Outside the project

A typical git-backed enterprise setup, for contrast:

```
spring:
  cloud:
    config:
      server:
        git:
          uri: https://git.company.com/platform/config-repo
          default-label: main
          search-paths: '{application}'
```

Config lives per-service in folders of a dedicated repo; the label is a branch, so `default-label: release-42` pins an environment to a release. The second widespread pattern skips the config-server entirely: Kubernetes ConfigMaps and Secrets mounted as env vars — which is exactly where this project is heading (Chapters 16 and 26); the config-server survives here mostly as a working example of the classic stack.

CHECK YOURSELF (answers: Appendix A)

1. In which order do these sources win for `spring.datasource.url`: packaged YAML, config-server document, Deployment env var?
2. What does `profiles.active: native` change about where the server finds its documents, and what does the git backend buy that native cannot?
3. Why is `optional: in spring.config.import` a double-edged sword? Describe the failure mode it permits.
4. The config-server's own config cannot come from a config-server. Where must it live, and what general bootstrapping principle is that?

Service Discovery: Eureka

3.1 The problem

The gateway must forward `/api/courses` to `course-service`. To which address? In Compose it could hard-code `course-service:8082` — but then every new replica, every moved service, every environment is a config change. Service discovery replaces addresses with *names resolved at runtime*: services register themselves in a registry; clients ask the registry.

3.2 How Eureka works

Eureka (from Netflix, the ancestral Spring Cloud stack) is a REST service holding a registry of instances:

- **Registration.** At startup each client POSTs its identity: app name (`spring.application.name`), host, port, health URL.
- **Heartbeats.** Every 30 s (default) the client renews its lease. Miss a few renewals and the instance is evicted.
- **Client-side caching.** Consumers fetch the registry and cache it (default refresh 30 s), then load-balance *locally* across the instances they know. The registry being briefly down does not stop traffic — callers keep using their cache.
- **Self-preservation.** If too many heartbeats vanish at once, Eureka assumes a network problem rather than mass death and stops evicting. Fewer surprises during network blips; staler data during real outages.

The consumer side in this project is the gateway's `lb://course-service` URI (Chapter 4): “look this name up in the registry, pick an instance.”

3.3 The server, and the one k8s-critical setting

The discovery-server module is Eureka with almost no configuration — a `@EnableEurekaServer` annotation and a port. The interesting configuration is on every *client*, and one line of it earns its own section because it decides whether the whole system works on Kubernetes:

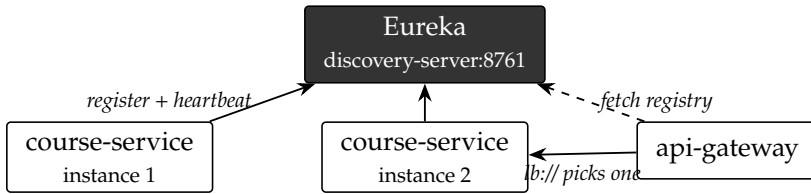


Figure 3.1: Registration, heartbeat, and client-side load balancing.

```
- name: EUREKA_INSTANCE_PREFER_IP_ADDRESS
  value: "true"
```

By default a client registers its *hostname*. On your laptop that is fine. Inside a Kubernetes pod, the hostname is the *pod name* — `course-service-7d9f6c-x2vlq` — which is not resolvable by other pods. Every registration would be an address no one can dial. Pod *IPs*, by contrast, are routable across the whole cluster. So every Deployment in `deploy/k8s/base/` sets this flag: register the IP, not the name.

PITFALL — EVERYTHING REGISTERS, NOTHING CAN CALL ANYTHING

Symptom: the Eureka dashboard shows all services UP, but every gateway-routed request fails with `UnknownHostException`. Cause: the registered hostnames are pod names (see above). This is the single most common Eureka-on-Kubernetes failure, and it is invisible until the first cross-service call.

DECISION — WHY EUREKA AT ALL, WHEN KUBERNETES HAS SERVICES

Kubernetes already gives every Service a DNS name (`course-service:8082`) with built-in load balancing — discovery is native to the platform. Eureka here is deliberate résumé-driven redundancy: the classic stack is still widespread in industry, and migrating *off* it (Chapter 27: replace `lb://` with plain Service DNS, delete two deployables) is itself the kind of modernization work enterprises pay for. Building both stacks teaches the migration.

3.4 Outside the project

Consul (HashiCorp) is the main non-JVM alternative: agent-based, language-agnostic, with health checks executed by the agent rather than self-reported heartbeats, plus a key-value store that can replace the config-server too. *Kubernetes-native discovery* is the other pole: no registry process at all — the

platform's controllers watch pod readiness and update Service endpoints; "registration" is simply a pod becoming Ready (Chapter 17 shows the machinery). Notice the pattern: the more the platform provides, the less the application must carry.

ON THIS HARDWARE

Eureka's conservative defaults (30 s heartbeat, 30 s client cache, eviction after 90 s) mean a crashed instance can receive traffic for up to a minute. On one replica per service that hardly matters — the caller would fail anyway — but it explains why rolling deployments in Chapter 23 wait on Kubernetes readiness, not on Eureka status.

CHECK YOURSELF (answers: Appendix A)

1. Trace what happens between course-service starting and the gateway successfully routing to it: which registrations, fetches, and caches are involved, and what is the worst-case delay?
2. Why does `prefer-ip-address` matter on Kubernetes but not on Compose? (What does each platform make resolvable?)
3. Self-preservation mode trades which property for which? Give a scenario where it helps and one where it hurts.
4. If Eureka were removed and the gateway pointed at `http://course-service:8082` directly, what Kubernetes machinery replaces each Eureka feature?

The API Gateway

4.1 The problem

Without a gateway, the frontend must know six service addresses, each service must implement its own authentication, and cross-cutting concerns (CORS, rate limits, tracing headers) are done six times or not at all. A gateway gives external clients *one* address and centralizes the edge concerns; behind it, services can trust their caller.

4.2 Spring Cloud Gateway's model

Three concepts compose every route:

Route a destination URI plus the conditions and rewrites below.

Predicates conditions deciding whether a request matches: path, method, header, host. First matching route wins.

Filters transformations applied on the way through: strip or add path segments, add headers, rate-limit, retry.

The gateway is built on the *reactive* stack (Project Reactor + Netty, web-application-type: reactive in its application.yml) rather than servlet Tomcat. A proxy spends its life waiting on other servers; an event-loop model holds thousands of in-flight requests without one thread each. Operationally this is why the gateway gets a smaller memory request than the MVC services in its Deployment (384 Mi vs. 512 Mi) and boots faster — which Chapter 17 exploits with a shorter startup probe.

4.3 The project's routes, annotated

From configurations/api-gateway.yml (abridged — the course-service block repeats for students, assignments, enrollments, submissions):

```
spring:
  cloud:
    gateway:
      routes:
```

```

- id: auth-service
  uri: lb://auth-service      ①
  predicates:
    - Path=/api/auth/**      ②
  filters:
    - StripPrefix=2          ③
- id: auth-oauth2
  uri: lb://auth-service
  predicates:
    - Path=/oauth2/**        ④
- id: course-service-courses
  uri: lb://course-service
  predicates:
    - Path=/api/courses/**
  filters:
    - StripPrefix=2
app:
  jwt:
    secret: ${JWT_SECRET:mySecret...} ⑤

```

- ① `lb://` = “resolve this name via the discovery client and load-balance.” The name is the target’s `spring.application.name`. Swap `lb://` for `http://` and you have a static proxy with no Eureka involved.
- ② Path predicate. `**` matches any depth.
- ③ `StripPrefix=2` removes two segments: `/api/auth/login` arrives at `auth-service` as `/login`. This is why `auth-service`’s `SecurityConfig` (Chapter 5) permits `/login`, not `/api/auth/login` — the service is unaware of its public prefix.
- ④ No `StripPrefix` here: Google’s OAuth2 redirect URI is registered with the full path, so it must arrive at `auth-service` unchanged.
- ⑤ The gateway holds the JWT *validation* key. Same secret as `auth-service`, which *signs* — one shared secret, two roles, and in Kubernetes one `Secret` object mounted into both (Chapter 16).

PITFALL — 404 FROM THE SERVICE, 200 FROM CURL-ON-THE-POD

Symptom: calling `/api/courses/list` through the gateway returns 404, but the same endpoint answers when called directly on the service. Cause: prefix mismatch — either `StripPrefix` removed segments the controller expects, or the controller mapping includes a prefix the gateway already stripped. Debug by logging the *rewritten* path: enable `spring.cloud.gateway` debug logging and compare with the controller’s `@RequestMapping`.

4.4 Two routers: the gateway and the Ingress

On Kubernetes this project has *two* L7 routers in series: Traefik (the Ingress controller, Chapter 18) at the cluster edge, then this gateway. The division of labor is deliberate:

- Traefik owns the *host*: TLS later, one public entry, path-prefix fan-out. It sends `/api`, `/oauth2`, `/login/oauth2` to the gateway and will send `/` to the frontend.
- The gateway owns the *application edge*: Eureka resolution, JWT validation, per-route rewrites.

DECISION — NO SIDE DOORS

Milestone 3 had an Ingress sending `/courses` straight to course-service — useful scaffolding, but it bypassed JWT validation: an unauthenticated path to protected data. It was deleted the day the gateway was deployed. The rule it teaches: *every* external route must pass the component that enforces authentication, or the authentication is decorative.

4.5 Advanced corners

Worth knowing, not yet used here: `RequestRateLimiter` filter (token bucket backed by Redis), `Retry` with backoff for idempotent GETs, `CircuitBreaker` filter (Resilience4j) returning fallbacks when a service is down, and default filters applied to all routes. Each is a few YAML lines — the gateway is where such policies belong, because it is the one place every external request passes.

4.6 Outside the project

The same three-route idea in a Kubernetes-native gateway (no Eureka, Service DNS instead) — an `nginx-ingress` annotation style:

```
nginx.ingress.kubernetes.io/rewrite-target: /$2
# path: /api/courses(/|$)(.*) -> service: course-service
```

And the cloud-managed version: an AWS API Gateway or GCP API Gateway route table doing predicates/filters as vendor config, with JWT validation delegated to the provider (Cognito, IAP). The concepts transfer 1:1; only the syntax and the billing change.

CHECK YOURSELF (answers: Appendix A)

1. Follow POST `/api/auth/login` from browser to controller method on Kubernetes: name every hop and every path rewrite.
2. Why must the OAuth2 routes not strip their prefix while the API routes must?
3. The gateway validates JWTs; auth-service signs them. What breaks, and with what symptom, if the two ever hold different secrets?
4. Why is a reactive stack the right choice for a proxy but not automatically for course-service?

Security: JWT, OAuth2, and the Trusted Edge

Security is where this project is most instructive, because it uses three mechanisms that industry uses constantly — stateless JWTs, OAuth2 login, TOTP two-factor — and wires them through a gateway in the standard “trusted edge” pattern.

5.1 Spring Security in one page

Spring Security is a chain of servlet filters in front of your controllers. Each request passes through the chain; filters authenticate (who are you?) and the authorization rules decide (may you?). The project’s auth-service chain, from its `SecurityConfig`:

```
http
    .csrf(AbstractHttpConfigurer::disable)                ①
    .sessionManagement(s ->
        s.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) ②
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/register", "/login", "/activate",
            "/refresh", "/verify-2fa", "/change-password",
            "/oauth2/**", "/login/oauth2/**",
            "/v3/api-docs/**", "/swagger-ui/**").permitAll() ③
        .anyRequest().authenticated())
    .oauth2Login(o -> o.successHandler(oAuth2LoginSuccessHandler))
    .addFilterBefore(gatewayHeaderAuthFilter,
        UsernamePasswordAuthenticationFilter.class);      ④
```

- ① CSRF protection defends browser cookie sessions; a stateless JWT API has no cookie session to ride, so it is disabled. Disable it for that reason — not because a tutorial did.
- ② STATELESS: no HTTP session is created. Every request must carry its own proof of identity. This is the property that lets any replica answer any request — the scaling prerequisite.
- ③ The public paths. Note they are the *stripped* paths (`/login`, not `/api/auth/login`) — Chapter 4 explains why. Note also what is *absent*: `/actuator/**`. That absence

decides this service’s Kubernetes probe strategy in Chapter 17.

- ④ The custom filter that trusts identity headers from the gateway — the subject of Section 5.3.

5.2 JWT: stateless identity

A JSON Web Token is three base64url parts, `header.payload.signature`. The payload carries *claims*: subject (user id), role, expiry. The signature is HMAC-SHA256 over header and payload with a shared secret — the 256-bit-minimum string both auth-service and the gateway hold.

The properties that matter:

- **Self-contained.** Validation needs no database call: check signature, check expiry, read claims. This is what lets the *gateway* authenticate without owning user data.
- **Readable.** Base64 is encoding, not encryption — anyone can decode the payload. Never put secrets in claims.
- **Irrevocable until expiry.** A signed token cannot be un-signed. Hence the two-token pattern: a short-lived *access token* (here 15 minutes) paired with a long-lived *refresh token* (7 days) that is checked against the database at `/refresh` — revocation happens there.

5.3 The trusted-edge pattern

The gateway’s `JwtAuthenticationFilter` validates the token, then forwards the request with identity as plain headers:

```
ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
    .header(JwtConstants.HEADER_USER_ID, userId)
    .header(JwtConstants.HEADER_USER_ROLE, role)
    .build();
```

Downstream, each service’s `GatewayHeaderAuthFilter` reads those headers and builds the Spring Security context from them — no re-validation. Services stay simple; cryptography happens once.

The pattern’s obligation: those headers are only trustworthy if *every* path to a service goes through the gateway. A client who can reach course-service directly could forge `X-User-Id`. This is the deep reason the direct `/courses` Ingress was deleted (Chapter 4), and why in-cluster traffic is the remaining soft spot — any pod can still call `course-service:8082` with invented headers. `NetworkPolicies`

or mTLS (service mesh) are the industrial hardening; Chapter 27 discusses when they earn their complexity.

5.4 OAuth2 login with Google

OAuth2's *authorization code* flow, as it runs here:

1. Browser hits `/oauth2/authorization/google` (via the gateway); Spring redirects to Google with the app's client-id and a `redirect_uri`.
2. The user consents at Google; Google redirects the browser to the `redirect_uri` — `/login/oauth2/code/google` — carrying a one-time *code*.
3. Auth-service exchanges `code` + `client-secret` for tokens server-to-server, reads the user's e-mail/profile, then the project's `OAuth2LoginSuccessHandler` takes over: find or create the local account and issue *our own* JWTs. Google authenticates; the application still authorizes with its own tokens.

PITFALL — REDIRECT_URI_MISMATCH

Symptom: Google shows “Error 400: redirect_uri_mismatch” instead of a consent screen. Cause: the URI the app sent is not byte-identical to one registered in the Google Cloud console. It changes per environment (`localhost:8080` in dev, `ts.local` in the cluster — which is why the Deployment overrides `...GOOGLE_REDIRECT_URI`), and every variant must be registered. The URI must reach auth-service *unstripped*, which is why the OAuth2 gateway routes carry no `StripPrefix`.

5.5 TOTP two-factor

Time-based One-Time Passwords (RFC 6238): at enrollment the server generates a secret, shows it as a QR code, and the authenticator app derives a 6-digit code from `HMAC(secret, time/30s)`. Login becomes two steps — password check issues a short-lived pending state, then `/verify-2fa` checks the code and issues the real tokens. Server and phone never communicate; they only share the secret and a clock.

5.6 Outside the project

The main industrial alternative moves token issuing out of your code entirely: an identity provider (Keycloak self-hosted, or Auth0/Cognito managed) issues to-

kens signed with an *asymmetric* keypair (RS256). Services validate with the *public* key fetched from a JWKS URL — no shared secret to distribute, key rotation built in:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: https://idp.company.com/realms/app/protocol/openid-connect/certs
```

Understanding the hand-rolled HS256 version — what this project builds — is precisely what makes that configuration meaningful: same claims, same validation, different key ceremony. Chapter 28 takes this design apart under load: where verification belongs, what asymmetric keys change, and how to keep tokens thin.

CHECK YOURSELF (answers: Appendix A)

1. Why can CSRF protection be disabled for a stateless JWT API but not for a cookie-session webapp?
2. An attacker steals an access token. How long is it useful, what cannot the server do about it meanwhile, and which endpoint finally stops the session?
3. In the trusted-edge pattern, state the exact invariant that must hold for X-User-Id to be trustworthy, and name two deployment mistakes that would break it.
4. In the Google flow, which secret never leaves the server, which value transits the browser, and why is the browser-visible one safe?
5. HS256 vs RS256: who can validate, who can sign, and why do identity providers choose RS256?

The Business Services and the Common Module

The previous chapters covered the plumbing services. This one maps the three that do the actual work, plus the shared `common` module — and, through them, the two ways microservices talk to each other.

6.1 course-service: the workhorse

Courses, students, assignments, enrollments, submissions — classic CRUD over Postgres (Chapter 7), plus file attachments stored in MinIO (Chapter 9). It *publishes* `ts.assignment.created` events and *listens* for `ts.user.activated` — the pattern to notice is that it reacts to another service’s business event without ever calling that service.

6.2 auth-service: identity

Everything Chapter 5 described: registration with e-mail activation, password and Google login, TOTP 2FA, JWT issuing, refresh tokens. It publishes three events: `ts.user.registered`, `ts.user.activated`, `ts.user.admin-provisioned`.

6.3 dictionary-service: the small one

Vocabulary lookups over MongoDB (Chapter 8). Deliberately simple — it exists partly to prove the architecture accommodates a document store and partly as the low-stakes service to try things on first.

6.4 notification-service: the pure consumer

No REST API of consequence, no database. It subscribes to three topics and turns events into e-mails (Maildev in dev, a real SMTP relay later):

Topic	Producer	Mail sent
ts.user.registered	auth-service	activation link
ts.user.admin-provisioned	auth-service	credentials notice
ts.assignment.created	course-service	new-assignment notice

Because its *only* input is Kafka, it is the one service that cannot usefully run before a broker exists — which is why its Kubernetes manifest shipped with the Kafka milestone (Chapter 25) rather than with the other services.

6.5 Synchronous vs. asynchronous, in this codebase

The project uses both interaction styles, each where it belongs:

Synchronous (HTTP through the gateway) when the caller needs the answer to proceed: the frontend fetching a course list. Coupled in time — both sides must be up.

Asynchronous (events through Kafka) when the caller only needs the fact recorded: a registration happened; whoever cares can act. auth-service does not know notification-service exists. Decoupled in time — mail goes out late, not never, if the consumer is down.

The rule of thumb the codebase follows: *commands* that need answers are HTTP; *facts* that others may react to are events, named in the past tense (UserRegistered, AssignmentCreated).

6.6 The common module

```
com/ts/common/
dto/      CourseDto, StudentDto, UserDto, ...
         UserRegisteredEvent, UserActivatedEvent,
         AssignmentCreatedEvent, AdminProvisionedUserEvent
exception/ ApiException, ErrorResponse
security/  JwtConstants    <- header names, claim names
```

Three kinds of sharing, with three different risk profiles:

- **Event records** are the *contract* between producer and consumer. Sharing the class guarantees both sides agree — at the price of lockstep deployment when the shape changes. (The industrial alternative — schema registry — appears in Chapter 11.)

- `JwtConstants` shares header and claim *names* between gateway and services. A typo'd header name would fail silently — requests simply arrive anonymous — so centralizing it is cheap insurance.
- **DTOs and exceptions** are convenience sharing: pleasant in a monorepo, and the first thing to duplicate-instead-of-share if services ever split into separate repositories, because shared convenience classes are how “independent” services end up released together.

DECISION — A SHARED MODULE AT ALL?

Microservice purists forbid shared code between services. The purist alternative — each service defines its own copy of every event — makes drift possible precisely where agreement is mandatory. This project draws the line at: share *contracts* (events, header names), think twice about sharing anything with behavior. `jib.skip=true` in `common`'s POM marks it as the one module that is a library, not a deployable.

PITFALL — THE LOCKSTEP TRAP

Symptom: after adding a field to `UserRegisteredEvent` and deploying only `auth-service`, `notification-service` starts throwing deserialization errors on every message — and because it shares a consumer group, it keeps retrying the same broken message. Cause: the contract changed on one side only, and the topic now holds events in two shapes. Rules that prevent it: only *add* optional fields, never rename or remove; deploy consumers before producers; and configure the consumer's error handling (Chapter 11) so one poison message cannot stall the topic.

CHECK YOURSELF (answers: Appendix A)

1. For each pair, say whether HTTP or an event fits and why: (a) frontend needs a student list; (b) a submission was graded and the student should be told; (c) course-service must verify a JWT.
2. Why is `notification-service` the only service whose Kubernetes manifest waits for the Kafka milestone, when `auth-service` also uses Kafka?
3. Name one thing that belongs in `common` and one that should never be there, with the reasoning.
4. What deployment-order rule makes an event schema change safe, and what property must the change itself have?

Part II

Data

Relational Data: Postgres, JPA, Flyway

Two of the services own a PostgreSQL database each: auth-service owns `ts_auth`, course-service owns `ts_course`. This chapter covers the three layers between a `@Service` method and bytes on disk — JPA/Hibernate, the connection pool, and Flyway — and the architectural rule that there are *two* databases at all.

7.1 Database-per-service

DECISION — TWO POSTGRES SERVERS, NOT TWO SCHEMAS IN ONE

Each service owns its data exclusively; the only way to another service's data is through its API or its events. This forbids the classic failure mode of shared databases — service A quietly joining service B's tables, coupling their schemas forever. The project takes it further than required: not just separate databases but separate server processes, so even accidental cross-connection is impossible and each can be tuned, backed up, or broken independently. Cost on this box: about 256 MiB per extra server. In a cloud account the same isolation is usually two databases on one managed RDS instance — isolation by credentials rather than by process — because managed instances are billed per server.

7.2 JPA and Hibernate

JPA is the specification (annotations like `@Entity`, `@OneToMany`, the `EntityManager`); Hibernate is the implementation Spring Boot auto-configures. Spring Data JPA sits on top, deriving queries from repository method names (`findByIdAndStudentId`) and generating the boilerplate.

The configuration from `configurations/course-service.yml`:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_course ①
    username: postgres
    password: root ②
  jpa:
```

hibernate:	
ddl-auto: none	③
show-sql: true	④
flyway:	
enabled: true	⑤

- ① The dev default. Compose overrides host to postgres-course; Kubernetes overrides identically via `SPRING_DATASOURCE_URL`. Note the k8s Service was deliberately *named* postgres-course so the override is the same string in both environments.
- ② Plaintext credentials are acceptable only because this file is the *dev default*; in the cluster, `SPRING_DATASOURCE_USERNAME/PASSWORD` arrive from a Kubernetes Secret (Chapter 16) and outrank these.
- ③ The most important line in the block — Section 7.3.
- ④ Every SQL statement is logged — a teaching tool and an N+1-query detector in dev; turn it off where log volume costs money.
- ⑤ Schema management belongs to Flyway, not Hibernate.

7.3 Why ddl-auto: none

Hibernate can generate schema from entities (`ddl-auto: update`), and every tutorial uses it. Production systems do not, because update is a one-way ratchet with no history: it adds columns but never drops or renames safely; two developers' databases diverge silently; there is no record of what changed when, and no way to review a schema change before it hits production.

7.4 Flyway: migrations as code

Flyway treats schema as an ordered series of immutable SQL scripts, applied exactly once each, tracked in a table (`flyway_schema_history`) inside the target database:

```
auth-service/src/main/resources/db/migration/
V1__create_users.sql
V2__create_refresh_tokens.sql
V3__add_admin_user_invitations.sql
V4__seed_default_admin.sql
```

At startup, Flyway compares the history table with the classpath scripts and applies the missing ones, in version order, before Hibernate ever initializes. The

rules that make it work: never edit an applied migration (each row stores a checksum — edits are detected and refused); only add new versions; and write migrations to be compatible with the *previous* application version when you deploy rolling (Chapter 15): add-then-migrate-then-remove, never rename in one step.

PITFALL — WORKS LOCALLY, CRASHLOOPBACKOFF IN THE CLUSTER

Symptom: a service boots on your machine but its pod restarts endlessly; logs show a Flyway checksum mismatch or a failed migration. Cause: the local database had been migrated by an older (or edited) script — your laptop’s history table and the cluster’s disagree. This is also why the Kubernetes startup probe (Chapter 17) targets an endpoint that only answers after the Spring context is up: “migrations applied” is part of what *ready* means. Fix by adding a corrective migration, never by editing history.

7.5 The connection pool

Opening a Postgres connection costs a TCP handshake, authentication, and a server process fork — far too much per request. HikariCP (Spring Boot’s default pool) keeps a small set of open connections that requests borrow and return. Default maximum: 10.

ON THIS HARDWARE

Pool sizing folklore says “more connections, more throughput”; the reality on a 2-core server is the opposite — Postgres does work *per connection*, and the practical formula ($\approx \text{cores} \times 2$) suggests the default 10 is already generous for two services sharing two cores. If Postgres ever shows too many clients, lower the pools before raising `max_connections`.

7.6 Outside the project

A managed-cloud equivalent of this chapter’s setup — AWS RDS Postgres with the same Flyway flow — changes only the URL and the trust model:

```
spring:
  datasource:
    url: jdbc:postgresql://app-db.xxxxx.eu-central-1.rds.amazonaws.com:5432/ts_course
    username: ${DB_USER}           # from AWS Secrets Manager
    password: ${DB_PASSWORD}
```

Backups, replication and failover become the provider's problem — which is why Chapter 19's warning about self-hosted state matters. The second industrial variant worth knowing: running Flyway as a separate CI step (`flyway migrate` against the target) instead of at application startup, so schema changes deploy independently of code and a slow migration cannot eat the startup probe's budget.

CHECK YOURSELF (answers: Appendix A)

1. Give two concrete failure modes of `ddl-auto: update` in a team setting that `none + Flyway` prevents.
2. Why must an applied migration never be edited, and what does Flyway use to detect it?
3. You need to rename a column without downtime under rolling deployments. Sketch the multi-release sequence.
4. Defend and attack the two-servers decision: one argument each way, and the setting in which each wins.

Document Data: MongoDB

8.1 The model

MongoDB stores *documents* — JSON-like structures (BSON) — in *collections*, without a fixed schema. Where Postgres normalizes a dictionary entry into rows across words, definitions, examples tables joined by keys, Mongo stores the whole entry as one nested document:

```
{ "word": "ubiquitous",
  "phonetic": "yoo-BIK-wi-tuhs",
  "meanings": [
    { "partOfSpeech": "adjective",
      "definitions": [
        { "definition": "present everywhere",
          "example": "smartphones are ubiquitous" } ] ] ] }
```

DECISION — WHY THE DICTIONARY IS THE MONGO SERVICE

A dictionary entry is read as a unit, written as a unit, and naturally nested — the document model’s best case. There are no cross-entry transactions and no joins to lose. Courses and enrollments, by contrast, are genuinely relational (a submission references a student *and* an assignment; consistency between them matters), so they stay in Postgres. “Polyglot persistence” is this: pick per service, by data shape — not one blessed database for everything, and not novelty for its own sake.

8.2 Spring Data MongoDB

The programming model mirrors JPA closely enough that the dictionary service is a good Rosetta stone: @Document instead of @Entity, MongoRepository instead of JpaRepository, derived query methods identical in spirit. What disappears: Flyway and schema DDL (collections spring into being on first write), the dialect, and the pool tuning (the driver manages its own connection pool). What appears: the single connection URI.

8.3 The configuration — all of it

Dictionary-service’s entire data config is one line, plus its cluster override:

```
# configurations/dictionary-service.yml (dev default)
spring:
  data:
    mongodb:
      uri: mongodb://localhost:27017/ts_dictionary

# deploy/k8s/base/dictionary-service.yaml (the override)
- name: SPRING_DATA_MONGODB_URI
  value: "mongodb://mongodb:27017/ts_dictionary"
```

The URI carries host, port, database, and (when enabled) credentials and options — one string to override per environment, which is exactly what the relaxed-binding machinery of Chapter 1 is for.

8.4 Schemaless cuts both ways

No migrations does not mean no schema — it means the schema lives *implicitly in the code*. Two documents written by two versions of the application can have different shapes in the same collection, and nothing but your reading code will notice. Disciplines that replace Flyway: keep document classes backward-readable (new fields optional), write data-fixing scripts for real shape changes, and validate at the boundary. Mongo can also enforce JSON Schema per collection — opt-in, and unused here.

PITFALL — THE COLLECTION THAT WAS NEVER CREATED

Symptom: the service starts, queries run, everything returns empty — no error anywhere. Cause: the URI pointed at a fresh database (a typo, or a new PVC in the cluster); Mongo silently created an empty `ts_dictionary` on first touch. Postgres would have failed loudly (database does not exist); Mongo’s create-on-write turns a wrong address into empty results. When a Mongo-backed service “loses” data, check *which database it is talking to* before anything else: `kubectl exec` into the pod’s node and run `mongosh --eval 'show dbs'`.

8.5 Operations in this cluster

The manifest (`deploy/k8s/base/mongodb.yaml`) follows the same `StatefulSet` pattern as Postgres (Chapter 15 explains the kind; 19 the storage), with two Mongo-

specific notes, both from its comments: the readiness probe execs `mongosh -eval "db.adminCommand('ping')"` — “accepting commands,” not merely “port open” — and it runs without authentication, acceptable only because it is a ClusterIP service with no Ingress: unreachable from outside the cluster. The moment that assumption changes (an exposed NodePort, multi-tenancy), auth comes first.

8.6 Outside the project

The managed version is MongoDB Atlas; the URI grows credentials and TLS and nothing else changes:

```
spring:
  data:
    mongodb:
      uri: mongodb+srv://app:${MONGO_PWD}@cluster0.abc.mongodb.net/ts_dictionary?
        ↪ retryWrites=true
```

For contrast, DynamoDB shows a document store with the opposite philosophy: schemaless like Mongo, but access patterns must be designed into the table (partition keys, no ad-hoc queries) — a reminder that “NoSQL” is a family of trade-offs, not one thing.

CHECK YOURSELF (answers: Appendix A)

1. Which properties of the dictionary data make it a good document case, and which properties of enrollments make them a bad one?
2. “Schemaless” moves schema enforcement from the database to where? Name two disciplines that replace migrations.
3. Why does a wrong database name fail loudly in Postgres but silently in Mongo, and which failure do you prefer in production?
4. The Mongo pod has no password while Postgres does, in the same cluster. Reconstruct the reasoning and state the condition that invalidates it.

Object Storage: MinIO and the S3 Model

9.1 The problem

Assignment submissions are files — PDFs, images, archives. Files in a relational database bloat backups and stream badly; files on a pod’s local disk vanish with the pod. The industry’s answer is *object storage*: an HTTP service storing immutable blobs in named *buckets* under string *keys*, metadata included, no directories (the / in a key is naming convention, not structure). Amazon S3 defined the API; MinIO is a self-hostable server speaking the same API — so course-service uses the standard AWS SDK and cannot tell the difference.

9.2 The configuration, annotated

From configurations/course-service.yml:

```
app:
  s3:
    endpoint: ${APP_S3_ENDPOINT:http://localhost:9000} ①
    public-endpoint: ${APP_S3_PUBLIC_ENDPOINT:http://localhost:9000} ②
    access-key: ${APP_S3_ACCESS_KEY:minioadmin} ③
    secret-key: ${APP_S3_SECRET_KEY:minioadmin}
    bucket: ${APP_S3_BUCKET:ts-submissions}
    presigned-url-expiry-minutes: ${APP_S3_PREIGNED_EXPIRY_MIN:15} ④
```

- ① Where *the service* talks to MinIO. In the cluster: `http://minio:9000`.
- ② Where *the browser* talks to MinIO — the subject of the pitfall below.
- ③ Credentials with dev defaults; the cluster injects real ones from the minio-app Secret. `app.s3.*` is custom (@ConfigurationProperties) config, not Spring-managed — the same override machinery applies regardless.
- ④ How long a presigned link stays valid.

9.3 Presigned URLs: the pattern worth knowing

Naive file download routes bytes *through* the service: browser → service → MinIO and back, tying up an application thread per transfer. The S3-native alternative: the service uses its credentials to *sign a URL* — bucket, key, expiry,

HMAC signature — and hands the URL to the browser, which downloads *directly* from storage. The service spends microseconds; storage does the streaming; the link self-expires. The same trick works for uploads.

PITFALL — THE TWO-ENDPOINTS PROBLEM

Symptom: uploads work, but clicking a download link fails with `ERR_NAME_NOT_RESOLVED` on `minio:9000`. Cause: a presigned URL embeds the endpoint *it was signed against*. The service signs against `minio:9000` — a name only resolvable inside the cluster network — and the browser lives outside it. Hence two configured endpoints: sign download links against public-endpoint, talk to storage over endpoint. Every containerized S3 setup meets this bug once; some meet it again with the signature itself, because the signature covers the host — you cannot sign against one host and rewrite the URL to another afterwards.

ON THIS HARDWARE

MinIO here is a single container with a single PVC — fine for a homelab’s submissions, nothing like production MinIO (multi-node erasure coding) or S3’s eleven nines of durability. The interesting part is that course-service cannot tell: the same SDK calls would hit real S3 with only the endpoint and credentials changed. That is the point of speaking a standard API.

9.4 Outside the project

The same service on AWS proper: set endpoint and public-endpoint to the real S3 URL (or leave the SDK’s default), replace static keys with an IAM role assumed by the pod (IRSA on EKS — no long-lived credentials at all), and turn on bucket versioning and lifecycle rules (auto-expire old submissions to cheaper storage). The concepts — buckets, keys, presigning, the identity that signs — transfer unchanged; what changes is who manages durability and how identity is proven.

CHECK YOURSELF (answers: Appendix A)

1. Why do presigned URLs scale better than proxying downloads through the service, and what new failure mode do they introduce?
2. Explain why endpoint and public-endpoint must differ inside the cluster, and why post-signing URL rewriting is not a fix.
3. A presigned URL leaks in a chat log. What limits the damage, by design?

4. What changes — and what does not — when this code moves from MinIO to real S3?

Part III

Messaging

Kafka: The Core Model

Kafka is the technology in this stack most worth learning deeply: it appears in a large fraction of backend job descriptions, and its model is genuinely different from a message queue's. This chapter is platform-independent Kafka; Chapter 11 is the Spring side.

10.1 A log, not a queue

Kafka's core abstraction is an *append-only log*. A *topic* (e.g. `ts.user.registered`) is split into *partitions*; each partition is an ordered, immutable sequence of *records*, each at a numbered *offset*. Records are not deleted when read — they expire by retention policy (this project: 168 hours). Reading is not destructive; a consumer just remembers *how far it got*.

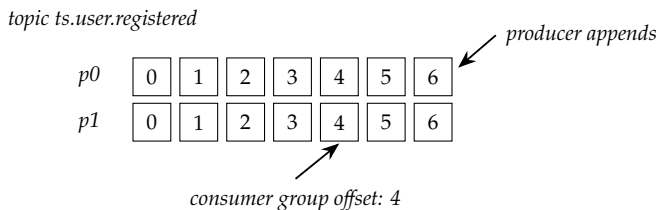


Figure 10.1: Two partitions; producers append at the head, consumer groups remember an offset per partition.

The consequences, each of which a queue lacks:

- **Replayable.** A new consumer can start from offset 0 and see history. Deploy a new analytics service next year; it can process last week's registrations.
- **Multi-reader.** Notification-service and course-service both read auth events independently, at their own pace, without stealing from each other — each *consumer group* keeps its own offsets.
- **Ordered per partition.** Records with the same *key* hash to the same partition and stay in order. This is why the publishers in this project key by `userId`: all events about one user arrive in sequence. Across partitions there is no order — design around per-key order only.

10.2 Consumer groups

A consumer group is a named team splitting work: each partition is assigned to exactly one consumer *within the group*. Two consumers in group notification-group on a 2-partition topic get one partition each; a third sits idle — *partitions are the unit of parallelism*. When a consumer joins or dies, the group *rebalances*: partitions are reassigned and members briefly pause. Different groups are invisible to each other, which is how the same event fans out to both notification-group and course-group.

10.3 Brokers, replication, KRaft

A production cluster runs several *brokers*; each partition has a leader on one broker and *replicas* on others (replication-factor, RF). Producers write to the leader; acks=all waits until in-sync replicas have it. Metadata coordination historically required ZooKeeper, a second distributed system; modern Kafka replaces it with *KRaft* — brokers running an internal Raft quorum. This project runs KRaft with one node playing both broker and controller roles, RF = 1: fine for a homelab, zero fault tolerance — one disk failure loses the log.

10.4 The advertised-listeners maze

The single most-Googled Kafka error. From docker-compose.yml:

```
KAFKA_LISTENERS: PLAINTEXT://:19092,CONTROLLER://:9093,EXTERNAL://:9092
    ↪ ①
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:19092,EXTERNAL://
    ↪ localhost:9092 ②
KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093 ③
```

- ① *Listeners*: the sockets the broker opens. Three: one for clients inside the Docker network, one for the KRaft controller, one published to the host.
- ② *Advertised listeners*: the addresses the broker *tells clients to use*. This is the trap: a Kafka client's first request is "where are the partition leaders?", and the broker answers with these strings. Containers must be told `kafka:19092` (resolvable on the Docker network); tools on your laptop must be told `localhost:9092`. One broker, two audiences, two advertised addresses.
- ③ KRaft quorum of one: node 1 is the whole election.

PITFALL — CONNECTS, THEN TIMES OUT

Symptom: a client connects to the bootstrap address successfully, then every send/poll fails with Connection to node -1 could not be established or hangs. Cause: bootstrap worked, but the *metadata answer* advertised an address the client cannot reach (e.g. your laptop was told kafka:19092). The bootstrap address only starts the conversation; the advertised listeners carry all subsequent traffic. Diagnose by asking what the broker advertises — e.g. `kcat -L -b localhost:9092` — and checking those exact strings resolve from where the client runs.

10.5 Delivery semantics

Kafka's default is *at-least-once*: a producer retry after a lost ack, or a consumer crash after processing but before committing its offset, both cause duplicates — not loss. Design consumers to be *idempotent* (processing the same event twice is harmless): notification-service sending one duplicate mail is acceptable; a payment service would instead track processed event IDs. Exactly-once exists (idempotent producers + transactions) but costs complexity; know that at-least-once + idempotent consumer is the industry's bread and butter.

10.6 Outside the project

Managed Kafka (AWS MSK, Confluent Cloud) removes broker operations and sells the same protocol — your clients change a bootstrap string and gain SASL/TLS. The other names in this space solve different problems: RabbitMQ is a smart-routing *queue* (work distribution, per-message acks, no replay); Redpanda reimplements the Kafka protocol without the JVM. When Chapter 25 moves this broker into the cluster under Strimzi, nothing in this chapter changes — Strimzi is *operations* for the same Kafka.

CHECK YOURSELF (answers: Appendix A)

1. Registration events for one user must be processed in order, but scaling matters. What do you key by, how many partitions, and what is the parallelism limit?
2. Two groups, notification-group and course-group, read `ts.user.activated`. Explain how both get every event.
3. Distinguish LISTENERS from ADVERTISED_LISTENERS, and explain

why the bug they cause appears *after* a successful connection.

4. Why does at-least-once delivery push idempotency onto the consumer, and what would notification-service's idempotency strategy reasonably be?

Spring Kafka: Producers, Listeners, Poison Pills

11.1 Producing

Publishing an event in this project is two moving parts: a `KafkaTemplate` and configuration choosing serializers. From `auth-service`:

```
kafkaTemplate.send("ts.user.registered", event.userId(), event);
//              topic              key          value
```

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
```

The key (`userId`) is a string; the value is the event record serialized to JSON. Keying by user id is the ordering decision from Chapter 10 made concrete. `send()` is asynchronous — it returns a future; the message sits in a client-side buffer until the broker acks. Two operational consequences: a service can *start* without a broker (sends fail later, per call — which is why `auth-service` could deploy to the cluster before Kafka), and a service that crashes right after `send()` returns may lose the buffered message — the outbox pattern below exists for when that matters.

11.2 Consuming

```
@KafkaListener(topics = "ts.user.registered",
                groupId = "notification-group")
public void on(UserRegisteredEvent event) {
    mailService.sendActivation(event);
}
```

Spring runs a polling loop per listener container, deserializes each record, invokes the method, and commits offsets after successful processing — at-least-

once out of the box. The consumer half of the YAML holds this chapter’s most production-relevant lines:

```
spring:
  kafka:
    consumer:
      group-id: course-group
      auto-offset-reset: earliest ①
      key-deserializer: ...ErrorHandlingDeserializer ②
      value-deserializer: ...ErrorHandlingDeserializer
      properties:
        spring.deserializer.value.delegate.class: ...JsonDeserializer
        spring.json.trusted.packages: com.ts.common.dto ③
```

- ① Where a *brand-new* group starts when it has no committed offset: `earliest` = from the beginning of retention; the default `latest` = only new messages. `earliest` means a consumer deployed late still processes events published before it existed — the right choice when events are facts to act on. Irrelevant once the group has offsets.
- ② The poison-pill defense — next section.
- ③ JSON deserialization instantiates classes named in message headers; trusting only `com.ts.common.dto` stops a malicious or buggy producer from making the consumer instantiate arbitrary classes. A security boundary, not boilerplate.

11.3 The poison pill

A *poison pill* is a message that fails deserialization forever — a producer bug, a renamed field, a corrupted payload. Without defenses the listener throws *before* the message can be skipped, the offset never advances, and the container retries the same record endlessly: the partition is stalled and the logs fill at full speed.

`ErrorHandlingDeserializer` wraps the real deserializer: on failure it emits a typed error instead of throwing inside the poll loop, letting Spring’s error handler skip past the record (optionally after retries, optionally publishing it to a dead-letter topic). The composition — error-handler outside, JSON delegate inside — is the standard production pattern, and it is why the config names *two* deserializer classes per side.

PITFALL — ONE BAD MESSAGE, TOTAL SILENCE

Symptom: notification mails stop entirely; the consumer logs the same deserialization stack trace thousands of times. Cause: a poison pill on one partition

with no error-handling deserializer — the loop can neither process nor pass it. This failure hides in systems that never tested a malformed message. Verify your defense once, deliberately: publish garbage to a dev topic and watch it get skipped.

11.4 Advanced corners

Dead-letter topics. `DefaultErrorHandler` + `DeadLetterPublishingRecordRecoverer`: after N failed attempts, the record lands on `<topic>.DLT` for inspection and replay — failure handling with an audit trail.

The outbox pattern. “Save the user *and* publish the event” is two systems and cannot be atomic directly. The outbox: write the event into an outbox table in the *same database transaction* as the user row; a relay (or Debezium reading the WAL) publishes from that table. If auth-service ever must not lose a registration event, this is the answer.

Schema registry. Chapter 6’s shared event classes solve producer/consumer agreement inside one repo. At organization scale the contract moves into a schema registry (Avro/Protobuf schemas, checked for compatibility at publish time) — the same idea, enforced by infrastructure instead of a shared jar.

CHECK YOURSELF (answers: Appendix A)

1. Why can auth-service start when Kafka is down, and at what moment does a Kafka outage become visible to it?
2. A new consumer group is deployed today. Explain what `auto-offset-reset: earliest` makes it do, and when this setting stops having any effect for that group.
3. Walk through the poison-pill failure without `ErrorHandlingDeserializer`, then with it. Where does the bad record end up in a full production setup?
4. Why is `spring.json.trusted.packages` a security control?
5. When is the outbox pattern necessary, and what pair of writes does it make effectively atomic?

Part IV

Containers

Docker: Images, Containers, Compose

12.1 What a container actually is

A container is not a small virtual machine. It is an ordinary Linux process, isolated by two kernel features: *namespaces* (its own view of processes, network, filesystem mounts, hostname) and *cgroups* (limits on CPU and memory). No guest OS, no hypervisor — which is why a container starts in milliseconds and why the JVM inside one must be told about its memory limit (Chapter 17’s `MaxRAMPercentage`) — from inside, it can see the *host’s* RAM.

An *image* is the packaged filesystem plus metadata (entrypoint, env, exposed ports), built as a stack of immutable *layers*. Layers are content-addressed and shared: fifty images `FROM eclipse-temurin:25-jre-alpine` store the JRE layer once. A *registry* (Docker Hub, GHCR, the project’s `registry:2`) stores images; `docker push/pull` move only layers the other side lacks.

12.2 The project’s Dockerfile, annotated

Every service carries the same five lines:

```
FROM eclipse-temurin:25-jre-alpine ①
WORKDIR /app
COPY target/*.jar app.jar ②
EXPOSE 8082 ③
ENTRYPOINT ["java", "-jar", "app.jar"]④
```

- ① Base image: a JRE (not JDK — no compiler needed at runtime) on Alpine Linux. Smaller base = smaller attack surface and faster pulls.
- ② Copies the Maven-built fat jar. Subtlety: this Dockerfile requires `mvn` package to have run *first* — the build happens outside. The consequence: one `COPY` of a ~60 MB jar makes one big layer, so *every* build re-pushes everything even for a one-line change. Jib (Chapter 13) exists to fix exactly this.
- ③ Documentation, not action — it opens nothing. Publishing ports happens at run time.
- ④ Exec-form entrypoint: `java` is PID 1, receives signals directly, so `docker stop` triggers a clean Spring shutdown.

12.3 Networking and Compose

Containers on a Docker *bridge network* reach each other by *service name* — Docker runs a DNS server answering `postgres-course`, `kafka`, `minio`. `localhost`, inside a container, is *that container* — the number-one beginner trap, and the reason every dev-default URL in the configs needed a per-environment override. Ports are only reachable from the host when *published* (-p 5434:5432).

Compose turns a fleet of docker run commands into one declarative file — the project’s `docker-compose.yml` is the whole dev stack: four datastores, Kafka, Zipkin, Maildev, and the seven services, with three idioms worth naming:

```
discovery-server:
  depends_on:
    config-server:
      condition: service_healthy ①
config-server:
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8888/actuator/health"] ②
postgres-course:
  ports:
    - "5434:5432" ③
  volumes:
    - postgres-course-data:/var/lib/postgresql/data
```

①+② encode startup order — and not merely “started” but “answering its health endpoint,” which is what makes the ordering real. ③ maps distinct host ports (5433, 5434) onto the two Postgres containers’ identical internal 5432 — names are per-network, host ports are global. The named volume keeps data across container recreation; Chapter 19 is the Kubernetes version of the same need.

DECISION — WHY COMPOSE SURVIVES ALONGSIDE KUBERNETES

The cluster is the deployment target, but Compose remains the inner loop: one command, laptop-local, debug ports mapped (every service opens a JWP port in Compose — attach the IDE debugger to 5082 and step through `course-service`). Kubernetes buys self-healing and rolling deploys at the cost of ceremony; a developer iterating on business logic wants neither cost. Two environments, one image contract between them.

PITFALL — WORKS IN COMPOSE, DIES IN KUBERNETES

Symptom: the stack is green locally; the same image crash-loops in the cluster. The usual suspects, all environmental: a URL still pointing at a Compose service name that has a different Service name in k8s; a `depends_on` ordering the cluster does not honor (Kubernetes has no startup ordering — Chapter 15); or memory limits present in the cluster but absent locally, turning a fat JVM into an OOMKill. The image is identical — diff the *environment*, not the code: `docker inspect` vs. `kubectl describe pod`.

CHECK YOURSELF (answers: Appendix A)

1. A container and a VM both isolate. Name two mechanisms the container uses, and two consequences of not having a guest kernel.
2. Why does `localhost:5432` fail inside a container that works on your laptop, and what two different fixes do Compose and Kubernetes offer?
3. Both Postgres containers listen on 5432 internally without conflict — explain. Where do conflicts become possible again?
4. What makes condition: `service_healthy` stronger than plain `depends_on`, and which project file supplies the health signal it waits for?

Images Without a Daemon: Jib and the Local Registry

13.1 Why not just `docker build` in CI?

The Dockerfiles work. Three reasons the pipeline builds images differently:

1. **CI needs no Docker daemon.** `docker build` requires a running daemon; giving Jenkins one means docker-in-docker or mounting the host's Docker socket — which is root on the host, handed to every build. Jib builds and pushes images in pure Java: no daemon, no privilege.
2. **Layering that matches how code changes.** The Dockerfile copies one fat jar: one huge layer, fully re-pushed per build. Jib splits the application into layers by volatility — dependencies / resources / classes — so a one-line code change re-pushes only the small classes layer. On two cores and Wi-Fi this is the difference between seconds and minutes per service.
3. **OpenShift-ready by accident.** Jib images run as a non-root user with group-0 file permissions — exactly what OpenShift's arbitrary-UID security model demands. Portability for free.

13.2 The configuration, annotated

Jib is configured once in the parent `pom.xml`'s `pluginManagement`; each runnable module opts in with a bare `<plugin>` reference, and common opts *out* with `jib.skip=true`:

```
<plugin>
  <groupId>com.google.cloud.tools</groupId>
  <artifactId>jib-maven-plugin</artifactId>
  <version>3.4.4</version>
  <configuration>
    <from><image>eclipse-temurin:25-jre-alpine</image></from> ①
    <to><image>${docker.registry}/${project.artifactId}:${project.version}</image></to>
    ↪ ②
    <allowInsecureRegistries>true</allowInsecureRegistries> ③
  </configuration>
  <container>
    <mainClass>${start-class}</mainClass> ④
    <creationTime>USE_CURRENT_TIMESTAMP</creationTime> ⑤
  </container>
</plugin>
```



```
</container>
</configuration>
</plugin>
```

- ① Same base as the Dockerfiles — runtime behavior unchanged, only the packaging path differs.
- ② Resolves to localhost:5000/course-service:1.0.0-SNAPSHOT. The name is written *from the cluster’s point of view* — Section 13.3.
- ③ The local registry is plain HTTP; Jib refuses insecure pushes unless told. Every tool in this chain (Jib, containerd) makes the same secure-by-default choice and needs the same explicit opt-out.
- ④ Set explicitly because Jib’s bundled bytecode scanner does not yet parse Java 25 class files — a real example of bleeding-edge Java taxing the toolchain.
- ⑤ Jib defaults to epoch-0 timestamps for reproducible builds; this trades that away so docker images shows honest build times.

13.3 The registry, and why localhost:5000

```
docker run -d --restart=unless-stopped --name registry \
  -p 5000:5000 -v registry-data:/var/lib/registry registry:2
```

The non-obvious fact that forces this architecture: **k3s does not see Docker’s images**. k3s runs its own containerd with its own image store; docker build on the host puts images somewhere k3s never looks. A registry is the honest interface between builder and cluster — which the pipeline wants anyway.

An image name is not just a name — it is *where to pull from*: localhost:5000/course-service means “pull from localhost:5000.” Since the puller is the k3s node itself, and pushing Jenkins shares the host’s network namespace (Chapter 22), the same string is valid for both — and localhost can never drift the way a LAN IP or host-name can. Two configuration hooks make it work:

```
# /etc/rancher/k3s/registries.yaml -- containerd assumes HTTPS otherwise
mirrors:
  "localhost:5000":
    endpoint: ["http://localhost:5000"]
```

PITFALL — HTTP: SERVER GAVE HTTP RESPONSE TO HTTPS CLIENT

The signature error of this chapter. Both Jib (pushing) and containerd (pulling) default to TLS; registry:2 speaks plain HTTP. If a pod sticks in ImagePullBackOff with this text in `kubectl describe pod`, the `registries.yaml` entry is missing — and note containerd reads it *only at startup*: `systemctl restart k3s` after editing.

DECISION — LOCAL REGISTRY VS. GHCR

GitHub Container Registry would make images pullable anywhere and add vulnerability scanning — for the price of pushing hundreds of megabytes up residential Wi-Fi on every build. A LAN-local registry keeps the build loop at disk speed. The decision flips the moment a second machine must pull images; nothing in the manifests would change except the image prefix.

13.4 Tags, digests, and the SNAPSHOT trap

A *tag* (:1.0.0-SNAPSHOT, :a1b2c3d) is a movable pointer; a *digest* (@sha256:...) is the immutable content hash. The pipeline pushes every build twice-labelled: the SNAPSHOT tag (which the manifests reference) and the git-SHA tag (which deploys pin — Chapter 23). Kubernetes caches images by tag; a re-pushed SNAPSHOT would be silently ignored on the node — which is why every Deployment sets `imagePullPolicy: Always`: re-check the registry each start; if the digest matches, the check costs one manifest fetch.

CHECK YOURSELF (answers: Appendix A)

1. Name the three CI problems Jib solves over daemon-based builds, and which one is a security issue.
2. Why must the image name be written from the *cluster's* viewpoint, and what breaks if Jenkins and k3s disagree about what `localhost:5000` means?
3. A colleague suggests `docker save | k3s ctr images import` instead of a registry. What does that cost per deploy, and which pipeline stage stops working naturally?
4. Explain how a re-pushed :1.0.0-SNAPSHOT plus default pull policy produces “I deployed but nothing changed,” and both fixes.

Part V

Kubernetes

Kubernetes: Cluster Anatomy and the Reconciliation Loop

14.1 The one idea everything else hangs on

You do not tell Kubernetes to *do* things; you tell it what should *be true*. You write *desired state* (“a Deployment named course-service with 1 replica of image X”) into its database via the API server; *controllers* run reconciliation loops forever comparing desired with actual and acting on the difference. Delete a pod: its controller notices $\text{actual} \neq \text{desired}$ and makes another. Nobody replays your commands — the state converges.

This explains behaviors that otherwise seem strange:

- `kubectl apply` is a merge into desired state, not a script run — apply the same file twice, nothing happens the second time (idempotence — the property the pipeline leans on in Chapter 23).
- There is no “startup order.” Everything starts at once; services that need absent dependencies crash and are restarted until the world is ready. Crash-looping *during* convergence is normal operation, not failure.
- Deleting from a manifest file does not delete from the cluster — apply adds and updates, it does not prune. Removing the old course-service Ingress took an explicit `kubectl delete`.

14.2 The moving parts

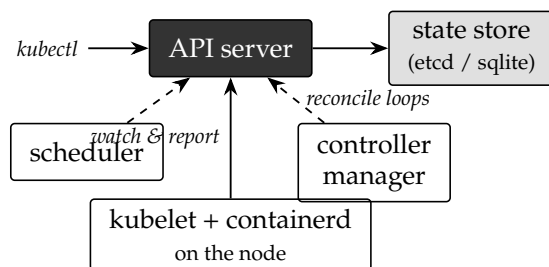


Figure 14.1: Everything talks only to the API server.

API server the single front door; `kubectl` is a REST client for it. Authentication, authorization (Chapter 22's RBAC), and validation happen here.

State store the desired-state database (etcd in standard clusters; k3s embeds sqlite on a single node).

Scheduler assigns unscheduled pods to nodes, honoring the resource *requests* of Chapter 17.

Controller manager the reconciliation loops: Deployment → ReplicaSet → pods, endpoints, and dozens more.

kubelet the node agent: watches for pods assigned to its node, drives containerd to run them, executes the probes, reports status.

k3s packages all of this into one binary — a CNCF-certified Kubernetes, minus a VM's overhead, plus batteries: Traefik as Ingress controller, local-path storage, containerd. Every manifest in this book would apply unchanged to EKS or OpenShift; that certification is why learning on k3s transfers.

14.3 Objects, labels, namespaces

Every object is a YAML document with the same skeleton — `apiVersion`, `kind`, metadata (`name`, `namespace`, `labels`), `spec` (desired), `status` (observed, written by controllers). Two organizing mechanisms:

Namespaces partition the cluster; this project puts everything in `ts`, keeping it separable from `kube-system` and making RBAC scopeable — Jenkins' permissions stop at the namespace edge (Chapter 22).

Labels arbitrary key/value pairs; *selectors* query them. Every wiring in Part V is a label selector: a Service finds pods by label, a Deployment owns pods by label. The kustomization stamps `app.kubernetes.io/part-of: teacher-supporter` onto everything so one selector can list — or delete — the whole application.

14.4 Getting kubectl to the cluster

`kubectl` reads `~/.kube/config`: cluster URL, CA certificate, and a credential. From Windows, this project reaches `https://100.85.66.43:6443` over Tailscale — which required installing k3s with `-tls-san 100.85.66.43`, because the API server's certificate is only valid for the names baked in at install time.

HANDS-ON — FEEL THE LOOP

```
kubectl -n ts get pods
kubectl -n ts delete pod -l app=course-service
```

```
kubectl -n ts get pods -w
```

Delete the pod, then watch: a replacement appears within seconds, created not by your command but by the ReplicaSet controller noticing a missing replica. You never ordered a restart — you made actual state diverge from desired, and the loop closed the gap.

PITFALL — CERTIFICATE IS VALID FOR... NOT 100.85.66.43

Symptom: `kubectl` from Windows fails TLS validation while working fine on the server. Cause: the API server certificate lacks the tailnet address as a SAN — it was installed without `-tls-san`. Fix at install time is one flag; fix afterwards means deleting the server TLS directory and restarting k3s. A concrete instance of a general rule: certificates bind to *names*, so decide every name a server will be called *before* issuing.

CHECK YOURSELF (answers: Appendix A)

1. Explain “declarative”: what do you write, who acts on it, and why does applying the same manifest twice change nothing?
2. Why is there deliberately no `depends_on` in Kubernetes, and what replaces startup ordering during convergence?
3. Trace `kubectl -n ts delete pod ...` through the components: who receives it, who notices, who acts, on which machine?
4. Why did removing the old Ingress from git not remove it from the cluster, and what two commands would each have removed it?

Workloads: Pods, Deployments, StatefulSets

15.1 Pods

The pod is the unit Kubernetes schedules: one or more containers sharing a network namespace (one IP, `localhost` between them) and volumes. Almost every pod here is a single container; the multi-container patterns (sidecars for proxies or log shippers) matter when you meet a service mesh. Pods are *mortal by design* — they are never healed, only replaced, and their IP dies with them. Everything else in Part V — controllers, Services, PVCs — exists to build durable behavior out of disposable pods.

15.2 Deployments and the rollout

You never create bare pods. A *Deployment* declares “N replicas of this pod template”; it manages a *ReplicaSet* (the counter), which manages the pods. The three-layer indirection is what makes updates elegant: change the template (a new image tag), and the Deployment creates a *new* *ReplicaSet*, scaling it up while the old scales down:

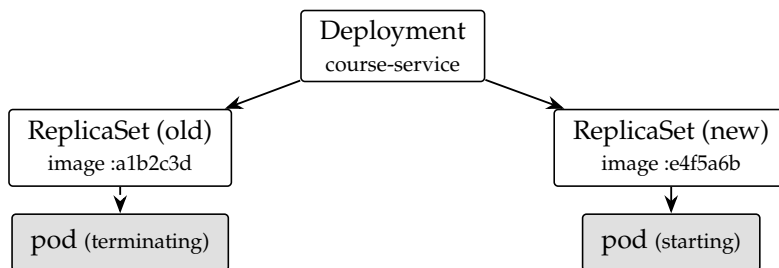


Figure 15.1: A rolling update mid-flight: two *ReplicaSets*, traffic shifting.

The rollout only proceeds as new pods become *Ready* (Chapter 17’s probes — this is where they earn their keep), so a bad image stops the rollout instead of taking the service down; and `kubectl rollout undo` points back at the previous

ReplicaSet, which is kept around precisely for this. The pipeline’s `kubectl rollout status` (Chapter 23) is CI waiting on this machinery to conclude.

With one replica — this cluster — a rolling update still means start-new-then-stop-old, a brief two-pod overlap costing a moment of double memory. `maxUnavailable`/`maxSurge` tune that dance at scale.

15.3 StatefulSets: when identity matters

For the databases, interchangeability is exactly wrong: whichever pod comes up must be *the same Postgres* with *the same data*. A StatefulSet provides what a Deployment refuses to: stable names (`postgres-course-0`, not a random suffix), per-replica PVCs via `volumeClaimTemplates` (Chapter 19) that survive the pod and re-attach to its successor, and ordered, one-at-a-time replacement. From `postgres-course.yaml`:

```
kind: StatefulSet
spec:
  serviceName: postgres-course
  replicas: 1
  volumeClaimTemplates: # a PVC per replica, not per manifest
    - metadata: { name: data }
      spec:
        accessModes: [ReadWriteOnce]
        resources: { requests: { storage: 5Gi } }
```

The decision rule: does the *process* hold state the next process must inherit? Databases yes; Spring services no — their state lives in those databases, which is what made them Deployments, and what makes them scalable and rollable at all. (The JWT statelessness of Chapter 5 is this same property one layer up.)

PITFALL — THE DEPLOYMENT-DATABASE DATA LOSS

Symptom: someone runs a database as a Deployment with a plain volume; it works for months; then a node problem creates a *second* replica pointing at the same files, and the database corrupts. Cause: a Deployment’s contract is “interchangeable replicas” and its rolling update briefly runs old and new *simultaneously* — fatal when both open the same data directory. StatefulSet ordering + per-replica claims exist to make this impossible. The lesson generalizes: choose the controller by its *contract*, not by which YAML you saw first.

15.4 Other workload kinds

For completeness, the remaining controllers, none yet used here: **DaemonSet** (one pod per node — log collectors, node metrics), **Job** (run to completion — Chapter 7’s Flyway-as-CI-step would be one), **CronJob** (Jobs on a schedule — backups, Chapter 19’s missing piece).

HANDS-ON — WATCH A ROLLOUT HAPPEN

```
kubectrl -n ts get rs -l app=course-service      # one ReplicaSet
kubectrl -n ts set image deployment/course-service \
  course-service=localhost:5000/course-service:1.0.0-SNAPSHOT
kubectrl -n ts rollout status deployment/course-service
kubectrl -n ts get rs -l app=course-service      # two -- old kept at 0
kubectrl -n ts rollout undo deployment/course-service
```

The second `get rs` is the point: the old `ReplicaSet` still exists, scaled to zero — the undo target, kept by default for the last ten revisions.

CHECK YOURSELF (answers: Appendix A)

1. Why does a `Deployment` manage `ReplicaSets` instead of pods directly? What feature falls out of that indirection?
2. State the contract difference between `Deployment` and `StatefulSet` in one sentence each, then classify: Eureka server, a Redis cache, a Kafka broker, the api-gateway.
3. During a rolling update, what condition gates old-pod termination, and which chapter’s machinery provides it?
4. Explain the exact mechanism by which running Postgres under a `Deployment` invites corruption where a `StatefulSet` does not.

Configuration and Secrets in the Cluster

16.1 Two injection mechanisms

Part I established that env vars outrank every file. Kubernetes offers two ways to set them, and the manifests use both deliberately:

```
env:                                # literal, visible in git
- { name: SPRING_DATASOURCE_URL,
  value: "jdbc:postgresql://postgres-course:5432/ts_course" }
envFrom:                            # every key of a Secret
- secretRef: { name: postgres-course-app }
- secretRef: { name: jwt }
```

The split *is* the security boundary: topology (hostnames, ports, feature flags) is not secret and belongs in reviewable git; credentials never appear in git at all. When you read a Deployment here, every env: entry is public configuration and every envFrom: line is a pointer to something created out-of-band.

16.2 Secrets: what they are and are not

A Secret is a namespaced object holding base64-encoded key/value pairs. Two honest statements about it:

- Base64 is *encoding*. `echo cm9vdA== | base64 -d` reveals the password. The protections are access control (RBAC decides who may read Secrets — notice Chapter 22 grants Jenkins no such verb), separate lifecycle from code, and optional encryption at rest.
- It is still the right container: every mechanism above it (Sealed Secrets, Vault, cloud secret managers) ultimately materializes a Secret for pods to consume. Learning the plain object first is learning the interface.

16.3 `create-secrets.sh`, annotated

Secrets here are created by an idempotent script, not manifests:

```
kubectrl -n "$NS" create secret generic postgres-course \
--from-literal=POSTGRES_USER="$PG_USER" \
```

```
--from-literal=POSTGRES_PASSWORD="$PG_PASSWORD" \
--from-literal=POSTGRES_DB=ts_course \
--dry-run=client -o yaml | kubectl apply -f - ①
```

```
kubectl -n "$NS" create secret generic postgres-course-app \
--from-literal=SPRING_DATASOURCE_USERNAME="$PG_USER" \
--from-literal=SPRING_DATASOURCE_PASSWORD="$PG_PASSWORD" \
--dry-run=client -o yaml | kubectl apply -f - ②
```

```
kubectl -n "$NS" create secret generic jwt \
--from-literal=JWT_SECRET="$JWT_SECRET" ③
```

- ① The idempotent-create idiom: `create` alone fails if the Secret exists; rendering with `--dry-run` and piping to `apply` makes the script safely re-runnable — `create` or `update`, either way converging. A tiny declarative-over-imperative lesson in shell form.
- ② The same credential twice, spelled per consumer: the Postgres image wants `POSTGRES_USER`; Spring wants `SPRING_DATASOURCE_USERNAME`. Two Secrets mean each pod's `envFrom` imports only names it understands — no pod carries a grab-bag of foreign variables.
- ③ One Secret consumed by two Deployments — `auth-service` signs JWTs with it, `api-gateway` validates. The shared-key contract of Chapter 5, expressed as infrastructure: rotating it is one script run and two pod restarts, and the two consumers *cannot* drift.

DECISION — SCRIPT, NOT MANIFESTS — AND NOT IN THE KUSTOMIZATION

Secret manifests in git would put base64'd credentials one base64 -d away from anyone with repo access, forever, in history. The script keeps values in the operator's environment (`JWT_SECRET=... ./create-secrets.sh`), defaults matching dev so behavior is identical, and git holding only the *shape*. The kustomization's comment says the same thing from the other side: secrets are deliberately absent from resources:.

16.4 ConfigMaps

The Secret's non-secret twin — same mechanics, no pretense of sensitivity, plus a talent Secrets share but ConfigMaps use more: mounting as *files*. This project has none yet for an honest reason: its non-secret config already flows through the config-server and env vars. The natural first ConfigMap arrives with Chapter 27's config-server retirement — the `configurations/*.yaml` files mounted into pods, let-

ting Spring read them natively and deleting a service.

PITFALL — CHANGED THE SECRET, NOTHING HAPPENED

Symptom: you re-run `create-secrets.sh` with a new password; services keep using the old one, until some pod restarts days later and starts failing authentication — an outage on a delay timer. Cause: env vars are injected *at container start*; running pods never see Secret updates. Rotation is therefore two steps by design: update the Secret, then `kubectl -n ts rollout restart deployment ...` for every consumer, immediately — while old and new credentials both work if the backing system allows it.

16.5 Outside the project

The gap between this setup and industrial practice is one concept: making secrets *deliverable by pipeline* without being readable in git. Three rungs on that ladder: **Sealed Secrets** (encrypt with the cluster’s public key; commit the sealed blob; a controller decrypts in-cluster — GitOps-compatible with one moving part); **External Secrets Operator** (a Secret manifest that is a *pointer* into Vault/AWS Secrets Manager, synced by a controller); and **Vault injection** (secrets never become Kubernetes objects at all). Each replaces the script; none changes how pods consume — `envFrom` survives every migration, which is why the manifests are already future-proof.

CHECK YOURSELF (answers: Appendix A)

1. Base64 is not encryption — so name the three things that actually protect a Secret, and where this project stands on each.
2. Why two Secrets for one Postgres credential? What failure does the split prevent?
3. Describe complete JWT-secret rotation on this cluster: commands, order, and the failure mode of doing only half.
4. Which config item in this project would move to a ConfigMap first, and what service does that retire?

Health Probes and Resource Management

This chapter is the operational heart of Part V: how the kubelet decides a pod is alive and ready, and how the scheduler and kernel share two cores among a dozen JVMs. Nearly every line was learned by watching a real pod misbehave.

17.1 The three probes

startupProbe “still booting — be patient.” While it fails, the other probes are suspended. Exhaust its budget and the container is restarted.

readinessProbe “may traffic come *right now*?” Failing pulls the pod out of Service endpoints — no restart, no drama, just no new requests until it passes again.

livenessProbe “is the process beyond saving?” Failing *restarts* the container. The dangerous one.

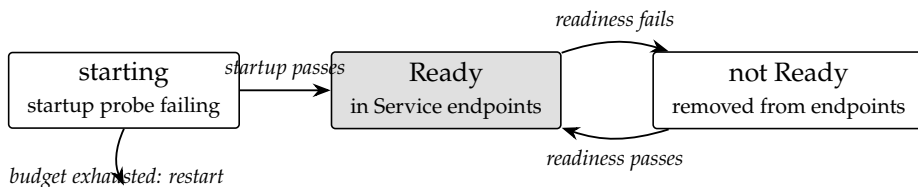


Figure 17.1: The pod’s life as the kubelet sees it.

From `course-service.yaml`:

```

startupProbe:
  httpGet: { path: /v3/api-docs, port: http } ①
  periodSeconds: 10
  failureThreshold: 12 ②
readinessProbe:
  httpGet: { path: /v3/api-docs, port: http }
  periodSeconds: 5
  failureThreshold: 3
# no livenessProbe ③
  
```

- ① The probe-path decision. `/actuator/health` is the purpose-built endpoint — but `course-service` and `auth-service` do not permit it through Spring Security, so it answers 401 and the `kubelet` counts that as failure. The one unauthenticated path that only answers 200 *after the full context is up* — `Flyway` migrated, `beans` built — is `springdoc's /v3/api-docs`. `Dictionary-service` and the gateway *do* permit `actuator` and probe it properly; and `notification-service` has no Spring Security at all, so its health endpoint is open by default — three services, three answers to the same question. Rule: a probe path must be unauthenticated *and* a true readiness signal; a `tcpSocket` check passing the moment `Tomcat` binds is neither.
- ② $12 \times 10\text{ s} = 2$ minutes of boot grace — sized to a JVM cold-starting on two contended cores.
- ③ Deliberate absence. Liveness kills; the only thing it would catch beyond readiness is a truly wedged JVM — rare — while a long GC pause on a loaded 2-core box could fail the probe and kill a *healthy* pod. Add liveness when you have observed the hang it is for.

PITFALL — STARTUP PROBE FAILED: STATUSCODE 404 — OR 403

Symptom (milestone 6, builds #6): `notification-service` logs show Kafka partitions assigned and a fully started application, yet the pod never goes Ready; the probe reports 404. `dictionary-service`, same story, but 403. Cause: the `config-server` files *expose* `health`, `info`, `metrics`, `prometheus` for every service — but only `config-server`, `discovery-server` and `api-gateway` had `spring-boot-starter-actuator` on the classpath. Exposure configuration for an absent library is silently inert; the URL is just a 404. The 403 variant is Spring Security's contribution: a 404 forwards to `/error`, `/error` is not in the `permitAll` list, and a stateless chain with no entry point answers 403 — so the symptom points at authorization while the cause is a missing jar. Fix: the starter in all four service POMs. Lesson: a probe path must be verified with `curl` from inside a pod *before* it is trusted in a manifest — the book's earlier claim that `dictionary-service` “probes health properly” was written from its `SecurityConfig`, not from a live request.

PITFALL — THE RESTART LOOP THAT CREATES ITSELF

Symptom: a service restarts every few minutes under load, each restart making the load worse. Cause: an aggressive liveness probe timing out during GC pauses or slow downstream calls — the probe *causes* the outage it was meant to detect. This failure mode is why “no liveness probe” is a defensible

default and why liveness, when added, should check only “is the event loop responding,” never downstream dependencies.

17.2 Requests, limits, and the JVM

resources:

requests: { memory: 512Mi, cpu: 200m } ①

limits: { memory: 768Mi } ②

env:

- { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" } ③

- ① *Requests* are the scheduler’s accounting: the pod is placed only where 512 Mi and 0.2 CPU are unclaimed. Requests *reserve*, they do not cap.
- ② *Limits* are kernel-enforced ceilings. Exceed the memory limit and the process is OOM-killed — status OOMKilled, exit code 137, no Java exception, no log line. Note the asymmetry: a memory limit, but *no CPU limit* — next decision box.
- ③ The container-aware JVM sizes its heap as a fraction of the *container’s* memory limit: 75% of 768 Mi for heap, the rest for metaspace, threads, and off-heap. Without this, defaults leave either waste or an OOM-kill margin of zero.

DECISION — WHY NO CPU LIMIT

CPU limits throttle: a container over its quota is paused until the next scheduling period — and a starting JVM is exactly when a service wants every cycle (class loading, JIT). Throttled startup turns 60 seconds into 200 and fails the startup probe: the limit manufactures the failure. Unlike memory, CPU is compressible — contention slows things instead of killing them — so the pattern used here is standard advice: **always request CPU, rarely limit it**. Memory gets a limit because the failure mode of unbounded memory is the kernel killing *someone*, and you want the boundary predictable.

ON THIS HARDWARE

The requests across all services sum to roughly what 19 GB minus Jenkins, k3s, and the datastores can hold — deliberately. Requests are promises to the scheduler; over-promising on a single node means pods that never schedule (Pending forever), under-promising means OOM roulette. On one node you are doing capacity planning by hand; a cloud autoscaler turns the same

arithmetic into node-pool sizing.

HANDS-ON — WATCH A STARTUP THROUGH THE PROBE'S EYES

```
kubectl -n ts delete pod -l app=course-service
kubectl -n ts get pods -w
# Running 0/1 <- startup probe still failing; no traffic yet
# Running 1/1 <- /v3/api-docs answered; endpoints updated
kubectl -n ts describe pod -l app=course-service | tail -n 12
```

The gap between 0/1 and 1/1 is Flyway plus context startup — the window in which, probe-less, requests would have hit a half-started service.

CHECK YOURSELF (answers: Appendix A)

1. For each probe: what does failing it do, and which is safe to be aggressive with?
2. Why is `/v3/api-docs` an honest readiness signal for `course-service` while `tcpSocket: 8082` is a lie? What makes `/actuator/health` unusable there, and usable on `dictionary-service`?
3. A pod shows `OOMKilled`, `exit 137`, clean logs. Explain the mechanism and why no Java exception exists. First two things to check?
4. Requests vs. limits: which does the scheduler read, which does the kernel enforce, and why does this project limit memory but not CPU?

Networking: Services, DNS, Ingress

18.1 The problem pods create

Pods have cluster-routable IPs — and lose them at every replacement. Chapter 15 made pods disposable; something stable must sit in front. That something is the *Service*: a fixed virtual IP and DNS name, backed by a live list of pod endpoints selected by label:

```
apiVersion: v1
kind: Service
metadata: { name: course-service }
spec:
  selector: { app: course-service } ①
  ports:
    - { port: 8082, targetPort: 8082 } ②
```

① The endpoint controller continuously lists *Ready* pods matching this selector — readiness (Chapter 17) is the membership test. ② Traffic to the Service’s port is forwarded to a chosen endpoint’s targetPort.

Cluster DNS gives every Service a name: `course-service` within the namespace, `course-service.ts.svc.cluster.local` from anywhere. This is why the Kubernetes env overrides look so much like the Compose ones — both platforms resolve a service’s name on a private network; the project deliberately named its k8s Services after its Compose services so the overrides match.

The default Service type, *ClusterIP*, is reachable *only inside* the cluster — a security property Part II leaned on for the password-less Mongo. *NodePort* and *LoadBalancer* exist to expose Services directly; this project instead funnels everything through one Ingress.

18.2 Ingress and Traefik

A Service exposes one thing; an *Ingress* is the L7 front door: host- and path-based HTTP routing, later TLS termination, declared as data and executed by an *Ingress controller* — in k3s, the bundled Traefik watching Ingress objects. From `api-gateway.yaml`:

```

kind: Ingress
spec:
  rules:
    - host: ts.local                ①
      http:
        paths:
          - path: /api              ②
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
# /oauth2 and /login/oauth2 likewise -> api-gateway

```

- ① Host-based rule: this applies to requests whose Host header says `ts.local` — a name that exists only in the Windows hosts file, pointing at the server’s tailnet IP. Multiple hosts can share the cluster’s single 80/443.
- ② Path fan-out. Only three prefixes enter, all to the gateway; `/` is left free for the frontend’s own Ingress (Chapter 26), which Traefik merges into the same host’s routing table.

18.3 The full path of one request

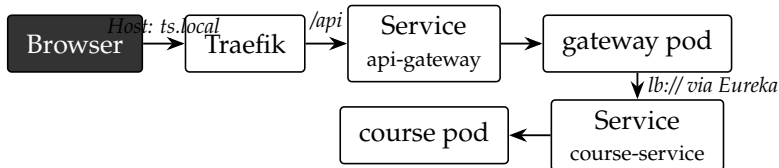


Figure 18.1: Two routers, two discovery systems, one request.

Worth pausing on: the gateway’s Eureka lookup returns a *pod IP* (Chapter 3’s `prefer-ip-address`), so hop four technically bypasses the `course-service` Service. Both discovery systems run in parallel — Eureka for gateway-to-service, Service DNS for everything else (config-server, databases, Kafka). Chapter 27’s simplification collapses them into one.

PITFALL — THE SERVICE THAT SELECTS NOTHING

Symptom: `curl` to a Service name times out; pods are Running and healthy.
Cause #1: label mismatch — the Service selector and the pod template labels differ by a typo; the endpoint list is empty and nothing complains. Check:

`kubectl -n ts get endpoints course-service` — an empty **ENDPOINTS** column is this bug. Cause #2: endpoints exist but the pods are not Ready — same command shows them under `NotReadyAddresses`. The Service abstraction fails *silently*; the endpoints object is where it becomes visible.

DECISION — ONE INGRESS HOST + HOSTS-FILE, NOT NODEPORTS

NodePorts (one weird port per service) would work over Tailscale without any DNS games — and produce URLs like `100.85.66.43:30082` that leak topology, dodge the gateway, and can't do TLS sanely. One Ingress host means one address to secure and a browser experience identical to production; the `ts.local` hosts-file entry is the only client-side cost. The upgrade path — a real domain plus cert-manager, or a Cloudflare Tunnel — changes the Ingress host and nothing else.

18.4 Outside the project

On EKS the same Ingress object, annotated for the AWS controller, provisions an actual ALB; on OpenShift it becomes a Route — same idea, different noun. The piece this cluster omits: **NetworkPolicies** — firewall rules for pod-to-pod traffic (“only the gateway may reach `course-service:8082`”), which is the in-cluster enforcement the trusted-edge pattern of Chapter 5 called its remaining soft spot. Flat-network trust is normal for a homelab and negligent for a bank.

CHECK YOURSELF (answers: Appendix A)

1. Pod IPs change at every replacement, yet `course-service:8082` always works. Name each mechanism between the name and the pod's socket.
2. How does readiness interact with Service endpoints, and what does that mean during a rolling update?
3. Diagnose: Service exists, pods Ready, `get endpoints` shows an empty list. What is wrong, and why did nothing error?
4. Why does `ts.local` require a hosts-file entry, and what replaces that entry in the two production upgrade paths?

Storage: Volumes, Claims, and State on a Single Node

19.1 The abstraction stack

Containers' filesystems die with them. Kubernetes separates the *request* for storage from its *provision* through three objects:

PersistentVolume (PV) an actual piece of storage — a directory, an EBS volume, an NFS export.

PersistentVolumeClaim (PVC) a workload's request: "5 Gi, ReadWriteOnce." The claim is what pods mount; it binds to a PV.

StorageClass the factory: how to make a PV on demand when a claim appears (*dynamic provisioning*).

The indirection is portability: `postgres-course.yaml` says only "I need 5 Gi"; *how* that becomes disk is the cluster's business. The same manifest gets a local directory on k3s and an EBS volume on EKS.

19.2 local-path on k3s

k3s bundles the `local-path` StorageClass: a PVC becomes a directory under `/var/lib/rancher/k3s/storage/` on the node's SSD. Simple, fast, and honest about its limits — the data is on *that node's disk*. On multi-node clusters that pins the pod to the node; on this single-node cluster it just means what it says.

ON THIS HARDWARE

`local-path` provides *persistence* (data survives pod and node restarts), not *durability* (one SSD failure loses everything) and not *backup* (a bad DELETE replicates to nowhere). The honest missing piece is a CronJob running `pg_dump` to MinIO or off-box. Rule of thumb worth keeping: a database without tested restores is a cache.

19.3 Two claim styles in the manifests

MinIO (a Deployment with one replica) uses a standalone PVC; the databases use volumeClaimTemplates inside their StatefulSets — a PVC *minted per replica* and, crucially, *re-bound to the replacement pod* rather than a second live pod. That distinction is the storage half of Chapter 15’s corruption pitfall.

```
volumeClaimTemplates:
- metadata: { name: data }
  spec:
    accessModes: [ReadWriteOnce] ①
    resources: { requests: { storage: 5Gi } }
# in the pod template:
volumeMounts:
- { name: data, mountPath: /var/lib/postgresql/data }
env:
- { name: PGDATA, value: /var/lib/postgresql/data/pgdata } ②
```

- ① ReadWriteOnce = mountable read-write by one *node* at a time — fits a database exactly. (RWX — many nodes — needs NFS-like storage and is the mode people want for shared uploads before discovering object storage, Chapter 9, is the better answer.)
- ② The PGDATA trick, a genuine classic: the Postgres image refuses to initialize into a non-empty directory, and ext4 volumes arrive with `lost+found`. Pointing PGDATA one directory deeper sidesteps it. You will meet this exact pattern in StackOverflow-years’ worth of Postgres-on-Kubernetes setups.

PITFALL — DELETED THE STATEFULSET, DATA STILL THERE — OR GONE

Two surprises in one. First: deleting a StatefulSet does *not* delete its PVCs — by design, data outlives orchestration mistakes; re-create the StatefulSet and it re-binds. Second, the reverse trap: deleting the PVC while experimenting *does* delete the PV and its directory (reclaimPolicy Delete, the dynamic-provisioning default). The asymmetry is worth memorizing before you clean up a namespace: workload objects are disposable; claims are where the data is.

HANDS-ON — SEE THROUGH THE ABSTRACTION

```
kubectrl -n ts get pvc           # claims and their PVs
kubectrl get pv                 # cluster-scoped, note the paths
ssh ts-server 'sudo ls /var/lib/rancher/k3s/storage/'
# pvc-... one directory per PV -- your databases, as plain files
```

Demystification is the point: a PVC on this cluster is a directory. On EKS the same two `kubectl` commands would show EBS volume IDs — same objects, different factory.

19.4 Outside the project

On managed Kubernetes, `StorageClasses` map to cloud disks (EBS/gp3 on EKS, Persistent Disks on GKE) with snapshots, encryption and resizing — `VolumeSnapshot` objects make backup part of the same declarative model. The industrial pattern this chapter points toward, though, is subtraction: teams increasingly keep *databases outside the cluster* (RDS, Cloud SQL) and let Kubernetes run only stateless things — trading Chapter 15’s `StatefulSet` care for a managed bill. Both sides of that trade are now visible to you.

CHECK YOURSELF (answers: Appendix A)

1. PV, PVC, `StorageClass`: which does a pod mount, which is cluster-scoped, and what does the indirection buy this project’s manifests on a future EKS?
2. Why is `ReadWriteOnce` sufficient for Postgres, and what kind of workload pushes people toward RWX — usually wrongly?
3. Explain both directions of the deletion asymmetry: `StatefulSet` gone/PVC stays, PVC gone/data gone.
4. What do persistence, durability, and backup each mean, and which does local-path provide?

Kustomize: Manifests as a Set

20.1 The problem

Part V produced a dozen YAML files. Applying them one by one invites forgetting one; and the moment a second environment exists (“prod, but with 2 replicas and real secrets”), copy-pasting manifests per environment guarantees drift. Two tools dominate this space; this project uses the one built into `kubectl`.

20.2 The kustomization, annotated

`kubectl apply -k deploy/k8s/base` reads one file:

```
# deploy/k8s/base/kustomization.yaml
namespace: ts
resources:
  - postgres-course.yaml
  - postgres-auth.yaml
  - mongodb.yaml
  - minio.yaml
  # kafka.yaml requires the Strimzi operator (one-time admin step:
  # ../install-strimzi.sh) -- without its CRDs, apply fails on kind Kafka.
  - kafka.yaml
  - kafka-topics.yaml
  - maildev.yaml
  - config-server.yaml
  - discovery-server.yaml
  - course-service.yaml
  - auth-service.yaml
  - dictionary-service.yaml
  - notification-service.yaml
  - api-gateway.yaml
commonLabels:
  app.kubernetes.io/part-of: teacher-supporter
```

- ① Stamped onto every resource — no manifest needs its own `namespace:` line, and retargeting the whole app to another namespace is a one-line change.
- ② The definitive list. Two deliberate absences teach more than the presences: `namespace.yaml` is out because it is cluster-scoped and Jenkins’ namespaced

Role could not apply it (Chapter 22); Secrets are out for Chapter 16’s reasons. A third object with the same admin-only status is not a file at all: the Strimzi operator and its CRDs, installed once by `install-strimzi.sh` (Chapter 25) — `kafka.yaml` is listed here but cannot apply until they exist. The listed order is documentation only — the API server accepts everything and reconciliation sorts out reality.

- ③ A label stamped on all objects, making the whole application one selector: `kubectl get all -l app.kubernetes.io/part-of=teacher-supporter` — or, in a disaster drill, one delete.

20.3 Bases and overlays

Kustomize’s real feature arrives with the second environment. A *base* holds the truth; *overlays* patch it:

```
# deploy/k8s/overlays/prod/kustomization.yaml
resources:
- ../../base
patches:
- target: { kind: Deployment, name: course-service }
  patch: |-
    - op: replace
      path: /spec/replicas
      value: 2
images:
- name: localhost:5000/course-service
  newTag: v1.4.2
```

No templates, no variables — overlays are *merges of plain YAML*. Every layer is valid Kubernetes YAML you can read, diff, and pipe through `kubectl kustomize` to inspect the final output before anything touches the cluster. The directory layout is already overlay-shaped (`base/`), so the day a prod overlay is needed, it is an `mkdir`, not a migration.

DECISION — KUSTOMIZE, NOT HELM

Helm is the other answer: manifests as Go templates (`replicas: {{ .Values.replicaCount }}`) with per-environment value files, packaged as versioned, shareable *charts*. Templating is strictly more powerful — loops, conditionals, one chart deploying anyone’s app — and that is the point of divergence: Helm shines when you *distribute* software to strangers’ clusters; for deploying your own app, templates make every manifest unreadable until rendered. Kustomize keeps plain YAML plain, ships inside `kubectl`, and covers replicas-

/images/labels patching — the whole need here. You will still *consume* Helm charts for third-party software (Strimzi, kube-prometheus-stack); writing one can wait until you have users.

PITFALL — SPEC.SELECTOR: FIELD IS IMMUTABLE

Symptom: the pipeline’s `kubectl apply -k` fails on one Deployment with this error, although the same file applies fine by hand. Cause: `commonLabels` stamps its label not only onto metadata but into every Deployment’s *selector* — and selectors are immutable once created. An object first created *outside* Kustomize (`kubectl apply -f maildev.yaml` during milestone 6’s setup, selector `{app: maildev}`) can never be updated by Kustomize afterwards (selector `{app: maildev, part-of: teacher-supporter}`). Fix: delete the hand-made object and let the pipeline recreate it. Rule: an object managed by a kustomization is applied *only* through it — for a one-off test, render with `kubectl kustomize` and pipe that, never the raw file. (Modern Kustomize’s labels: with `includeSelectors: false` avoids selector injection — but switching now would break every selector already created the old way.)

PITFALL — IT WORKED YESTERDAY: THE PRUNE ILLUSION

Symptom: you delete a manifest from `resources;`, re-apply, and the object is still running — for months, until it conflicts with something. Cause: apply-the-set is still apply — Chapter 14’s rule that apply never deletes holds for Kustomize too. The removed Ingress of Chapter 4 needed a manual `kubectl delete`. The systematic fixes: `kubectl apply -k --prune` (still gated behind flags for good reason — a mislabeled selector can mass-delete), or GitOps controllers (Chapter 27) whose whole job is making git and cluster converge, deletions included.

HANDS-ON — RENDER BEFORE YOU TRUST

```
kubectl kustomize deploy/k8s/base | less    # the final YAML, merged
kubectl kustomize deploy/k8s/base | grep -c 'kind:' # object count
kubectl apply -k deploy/k8s/base --dry-run=server # validate, no changes
```

`kubectl kustomize` is pure text transformation — reading its output is how you review what an overlay actually changes, and `--dry-run=server` is the closest thing to a manifest unit test.

CHECK YOURSELF (answers: Appendix A)

1. What two problems does a kustomization solve over `kubectl apply -f` per file, even with zero overlays?
2. Why are the namespace and the Secrets deliberately missing from resources? (Two different reasons.)
3. Helm vs. Kustomize: state each tool's home turf in one sentence, and name the third-party software in this project's future that will arrive as a chart.
4. A teammate removes `mongodb.yaml` from the list to "decommission" it. What actually happens, and what are the two correct procedures?

Part VI

CI/CD

CI/CD: The Concepts

21.1 Two practices, one pipeline

Continuous Integration every change is merged into the shared branch frequently and *verified automatically* — compiled, tested, packaged. CI's product is confidence: the main branch is always in a known-good state, and a breakage is caught minutes after the commit that caused it, while its author still has context.

Continuous Delivery/Deployment every verified change is automatically prepared for release (Delivery) or actually released (Deployment — no human gate). This project practices the stronger form: a push to `main` ends as running pods, no button pressed.

The pipeline is the machine that enforces both:

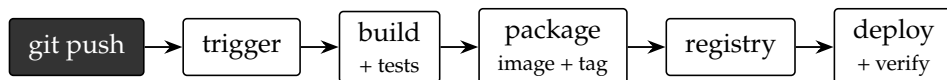


Figure 21.1: The universal pipeline shape. Tool names vary; stages do not.

Whether it is Jenkins+Jib+k3s (this project), GitHub Actions+Docker+EKS, or GitLab CI+Kaniko+OpenShift — source in, tested artifact into a registry, orchestrator told to converge on it, verification that it did. Learn the shape once and every employer's pipeline is a renaming exercise.

21.2 The vocabulary

Trigger what starts a run. *Webhook*: the git host calls the CI server on push — instant, but requires the CI server to be reachable from the internet. *Polling*: CI asks the repo “anything new?” on a schedule — reachable-from-nowhere, at the cost of latency. This project polls (H/5 * * * *: about every five minutes) precisely because Jenkins sits behind home NAT with no public address — topology decided the trigger, a pattern you will see in every locked-down corporate network.

Stage / step the pipeline's units: stages are the named phases (visible in the UI, gating progress); steps are the commands inside.

- Artifact** anything a build produces worth keeping: the jar, the image, test reports. Artifacts must be *immutable and traceable* — hence tagging images with the git SHA, so a running pod can be traced to the exact commit it runs.
- Test gate** the property that later stages run only if earlier ones passed. A pipeline that deploys despite failing tests is a deployment script with extra steps.
- Environment promotion** the same artifact moving dev → staging → prod. The iron rule: *promote artifacts, never rebuild them* — a rebuild “for prod” is a different binary than the one you tested. Chapter 1’s env-var machinery exists exactly so one image fits all environments.
- Rollback** the escape hatch. Because the registry keeps SHA-tagged images and Kubernetes keeps old ReplicaSets (Chapter 15), rolling back is re-pointing at a previous artifact — seconds, not a rebuild.

DECISION — DEPLOY ON EVERY PUSH TO MAIN — FOR ONE DEVELOPER

Continuous Deployment without approvals is the right call here: one developer, a homelab blast radius, and the fastest possible feedback loop. The same choice at a bank would be wrong not because automation is risky but because *regulated* changes need evidence of review. The graduation path keeps the pipeline and adds gates: a manual approval stage before prod, or — Chapter 27 — GitOps, where “deploy” becomes a reviewable pull request to a config repo. The pipeline shape survives; only the gate policy changes.

PITFALL — THE PIPELINE THAT LIES GREEN

Symptom: every run is green; production still breaks. The classic causes: tests skipped in the build command (`-DskipTests` — which *this* pipeline does in its build stage, accepted consciously because `mvn verify` runs in CI on GitHub Actions — know your own gates); deploy stage not verifying rollout (this one does — rollout status fails the build if pods never go Ready); or success measured as “commands exited 0” rather than “service answers requests.” A green pipeline is a claim; know precisely what it claims.

CHECK YOURSELF (answers: Appendix A)

1. CI and CD make different promises. State each in one sentence, and identify which stage of this project’s pipeline delivers which.
2. Webhook vs. polling: name the deciding constraint in this project, and a corporate setting with the same constraint.
3. Why must a promoted artifact never be rebuilt per environment, and which mechanism from Chapter 1 makes single-artifact promotion pos-

sible?

4. Your pipeline is green and prod is down. List three distinct ways a pipeline can lie, and the fix for each.

Jenkins: The CI Server

22.1 Architecture

Jenkins is a Java web application — the *controller* — that stores jobs, credentials, plugins and build history in one directory, `JENKINS_HOME`, and executes builds either on itself (via *executors*, its parallel-build slots) or on remote *agents*. The enterprise pattern is a controller that builds nothing, farming work out to disposable agents (often pods, via the Kubernetes plugin).

DECISION — BUILDS ON THE CONTROLLER, JENKINS OUTSIDE THE CLUSTER

Two homelab inversions of enterprise practice, both CPU arithmetic. Agent pods would cost scheduling and image pulls per build on the same two cores the build needs; the controller's two executors spend those cycles on Maven instead. And Jenkins runs in Docker on the host, not in k3s: if you break the cluster — and while learning, you will — the thing that can rebuild the cluster must not have died with it. Both revert cleanly at enterprise scale: in-cluster Jenkins with pod agents is Chapter 27 material, and the migration itself is the lesson.

22.2 The controller image, annotated

Rather than clicking tools into a stock Jenkins, the project bakes its toolchain into an image — `deploy/jenkins/Dockerfile`:

```
FROM jenkins/jenkins:lts-jdk21 ①
USER root
RUN ... temurin-25-jdk ... ②
RUN ... apache-maven-3.9.9 ... ③
RUN wget -qO /usr/local/bin/kubectl ... ④
ENV JDK25_HOME=/usr/lib/jvm/temurin-25-jdk-amd64 ⑤
USER jenkins
RUN jenkins-plugin-cli --plugins \
    git workflow-aggregator ... credentials-binding ⑥
```

① LTS Jenkins running on JDK 21 — the version *Jenkins itself* runs on.

- ② The project targets Java 25, so a second JDK is installed for *builds*. Note `JAVA_HOME` is *not* changed globally ⑤ — the `Jenkinsfile` sets it per pipeline. Controller runtime and build toolchain are independent concerns; this `Dockerfile` keeps them visibly separate.
- ③ Maven pinned to 3.9.9 — distro packages lag.
- ④ `kubectl`: the deploy stage is just this binary plus a `kubeconfig`.
- ⑥ Plugins pinned in the image, not clicked in the UI: pipeline support, git, JUnit reporting, and `credentials-binding` (used by `withCredentials` in Chapter 23). The whole point: *the CI server itself is now reproducible* — destroy the container, rebuild from the `Dockerfile`, and only `JENKINS_HOME` (a named volume) carries state.

One run-time flag from `run.sh` deserves its own paragraph: `-network host`. Jib inside Jenkins pushes to `localhost:5000` — and inside a bridge-networked container, `localhost` would be the *container*. Host networking makes Jenkins share the host's network namespace, so its `localhost:5000` is the same registry k3s pulls from (Chapter 13's naming contract). Not a shortcut — a requirement of the image-naming scheme.

22.3 Credentials

Jenkins stores secrets in an encrypted store, referenced by ID; pipelines bind them to environment variables only for the duration of a block (`withCredentials`), and the log masker redacts their values from console output. This project stores one credential: `kubeconfig-ts`, a file credential holding the `kubeconfig` Jenkins uses to deploy — which is the interesting part.

22.4 The deploy identity: a `ServiceAccount`, not admin

The lazy path is giving Jenkins the k3s admin `kubeconfig` — `cluster-admin`, licensed to delete every namespace. Instead, `deploy/k8s/jenkins-rbac.yaml` builds a minimal identity:

```
kind: ServiceAccount      # the identity
metadata: { name: jenkins, namespace: ts }
---
kind: Role                # namespaced permissions
metadata: { name: jenkins-deployer, namespace: ts }
rules:
- apiGroups: ["apps"]
  resources: ["deployments", "statefulsets"]
  verbs: ["get", "list", "watch", "create", "patch", "update"] ①
```



```

- apiGroups: ["apps"]
  resources: ["replicasets"]
  verbs: ["get", "list", "watch"]
# + services, PVCs, secrets, configmaps, ingresses, pods/log ...
- apiGroups: ["kafka.strimzi.io"]
  resources: ["kafka", "kafkanodepools", "kafkatopics"]
  verbs: ["get", "list", "create", "patch", "update"]
---
kind: RoleBinding          # identity <- permissions

```

- ① The verbs are derived from the pipeline's actual commands: `kubectl` apply needs `get+create+patch`; `set image` needs `get+patch`. *No delete anywhere*: CI rolls forward; a human deletes. A compromised pipeline can thus corrupt deployments in `ts` but cannot wipe them, and cannot see beyond the namespace — a Role, not a ClusterRole, is the blast-radius decision.
- ② Discovered the honest way: `rollout status` watches the Deployment's ReplicaSets, so read access to them is required — the kind of permission you find by running the pipeline against the Role and reading the first Forbidden.
- ③ Added with milestone 6: the pipeline may declare Kafka clusters and topics (namespaced custom resources), but installing the operator and its CRDs — cluster-scoped — stays an admin act (Chapter 25). RBAC grows one rule per API group the pipeline touches; the Role file is a changelog of what CI has been trusted with.

The kubeconfig stored in Jenkins simply pairs the cluster address with this ServiceAccount's token instead of the admin certificate.

PITFALL — FORBIDDEN: CANNOT CREATE RESOURCE "NAMESPACES"

Symptom: the deploy stage dies with a Forbidden error on namespaces although the RBAC file looks complete. Cause: namespaces are *cluster-scoped*, and a namespaced Role cannot grant them — which is exactly why `namespace.yaml` is excluded from the kustomization (Chapter 20) and the namespace is created once by an admin. RBAC debugging in general: `kubectl auth can-i patch deployments -n ts -as system:serviceaccount:ts:jenkins` answers "would this be allowed" without running the pipeline.

HANDS-ON — PROVE THE BOUNDARY EXISTS

```

kubectl auth can-i patch deployments -n ts \
--as system:serviceaccount:ts:jenkins      # yes
kubectl auth can-i delete deployments -n ts \
--as system:serviceaccount:ts:jenkins      # no

```

```
kubectl auth can-i list pods -n kube-system \
--as system:serviceaccount:ts:jenkins      # no
```

CHECK YOURSELF (answers: Appendix A)

1. What lives in `JENKINS_HOME`, and why does baking plugins into the image plus a volume for home make the CI server disposable?
2. Why does the controller run two JDKs, and where is each selected?
3. Reconstruct the RBAC verb list for a pipeline that only ever runs `kubectl apply -k` and `rollout status` — which verb in the project's Role could then be dropped?
4. Explain `--network host` on the Jenkins container in terms of Chapter 13's image-naming contract.
5. ServiceAccount, Role, RoleBinding: which is identity, which is capability, which is the join — and why is Role-not-ClusterRole the security headline?

The Pipeline, Line by Line

Everything in this book converges on one 70-line Groovy file. A declarative Jenkinsfile lives in the repository it builds — pipeline as code, versioned and reviewed like everything else.

23.1 Preamble and environment

```

pipeline {
  agent any                                ①
  options {
    timestamps()
    disableConcurrentBuilds()              ②
    buildDiscarder(logRotator(numToKeepStr: '10'))
  }
  triggers { pollSCM('H/5 * * * *') }      ③
  environment {
    JAVA_HOME = "${env.JDK25_HOME}"         ④
    PATH       = "${env.JDK25_HOME}/bin:${env.PATH}"
    TAG        = "${env.GIT_COMMIT.take(7)}" ⑤
    NS         = 'ts'
    REGISTRY   = 'localhost:5000'
    SERVICES   = 'config-server discovery-server course-service api-gateway auth-service
                  ↪ dictionary-service notification-service'
    MODULES    = 'common,config-server,discovery-server,course-service,api-gateway,auth-
                  ↪ service,dictionary-service,notification-service'
  }
}

```

- ① Any executor on the controller — Chapter 22’s homelab compromise in one word.
- ② Two builds at once on two cores would thrash; queued runs also collapse (a new commit during a build supersedes waiting ones).
- ③ Polling because NAT blocks webhooks. The H is a hash-spread: “some fixed minute in each 5,” so many jobs don’t all poll at :00.
- ④ The per-pipeline JDK switch the controller image prepared: builds see Java 25, Jenkins itself stays on 21.
- ⑤ The deploy tag is the short git SHA — every image traceable to its commit, every deploy pinned to an immutable tag (Chapter 13’s tags-vs-digests lesson

applied).

SERVICES/MODULES the pipeline's registry of what exists: MODULES feeds Maven (comma-separated, includes common which is built but produces no image); SERVICES feeds kubectl (space-separated, deployables only). Adding a service to the cluster means extending both — the one manual step per new service.

23.2 The three stages

```
stage('Build') {
  steps {
    dir('Project/TeacherSupporter') {
      sh 'mvn -B -DskipTests -pl ${MODULES} -am verify'
    }
  }
}
stage('Push images') {
  steps {
    dir('Project/TeacherSupporter') {
      sh 'mvn -B -DskipTests -pl ${MODULES} -am package jib:build -Djib.to.tags=${TAG}'
      ↔
    }
  }
}
stage('Deploy') {
  steps {
    dir('Project/TeacherSupporter') {
      withCredentials([file(credentialsId: 'kubeconfig-ts',
        variable: 'KUBECONFIG')]) {
        sh '''
          set -eu
          kubectl apply -k deploy/k8s/base
          for s in ${SERVICES}; do
            kubectl -n ${NS} set image deployment/${s} \
              ${s}=${REGISTRY}/${s}:${TAG}
          done
          for s in ${SERVICES}; do
            kubectl -n ${NS} rollout status \
              deployment/${s} --timeout=300s
          done
          ...
        '''
      }
    }
  }
}
```

- ① The repository root is a study monorepo; the project lives in a subdirectory, so every stage `dir()`s into it.
- ② `-pl` (project list) + `-am` (also make dependencies): build only the listed modules and what they need — monorepo economics on two cores. `-B` is batch mode (no interactive output). `-DskipTests` is the consciously accepted gate-

weakening from Chapter 21’s pitfall box.

- ③ One command builds and pushes all seven images — Chapter 13 in a single line. `-Djib.to.tags` adds the SHA tag alongside the SNAPSHOT tag. The package in front of `jib:build` is not decoration — see the pitfall below; it cost two failed builds on the day milestones 5 and 6 shipped.
- ④ The file credential materializes as a temp file; KUBECONFIG is the env var kubectl reads. Outside this block the file does not exist; in the log its content never appears.
- ⑤ Fail on any error and on unset variables — without it, a failed apply would be sailed past and “deployed” reported.
- ⑥ Converge the cluster on the manifests — idempotent (Chapter 14), so it is safely re-run every build whether or not anything changed.
- ⑦ Repoint each Deployment at the SHA-tagged image. This is what actually triggers rollouts — and note the manifests themselves keep the SNAPSHOT tag, so *apply* alone would not redeploy; the pipeline’s pinning and the manifests’ default coexist.
- ⑧ The verification gate: block until each rollout completes; fail the build if any pod never goes Ready within five minutes. This line is what makes the pipeline’s green mean “running,” not “commands executed” — it delegates truth to the probes of Chapter 17.

PITFALL — COULD NOT FIND ARTIFACT COM.TS:COMMON:JAR:1.0.0-SNAPSHOT

Symptom: the Build stage passes, then the Push stage fails on the first module that depends on `common`, unable to resolve a jar that the previous stage just built. Cause: `mvn jib:build` is a *bare goal* — Maven runs it directly with no lifecycle phases, so nothing in the reactor is compiled or packaged in that session, and sibling dependencies fall back to the local repository, where `common` was never installed (`verify` stops short of `install`). Modules without a sibling dependency (`config-server`, `discovery-server`) sail through, which makes the failure look module-specific. Fix: put a phase in front — `package jib:build` — so the reactor packages `common` before Jib asks for it. General rule: a plugin goal that needs other modules’ artifacts must ride a lifecycle phase, not be invoked alone.

Two loops, not one, is a small correctness point: all rollouts start first, then all are awaited — otherwise service N’s wait serializes service N+1’s start, stretching total deploy time for no gain.

23.3 Failure handling

```
post {
  success { echo "Deployed ${TAG} to namespace ${NS}." }
  failure {
    dir('Project/TeacherSupporter') {
      withCredentials([...]) {
        sh 'kubectl -n ${NS} get pods || true'
      } } } }
```

On failure, the build log ends with a pod listing — the first command a human would run, already run. The `|| true` keeps the diagnostic itself from masking the build's real exit status.

23.4 The same pipeline, elsewhere

For calibration, the identical shape as GitHub Actions:

```
on: { push: { branches: [main] } }      # webhook, not polling
jobs:
  deploy:
    runs-on: ubuntu-latest                # disposable agent, not controller
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 25 }
      - run: mvn -B -pl $MODULES -am verify
      - run: mvn -B -pl $MODULES -am jib:build -Djib.to.tags=${GITHUB_SHA::7}
      - run: kubectl apply -k deploy/k8s/base && ...
      env: { KUBECONFIG_DATA: ${ secrets.KUBECONFIG } }
```

Same stages, same tag scheme, same secret-binding idea. The differences are the infrastructure trade: GitHub's runners are disposable VMs (agents done right, for free) but they live on the internet — so this variant would need the registry and cluster reachable from outside, the exact constraint that shaped the homelab design. Pipelines are portable; *network topology is what actually differs between shops*.

CHECK YOURSELF (answers: Appendix A)

1. Why tag deploys with the git SHA when the SNAPSHOT tag also exists? Name the failure mode SNAPSHOT-only deploys invite (Chapter 13) and what traceability the SHA adds.
2. Explain `-pl ... -am` and what it saves in a monorepo. What breaks if you

forget to extend MODULES for a new service?

3. Which single line makes this pipeline's success mean "pods are Ready," and which chapter's machinery does it lean on?
4. Why two loops in the deploy stage instead of set-image-then-wait per service?
5. List three things that change and three that do not when this Jenkinsfile is translated to GitHub Actions.

Part VII

Operations and the Road Ahead

Operating the Stack: A Debugging Field Manual

This chapter is the one to reread with a broken cluster in front of you. It assembles the debugging workflow the earlier chapters earned, ordered the way real incidents unfold.

24.1 The four questions

Almost every Kubernetes incident is located by asking, in order:

1. **Is the pod running?** `kubectl -n ts get pods` — read the STATUS and RESTARTS columns; the taxonomy below.
2. **What does the platform say about it?** `kubectl -n ts describe pod <pod>` — the Events section at the bottom is the kubelet and scheduler narrating their decisions: image pulls, probe failures, OOM kills, scheduling refusals.
3. **What does the app say?** `kubectl -n ts logs <pod>` (add `-previous` for the container that just died — the single most-forgotten flag in incident response).
4. **Can I reproduce its view of the world?** `kubectl -n ts exec -it <pod> - sh` and try the connection the app claims is failing.

24.2 A status taxonomy

HANDS-ON — BREAK IT ON PURPOSE — THREE DRILLS

Deliberate sabotage, on a quiet day, teaches more than any outage:

```
# 1. wrong image tag -> watch ImagePullBackOff, then roll back
kubectl -n ts set image deployment/course-service \
  course-service=localhost:5000/course-service:nope
kubectl -n ts rollout undo deployment/course-service

# 2. scale the database down -> watch dependents degrade honestly
kubectl -n ts scale statefulset/postgres-course --replicas=0
kubectl -n ts get pods -w      # course-service readiness reacts
kubectl -n ts scale statefulset/postgres-course --replicas=1
```

Status	Meaning and first move
Pending	unschedulable: requests exceed free resources, or PVC unbound. describe shows which.
ImagePullBackOff	registry unreachable, name typo'd, or the HTTPS-vs-HTTP trap of Chapter 13.
CrashLoopBackOff	process exits soon after start; backoff doubles between retries. logs –previous.
OOMKilled (exit 137)	memory limit hit; no Java trace exists — Chapter 17's kernel kill. Raise limit or shrink heap.
Running 0/1	alive but readiness failing; normal during startup; a problem when persistent — probe the probe's own URL.
Completed/Error	for Jobs; for a service pod, Error means the process exited nonzero once.

Table 24.1: What the STATUS column is trying to tell you.

```
# 3. kill Eureka -> verify existing routes survive on client caches
kubectl -n ts delete pod -l app=discovery-server
```

Each drill pairs a chapter's theory with its observable behavior: 13 (naming), 17–18 (probes and endpoints), 3 (client-side caching).

24.3 Reading a service's world from inside

When logs claim a connection failure, verify the claim from the pod's own network position — DNS and reachability differ between your machine and the pod:

```
kubectl -n ts exec -it deploy/course-service -- sh
wget -qO- http://config-server:8888/actuator/health # DNS + service up?
nc -zv postgres-course 5432 # TCP reachable?
env | grep SPRING # what was injected?
```

That last command settles arguments fast: it prints what configuration the pod *actually* received, which beats re-reading manifests.

24.4 The routine that keeps weekends free

After every deploy the pipeline already verifies rollouts; glance at `kubectl -n ts get pods` anyway — RESTARTS creeping upward is tomorrow’s outage announcing itself.

Weekly `kubectl top pods -n ts` (requires metrics-server, bundled with k3s): compare actual usage against the requests of Chapter 17 and correct the fiction. Check disk: `df -h` on the server — registries and log files grow; registry:2 needs occasional garbage collection.

Before demos *do not build* — Chapter 5’s hardware box: two cores mean a Maven build and a demo contend for the same CPU that keeps readiness probes answering.

ON THIS HARDWARE

`ufw` on the server must exempt the pod and service CIDRs (10.42.0.0/16, 10.43.0.0/16) — firewalling them breaks DNS and pod-to-pod traffic in ways that impersonate application bugs: intermittent timeouts, resolvable-sometimes names. When *weird* networking happens on k3s, suspect the host firewall before the app. And k3s reads `registries.yaml` only at startup — after editing, `systemctl restart k3s`, which briefly restarts every pod: not during a demo.

PITFALL — DEBUGGING THE WRONG LAYER

The most expensive habit in Kubernetes debugging is starting where the symptom appeared instead of where the cause lives. A browser 502 can be Traefik (no endpoints), the gateway (Eureka staleness), the service (readiness failing), the database (pod evicted), or the disk (full). The four questions exist to walk the layers in order; resist the temptation to open application logs first just because they are the most familiar layer. `describe` before logs — the platform usually already knows.

CHECK YOURSELF (answers: Appendix A)

1. Order the four questions and name the command for each. Which one uses a flag people forget, and when does that flag matter?
2. `CrashLoopBackOff` vs. `Running 0/1`: different layers are failing — which probe concept distinguishes them?
3. A service claims it cannot reach Postgres; from your laptop Postgres answers. What is the next diagnostic step and why is your laptop’s success irrelevant?

4. Why can a host firewall change masquerade as an application bug on k3s, and which two CIDRs must stay exempt?

Kafka in the Cluster: Strimzi and the Mail Flow

Milestone 6 moves the broker of Chapter 10 from Compose into k3s and deploys the last service — notification-service — so the first end-to-end *event* flow runs in the cluster: register a user, receive a mail. It is also the chapter where Chapter 14’s reconciliation idea grows its most important extension: the *operator*.

25.1 Operators and custom resources

Kubernetes ships controllers for its built-in kinds — Deployments, Services. An *operator* is a controller for kinds you add: a *CustomResourceDefinition* (CRD) teaches the API server a new noun (Kafka, KafkaTopic), and the operator’s reconciliation loop turns instances of it into ordinary objects — pods, Services, ConfigMaps — and keeps them converged. Operational knowledge that used to live in runbooks (how to roll a broker safely, how to compute advertised listeners, how to add a partition) is encoded in the operator’s code. Strimzi is the Kafka operator; the pattern is universal (Postgres operators, Prometheus operator, cert-manager).

25.2 Installing the operator: an admin act

```
# deploy/k8s/install-strimzi.sh (abridged)
kubectl apply --server-side --force-conflicts \
  -f "https://strimzi.io/install/latest?namespace=${NS}" -n "$NS" ①
kubectl -n "$NS" rollout status deployment/strimzi-cluster-operator
```

- ① The bundle contains the CRDs, the operator Deployment, and its RBAC — much of it *cluster-scoped*. Two consequences: it is applied by an admin, not by Jenkins (whose Role, Chapter 22, stops at the namespace), and it is applied *server-side* because the CRD documents are too large for the client-side last-applied annotation. Idempotent, so it doubles as the upgrade command. The `?namespace=` parameter scopes the operator to watch `ts` only.

The kustomization (Chapter 20) lists `kafka.yaml`, but that file cannot apply until this script has run once — the third member of the “admin prerequisites” set, after the namespace and the secrets.

PITFALL — NO MATCHES FOR KIND “KAFKA” IN VERSION v1BETA2

Symptom: applying a Kafka manifest copied from any tutorial fails with this error even though the operator is installed and healthy. Cause: Strimzi 1.x serves only `kafka.strimzi.io/v1`; the `v1beta2` version that a decade of blog posts use is gone. Check what a CRD actually serves before trusting an example: `kubectrl get crd kafkas.kafka.strimzi.io -o jsonpath={.spec.versions[*].name}`. This project hit it live on installation day.

25.3 The cluster, declared

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool                                ①
metadata:
  name: dual-role
  labels: { strimzi.io/cluster: ts-kafka } ②
spec:
  replicas: 1
  roles: [controller, broker]
  storage: { type: persistent-claim, size: 5Gi, deleteClaim: false } ③
  resources:
    requests: { memory: 768Mi, cpu: 200m }
    limits: { memory: 1Gi }
---
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: ts-kafka
spec:
  kafka:
    listeners:
      - { name: plain, port: 9092, type: internal, tls: false } ④
    config:
      offsets.topic.replication.factor: 1 ⑤
      default.replication.factor: 1
      min.insync.replicas: 1
      log.retention.hours: 168
      jvmOptions: { -Xms: 256m, -Xmx: 512m } ⑥
  entityOperator:
    topicOperator: {}                                ⑦
```

- ① Two objects: the *pool* says where Kafka runs (replicas, roles, storage, resources); the *Kafka* resource says how it behaves. One dual-role KRaft node is the Kubernetes spelling of Compose's `process.roles=broker,controller`.
- ② The label joins pool to cluster; the operator matches on it. A mismatch is silent — the pool simply never materializes.
- ③ A PVC via local-path, exactly like the databases (Chapter 19). `deleteClaim: false`: deleting the Kafka resource keeps the log — the same asymmetry *StatefulSets* have by default.
- ④ Compare with the three-listener, two-advertised-address block in `docker-compose.yml`: gone. You state *one internal listener*; the operator computes bootstrap and per-broker *Services*, the advertised addresses, and the KRaft controller listener. Chapter 10's maze was operational knowledge; it now lives in the operator.
- ⑤ Single broker means every replication factor must be 1, or the first topic creation fails asking for brokers that do not exist.
- ⑥ Kafka's default heap assumes a server; half a GiB serves a homelab trickle and leaves memory for the JVM services.
- ⑦ The entity operator's topic half watches *KafkaTopic* objects and reconciles them into real topics. The user half is omitted: no authentication, no users to manage.

Strimzi names the bootstrap Service `<cluster>-kafka-bootstrap`; every producer and consumer therefore gets `SPRING_KAFKA_BOOTSTRAP_SERVERS=ts-kafka-kafka-bootstrap:9092` in its Deployment — the `localhost:9092` config default of Part III overridden the way every other address was.

DECISION — OPERATOR, NOT A HAND-WRITTEN STATEFULSET

A plain KRaft *StatefulSet* (one pod, one PVC, the compose env block copied in) would cost less memory — no operator, no entity operator — and it is the honest fallback if CPU contention ever bites. The operator wins because it is what Kubernetes shops run (Red Hat sells it as AMQ Streams), because topics become declarative objects, and because broker upgrades and rolling restarts become someone else's tested code. Learning it is the point; the fallback stays documented in `STACK-K3S-JENKINS.md`.

25.4 Topics as objects

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
```

```
metadata:
  name: ts.user.registered
  labels: { strimzi.io/cluster: ts-kafka }
spec: { partitions: 1, replicas: 1 }
```

Kafka would auto-create these on first use; declaring them makes partition count a reviewed decision, removes the day-one race between producer and topic, and turns `kubectl get kafkatopics` into the inventory of every event contract in the system (four, matching Chapter 6’s table). One partition per topic: total order, which trivially preserves the per-user ordering the publishers key for. Partitions can be added later, never removed.

25.5 The last service, and its mailbox

notification-service’s manifest is the standard Deployment of Chapter 15 with three specifics: the bootstrap address above; `SPRING_MAIL_HOST=maildev / PORT=1025` overriding the config’s `#{MAIL_HOST:localhost}` default via relaxed binding; and probes on `/actuator/health` — this service has no Spring Security at all, so the endpoint is open by default (the third answer to Chapter 17’s probe question). Maildev is a tiny Deployment plus Service (ports 1025 SMTP, 1080 web); it delivers nothing and shows everything it caught. No Ingress: it is an operator’s tool, reached by port-forward. It also supplied this milestone’s second live lesson: applied by hand before the pipeline ran, its selector lacked the label `Kustomize injects`, and the first pipeline deploy died on “field is immutable” — Chapter 20’s pitfall, learned the expensive way.

PITFALL — NOTREADY — FATALPROBLEM, WHILE THE POD IS RUNNING

Symptom: minutes after applying, `kubectl get kafka` shows `READY` empty and a condition reading “Pod is unschedulable or is not starting” — yet `get pods` shows the broker 1/1 Running. Cause: the operator’s reconciliation timed out while the 600 MB Kafka image was still pulling over Wi-Fi, and wrote a failure status; the *next* periodic loop (every two minutes) observed the running pod and flipped it to Ready. Operator status is eventually honest. Before reacting to a red condition, check the pod it is describing — and check the operator log, where “reconciled” lines show the loops still turning.

ON THIS HARDWARE

Operator + broker + entity operator cost about 1.5 GB here, with the broker’s heap capped at 512 MiB. The first image pull is the expensive part — expect

ten minutes on residential Wi-Fi, and the stale-status pitfall above. Everything afterwards is fast: the image is cached on the node, which the smoke test below exploits.

HANDS-ON — PROVE THE BROKER, THEN PROVE THE FLOW

A throwaway pod with the Kafka CLI, in the same network position as the services, using the image the node already holds:

```
kubectrl -n ts run kafka-smoke --rm -i --restart=Never \
  --image=quay.io/strimzi/kafka:1.2.0-kafka-4.3.1 -- sh -c '
  B=ts-kafka-kafka-bootstrap:9092
  bin/kafka-topics.sh --bootstrap-server $B --create --topic smoke \
    --partitions 1 --replication-factor 1
  echo hello | bin/kafka-console-producer.sh --bootstrap-server $B --topic smoke
  bin/kafka-console-consumer.sh --bootstrap-server $B --topic smoke \
    --from-beginning --max-messages 1 --timeout-ms 20000
  bin/kafka-topics.sh --bootstrap-server $B --delete --topic smoke'
```

Use a scratch topic, not a real one: a plain-text “hello” on `ts.user.registered` is a poison pill for the JSON consumer (Chapter 11) — skipped gracefully thanks to `ErrorHandlingDeserializer`, but why find out in production. Then the real flow, once the pipeline has deployed the services:

```
curl -X POST http://ts.local/api/auth/register -H 'Content-Type: application/json' \
  -d '{"email":"t@example.com","password":"Passw0rd!","firstName":"Test",
    "lastName":"User","role":"TEACHER","activationMethod":"EMAIL"}'
# -> 201 {"message":"Registration successful. Check your email ..."}
kubectrl -n ts logs deploy/notification-service | grep -i registered
# maildev's REST API, from inside the cluster (or port-forward 1080 for the UI):
kubectrl -n ts run mailcheck --rm -i --restart=Never \
  --image=curlimages/curl:8.10.1 -- -s http://maildev:1080/email
# -> [{"subject":"Activate your TeacherSupporter account","to":"t@example.com",...
```

That request crosses every part of this book: Ingress → gateway → auth-service → Postgres → Kafka → notification-service → SMTP. It ran for the first time on 2026-08-25, pipeline build #7 — after builds #3–#6 each taught one of this chapter’s and Chapters 17, 20 and 23’s pitfalls. One loose end it exposed: the activation link in the mail still says `localhost:8080`, the dev default of the link base URL — an env override for milestone 7, when the frontend gives the cluster a real public address.

25.6 Outside the project

Managed Kafka — AWS MSK, Confluent Cloud, Azure Event Hubs' Kafka endpoint — is the operator's operator: the same Kafka-shaped declaration made in a cloud console, with brokers you never see. Migration from here is a bootstrap string plus SASL/TLS on the listener; the `KafkaTopic` objects would give way to Terraform resources saying exactly the same thing. The other lesson to carry: the operator pattern is how Kubernetes hosts *any* stateful platform component — Postgres (CloudNativePG), Redis, Elasticsearch — and having installed, mis-versioned, and status-debugged one, you know the shape of all of them.

CHECK YOURSELF (answers: Appendix A)

1. Define an operator in terms of Chapter 14's controllers, and name the two kinds of object Strimzi added to this cluster.
2. Why is `install-strimzi.sh` an admin step while `kafka.yaml` is applied by Jenkins? Which RBAC concept draws the line?
3. List three things Compose's Kafka block configured by hand that the Kafka resource does not mention, and say who computes them now.
4. You see `NotReady` / `FatalProblem` on the Kafka resource. What two things do you check before acting, and why might the status be wrong?
5. Why declare topics as objects when Kafka auto-creates them, and why one partition each?

The Frontend and the Single Host

Milestone 7 puts the last deployable — the React frontend — into the cluster and claims the `/` path on `ts.local` that Chapter 18’s Ingress deliberately left free. It is a short milestone in lines of YAML and a long one in concepts: it is the only image the pipeline builds from a Dockerfile, the only pod that is not a JVM, and the change that turns “services reachable with `curl`” into “an application usable in a browser”.

26.1 What a frontend is, to an operator

A built React app is not a server. `npm run build` (TypeScript check, then Vite bundling) produces a directory of static files: one `index.html`, one content-hashed JavaScript bundle (`index-CqZING8N.js`), one CSS file, a favicon. Nothing in it runs on the server; every line of it runs in the visitor’s browser, which then calls `/api/...` over HTTP like any other client. So “deploying the frontend” means deploying a *file server* with those files inside — nginx here — and telling the edge router which requests are files and which are API.

That framing decides everything else in this chapter. No config server (the app has no properties to import), no Eureka (nobody calls it by name), no secret (the browser can read anything the frontend holds, so nothing secret may be there), and a resource footprint smaller than the smallest JVM’s class-loading overhead.

26.2 Two stages, one Dockerfile

Jib (Chapter 13) builds JVM images from `target/classes`; it cannot run `npm`. The frontend therefore gets the classic multi-stage Dockerfile — and the reason it has two stages is the same reason the JVM images use a JRE base rather than a JDK:

```
FROM node:24-alpine AS build      ①
WORKDIR /app
COPY package.json package-lock.json ./ ②
RUN npm ci --no-audit --no-fund
COPY . .
RUN npm run build

FROM nginx:1.27-alpine           ③
```

```
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html ④
EXPOSE 80
```

- ① The build stage has Node, the TypeScript compiler, and after `npm ci` roughly 300 MB of `node_modules`. None of it is needed to *serve* the result.
- ② Lockfile first, sources second — Chapter 12’s layer cache rule. `npm ci` installs exactly the versions pinned in `package-lock.json` and refuses to modify it; a source edit invalidates only the layers from `COPY . .` down, so the dependency download is skipped on every rebuild that does not touch `package.json`.
- ③ A fresh base. The final image is `nginx` plus static files: about 15 MB, no Node, no compiler, no package manager — a smaller attack surface and a faster pull, for free.
- ④ `--from=build` is the multi-stage copy: only `dist/` crosses from stage one to stage two. Everything else in stage one is discarded when the build ends.

DECISION — WHY NOT ADD NODE TO THE JENKINS IMAGE AND USE JIB?

Jib can wrap any directory into an image, so a “Node in Jenkins, Jib for packaging” route exists. It was rejected because it hides the most common image-building technique behind the least common one. Every team that ships a frontend has this Dockerfile; being able to read and write it is a job-market skill in itself, and having one non-Jib image in the pipeline also proves the pipeline is not secretly Jib-shaped. The cost is a Docker CLI in the Jenkins image and a mounted daemon socket — Section 26.6.

26.3 nginx: the fallback that makes a SPA work

```
server {
  listen 80;
  root /usr/share/nginx/html;
  location / {
    try_files $uri $uri/ /index.html; ①
  }
  location /assets/ {
    add_header Cache-Control "public, max-age=31536000, immutable"; ②
  }
  location = /healthz { return 200 "ok\n"; } ③
}
```

- ① The line that every single-page app needs and every first deployment forgets. React Router (BrowserRouter in main.tsx) owns URLs like `/courses/42` purely in the browser: clicking a link never asks the server. But a refresh, a bookmark or a pasted link *does* ask the server for `/courses/42`, no such file exists, and a plain file server answers 404. `try_files` tries the literal path, then a directory, then falls back to `index.html` — the app boots and its router reads the URL. The OAuth callback (`/oauth/callback`) relies on this too: Google’s redirect is a cold request for a route that exists only in JavaScript.
- ② Vite writes a content hash into every asset filename, so a changed file always has a new name. That makes “cache forever” safe for `/assets/` — and *unsafe* for `index.html`, which must be re-fetched to learn the new names after a deploy. Caching the HTML is how users end up with a stale app that requests bundles the server no longer has.
- ③ A probe target that costs nothing and never touches disk (Chapter 17’s probes need something to call).

Note what the file does *not* do: proxy `/api`. On a laptop, `vite.config.ts` proxies `/api` to the gateway on `localhost:8080` so the dev server and the API look like one origin. In the cluster that job belongs to Traefik — nginx never sees an API request at all.

26.4 Claiming `/`: one origin

```
kind: Ingress
metadata: { name: frontend }
spec:
  rules:
    - host: ts.local
      http:
        paths:
          - path: / ①
            pathType: Prefix
            backend: { service: { name: frontend, port: { number: 80 } } }
```

- ① A catch-all, in a *second* Ingress object next to the gateway’s. Traefik merges every Ingress rule for a host and routes by *longest matching prefix*: `/api/courses` matches both `/` and `/api`, and `/api` wins. The gateway’s three prefixes therefore keep working unchanged, and everything else — `/`, `/login`, `/courses/42`, `/assets/...` — falls through to nginx. Two objects, one merged routing table; Chapter 20’s “apply never deletes” still applies to each.

The consequence is the reason this milestone exists at all: **one origin**. The

browser loads the page from `http://ts.local` and calls `http://ts.local/api/...` — same scheme, host and port. Compare Compose, where the page came from `localhost:3000` and the API from `localhost:8080`: a different port is a different origin, so every API call was preceded by a CORS preflight and the gateway's `CorsConfig` had to list `localhost:3000` explicitly. In the cluster `CorsConfig` is simply never consulted — there is no cross-origin request to police. Cookies, the OAuth redirect, and any future SameSite policy all get simpler for the same reason.

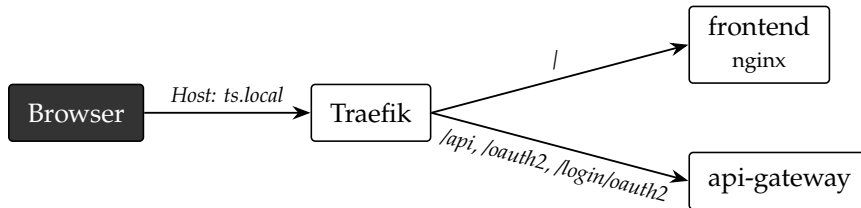


Figure 26.1: Longest prefix wins: one host, two backends.

One property follows the host change. `OAuth2LoginSuccessHandler` finishes Google login by redirecting the browser to `app.oauth2.success-redirect-uri`, which `config-server`'s `auth-service.yml` sets to `localhost:3000/oauth/callback` — correct on a laptop, wrong in the cluster. `auth-service.yml` overrides it with the env var `APP_OAUTH2_SUCCESS_REDIRECT_URI=http://ts.local/oauth/callback` — Chapter 16's precedence rule (env var beats config server) doing the same job it did for the datasource URL.

26.5 The manifest

The Deployment is Chapter 15's shape with almost everything removed: no `envFrom`, no `SPRING_*` variables, no `startupProbe` (nginx is up in milliseconds), and `requests: { memory: 32Mi, cpu: 10m }` — one-twelfth of the smallest JVM. `imagePullPolicy`: Always with the `:latest` tag as the manifest default, overridden to the commit tag by the pipeline exactly as for the JVM services.

26.6 The pipeline grows a stage

```

stage('Frontend image') {
  steps {
    dir('Project/TeacherSupporter') {
      sh '''
        docker build -t ${REGISTRY}/frontend:${TAG} \

```

```

        -t ${REGISTRY}/frontend:latest frontend ①
    docker push ${REGISTRY}/frontend:${TAG}
    docker push ${REGISTRY}/frontend:latest
    ...
}
}
}

```

- ① The build context is `frontend/`, trimmed by a `.dockerignore` that excludes `node_modules` and `dist`: the Windows laptop's native modules must not be sent to the daemon, and the image builds its own. `frontend` joins `SERVICES` (kubect'l's list) but not `MODULES` (Maven's) — the Deploy stage's `set image / rollout status` loops handle it without change.

For `docker build` to work inside the Jenkins container, two things change in Chapter 22's setup. The image gains the Docker CLI (`docker-ce-cli` — the client only), and `run.sh` mounts the host's `/var/run/docker.sock` into the container. The CLI talks over that socket to the *host's* daemon; there is no daemon inside Jenkins and no Docker-in-Docker. The resulting image lands in the host's image store, and `docker push localhost:5000/...` reaches the registry over the host network the container already shares.

PITFALL — PERMISSION DENIED ON `/VAR/RUN/DOCKER.SOCK`

Symptom: the CLI is installed, the socket is mounted, and every `docker` command fails with a permission error. Cause: the socket is owned by the host's `docker` group — `GID 984` on `ts-server` — and that number means nothing inside the image, where the `jenkins` user belongs to a group of the same name with a different `GID` (or none). Fix in `run.sh`: `-group-add "$(stat -c %g /var/run/docker.sock)"` adds the host's numeric `GID` to the container's user at run time, without baking a machine-specific number into the Dockerfile.

DECISION — THE SOCKET IS ROOT

Mounting the Docker socket grants root-equivalent control of the host: whoever can run `docker run -v /:/host` owns the machine. For a single-user home-lab whose Jenkins is reachable only over the tailnet this is an accepted trade, stated out loud. Production builds images without a daemon at all — Kaniko or BuildKit running as an unprivileged pod, or Jib for the JVM — and the Chapter 27 migration of Jenkins into the cluster is where this compromise would be paid back.

ON THIS HARDWARE

The frontend pod is the cheapest in the namespace (32 Mi requested), but the *build* is not: `npm ci` plus `tsc` plus Vite runs near a CPU-minute and a gigabyte of RAM on the controller, while Maven may still be running in the same 2 GB container. The Jenkinsfile keeps the stages sequential for that reason; on this box, parallel stages would only compete.

26.7 Verifying the milestone

HANDS-ON — THE MILESTONE-7 CHECKLIST

1. `kubectl -n ts get ingress` — two objects on `ts.local`; `kubectl -n ts get pods -l app=frontend` Ready.
2. Open `http://ts.local/` — the login page. Then paste `http://ts.local/courses` directly into the address bar: a page, not nginx’s 404 — the SPA fallback works.
3. Register, log in, create a course: the browser’s network tab shows `/api/...` requests with no OPTIONS preflights — same origin.
4. `curl -sI http://ts.local/assets/index-*.js | grep Cache-Control` — immutable; the same for `/` — no cache header.
5. Change any text in a page, push, and watch the pipeline: the Frontend image stage reuses the `npm ci` layer, and the rollout swaps a 15 MB image.

Build #9 was the first run of the new stage: `docker build` completed in 90 seconds on the controller, the rollout swapped `frontend:f5d9213` in, and the checklist above passed as written — `/courses` answered `index.html`, `/api/courses` answered the gateway’s 401 JSON, the asset carried `immutable` and `/` carried no cache header.

PITFALL — THE INGRESS THAT OUTLIVED ITS MANIFEST

`kubectl get ingress` on that day listed *three* objects on `ts.local`: the gateway’s, the frontend’s — and `course-service`, eight days old, routing `/courses` straight to the service with no JWT check. Its manifest had been removed from `course-service.yaml` back in Chapter 18 with a comment saying so, but Chapter 20’s rule held: `apply` never deletes what it no longer sees. It took a manual `kubectl -n ts delete ingress course-service`. The lesson is not the command but the audit: after every milestone, list what the cluster *has*, not only what git *says* — until GitOps pruning (Chapter 27) makes the two the same thing.

26.8 Reaching it from anywhere: the tailnet name

Everything above answered on `ts.local` — a name that exists only in one laptop’s hosts file, on one LAN. “Live” means a URL that resolves on every device you own, and the machine already has one: Tailscale gives each node a MagicDNS name, `ts-server.tailbfe002.ts.net`, that every peer on the tailnet resolves without any hosts-file edit. Traefik routes on the Host header (Chapter 18), so a name it does not list gets a 404 even though the packets arrive. The fix is one more rules entry with the same path list in both Ingress objects (Appendix D shows them) — applied by hand with a filter that extracts only the Ingress documents from `kubectl kustomize` output, because a full `apply -k` would reset every Deployment’s image tag to the manifests’ 1.0.0-SNAPSHOT and roll the whole namespace (Chapter 23’s `set image lives after` apply for exactly this reason).

The name then surfaced a bug the LAN had hidden. A registration through the new host produced its activation mail (Chapter 25’s chain, end to end over the tailnet), but the link inside read `http://localhost:8080/api/auth/activate?code=...`: a host no phone can reach, and a POST endpoint that a mail client’s GET could never have used anyway. EmailService built every link — activate, invite, login — from string literals. The correction is the Chapter 16 rule applied to URLs: the public origin is *configuration*. `app.frontend.base-url` defaults to the Vite dev server in `config-server’s notification-service.yml`; the manifest overrides it with `APP_FRONTEND_BASE_URL` set to the tailnet name, the same relaxed-binding trick as `SPRING_MAIL_HOST`; and the links now target the frontend’s `/activate` page, which does the POST. The two OAuth variables moved to the same host, which means one more entry under Google’s “Authorized redirect URIs” — the registration that Chapter 5 called the one thing no manifest can automate.

PITFALL — A LINK IS AN EXTERNAL CONTRACT

Anything that leaves the system — an email, a webhook payload, a QR code — carries a URL that must work from *outside*, so it can never be a Service name and should never be a literal. The test is cheap: register from a device that is not the build laptop and click. On this stack it was the first request through the new hostname that exposed a bug present since the first commit.

26.9 Outside the project

The pattern — edge router splits static from API by path, static comes from a dumb file server — is universal, with the file server usually replaced by object storage plus a CDN (S3 + CloudFront, or a static host such as Netlify) and the split

done by the CDN's origin rules. The build stage stays exactly this Dockerfile's first half. Two habits transfer directly: hash-and-cache-forever for assets with an uncached `index.html`, and the SPA fallback — which in CloudFront terms is “map 403/404 to `/index.html` with status 200”, the same idea with worse ergonomics.

CHECK YOURSELF (answers: Appendix A)

1. Why does the frontend image have two FROM lines, and what exactly crosses between them?
2. A user bookmarks `http://ts.local/courses/7` and opens it the next morning. Trace the request through Traefik and nginx, and say which line makes it succeed.
3. Both Ingress objects declare `host: ts.local`. How does Traefik decide that `/api/courses` goes to the gateway and `/assets/x.js` to nginx?
4. Explain why the gateway's `CorsConfig` became irrelevant in the cluster while remaining necessary on a laptop.
5. What does `-group-add "$(stat -c %g /var/run/docker.sock)"` fix, and why is the value computed instead of written down?
6. The activation mail's link was wrong for months and nobody noticed. Name the two separate defects in `http://localhost:8080/api/auth/activate?code=...`, and say which environment (laptop, compose, cluster) each one would have broken.

The Road Ahead

With the frontend served from the cluster (Chapter 26), every deployable the project has is deployed and the application is usable in a browser. The remaining milestones each swap something hand-rolled for the industrial mechanism — and each makes most sense now that you know what it replaces.

27.1 Observability

Beyond `kubectl logs` lies the standard trio: **metrics** (Prometheus scraping the `/actuator/prometheus` endpoints every service already exposes; Grafana dashboards over the RED trio — rate, errors, duration), **traces** (the Zipkin wiring already in every config, currently sampled at zero in the cluster — one env var turns it on once a Zipkin pod exists), and **alerts** (Alertmanager: a page when a service’s error rate or a PVC’s fullness crosses a line). This lands via the `kube-prometheus-stack` Helm chart — the promised moment where you *consume* Helm without authoring charts (Chapter 20’s decision box, honored). Strimzi can export broker metrics into the same Prometheus with a few lines of `metricsConfig` on the Kafka resource — consumer-group lag is the first Kafka alert worth having.

ON THIS HARDWARE

The full stack costs ~1.5 GB and nontrivial CPU — the budget says it fits, but it is the first thing to shed under load. Droppable observability is a homelab luxury; production teams size for it first.

27.2 GitOps: Argo CD

The pipeline *pushes* deployments (Jenkins holds a `kubeconfig` and runs `kubectl`). GitOps inverts the arrow: an in-cluster agent (Argo CD) *pulls* — continuously comparing a git repository’s manifests with the cluster and converging. What that buys, in the vocabulary you now have: the cluster credential never leaves the cluster (Chapter 22’s RBAC concern dissolves — CI only pushes images and commits a tag bump); deletions finally propagate (Chapter 20’s prune pitfall, solved by design); drift becomes visible (a hand-edited Deployment shows as “OutOfSync”); and rollback is `git revert`. Jenkins keeps CI — build, test, push, update

the tag in git — Argo CD takes CD. The Jenkinsfile’s deploy stage, this book’s Chapter 23, becomes a commit.

27.3 Retiring the classic stack

The deliberate redundancy of Chapters 2 and 3 can now be paid off, service by service:

- **Eureka out:** set the gateway’s routes from `lb://course-service` to `http://course-service:8082` — Service DNS and readiness-gated endpoints do everything the registry did here. Delete a deployable, free 512 Mi.
- **Config-server out:** mount the `configurations/` files as a ConfigMap and point `spring.config.import` at the mounted directory (optional: `file:/config/`). Same files, one fewer service, and config changes become manifest changes flowing through GitOps.

Both migrations are résumé material precisely because they run *toward* the platform: knowing why Eureka exists *and* why Kubernetes obsoletes it is the difference between using either and understanding both.

27.4 Jenkins into the cluster

Chapter 22’s homelab inversion — builds on the controller, Jenkins outside k3s — reverses cleanly once the pipeline is stable: the Kubernetes plugin spawns an agent pod per build, and Jenkins itself becomes a StatefulSet with `JENKINS_HOME` on a PVC. The migration touches every concept in Part V at once, which is why it is scheduled last.

27.5 OpenShift, briefly

The manifests were kept portable on purpose: Jib’s non-root images (Chapter 13) satisfy OpenShift’s arbitrary-UID security model, so deploying to a free OpenShift sandbox should need only the Ingress-to-Route translation (Chapter 18). Running that experiment once validates the portability claims this book has been making — and OpenShift vocabulary (Routes, SCCs, BuildConfigs) maps one-to-one onto chapters you have read.

27.6 Where this leaves you

Strip the project names away and the inventory reads like a platform job description: the property-resolution machinery that makes one artifact run anywhere; logs, consumer groups and delivery semantics; images, layers and registries; reconciliation, probes, RBAC, storage classes and operators; pipelines, RBAC-scoped deploy identities, and the GitOps inversion. The homelab constraints were not limitations — they were the syllabus: every “on this hardware” box forced a decision that money usually hides.

CHECK YOURSELF (answers: Appendix A)

1. Which metric would you alert on first for the Kafka path, and what does it measure in Chapter 10’s terms?
2. Push-based vs. pull-based deployment: locate the cluster credential in each, and explain which chapter’s pitfall GitOps pruning resolves.
3. Sketch the Eureka retirement: what changes in the gateway’s config, which two Kubernetes mechanisms replace registration and liveness, and what can be deleted?
4. Why do Jib’s images make the OpenShift experiment cheap, and which object translation remains?

Part VIII

Deep Dives

Deep Dive: JWT at Scale — Where to Verify, Fat Tokens, and Asymmetric Keys

Chapter 5 taught the project’s security design as it is: HS256 tokens, verified once at the gateway, identity forwarded as headers. That design is correct for a system with one gateway and five services. This chapter is the first of the book’s *deep dives*: it takes one design already in the project and pushes it until it breaks, the way an interviewer or a tenfold traffic increase would. The questions come from a widely shared Vietnamese article on JWT in microservices;¹ the answers come from reading the project’s code.

Three questions organize the chapter:

1. **Where should a token be verified** — at the gateway once, or in every service?
2. **What happens when the token gets fat** — and what is the *phantom token* pattern that keeps it thin?
3. **What does a signature actually guarantee**, and why does the choice between HS256 and RS256 decide who can be trusted with what?

28.1 Where to verify: gateway or every service?

28.1.1 Model 1: centralized verification (what the project does)

Re-read the two halves of the trusted-edge pattern. At the edge, in the gateway’s `JwtAuthenticationFilter`:

```
Claims claims = Jwts.parser()
    .verifyWith(signingKey)           ①
    .build()
    .parseSignedClaims(token)
    .getPayload();

String userId = claims.getSubject();
String role = claims.get(JwtConstants.CLAIM_ROLE, String.class);
```

¹“Lột trần sự thật về JWT — Phần 2: Trận chiến Microservices và nỗi ám ảnh Token béo phì”, viblo.asia, 2026.

```
ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
    .header(JwtConstants.HEADER_USER_ID, userId) ②
    .header(JwtConstants.HEADER_USER_ROLE, role)
    .build();
```

And behind it, in every service’s `GatewayHeaderAuthFilter` (course-service’s copy shown):

```
String userId = request.getHeader("X-User-Id");
String userRole = request.getHeader("X-User-Role");

if (userId != null && userRole != null) { ③
    var authorities = List.of(
        new SimpleGrantedAuthority("ROLE_" + userRole));
    var authentication = new UsernamePasswordAuthenticationToken(
        Long.valueOf(userId), null, authorities);
    SecurityContextHolder.getContext().setAuthentication(authentication);
}
```

- ① Cryptography happens exactly once, here. The key is the same `app.jwt.secret` that auth-service signs with.
- ② Identity leaves the gateway as two plain HTTP headers. Nothing downstream can tell whether the gateway wrote them or someone else did.
- ③ The service performs *no* check. Two headers present means authenticated. This is the whole contract.

The advantages are real: one place holds the secret, services stay free of JWT libraries and key handling, and the per-request CPU cost of signature checking is paid once. Most Spring Cloud tutorials stop here.

28.1.2 The “trusted internal network” fallacy

The pattern rests on one invariant — *every* path to a service passes through the gateway. In the project’s Compose file that invariant is already false. `docker-compose.yml` publishes auth-service on 8081 and course-service on 8082 directly to the host, so that you can debug them. Anyone on the host can therefore skip the gateway:

HANDS-ON — FORGE AN IDENTITY IN ONE LINE

With the stack running, ask course-service for a teacher’s courses without ever presenting a token:

```
$ curl -s -H "X-User-Id: 1" -H "X-User-Role: TEACHER" \
    http://localhost:8082/courses | head -c 300
```


It answers. Now repeat through the gateway on 8080 with the same two headers and no Authorization:

```
$ curl -s -H "X-User-Id: 1" -H "X-User-Role: TEACHER" \
  http://localhost:8080/api/courses
{"error":"Unauthorized","message":"Missing or invalid Authorization header"}
```

The difference between the two outputs is the entire security model.

In Kubernetes the published ports go away (Chapter 18), but the flat pod network remains: any pod in the namespace can reach `course-service:8082`. The fallacy is assuming that “inside” means “trusted”. One compromised container — a vulnerable image, a leaked kubeconfig, a supply-chain dependency — and the attacker forges `X-User-Id: 1` to every service at once.

PITFALL — IDENTITY HEADERS SURVIVE PUBLIC PATHS

Symptom: nothing visible, which is the problem. Look at the top of `JwtAuthenticationFilter.filter`: a request to a public path (`/api/auth/refresh`, `/actor/**`) is forwarded *unmodified* — the gateway neither verifies a token nor touches headers. A client can therefore send `X-User-Id: 1` to a public path and the header arrives at the service intact. Today no public endpoint reads it, so nothing is exploitable; but the invariant “only the gateway writes these headers” is not enforced, it is merely true by luck. The fix is one default filter in `api-gateway.yml` that strips both headers from every *incoming* request before the JWT filter sets them:

```
spring:
  cloud:
    gateway:
      default-filters:
        - RemoveRequestHeader=X-User-Id
        - RemoveRequestHeader=X-User-Role
```

Now the headers can only originate at the gateway. This closes the external door; it does nothing for the in-cluster one.

28.1.3 Model 2: zero trust — every service verifies

The alternative removes the trust: every service parses and verifies the JWT itself, and ignores any identity header. A forged header is now worthless because the service never reads it; a forged token fails signature verification. The gateway

may still verify — rejecting bad tokens early is cheap — but nothing downstream *depends* on it.

The cost appears immediately: every verifier needs the key. With HS256 that means copying the *signing* secret to every service, which is worse than the problem it solves (Section 28.3). Zero trust at the application layer is only reasonable with asymmetric keys, which is why the two halves of this chapter belong together.

DECISION — GATEWAY-VERIFIED HEADERS, FOR NOW

The project keeps Model 1. It has exactly two key holders, no third-party services, and every service is deployed by the same pipeline — the blast radius of the trusted-edge assumption is the single namespace. Three signals would flip the decision: a service written by another team, a service reachable by any caller other than the gateway (a Kafka consumer that also exposes HTTP, a batch job), or a compliance requirement that identity be verifiable at the point of use. At that moment the move is RS256 plus per-service verification, not more headers. Production systems usually take a third path: keep the headers, and make the *transport* prove the sender with mutual TLS from a service mesh (Istio, Linkerd). The gateway then becomes a cryptographically identified peer instead of an unverifiable neighbor.

28.2 What a signature guarantees

A JWT is `base64url(header).base64url(payload).signature`. The first two parts are *encoded*, not encrypted: paste any project token into a decoder and sub, email, role are readable. The signature adds exactly two properties:

- **Integrity.** Change one byte of the payload (`"role":"STUDENT"` to `"TEACHER"`) and verification fails.
- **Authenticity.** The token was produced by a holder of the signing key — and by nobody else.

It adds no confidentiality. Both HS256 and RS256 compute the signature over the same string, `base64url(header) + "." + base64url(payload)`. They differ only in *which key creates the signature and which key checks it*. That single difference decides the whole trust topology of a system.

28.3 HS256 versus RS256

28.3.1 HS256: one key, two roles

HS256 is HMAC-SHA256: a hash with a key mixed in. You do not need the construction (two nested SHA-256 passes with the key XORed against fixed pads); you need one property. *The same key both creates and checks the signature.* Verifying means recomputing the HMAC with your copy of the key and comparing bytes. Without the key you cannot compute it, so you cannot forge. With the key you can compute it, so you *can* forge. Verifier and signer are indistinguishable.

The project's two copies of that key:

```
// auth-service/.../service/JwtService.java
this.signingKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));

// api-gateway/.../filter/JwtAuthenticationFilter.java
this.signingKey = Keys.hmacShaKeyFor(jwtSecret.getBytes(StandardCharsets.UTF_8));
```

Both read `app.jwt.secret`; in the cluster both mount the same `jwt Secret` (Chapter 16). The gateway only ever needs to *verify*, yet it holds a key that can *mint* a token for any user with any role. Compromise the gateway pod and you own every identity in the system. Extend Model 2 naively — copy the secret to five services — and you have six such places.

28.3.2 RS256: two keys, one role each

RS256 is an RSA signature over a SHA-256 digest. RSA rests on an asymmetry in arithmetic: multiplying two large primes is instant; recovering them from the product is infeasible.

Key generation, once:

1. Pick two large primes p, q (about 1024 bits each for a 2048-bit key). Compute $n = p \times q$.
2. Pick a public exponent e ; in practice always 65537. The pair (n, e) is the **public key**.
3. Compute d such that $e \times d \equiv 1 \pmod{(p-1)(q-1)}$. The pair (n, d) is the **private key**. Computing d requires p and q ; knowing only n does not give them.

Signing and verifying:

```
h = SHA256(header " " payload), padded (PKCS#1 v1.5)
sign:  signature = h^d mod n    -- needs d: only auth-service
verify: signature^e mod n == h ? -- needs e, n: anyone
```

Because $e \times d \equiv 1$, raising to d and then to e returns h . Anyone with the public key can check a signature; nobody without d can produce one. That is the asymmetry HS256 lacks, and it changes who may hold what:

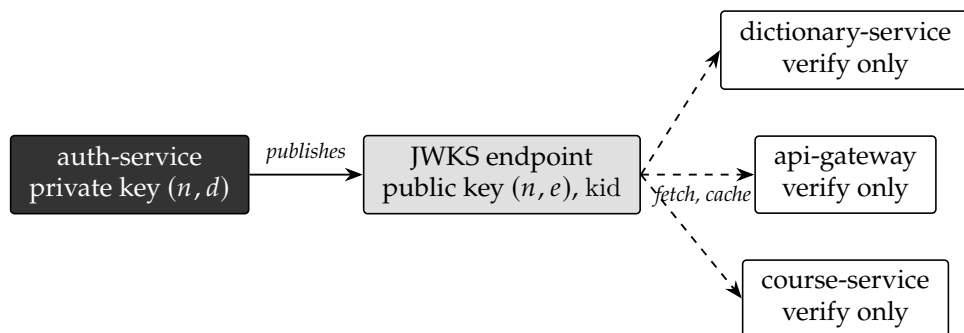


Figure 28.1: RS256 key topology: one signer, any number of verifiers. Leaking a verifier leaks nothing.

- Auth-service holds the private key and is the *only* place a token can be issued.
- Gateway and every service hold the public key. Leaking it is harmless — it is public by definition. You could print it in this book.
- Key distribution stops being a secret-management problem: publish the public key at a well-known URL (a JWKS, JSON Web Key Set), and any verifier fetches it at startup and caches it.

You already consume this topology. The Google login of Chapter 5 returns an `id_token` signed RS256; Spring’s OAuth2 client verifies it against Google’s JWKS at <https://www.googleapis.com/oauth2/v3/certs>. Google never hands you its private key, and you never need it.

28.3.3 The costs

	HS256	RS256 (2048-bit)
Key material	one secret, ≥ 32 bytes	private + public, ≈ 1.7 KB PEM each
Signature size	32 bytes (43 chars)	256 bytes (342 chars)
Sign	microseconds	about 1 ms
Verify	microseconds	tens of microseconds
Who can sign	any key holder	private-key holder only
Who can verify	any key holder	anyone

RSA verification is far cheaper than signing because $e = 65537$ is tiny while d is 2048 bits wide. That asymmetry fits the topology: one signer, many verifiers. If signature size matters, ES256 (ECDSA on curve P-256) gives 64-byte signatures and much smaller keys, and is the modern default for new identity providers.

PITFALL — ALGORITHM CONFUSION

Symptom: a token that should be rejected is accepted — and your logs show nothing wrong. Cause: older libraries read the `alg` field of the header to decide *how* to verify. An attacker takes a legitimate RS256 token, rewrites the header to `"alg":"HS256"`, and signs it with HMAC using the *public key bytes* as the secret — bytes that are, by design, public. A naive verifier sees HS256, uses the public key it has as an HMAC secret, and the check passes. JJWT prevents this by letting the *key type* choose the algorithm, not the header: `verifyWith(SecretKey)` rejects any asymmetric token and `verifyWith(PublicKey)` rejects any HMAC token. That is why `JwtAuthenticationFilter` is safe today, and why you must never verify with a method that accepts “whatever the header says”.

28.3.4 Converting the project

The change is small enough to read in full. Generate the pair once:

```
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 \
  -out jwt-private.pem
$ openssl pkey -in jwt-private.pem -pubout -out jwt-public.pem
```

The private PEM goes only into `auth-service`’s `Secret`; the public PEM can go into a `ConfigMap`, because it is not secret. `JwtService` signs with the private key:

```
@Service
public class JwtService {

    private final PrivateKey privateKey; // sign
    private final PublicKey publicKey; // self-verify where needed

    public JwtService(@Value("${app.jwt.private-key-pem}") String privPem,
                     @Value("${app.jwt.public-key-pem}") String pubPem)
        throws GeneralSecurityException {
        KeyFactory kf = KeyFactory.getInstance("RSA");
        this.privateKey = kf.generatePrivate(
            new PKCS8EncodedKeySpec(decodePem(privPem)));
        this.publicKey = kf.generatePublic(
            new X509EncodedKeySpec(decodePem(pubPem)));
    }
}
```

```
public String generateAccessToken(User user) {
    return Jwts.builder()
        .header().keyId("2026-09-key1").and()           ②
        .subject(String.valueOf(user.getId()))
        .claim("email", user.getEmail())
        .claim("role", user.getRole().name())
        .issuedAt(new Date())
        .expiration(new Date(System.currentTimeMillis()
            + accessTokenExpiryMs))
        .signWith(privateKey, Jwts.SIG.RS256)           ③
        .compact();
}

private static byte[] decodePem(String pem) {
    String body = pem.replaceAll("-----BEGIN [A-Z ]+-----", "")
        .replaceAll("-----END [A-Z ]+-----", "")
        .replaceAll("\\s", "");
    return Base64.getDecoder().decode(body);
}
}
```

- ① PKCS#8 is the container format openssl genpkey writes for private keys; X.509 SubjectPublicKeyInfo is what -pubout writes. Strip the PEM armor, base64-decode, done.
- ② kid (key id) names the key that signed this token. It costs a few bytes now and makes rotation possible later.
- ③ JJWT infers RS256 from the key type; naming it explicitly documents the intent.

The gateway — and any service that wants to verify for itself — holds only the public key:

```
public JwtAuthenticationFilter(@Value("${app.jwt.public-key-pem}") String pubPem)
    throws GeneralSecurityException {
    this.publicKey = KeyFactory.getInstance("RSA")
        .generatePublic(new X509EncodedKeySpec(decodePem(pubPem)));
}

Claims claims = Jwts.parser()
    .verifyWith(publicKey)    // PublicKey: accepts RS/PS/ES only
    .build()
    .parseSignedClaims(token)
    .getPayload();
```

Nothing else changes: same claims, same headers, same GatewayHeaderAuth-

Filter. The gateway can no longer mint tokens, and `api-gateway.yml` no longer needs `app.jwt.secret` at all.

28.3.5 Rotation, and what `kid` is for

Keys age: policy says rotate every 90 days, or a laptop with the PEM on it is lost. Rotation without logging every user out is a four-step ceremony:

1. Generate a new pair; add its public key to the JWKS alongside the old one, under a new `kid`.
2. Wait until every verifier has refreshed its cache (or restart them).
3. Switch auth-service to sign with the new private key. New tokens carry the new `kid`; old tokens, still in flight, carry the old one — and verifiers pick the matching public key by `kid`.
4. After the longest token lifetime has elapsed (here 15 minutes for access tokens), remove the old key from the JWKS.

Without `kid` a verifier would have to try every key it knows; with it, the lookup is exact and the old key can be retired with confidence. The project's static-PEM version above has no JWKS endpoint; the natural next step is to serve `/well-known/jwks.json` from auth-service and have verifiers use Spring Security's `NimbusJwtDecoder.withJwkSetUri(...)`, which caches and refetches on an unknown `kid` automatically.

28.4 Payload bloat

Statelessness is seductive: if the token carries everything a service needs, no service ever calls another to ask “may this user do this?” Teams follow that logic until the token carries the user's roles, permissions, groups, feature flags, and profile — and weighs 4 KB. The arithmetic then turns against them. A dashboard page firing 20 parallel API calls ships 80 KB of token before a single byte of business data. Multiply by tens of thousands of concurrent users and the authentication header is a measurable share of ingress bandwidth. Cookies, if that is where the token lives, have a 4 KB ceiling; some proxies cap header size at 8 KB; logs that record headers bloat in step.

The project's access token is thin. `JwtService.generateAccessToken` writes `sub`, `email`, `role`, `iat`, `exp` — about 200 bytes signed. Keep it that way by rule: **a token carries identity; entitlements are looked up**. The moment course-service needs to know “which courses may this teacher edit?”, the temptation to add a `courseIds` claim appears. Resist it. That list changes when a course is created, which the token cannot know until it expires; and it grows without bound. The service

already owns the enrollment data (Chapter 7) — one indexed query answers the question correctly and currently.

28.5 The phantom token pattern

Suppose the fat token is unavoidable — a legacy authorization model, a partner integration. The phantom token pattern keeps the *external* token tiny while the *internal* one stays self-contained:

1. Auth-service creates the full JWT but does *not* return it. It stores the JWT in a fast store (Redis) under a random opaque identifier, and returns only that identifier — a UUID, 36 bytes.
2. The gateway receives the UUID, looks it up, and swaps in the real JWT before forwarding. Internal services see a normal signed token.
3. Internal services verify it themselves (RS256, JWKS). Inside the perimeter the system is stateless and zero-trust; outside, the client carries a token with no readable content at all.

Logout becomes trivial and *immediate*: `DEL <uuid>` in Redis, and the next request with that identifier fails at the gateway with 401, no matter how long the inner JWT would have lived. This is the pattern's real prize — it repairs JWT's one structural weakness, irrevocability.

The project already runs half of it. Look at `JwtService.generateRefreshTokenValue`:

```
public String generateRefreshTokenValue() {  
    return UUID.randomUUID().toString();  
}
```

The refresh token is not a JWT. It is an opaque identifier stored in the `refresh_tokens` table (Flyway V2) and revoked by deleting the row — `AuthService.logout` does exactly that. The access token remains a self-contained JWT. So the project's revocation latency is bounded by the access token's 15 minutes, and its revocation *mechanism* is the phantom pattern applied to the long-lived credential only.

PITFALL — REFRESH WITHOUT ROTATION

Read `AuthService.refreshToken`: it returns the *same* refresh token it was given. A refresh token stolen once is therefore valid for its full seven days, and the theft is undetectable. The industry practice is *rotation*: every `/refresh` issues a new refresh token and invalidates the old one. If an old token is ever presented again, two parties hold copies — revoke the whole family. The table already has the columns to do this; the change is a delete-and-insert in one

transaction.

DECISION — THREE WAYS TO REVOKE AN ACCESS TOKEN

When an admin disables an account, the project's answer is "within 15 minutes". The alternatives, in ascending cost:

1. **Wait it out.** Short access tokens plus stateful refresh tokens. Zero extra infrastructure; revocation lag equals access-token lifetime. Acceptable for most applications, and what the project does.
2. **Deny-list.** Add a `jti` (token id) claim; on logout or disable, write the `jti` to Redis with a TTL equal to the token's remaining life; the gateway checks Redis on every request. One fast lookup per request, and only revoked tokens are stored.
3. **Phantom token.** Opaque identifier outside, JWT inside, Redis as the mapping. Immediate revocation and no readable token in the browser — but Redis is now in the critical path of *every* request. Redis down means every user is logged out at once.

A senior engineer's objection to the article's enthusiasm: option 3 trades "cannot revoke" for "one new single point of failure". Most teams should reach for option 2 first and treat option 3 as a fit for banking-grade requirements, where the article itself admits sessions may be the better tool anyway.

28.6 What breaks at 3 a.m.

Failure analysis is the fastest way to tell the two key models apart. Auth-service goes down at 3 a.m.; nothing else does.

- **HS256, as deployed.** The gateway holds the secret and verifies locally: every logged-in user keeps working until their access token expires. Refresh calls fail (auth-service is the `/refresh` endpoint), so sessions die within 15 minutes. New logins fail. Recovery is instant when auth-service returns.
- **RS256 with cached JWKS.** Identical for running verifiers: they cached the public key at startup and need nothing from auth-service. The new failure is a *cold* verifier — a gateway pod restarted during the outage cannot fetch the JWKS, and depending on its configuration either refuses all tokens (fail closed) or crashes at boot. Mitigations: ship the current public key in configuration as a fallback, or give the JWKS cache a long grace period.
- **Phantom token.** Add Redis to the list of things that must be up. Auth-service down has the same effects as above; Redis down is a total outage of

authenticated traffic.

The lesson generalizes: statelessness buys you independence from the issuer at request time. Every mechanism you add to regain control — JWKS, deny-lists, opaque mappings — reintroduces a dependency, and you should be able to name it and say what happens when it is unavailable.

28.7 When JWT is the wrong tool

The article closes with a warning worth repeating, because the project’s own path (Chapter 27) includes a monolithic alternative. JWT earns its complexity when *many independent parties* must answer “who is calling?” without sharing a session store: microservices, single sign-on across applications, OAuth2/OIDC federation. It is the wrong tool when the system is a single deployable with room for its sessions in memory, when permissions change often and must take effect immediately, or when a regulator requires that a session be terminable in milliseconds. In those cases a server-side session with a cookie is simpler, revocable, and boring — and boring is a feature.

28.8 Outside the project

Keycloak, Auth0, Cognito, Okta and Entra ID all issue RS256 or ES256 tokens and publish JWKS; every Spring resource server in the industry verifies with the one-line `jwk-set-uri` configuration shown in Chapter 5. Service meshes (Istio, Linkerd) implement the mTLS variant of zero trust so that application code keeps reading headers while the transport proves who wrote them. The phantom token pattern is productized by Curity and by API gateways such as Kong and Apigee under names like “token introspection” and “opaque token exchange”. Knowing the hand-rolled version — one HS256 secret, two holders — is what makes those products legible.

28.9 Questions from real interviews

These are asked as open problems, not definitions. The answers below are what a senior engineer says in the first two minutes; the interviewer then digs into whichever trade-off you named.

Q: How do you invalidate a JWT before it expires?

You do not; you bound the damage. Access tokens live minutes, not hours, so “wait it out” is the baseline. For immediate revocation add a `jti` claim and a deny-list in Redis with TTL equal to the token’s remaining life — one lookup per request, only revoked tokens stored, and Redis unavailability can fail open or closed by policy. Full opacity (phantom token) makes revocation instant but puts Redis on every request’s critical path. In this project the refresh token is already opaque and database-backed, so “log out everywhere” is `DELETE FROM refresh_tokens WHERE user_id = ?` plus, at most, 15 minutes of residual access.

Q: Login traffic goes from 10 to 10 000 per second. What breaks?

Signing first: RS256 signing is about 1 ms of CPU per token, so ten thousand logins per second is ten cores of pure signing, while HS256 would be negligible. Then password hashing, which is deliberately slow (bcrypt at cost 10 is roughly 100 ms) — that is the real wall, and the answer is horizontal scaling of auth-service plus rate limiting per IP and per account at the gateway. Verification is not the problem: RS256 verify is tens of microseconds and needs no network. Also check that the refresh-token insert is not on a single hot index and that the `refresh_tokens` table is pruned of expired rows.

Q: Two regions, one auth system. What has to be shared?

For HS256, the secret, which is a distribution problem you do not want across regions. For RS256, only the public key set, which is cacheable, public, and can lag safely for the length of the rotation overlap. Clocks matter more than keys: `exp` and `iat` are compared to the verifier’s clock, so allow a small skew (JWT’s `clockSkewSeconds`, typically 30–60 s) and run NTP everywhere. Refresh tokens are stateful, so either pin refresh to the issuing region or replicate the table; the project’s single Postgres would be the first thing to change.

Q: A token was stolen from a user’s browser. What do you do?

Contain, then detect. Contain: revoke the user’s refresh tokens; the access token dies within its lifetime. Detect: refresh-token rotation turns a replayed old token into a signal — two parties presenting the same family means theft, revoke the family. Prevent: keep the refresh token in an `HttpOnly` cookie so injected scripts cannot read it, and keep the access token in memory rather than `localStorage` (Chapter 38). Add device or IP binding only if you accept the false-positive cost for mobile users.

Q: Why not just check the database on every request?

You can, and monoliths do — that is a session. The reason not to in a service architecture is that every service would need the user store or a call to the one that has it, which puts auth-service on the critical path of everything and couples every deploy to it. A signed token lets each hop answer “who is this” locally. The honest answer names the cost — revocation lag — and the price you paid to buy back control: short lifetimes, a stateful refresh token, optionally a deny-list.

Q: What is in your token, and what is deliberately not?

Subject, role, issued-at, expiry, and here an e-mail — identity, not entitlements. Not in it: permissions lists, group memberships, anything that changes more often than the token expires, anything another service owns, and anything secret (the payload is readable). If an interviewer pushes for “put the permissions in so we skip a query”, the senior answer is the arithmetic of Section 28.4: 4 KB times every request, and a stale authorization decision frozen for the token’s lifetime.

CHECK YOURSELF (answers: Appendix A)

1. After the RS256 conversion, `api-gateway.yml` no longer contains `app.jwt.secret`. If the entire config-server were leaked, could the attacker forge a token? Explain which file would have to leak instead.
2. Describe the 90-day rotation ceremony and state precisely which problem kid solves in step 3.
3. Thirty services each verify RS256 tokens with a JWKS fetched at startup. Auth-service dies at 3 a.m. Which users are affected, when, and which single event during the outage would make a verifier reject *all* traffic?
4. A colleague proposes adding a `courseIds` claim so that course-service can authorize without a query. Give two reasons to refuse that are independent of token size.
5. The gateway forwards a request to a public path without touching headers. Explain why this is a latent rather than an active vulnerability today, and name the one-line configuration that removes the latency.
6. An admin disables a user who is logged in. Give the revocation lag under the project’s current design, and name the extra dependency each of the two faster designs introduces.

Deep Dive: Configuration — Config Server, Secrets, and Refresh at Scale

Chapter 2 introduced the config-server as “one more property source in the stack”: a small HTTP service that hands each application its YAML at startup. That description is complete for a laptop and dangerously incomplete for anything else. This chapter takes the project’s configuration path apart — what the server actually serves, who can read it, what happens when it changes, and what happens when it is not there — and then rebuilds it two different ways.

Three questions organize the chapter:

1. **What problem does centralized configuration solve**, and which of its three backends (native, git, platform) solves it best for which system?
2. **What is the blast radius of a configuration file** — who reads it, what precedence rules apply, and what a default value in a git-tracked file really means?
3. **What breaks** when the server is down, when a value must change at runtime, and when a secret must rotate?

29.1 Why centralize, and what it solves

The twelve-factor rule is *strict separation of config from code*: an image is built once and promoted unchanged through environments; only the configuration around it differs. Every configuration mechanism is a way of satisfying that rule, and the config-server is the first one the Spring ecosystem produced. It solves three concrete problems at once:

- **Duplication.** Five services share Kafka addresses, tracing endpoints, logging patterns. One document per service in one place beats five copies that drift.
- **Environment awareness.** The same service asks for `/course-service/prod` or `/course-service/default` and gets the right document, from the same image.
- **Change without rebuild.** With a git backend, a config change is a commit — reviewed, audited, revertible — and takes effect on the next restart or refresh, with no image rebuilt.

The project gets the first benefit fully, the second not at all (there is one profile, default), and the third only in principle: with the `native` backend the documents are inside the config-server's own jar, so changing a value *does* mean a rebuild.

DECISION — THREE BACKENDS, THREE SYSTEMS

Native classpath (this project): config and code in one repo, one pipeline, one version. Right when one team owns everything and every config change ships with a code change anyway. **Git backend**: config in its own repository with its own history; right when operations and development are different people, when the same service runs in many environments, or when an auditor asks “who changed the timeout and when”. **Platform-native** (ConfigMap and Secret, no server): right when every deployment target is Kubernetes, because the platform already offers versioned, mountable, RBAC-protected configuration and one fewer JVM to run. The project sits on the first and is heading for the third (Section 29.6); the second is what most enterprises you interview at actually run.

29.2 Reading the real path

Follow one property, `spring.datasource.url` for `course-service`, from the file to the running pod. It starts in the config-server's jar:

```
# config-server/src/main/resources/configurations/course-service.yml
server:
  port: 8082
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_course ①
    username: postgres
    password: root ②
  app:
    s3:
      endpoint: ${APP_S3_ENDPOINT:http://localhost:9000} ③
      secret-key: ${APP_S3_SECRET_KEY:minioadmin}
```

The service's packaged `application.yml` says where to fetch that document from:

```
# course-service/src/main/resources/application.yml
spring:
  application:
    name: course-service ④
  config:
```

```
import: optional:configserver:http://localhost:8888 ⑤
```

And the container overrides both the address and the value:

```
# deploy/k8s/base/course-service.yaml (env excerpt)
- { name: SPRING_CONFIG_IMPORT,
  value: "configserver:http://config-server:8888" } ⑥
- { name: SPRING_DATASOURCE_URL,
  value: "jdbc:postgresql://postgres-course:5432/ts_course" }
envFrom:
- secretRef: { name: postgres-course-app } ⑦
```

- ① A literal, not a placeholder: the served document says `localhost`. It is correct only on a laptop.
- ② A database password in cleartext, in a git-tracked file, served over HTTP. Dev-only in intent; nothing in the file says so.
- ③ The `$VAR:default` idiom: the config-server serves the placeholder *unresolved*; each client resolves it against its own environment at startup. This is why the same document works in Compose and in Kubernetes — and why a default in it is a value every environment will silently use unless told otherwise.
- ④ The lookup key. The server maps `course-service` to `course-service.yml`.
- ⑤ `optional:` — if nothing answers on 8888, boot anyway from the packaged file. Right for a developer running one service in an IDE; wrong for a container.
- ⑥ The container replaces the whole import line: new host, and *no* `optional:`. An unreachable config-server is now fatal at startup. Compose masks this with `depends_on:` `condition: service_healthy`; Kubernetes does not, and the pod crash-loops until the server is Ready (Chapter 15).
- ⑦ The password from ② is overridden by the Secret's `SPRING_DATASOURCE_PASSWORD`. The cleartext default is therefore harmless in the cluster — *as long as* the Secret exists. Remove it and the service does not fail; it connects with `root`.

29.2.1 Precedence, precisely

Spring Boot builds one `Environment` from an ordered list of property sources; the first source that defines a key wins. For this project the order that matters, highest first:

1. Command-line arguments and `SPRING_APPLICATION_JSON` (unused here).

2. OS environment variables — the Deployment’s `env` and `envFrom`, with relaxed binding turning `SPRING_DATASOURCE_URL` into `spring.datasource.url`.
3. The document imported from the config-server.
4. The packaged `application.yml` that did the importing.

Two consequences are easy to get wrong in an interview. First, an imported document outranks the file that imports it: the config-server can override anything packaged in the jar, but never anything set by the platform. Second, a `$VAR:default` placeholder is *not* a precedence rule; it is resolved after precedence has already chosen the document. The env var `APP_S3_SECRET_KEY` wins over the default not because env vars outrank the config-server, but because the config-server’s own text asked for it.

HANDS-ON — READ WHAT THE SERVER SERVES

The config-server has no authentication and Compose publishes its port. Ask it for course-service’s document exactly as the client does:

```
$ curl -s http://localhost:8888/course-service/default | \
  python -m json.tool | head -20
{
  "name": "course-service",
  "profiles": ["default"],
  "label": null,
  "version": null,
  "propertySources": [
    {
      "name": "classpath:/configurations/course-service.yml",
      "source": {
        "server.port": 8082,
        "spring.datasource.url": "jdbc:postgresql://localhost...",
        "spring.datasource.username": "postgres",
        "spring.datasource.password": "root",
```

Note version: null — the native backend has no commit id to report. A git backend fills it with the SHA, which is how you answer “which config is this pod running?” in production.

29.3 Pros and cons, honestly

Strength	In this project
One document per service	Real: five files, zero duplication of Kafka or Zipkin addresses.
Environment placeholders	Real: the same files serve Compose and Kubernetes untouched.
Startup-time fetch, no runtime dependency	Real: once up, a service never talks to the server again.
Weakness	
No authentication	Anyone reaching 8888 reads every document, defaults and passwords included.
Native = rebuild to change	A timeout tweak is a config-server image build and rollout.
Working defaults for secrets	JWT_SECRET and root have usable fallbacks in git.
Ordering constraint	Every pod depends on one pod that has nothing to do after boot.
One profile only	default everywhere; environments differ by env var alone.

The last row is worth a sentence. Spring profiles were designed for exactly the dev/staging/prod split, and the config-server serves `course-service-prod.yml` on top of `course-service.yml` automatically when `SPRING_PROFILES_ACTIVE=prod`. The project never uses this; every environmental difference is an env var. That is a defensible simplification — one mechanism instead of two — but it means the served documents can never say “this value is for development only”.

29.4 What breaks

PITFALL — THE SIGNING KEY IS IN THE REPOSITORY

Symptom: none, which is the point. Open the served `api-gateway.yml` and `auth-service.yml`:

```
app:
  jwt:
    secret: ${JWT_SECRET:mySecretKeyThatIsAtLeast256Bits...}
```

Cause: the default is a *valid* HS256 key, committed to a public repository, and `create-secrets.sh` defaults `JWT_SECRET` to the same string. Any deployment that forgets to set the variable signs tokens with a key the whole internet can read — and with HS256 (Chapter 28) reading the key means minting tokens for any user. The fix has two halves: remove the default so a missing variable fails at startup (`$JWT_SECRET` with no colon — Spring refuses to resolve it), and treat every `$VAR:value` in a served document as a question: “would I be fine if production ran with this?” Passwords and keys never pass that test; ports and hostnames usually do.

PITFALL — REFRESHED ONE SERVICE, BROKE ALL OF THEM

Symptom: after “rotating the JWT secret” with `POST /actuator/refresh` on `auth-service`, every request through the gateway returns 401 until the gateway is restarted. Cause: refresh is *per instance*. `Auth-service` rebuilt its `@RefreshScope` beans and now signs with the new key; the gateway still verifies with the old one. Even if both are refreshed, there is no overlap window — every token issued before the refresh is invalid the moment the gateway refreshes. A symmetric key shared by a signer and a verifier cannot be hot-swapped at all; it needs the kid-based ceremony of Chapter 28. The general rule: refresh is for values whose old and new readings can coexist (a feature flag, a log level, a timeout), not for values that two parties must agree on.

PITFALL — HEALTHY BUT WRONG

Symptom: `config-server` reports UP, services start, the wrong values are in effect. Cause: the `config-server`’s health indicator checks that the *backend* answers — for native, that the classpath directory exists. It says nothing about content. A typo in a YAML key name (`acess-key`) is served happily and ignored happily by the client, which then uses its default. Neither side logs an error. The defence is in the client: bind configuration to a `@ConfigurationProperties` record with `@Validated` and `@NotBlank` on required fields, so a missing or misspelled key fails at startup with the property name in the message.

29.5 What breaks at 3 a.m.

The `config-server` pod is evicted at 3 a.m.; nothing else is.

- **Running services: nothing.** Each fetched its document once at boot and holds it in memory. No request path touches the `config-server`. This is the

property that justifies a single replica.

- **A service that restarts during the outage: crash loop.** Its import is non-optional; the fetch fails; the JVM exits; kubelet restarts it with exponential back-off (Chapter 24). When the config-server returns, the next restart succeeds — after up to five minutes of back-off delay. The system converges; it does not converge quickly.
- **A rolling update during the outage: stalls.** The new pod never becomes Ready, the old pod keeps serving (Chapter 15's `maxUnavailable`), and the roll-out waits. Correct behaviour, but a deploy pipeline will report a timeout.
- **With optional: restored: worse.** The restarting pod boots from packaged defaults — `localhost:5432`, `localhost:9092` — becomes Ready (its probe is `/v3/api-docs`, which needs no database), receives traffic, and fails every request. An outage that looked like “config-server down” is now “course-service returning 500s”, two hops from the cause.

Retry with back-off is the middle ground: keep the import mandatory, but let the client try for a while before dying. It needs `spring-retry` and `spring-boot-starter-aop` on the classpath, then:

```
spring:
  cloud:
    config:
      fail-fast: true           # unreachable server is fatal ...
      retry:
        max-attempts: 20       # ... after 20 tries
        initial-interval: 2000 # 2 s, then x1.5 each time
        multiplier: 1.5
        max-interval: 15000    # capped at 15 s: ~4 minutes total
```

Compare with letting kubelet do the retrying: the JVM restart costs twenty seconds of class loading each time and the back-off grows to five minutes; in-process retry costs nothing and caps at fifteen seconds. Both converge; one wastes a CPU the two-core box does not have.

29.6 Enhancing the resolution

Two upgrades, in the order an enterprise would do them.

29.6.1 Upgrade 1: git backend with encrypted values

Keep the config-server, move its documents to a repository, and stop committing secrets in cleartext. The server side:

```
# config-server/src/main/resources/application.yml
server:
  port: 8888
spring:
  application:
    name: config-server
  profiles:
    active: git ①
  cloud:
    config:
      server:
        git:
          uri: https://git.example.com/ts/config-repo.git
          default-label: main ②
          search-paths: '{application}' ③
          clone-on-start: true ④
          username: ${CONFIG_GIT_USER}
          password: ${CONFIG_GIT_TOKEN}
  encrypt:
    key: ${CONFIG_ENCRYPT_KEY} ⑤
  management:
    endpoints:
      web:
        exposure:
          include: health
```

- ① The profile selects the backend. `native` and `git` are the two built in; `Vault` and `JDBC` exist as well.
- ② The label is a branch or tag. Pin a production environment to `release-2026.09` and a config change is a merge, not an edit.
- ③ One folder per service in the repository: `course-service/application.yml`. The same file names as today, one directory deeper.
- ④ Clone at startup rather than on first request, so the health check proves the credentials work before any client asks.
- ⑤ A symmetric key for the `/encrypt` and `/decrypt` endpoints. It is the one secret the config-server must hold, and it must come from the platform (a Kubernetes Secret), never from a file in either repository.

With the key in place, encrypting a value is one request, and the repository stores only the ciphertext:

```
$ curl -s -X POST http://localhost:8888/encrypt \
  -d 'a-real-256-bit-signing-key-generated-with-openssl'
AQBt3s6...b3f0e # ciphertext; safe to commit
```

```
# config-repo/api-gateway/application.yml
app:
  jwt:
    secret: '{cipher}AQbt3s6...b3f0e' # decrypted by the server on serve
```

The {cipher} prefix tells the server to decrypt before serving. The repository now holds no usable secret, the default-value trap is gone (there is no default), and git log – api-gateway/ answers who changed the key and when. What it does *not* fix: the served document still crosses the network in cleartext to an unauthenticated endpoint. Add spring-boot-starter-security to the config-server with HTTP basic credentials, and give each client spring.cloud.config.username/password from its own Secret.

29.6.2 Upgrade 2: retire the server

On Kubernetes the platform can serve the documents itself. Kustomize generates a ConfigMap per service straight from the existing files, so the git history is unchanged:

```
# deploy/k8s/base/kustomization.yaml (excerpt)
configMapGenerator:
- name: course-service-config
  files:
    # key in the ConfigMap = application.yml, value = the file
    - application.yml=../..//config-server/src/main/resources/configurations/course-service.
      ↪ yml
generatorOptions:
  disableNameSuffixHash: false # hash suffix: a changed file = a
                                # new ConfigMap name = a rollout
```

The Deployment mounts it as a directory and points Spring at the directory instead of the server:

```
# deploy/k8s/base/course-service.yaml (excerpt)
spec:
  template:
    spec:
      containers:
        - name: course-service
          env:
            # replaces configserver:http://config-server:8888
            - { name: SPRING_CONFIG_IMPORT, value: "file:/config/" }
            - { name: SPRING_DATASOURCE_URL,
              value: "jdbc:postgresql://postgres-course:5432/ts_course" }
          envFrom:
```

```
- secretRef: { name: postgres-course-app }
volumeMounts:
- { name: config, mountPath: /config, readOnly: true }
volumes:
- name: config
  configMap: { name: course-service-config }
```

Spring Boot treats a trailing slash as a directory import and loads `application.yml` from it, with exactly the precedence the `config-server` document had: above the packaged file, below `env` vars. Nothing in the service’s code or jar changes; Compose keeps working because the packaged `optional:configserver: import` still points at the (still running, in Compose) server. Then delete `config-server.yaml` from the base, and the cluster has one fewer JVM, one fewer startup dependency, and one fewer unauthenticated endpoint.

The name-suffix hash is the detail that makes this better than the server, not merely equivalent. A `ConfigMap` named `course-service-config-7b9c4` changes name when its content changes; the `Deployment` referencing the new name is a new pod template; Kubernetes rolls the pods. Config change, rollout, and rollback are the same operation as a code change — which is what Chapter 27’s `GitOps` section wants.

DECISION — WHEN TO DROP THE CONFIG-SERVER ENTIRELY

Drop it when three things are true: every runtime is Kubernetes, no service needs a value to change without a pod restart, and secrets already live in the platform (they do — Chapter 16). Keep it when any is false: services on VMs or in Compose in production, a genuine runtime-refresh requirement (rare; feature flags are better served by a purpose-built system), or an organization whose configuration audit trail must be independent of the deployment repository. An interviewer who asks “would you use Spring Cloud Config on Kubernetes?” is listening for this list, and for the sentence “the platform already provides it”.

29.7 Outside the project

The git-backed `config-server` with `{cipher}` values was the standard from about 2015 to 2020 and is still what you will find in most Java shops. Two things have replaced parts of it. HashiCorp Vault (and its cloud equivalents, AWS Secrets Manager, Azure Key Vault) holds secrets with leases and rotation; Spring Cloud Vault or the External Secrets Operator delivers them into pods, and the config repository holds only references. Feature-flag services (LaunchDarkly, Unleash,

OpenFeature) took over the runtime-refresh use case with targeting rules and audit that `@RefreshScope` never had. What survived unchanged is the discipline: config outside the image, precedence you can state from memory, and no working secret in any file you commit.

29.8 Questions from real interviews

Q: You have to rotate a database password across fifty services with no downtime. Walk me through it.

Rotation is never a single write; it is an overlap. First, create the new credential in the database while the old one still works — Postgres lets two roles or two passwords coexist, and managed services (RDS, Cloud SQL) expose “dual password” for exactly this. Second, put the new value into the platform Secret (or the config repository as a {cipher} value) and trigger the consumers: on Kubernetes that is a rollout restart per Deployment, which the ConfigMap name-hash trick does for you; with Spring Cloud Bus it is one POST `/actuator/busrefresh` that every instance receives over Kafka. Third, watch the database’s connection log until no session uses the old role, then revoke it. The project’s pitfall in Chapter 16 — “changed the Secret, nothing happened” — is what step two prevents. The senior answer names the overlap explicitly and says which step is allowed to fail: revocation, which you simply postpone.

Q: A config change must reach every instance within a minute. Restarting is too slow. What do you do?

Distinguish the value kinds first. A log level, a rate limit, a feature flag: mark the bean `@RefreshScope`, expose `/actuator/busrefresh`, add `spring-cloud-starter-bus-kafka` — the project already has Kafka — and one POST broadcasts a `RefreshRemoteApplicationEvent`; each instance re-fetches its document and rebuilds refresh-scoped beans within seconds. A database URL, a Kafka bootstrap address, a signing key: do *not* refresh those, because the connection pool, the producer, or the verifier holds the old value in objects that are not rebuilt cleanly; restart instead. If the ask is really “flags for product experiments”, say that a feature-flag service (Unleash, LaunchDarkly) with an SDK that polls every few seconds is the purpose-built tool, and `@RefreshScope` is the ten-year-old workaround.

Q: The config server is a single point of failure. How do you handle that?

Start by narrowing what it is a SPOF *for*: startup, not traffic. A running service never calls it again, so a config-server outage affects only restarts and scale-outs. Then reduce even that: two replicas behind the Service (it is stateless with a native or git backend — a git clone per replica is fine), fail-fast with retry so a booting pod waits up to a few minutes instead of dying, and a readiness probe on the config-server so the Service never routes to a replica still cloning. The strongest answer removes the dependency: serve the documents from ConfigMaps, and the “SPOF” becomes the Kubernetes API server, which you already depend on for everything else.

Q: How do you keep secrets out of git while still having config in git?

Three layers, each acceptable at a different size. Smallest: the config-server’s /encrypt with a symmetric key held only by the server; git stores {cipher} text, and a leaked repository is useless without the key. Middle: Kubernetes Secrets created out of band (the project’s `create-secrets.sh`) and referenced by name in the manifests, so git holds the shape but never the value. Largest: a secrets manager — Vault, AWS Secrets Manager — with the External Secrets Operator synchronizing into Kubernetes Secrets, so rotation and audit live where the secret does. The trap to name: a `$VAR:default` with a working default is a secret in git no matter which layer you chose.

Q: Two pods of the same service are running with different configuration. How did that happen, and how do you prevent it?

The usual causes: a rolling update that stalled halfway (old pod on the old document, new pod on the new one — normal and temporary); a config-server whose git clone is stale on one replica because clone-on-start was off and one replica never refreshed; or, the subtle one, a /actuator/refresh sent to one instance through a load balancer that happened to pick it. Prevention is to make configuration part of the pod template: mount it from a ConfigMap whose name carries a content hash, so “which config” is answered by “which ReplicaSet”. Detection is to expose the config version — the git SHA the config-server returns as `version`, or the ConfigMap name — in /actuator/info, and to compare across pods before believing any “it works on one of them” report.

CHECK YOURSELF (answers: Appendix A)

1. The served `course-service.yml` says `password: root` and the Deployment's Secret says otherwise. State which wins and why, then describe what happens if the Secret is deleted and the pod restarts.
2. Explain why `$APP_S3_SECRET_KEY:minioadmin` in a served document is not a precedence rule, and what the config-server actually sends to the client for that key.
3. A colleague runs `POST /actuator/refresh` on both `auth-service` and `api-gateway` after changing `JWT_SECRET`. List the two distinct failure windows and name the mechanism from Chapter 28 that a hot rotation actually requires.
4. The config-server dies; course-service is scaled from one to two replicas during the outage. Describe what the new pod does with `configserver:` (non-optional), with `optional:configserver:`, and with `fail-fast plus retry`.
5. After the ConfigMap migration, what causes pods to restart when a value changes, and why does that make the migration *more* capable than the server it replaced rather than merely equivalent?
6. Interview: "Would you deploy Spring Cloud Config on Kubernetes?" Give the three conditions under which you would remove it and the one condition under which you would keep it.

Deep Dive: Service Discovery — Eureka Internals, Failure Modes, and the Kubernetes Alternative

Chapter 3 explained Eureka in four bullet points — register, heartbeat, cache, self-preserve — and Chapter 27 sketched its retirement in two lines. Between those two lies everything an operator actually meets: the timers that decide how long a dead instance keeps receiving traffic, the threshold that switches the registry into a mode where it stops telling the truth, and the awkward fact that on Kubernetes the project runs two discovery systems at once and only one of them is consulted by the gateway. This chapter measures those things, breaks them, and then shows the exact configuration that removes Eureka without breaking Compose.

Three questions organize the chapter:

1. **What does a registry solve that DNS does not**, and why did the classic stack put the lookup on the *client*?
2. **How stale can the registry be** — in seconds, with the arithmetic — and what does staleness do during a deploy?
3. **What replaces each Eureka feature** when the platform is Kubernetes, and how do you migrate without a flag day?

30.1 Why a registry, and why on the client

The problem is addressing. Instances have IPs that change with every restart; callers need a stable name. Three families of answer exist:

- **Server-side load balancer.** Callers dial one fixed address (an nginx, an AWS ELB); it knows the instances. Simple for callers, but every hop adds latency and the balancer must itself be kept current.
- **Client-side registry** (Eureka, Consul). Each instance registers; each caller downloads the list and balances locally. No extra hop, no balancer to scale, callers survive a registry outage on their cache — and every caller must embed a client library.
- **Platform DNS** (Kubernetes Services). The platform watches pod readiness and keeps a virtual IP's endpoint list current; callers use plain DNS and

know nothing. No library, no registry process, but only inside that platform.

Netflix built the second in 2012 because it ran on EC2 with no platform DNS to lean on and thousands of JVM callers that could carry a library. Spring Cloud inherited it, and this project inherits it from Spring Cloud. That history is the honest answer to “why Eureka”: not because it is the best fit for a Kubernetes cluster in 2026, but because the classic stack is still what most Java shops run and what most Java interviews ask about.

DECISION — CLIENT-SIDE, THEN PLATFORM-SIDE

The project keeps Eureka through Compose and the first Kubernetes milestones for two reasons: Compose has no readiness-gated DNS, so a registry is the only way for the gateway to skip a service that is up but not ready; and running both systems side by side is how you learn what the platform replaces. The moment Compose stops being a deployment target — it is already only a dev tool — the registry is pure cost: one more JVM, one more startup dependency, and a second opinion about which pods are alive that disagrees with the kubelet’s for up to four minutes. Section 30.6 makes the switch.

30.2 Reading the real configuration

The server is a port and an annotation. All the behaviour lives in defaults the project never sets, so read the client side first:

```
# course-service/src/main/resources/application.yml
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/ ①
```

```
# deploy/k8s/base/course-service.yaml (env excerpt)
- { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE,
  value: "http://discovery-server:8761/eureka/" } ②
- { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" } ③
```

And the consumer, the gateway’s routes:

```
# configurations/api-gateway.yaml (excerpt)
- id: course-service-courses
  uri: lb://course-service ④
  predicates:
```

```
- Path=/api/courses/**
filters:
- StripPrefix=2
```

- ① The only address a client is told. Everything else — where `course-service` is, where `auth-service` is — comes back from this URL as a registry document.
- ② On Kubernetes `discovery-server` is resolved by *platform DNS* to a ClusterIP. The registry itself is found by the mechanism it is about to duplicate.
- ③ Register the pod IP, not the pod name (Chapter 3). The registered address is what the gateway will dial, bypassing the `course-service` Service entirely.
- ④ `lb://` hands the name to Spring Cloud LoadBalancer, which asks the Eureka client’s local cache for instances and picks one round-robin. The Kubernetes Service `course-service:8082` is never consulted on this path.

Point ④ is the sentence to remember from this chapter. On the cluster, *readiness probes protect the Kubernetes Service, and the gateway does not use the Kubernetes Service*. The only thing between the gateway and a pod that is not ready is the Eureka status — which the pod reports about itself.

30.2.1 The timers

Everything below is a default the project inherits. Each is a property you can set; each has a reason for its value.

Property (eureka.*)	Default	Meaning
<code>instance.lease-renewal-interval-in-seconds</code>	30 s	heartbeat period
<code>instance.lease-expiration-duration-in-seconds</code>	90 s	no heartbeat for this long = expired
<code>server.eviction-interval-timer-in-ms</code>	60 s	how often the server sweeps expired leases
<code>server.response-cache-update-interval-ms</code>	30 s	server’s read-only registry cache refresh
<code>client.registry-fetch-interval-seconds</code>	30 s	client re-downloads the registry
<code>spring.cloud.loadbalancer.cache.ttl</code>	35 s	LoadBalancer’s own instance-list cache
<code>server.renewal-percent-threshold</code>	0.85	self-preservation trigger

Now the arithmetic an interviewer wants.

A new instance receiving traffic. Spring Cloud registers as soon as the web server starts (the raw Netflix client would wait 40 s; Spring's `EurekaAutoServiceRegistration` forces an on-demand registration). The server's read-only cache may be up to 30 s stale, the gateway's registry fetch up to 30 s behind that, and its Load-Balancer cache up to 35 s behind that. Worst case: about **95 seconds** between "Tomcat is listening" and "the gateway knows". Typical: half that.

A dead instance no longer receiving traffic. The pod is killed without a graceful shutdown (OOM, node loss). The lease expires after 90 s without a heartbeat; the eviction sweep runs every 60 s, so up to 150 s after death the server still lists it; add the same 30 + 30 + 35 s of caches: up to **245 seconds** — four minutes — during which the gateway keeps dialing an IP that no longer answers. A graceful shutdown sends a DELETE on the way out and skips the first 150 s, leaving up to 95 s of stale caches.

Self-preservation. Every minute the server compares heartbeats received against heartbeats expected: $\text{instances} \times 2$ (one per 30 s) $\times 0.85$. Below the line, it stops evicting anything and the dashboard shows the red banner. The project registers five clients: expected $5 \times 2 = 10$, threshold $\lceil 10 \times 0.85 \rceil = 8$, and the condition is "more than 8". Kill *one* service and the count drops to 8 — not more than 8 — and the registry enters self-preservation. On this stack, a single crashed service means *no* instance is ever evicted again until it comes back. Netflix tuned that threshold for hundreds of instances, where losing 15 % in a minute really does mean a network partition; on five, it means Tuesday.

HANDS-ON — WATCH A DEAD INSTANCE STAY ALIVE

In Compose, stop `course-service` without letting it deregister, then ask the registry about it a minute later:

```
$ docker compose kill course-service
$ sleep 60
$ curl -s -H 'Accept: application/json' \
  http://localhost:8761/eureka/apps/COURSE-SERVICE | \
  python -c "import sys,json; a=json.load(sys.stdin)['application']; \
    print([i['status'] for i in a['instance']])"
['UP']
```

Now open `http://localhost:8761`. After the next sweep, the instance is either gone — or, because four remaining clients send 8 renewals a minute, the page carries the banner **EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT**, and `course-service` stays UP for as long as you care to wait. Meanwhile `curl localhost:8080/api/courses` returns a 503 from the gateway on every attempt that picks the dead address.

30.3 Pros and cons, honestly

- **For.** No extra network hop; callers keep working through a registry outage on their cache; one mechanism that is identical on a laptop, in Compose, and in the cluster; a dashboard that shows who is registered; and an instance can mark itself `OUT_OF_SERVICE` for a controlled drain.
- **Against, in this project.** A second source of truth about liveness that lags the kubelet's by up to four minutes; readiness probes bypassed on the gateway path (④ above); self-preservation tuned for a fleet a hundred times larger; a 512 Mi JVM that does nothing for traffic; `prefer-ip-address` as a platform-specific workaround every manifest must carry; and every service compiled against a client library whose upstream is in maintenance mode.
- **Neutral.** Eureka is AP by design: it prefers answering with stale data to not answering. That is the right choice for a registry — a caller with a slightly wrong list can retry; a caller with no list can do nothing — and it is also why every number above is “up to”.

30.4 What breaks

PITFALL — ROLLING DEPLOY, THIRTY SECONDS OF 503s

Symptom: `kubectl rollout restart deployment/course-service` succeeds, readiness passes, and for half a minute the gateway returns 503 Service Unavailable on `/api/courses`. Cause: the old pod deregistered on `SIGTERM` and exited; the new pod registered with a new IP; but the gateway's LoadBalancer still holds the old IP for up to 95 s of cache. The Kubernetes Service switched instantly and nobody asked it. Fix, in ascending cost: a `preStop` hook that sleeps 20–30 s so the old pod keeps answering while caches drain (`terminationGracePeriodSeconds` must exceed it); shorter caches (`registry-fetch-interval-seconds: 5`, `spring.cloud.loadbalancer.cache.ttl: 5s`) at the price of registry load; or a gateway `Retry` filter on connection errors so the second attempt lands on the live IP. Or remove the registry from the path (Section 30.6) and let the Service do what it is for.

PITFALL — REGISTERED, UP, AND RETURNING ERRORS

Symptom: the dashboard shows `course-service` UP; requests fail with database errors. Cause: Eureka status is self-reported, and by default it is UP

the moment registration succeeds — it does not consult the actuator health. Flyway may have failed after registration, Postgres may have gone away, and the instance still says UP every 30 s. Fix: `eureka.client.healthcheck.enabled: true` propagates actuator health to the registry status, so a DOWN health check becomes DOWN in Eureka on the next heartbeat — 30 s later, plus caches. Kubernetes readiness would have pulled the pod in 5 s; on the gateway path, it did not get the chance.

PITFALL — TWO EUREKA REPLICAS, TWO ANSWERS

Symptom: after scaling discovery-server to two replicas for availability, some callers see an instance the others do not. Cause: peers replicate registrations to each other asynchronously; a client whose `defaultZone` lists both addresses talks to the first that answers and may read a replica that has not yet received the replication. Each server also counts renewals independently, so one may enter self-preservation while the other evicts. Fix: list both peers in every client's `defaultZone` and in each server's `defaultZone` (they must register with *each other*), accept seconds of inconsistency as the price of AP, and never scale a registry you are about to remove.

30.5 What breaks at 3 a.m.

The discovery-server pod dies at 3 a.m.; nothing else does.

- **Gateway traffic: unaffected.** The gateway's Eureka client keeps its last registry in memory and the LoadBalancer keeps routing to it. The registry fetch fails every 30 s and logs a warning; the cache does not expire. This is the AP promise kept.
- **Heartbeats: fail silently.** Every client logs a renewal failure each 30 s. Nothing else changes; leases are a server concept and the server is gone.
- **A service restarted during the outage: invisible.** It boots, registration fails, it retries in the background — and it serves nothing through the gateway until the registry returns and the 95 s of caches drain. Its Kubernetes Service has it as an endpoint within seconds; nobody uses that Service.
- **A service that died during the outage: still routed to.** No server, no eviction. The gateway keeps the dead IP until the registry is back *and* has evicted it *and* the caches have turned over: the full 245 s, starting from when the server returns.

- **When discovery-server returns: a cold registry.** It starts empty. For its first wait-time-in-ms-when-sync-empty (5 minutes by default) it refuses to serve a registry it considers incomplete, returning what it has; clients re-register on their next heartbeat within 30 s. Full recovery: about a minute after the pod is Ready.

Compare with the platform path: a Service's endpoint list is maintained by the control plane, which is replicated and which every other component already depends on. There is no separate 3 a.m. story for Kubernetes DNS because it has no separate process to lose.

30.6 Enhancing the resolution

The goal: on Kubernetes, route by Service DNS and let readiness probes gate traffic; in Compose, keep Eureka so the developer experience is unchanged. A Spring profile splits the two, and the config-server serves the profile document on top of the base one.

30.6.1 Step 1: a profile document for the gateway

The base api-gateway.yml keeps its lb:// routes. A new api-gateway-k8s.yml beside it replaces them — the whole list, because YAML lists override by index and a partial override is a trap:

```
# configurations/api-gateway-k8s.yml -- served when profile k8s is active
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service
          uri: http://auth-service:8081
          predicates:
            - Path=/api/auth/**
          filters:
            - StripPrefix=2
        - id: auth-oauth2
          uri: http://auth-service:8081
          predicates:
            - Path=/oauth2/**
        - id: auth-oauth2-callback
          uri: http://auth-service:8081
          predicates:
            - Path=/login/oauth2/**
        - id: course-service
```

①


```

uri: http://course-service:8082
predicates:
  - Path=/api/courses/**,/api/students/**,/api/assignments/**,/api/enrollments
  ↪ /**,/api/submissions/**
filters:
  - StripPrefix=2
- id: dictionary-service
  uri: http://dictionary-service:8083
  predicates:
    - Path=/api/dictionary/**
  filters:
    - StripPrefix=2
httpClient:
  connect-timeout: 2000           ②
  response-timeout: 10s
eureka:
  client:
    enabled: false                ③

```

- ① A Service name and port. CoreDNS resolves it to the ClusterIP; kube-proxy balances across Ready endpoints. Readiness probes are now on the gateway's path.
- ② With no registry to consult, the gateway's only defence against a dead endpoint is a fast connect timeout; kube-proxy will have removed the endpoint within seconds anyway.
- ③ The Eureka client stays on the classpath but does nothing: no registration, no fetch, no heartbeat warnings. This is what lets one image serve both environments.

30.6.2 Step 2: Feign without the registry

course-service's `AuthServiceClient` resolves auth-service through the LoadBalancer. Give it an optional fixed URL:

```

// course-service/.../client/AuthServiceClient.java
@FeignClient(
    name = "auth-service",
    url = "${app.clients.auth-service:}", // empty -> use lb://
    fallback = AuthServiceFallback.class)
public interface AuthServiceClient {
    @GetMapping("/users/{id}")
    UserDto getUserById(@PathVariable Long id);
}

```

When url resolves to an empty string Feign falls back to load-balanced res-

olution by name; when it resolves to `http://auth-service:8081` the LoadBalancer is bypassed. The profile document for `course-service` sets it:

```
# configurations/course-service-k8s.yml
app:
  clients:
    auth-service: http://auth-service:8081
eureka:
  client:
    enabled: false
```

30.6.3 Step 3: the manifests

The diff against `deploy/k8s/base/api-gateway.yaml`; every service manifest changes the same way:

```
env:
+ - { name: SPRING_PROFILES_ACTIVE, value: "k8s" }
- { name: SPRING_CONFIG_IMPORT,
  value: "configserver:http://config-server:8888" }
- - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE,
-   value: "http://discovery-server:8761/eureka/" }
- - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }
```

And in `kustomization.yaml`:

```
resources:
- config-server.yaml
- discovery-server.yaml
- api-gateway.yaml
```

Compose is untouched: no profile, the base documents, `lb://`, Eureka. The same images run in both. When Compose is finally retired, delete `spring-cloud-starter-netflix-eureka-client` from every `pom.xml`, the `discovery-server` module, and the base documents' `lb://` routes — in that order, each a separate commit the pipeline can verify.

HANDS-ON — PROVE THE PLATFORM PATH

After applying the profile, confirm the gateway no longer dials pod IPs directly — it dials the Service, whose endpoints follow readiness:

```
$ kubectl -n ts get endpoints course-service
NAME           ENDPOINTS          AGE
course-service  10.42.0.57:8082    3d
```

```
$ kubectl -n ts rollout restart deployment/course-service
$ while true; do curl -s -o /dev/null -w '%{http_code}\n' \
  -H "Authorization: Bearer $TOKEN" \
  http://ts.local/api/courses; sleep 1; done
200
200
200 # no 503 window: the endpoint switched with readiness
```

Run the same loop before the change and count the 503s.

30.7 Outside the project

Consul is the registry you meet outside the JVM: an agent per host runs health checks itself instead of trusting self-reported heartbeats, and serves the result over DNS as well as HTTP, so callers need no library. Service meshes (Istio, Linkerd) go one step further and put a sidecar proxy next to every pod that does discovery, retries, timeouts, and mTLS, leaving the application with `http://course-service`. In Spring, `spring-cloud-kubernetes` lets `lb://` names resolve against the Kubernetes API instead of Eureka — useful during a migration, unnecessary at its end. And AWS Cloud Map, the managed descendant of Eureka’s idea, exists mostly for ECS, where there is no kube-proxy to make it redundant.

30.8 Questions from real interviews

Q: Your gateway keeps sending traffic to an instance that died ninety seconds ago. Why, and how do you cut that down?

Because nothing on the path noticed. Eureka learns of an ungraceful death only by a missing heartbeat: 90 s lease expiry, then a sweep that runs every 60 s, then a 30 s server cache, a 30 s client fetch, and a 35 s LoadBalancer cache — up to four minutes end to end, and if the registry has slipped into self-preservation, forever. First cut the tail: shorten `registry-fetch-interval-seconds` and the LoadBalancer TTL to 5 s, which costs registry traffic you can afford below a few hundred instances. Then cut the head: make death graceful (SIGTERM handling, `terminationGracePeriodSeconds` long enough) so the DELETE goes out and the 150 s of lease logic never runs. Then make the caller resilient regardless: a Retry filter on connect failures, so a stale address costs one retry, not an error. The senior answer ends with “or stop routing by registry on Kubernetes, because kube-proxy removes the endpoint within seconds of the readiness probe failing”.

Q: How does a new instance start receiving traffic during a rolling deploy with zero errors?

Two conditions and an overlap. The new instance must not receive traffic before it can serve: on Kubernetes that is the readiness probe, on Eureka it is registering only after the context is fully up — Spring does this, but a health check that passes before Flyway finishes would not, so probe something that depends on the database. The old instance must keep serving until every caller has stopped sending: on Kubernetes the endpoint is removed at SIGTERM but in-flight requests finish inside the grace period, and a `preStop` sleep covers callers with stale caches. The overlap is `maxSurge: 1, maxUnavailable: 0`: bring the new one up, wait for Ready, then take the old one down. Errors during a rollout are almost always one of three things — a probe that lies, a cache that outlives the pod, or a grace period shorter than the slowest request — and you name which one from the shape of the error.

Q: Eureka versus Kubernetes DNS. One minute.

Both map a name to a set of addresses; they differ in who maintains the set and who does the lookup. Eureka: the instances register themselves, report their own health, and every caller carries a client that downloads and caches the list — no platform needed, works on VMs, AP with minutes of staleness, and every caller is a JVM with a library. Kubernetes: the control plane maintains a Service's endpoints from label selectors and readiness probes it runs itself, kube-proxy balances, callers use DNS and know nothing — seconds of staleness, no library, no registry process, but only inside the cluster. On Kubernetes, Eureka is a second, slower opinion about liveness. Keep it only if you must run the same code somewhere the platform is not.

Q: You run two Eureka replicas and they disagree about whether an instance is alive. What now?

Expect it; design for it. Peer replication is asynchronous and each replica evicts and self-preserved on its own counters, so a window of disagreement is inherent — Eureka is AP and the disagreement is the cost of answering during a partition. What matters is that clients tolerate it: every client lists both replicas, retries the second on failure, and treats the registry as a hint that a connect timeout can override. If the disagreement persists past a few minutes, one replica is in self-preservation and the other is not; check the renewal threshold against the actual instance count, and on a small fleet lower `renewal-percent-threshold` or disable self-preservation outright. The wrong answer is trying to make the registry consistent;

the right one is making callers indifferent to which replica they read.

Q: Two hundred services, each fetching the registry every thirty seconds. The registry is melting. What do you change?

Four hundred fetches a minute of a document listing two hundred applications is the first bottleneck, and Eureka's answer is delta fetches: clients ask for `/apps/delta` after the first full download, which is on by default and only fails when the server's delta queue is disabled. Second, the read-only response cache exists precisely so the server serializes the registry once per 30 s, not once per request — keep it. Third, split by zone: Eureka's `availability-zone` configuration lets clients prefer a local registry and a local instance, so each server carries a fraction of the fetches. Beyond a thousand instances, Netflix's own answer was more Eureka servers behind DNS, and today's answer is that at that size you are on a platform that does discovery for you.

CHECK YOURSELF (answers: Appendix A)

1. Compute the worst-case delay before the gateway routes to a freshly started course-service pod, naming each cache in the chain and its default.
2. An instance is killed by the OOM killer. List the timers that must elapse before the gateway stops dialing it, and state which of them a graceful shutdown would skip.
3. With five registered clients, show why the loss of one puts the registry into self-preservation, and give the one-line configuration that changes that on a small fleet.
4. On Kubernetes, which mechanism gates traffic to a pod on the gateway path today, and why do readiness probes not? What single configuration change puts readiness back on the path?
5. After `eureka.client.enabled: false` under the `k8s` profile, explain how the same course-service image still registers with Eureka in Compose, and what `url = "$app.clients.auth-service:"` does in each environment.
6. Interview: "Why does a registry keep serving a dead instance while a Kubernetes Service does not?" Answer in terms of who observes liveness and how.

Deep Dive: The Gateway Under Load — Reactive Internals, Rate Limits, and the Edge’s Job

Chapter 4 taught the gateway as a route table: predicates, `lb://` URIs, `StripPrefix`, one JWT filter. That is the gateway on a quiet day. This chapter is the gateway on a bad one — a slow course-service, a client in a retry loop, a pod restart at the wrong moment — and it asks what the reactive stack actually buys, what the project’s edge is missing, and where the line runs between an edge’s job and everybody else’s.

Three questions organize the chapter:

1. **Why is the gateway non-blocking**, and what exactly goes wrong if a filter blocks?
2. **What breaks first under load** — and why the answer is “the thing without a timeout”?
3. **What belongs at the edge** — rate limits, circuit breakers, CORS — and what never should?

31.1 Why a gateway, and what it solves

Without the gateway the frontend would hold six addresses, each service would verify JWTs, and CORS, rate limits and tracing headers would be implemented six times or zero times. The gateway collapses that into one address and one place for every *cross-cutting* concern — concerns that are about *every* request rather than about courses or users. The interview form of the question is “what does an API gateway do that a load balancer does not?” and the senior answer is: a load balancer picks a backend; a gateway *understands the request* — it can authenticate, rewrite, rate-limit, and fail over per route, because it runs application code on the path. That power is also the risk: whatever you put on that path, every request pays for.

DECISION — AN APPLICATION GATEWAY BEHIND AN INFRASTRUCTURE ROUTER

The cluster already has Traefik (Chapter 18). Why keep Spring Cloud Gateway behind it? Because the two do different jobs: Traefik owns the host, TLS, and the path fan-out; the gateway owns things only the application knows — which Eureka name serves `/api/courses`, what a valid JWT looks like, which headers to inject. Merging them would mean teaching Traefik the JWT secret and the header contract, which is possible (ForwardAuth middleware) but moves application security into infrastructure config. The decision flips if the platform provides identity itself (a cloud API Gateway with Cognito or IAP): then the Spring gateway is a second hop paid for nothing.

31.2 The reactive stack, read from the filter

The gateway declares `web-application-type: reactive` in its `application.yml`. That one line swaps Tomcat's thread-per-request model for Reactor Netty's event loop, and every line of the JWT filter is written for that model:

```
@Override
public Mono<Void> filter(ServerWebExchange exchange,
                        GatewayFilterChain chain) { ①
    String path = exchange.getRequest().getURI().getPath();
    if (isPublicPath(path)) {
        return chain.filter(exchange); ②
    }
    String authHeader = exchange.getRequest().getHeaders()
        .getFirst(HttpHeaders.AUTHORIZATION);
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        return unauthorizedResponse(exchange, "Missing ..."); ③
    }
    String token = authHeader.substring(7);
    try {
        Claims claims = Jwts.parser().verifyWith(signingKey)
            .build().parseSignedClaims(token).getPayload(); ④
        ServerHttpRequest mutated = exchange.getRequest().mutate()
            .header(JwtConstants.HEADER_USER_ID, claims.getSubject())
            .header(JwtConstants.HEADER_USER_ROLE,
                claims.get(JwtConstants.CLAIM_ROLE, String.class))
            .build();
        return chain.filter(exchange.mutate().request(mutated).build());
    } catch (Exception e) {
        return unauthorizedResponse(exchange, "Invalid or expired token");
    }
}
```

- ① The return type is the whole story. A servlet filter returns `void` *after* the request is done; this returns a `Mono<Void>` — a *description* of work that completes later. The thread that called `filter` is free the moment this method returns.
- ② `chain.filter(exchange)` does not forward the request; it returns the next stage's `Mono`. Nothing has happened yet. The chain is assembled first and subscribed once.
- ③ Even the error path is reactive: `writeWith` returns a `Mono` that completes when Netty has flushed the body.
- ④ The one piece of real work, and it is CPU-bound and microseconds long: an HMAC over a few hundred bytes. This is the kind of work that *may* run on an event-loop thread. A database call is not.

31.2.1 Threads: four, not four hundred

Tomcat answers concurrency with threads: 200 by default, one per in-flight request, each parked while its backend answers. A proxy's requests are almost entirely waiting, so a Tomcat proxy under 1,000 in-flight requests needs 1,000 threads and about a gigabyte of stacks. Reactor Netty runs a small fixed pool of *event-loop* threads, each multiplexing thousands of sockets with non-blocking I/O: a request arrives, the filter chain is assembled, the upstream write is issued, and the thread moves on. When the upstream answers, whichever loop owns that socket resumes the chain.

ON THIS HARDWARE

Reactor Netty sizes the loop as `max(availableProcessors, 4)`. On `ts-server`'s two cores that is **four** event-loop threads for the whole gateway. Four threads serving every request is the reason the gateway's Deployment asks for less memory than an MVC service (384 Mi vs. 512 Mi, Chapter 17) — and the reason one blocked thread is not a 0.5 % problem but a 25 % one.

31.2.2 What blocking does to an event loop

Suppose someone “improves” the filter by checking a deny-list in Postgres:

```
// DO NOT: a JDBC call inside a WebFlux filter
boolean revoked = jdbcTemplate.queryForObject(
    "select exists(select 1 from revoked where jti = ?)",
    Boolean.class, claims.getId());
```

JDBC is blocking by specification: the calling thread waits for the socket. That thread is one of four event loops, and while it waits it is not servicing the hun-

dreds of other sockets it owns. Their requests stall — not fail, *stall* — until the query returns. Under a slow database, all four loops are pinned within seconds and the gateway accepts connections it cannot process: every route, every user, one symptom. The same is true of `.block()` on a `Mono`, `Thread.sleep`, a synchronous `RestTemplate` call, or a file read. The reactive alternative moves the work off the loop and *composes* it into the chain:

```
return Mono.fromCallable(() -> deny.isRevoked(claims.getId()))
    .subscribeOn(Schedulers.boundedElastic()) // off the loop
    .flatMap(revoked -> revoked
        ? unauthorizedResponse(exchange, "Token revoked")
        : chain.filter(exchange.mutate().request(mutated).build()));
```

Better still is a non-blocking client (reactive Redis, R2DBC) so no thread waits at all. The rule for anything on the gateway path: *if the library's method returns a value instead of a `Mono/Flux`, it does not belong in a filter*. Reactor's `BlockHound` agent turns that rule into a test failure.

31.2.3 Predicates, order, and the `StripPrefix` arithmetic

Routes are tried in declaration order (all have the default order of 0, and the sort is stable); the first predicate match wins. The project's table has no overlaps, but a future `Path=/api/**` catch-all placed *above* the specific routes would swallow them silently — put catch-alls last or give them a high order. Filters attached to a route are numbered in sequence and run *after* global filters with a lower order; the JWT filter's `getOrder()` of `-1` places it before every route filter, which is what lets a route's rate limiter (below) read the `X-User-Id` header the JWT filter just wrote.

PITFALL — THE DOUBLED SEGMENT

Symptom: the frontend calls `/api/courses/courses/5` and it works, but a hand-written `/api/courses/5` returns 404 from course-service. Cause: `StripPrefix=2` removes exactly two segments, `/api` and `/courses`, so `/api/courses/5` arrives as `/5` — and no controller is mapped there. The first `courses` selects the *route*; the second survives the strip and matches `@RequestMapping("/courses")`. The frontend repeats the segment on purpose. The cleaner design is `StripPrefix=1` with controllers keeping their resource prefix; the project chose to keep services unaware of `/api` at the cost of the doubled URL.

31.3 CORS at the edge

CorsConfig registers one CorsWebFilter for `/**`: origins `localhost:3000` and `5173`, six methods, any header, credentials allowed. Three things to understand about where it lives.

Why here and not in each service. CORS is a browser rule about *origins*, and the origin the browser talks to is the gateway. A service answering `Access-Control-Allow-Origin` would be answering a question nobody asked it; and if two services disagreed, the browser would see inconsistent policy for one API. Edge concerns go at the edge.

What a preflight costs. For any request that is not “simple” — a JSON POST, anything with an `Authorization` header — the browser first sends an `OPTIONS` request and waits for the answer. That is one extra round trip per call, and every API call in this project carries `Authorization`. The gateway answers preflights itself without touching a route, which is cheap, but the round trip is paid by the user. `Access-Control-Max-Age` lets the browser cache the answer; CorsConfig does not set it, so Chrome falls back to five seconds. One line, `config.setMaxAge(3600L)`, removes almost every preflight from a session.

When it is never consulted. In the cluster the frontend and `/api` share one origin behind Traefik (Chapter 26), so no request is cross-origin and the filter is idle. Dev is the only place it runs — and Vite’s proxy could make even dev same-origin. CORS config is therefore not a security feature here; it is a development convenience that must not be widened carelessly: `allowCredentials(true)` with a wildcard origin is refused by browsers precisely because it would let any site call the API with the user’s cookies.

31.4 The edge’s job, and what is not

Belongs at the edge	Does not
Authentication, identity headers	Authorization rules per resource
Rate limiting, quotas	Business validation
Timeouts, retries (idempotent), circuit breaking	Response aggregation across services
CORS, TLS termination, compression	Calling a database
Tracing headers, access logs	Data transformation beyond headers/-paths

The right-hand column is the “smart pipes” anti-pattern the ESB era taught: once the gateway knows business rules, every feature ships in two places and the gateway team becomes the bottleneck for everyone. The one legitimate exception

— composing several service calls into one screen-shaped response — is a service of its own (a *backend for frontend*), deployed and scaled separately from the edge.

31.5 Pros and cons, honestly

- **Pro:** one address, one JWT filter, one CORS policy; routes are configuration, changeable without a rebuild (Chapter 2).
- **Pro:** the reactive model holds thousands of in-flight requests on four threads and boots in seconds.
- **Con:** the project's gateway sets *no* HTTP client timeouts. Reactor Netty's defaults are a 30-second connect timeout and *no* response timeout at all.
- **Con:** no circuit breaker, no rate limiter — every client can send as fast as it likes, and a dead service is discovered by each request individually.
- **Con:** replicas: 1 in Kubernetes. The component every request passes is the one with a single copy.
- **Con:** the starter it uses, `spring-cloud-starter-gateway`, is deprecated in Spring Cloud 2025.0 in favor of `spring-cloud-starter-gateway-server-webflux`; the rename is harmless today and a build break in a future upgrade.

31.6 What breaks

PITFALL — A SLOW DOWNSTREAM TAKES THE WHOLE ROUTE WITH IT

Symptom: `course-service` is slow (a lock, a full GC), and within a minute *all* requests to `/api/courses/**` fail with 503 or hang — including cheap ones that would have succeeded. Cause: the gateway keeps a per-host connection pool, sized $\max(\text{cores}, 8) \times 2 = 16$ on this box. With no response timeout, each slow request holds one pooled connection for as long as `course-service` takes; the seventeenth request waits in the acquire queue for 45 seconds and then fails. Every user is queued behind the slowest one. Fix: a response timeout (Section 31.8) so a stuck connection is reclaimed, and a circuit breaker so the route fails fast once failures cluster.

PITFALL — 431 REQUEST HEADER FIELDS TOO LARGE

Symptom: a user with many cookies, or a token with fat claims (Chapter 28), gets an immediate 431 from the gateway with nothing in the service logs. Cause: Netty caps the request *header block* at 8 KB by default; the gateway rejects the request before any filter runs. Fix: keep tokens thin. Raising

`server.max-http-request-header-size` is the second-best answer, because every hop behind the gateway has its own cap.

PITFALL — THE GATEWAY IS THE SINGLE POINT OF FAILURE

Symptom: a rolling update of the gateway, a node reboot, or an OOM kill — and the entire application is down, while every service behind it is healthy. Cause: one replica. The gateway is stateless, so the fix is replicas: 2 plus a `PodDisruptionBudget`; the only thing that would make scaling harder is per-instance state such as an in-memory rate limiter, which is exactly why the enhancement below keeps that state in Redis.

31.7 What breaks at 3 a.m.

course-service's Postgres starts a long autovacuum; queries that took 20 ms take 4 s. Walk the path.

1. The frontend fires a dashboard's six requests. Four hit course-service and take 4 s each. The gateway holds four pooled connections for 4 s.
2. Users retry. Twenty users, four requests each: 80 requests compete for 16 connections. Acquire queue grows; after 45 s waiting requests fail with 503.
3. Requests to auth-service and dictionary-service are *unaffected* — the pool is per host — until users' access tokens expire and `/refresh` succeeds but the dashboard still fails, so they log out and back in, adding load.
4. Nobody is paged, because the gateway is "healthy": `/actuator/health` answers 200 and its event loops are not blocked. The failure is invisible at the edge and visible only as latency in Zipkin (Chapter 24).

With a 5 s response timeout and a circuit breaker on the route, step 2 becomes: slow calls are cut at 5 s and counted as failures; after ten of them the breaker opens; for the next 10 s every `/api/courses` request fails in under a millisecond with a fallback body; the pool is never exhausted; auth and dictionary keep working; the breaker's state change is a metric you can alert on. The database is still slow — resilience does not fix causes — but the blast radius is one route instead of the site.

31.8 Enhancing the resolution

Three additions, all configuration plus one bean. Dependencies first (gateway `pom.xml`):

```
# spring-cloud-starter-circuitbreaker-reactor-resilience4j
# -> the reactive CircuitBreaker gateway filter
# spring-boot-starter-data-redis-reactive
# -> backing store for RequestRateLimiter (non-blocking client)
```

Then api-gateway.yml in the config-server:

```
spring:
  data:
    redis:
      host: ${REDIS_HOST:localhost}      # redis:6379 in compose
  cloud:
    gateway:
      httpclient:
        connect-timeout: 2000            ①
        response-timeout: 5s
        pool:
          max-connections: 64            ②
          acquire-timeout: 2000
      default-filters:
        - RemoveRequestHeader=X-User-Id    ③
        - RemoveRequestHeader=X-User-Role
      routes:
        - id: course-service-courses
          uri: lb://course-service
          predicates:
            - Path=/api/courses/**
          filters:
            - StripPrefix=2
            - name: CircuitBreaker          ④
              args:
                name: courseCb
                fallbackUri: forward:/fallback/course
                statusCodes: "500,502,503,504"
            - name: RequestRateLimiter      ⑤
              args:
                redis-rate-limiter.replenishRate: 20
                redis-rate-limiter.burstCapacity: 40
                redis-rate-limiter.requestedTokens: 1
                key-resolver: "#{@userKeyResolver}"
resilience4j:
  circuitbreaker:
    instances:
      courseCb:
        sliding-window-size: 10
        minimum-number-of-calls: 10
        failure-rate-threshold: 50
        slow-call-duration-threshold: 3s    ⑥
```

```
slow-call-rate-threshold: 80
wait-duration-in-open-state: 10s
permitted-number-of-calls-in-half-open-state: 3
timelimiter:
  instances:
    courseCb:
      timeout-duration: 5s
```

⑦

- ① Connect in 2 s or give up; a full response within 5 s or the connection is closed and returned to the pool. Per-route overrides are possible via route metadata for a known-slow endpoint such as file upload.
- ② Sixteen connections per host is tight once timeouts make requests short; 64 is comfortable on this box and still bounded.
- ③ The Chapter 28 fix: identity headers can only be written by the gateway.
- ④ The reactive Resilience4j filter. Failures (and the listed status codes) count toward the breaker; when open, requests are forwarded to `/fallback/course` *inside the gateway*.
- ⑤ Token bucket in Redis: 20 tokens per second refill, up to 40 stored, one spent per request. Exceeding it returns 429 with `X-RateLimit-Remaining: 0`. Redis, not memory, so two gateway replicas share one bucket per user.
- ⑥ Without this, slow calls are not failures — the default slow-call threshold is 60 s. Three seconds means “slower than we designed for”.
- ⑦ The gateway’s breaker wraps every call in a `TimeLimiter` whose default is **one second**. Left unset, a healthy 1.2 s upload would trip the breaker. Set it explicitly and keep it $\geq$ the response timeout.

The key resolver and the fallback endpoint:

```
@Configuration
public class RateLimitConfig {

    /** Bucket per authenticated user; anonymous callers share
     *  a bucket per client IP (public paths: login, refresh). */
    @Bean
    public KeyResolver userKeyResolver() {
        return exchange -> {
            String user = exchange.getRequest().getHeaders()
                .getFirst(JwtConstants.HEADER_USER_ID);
            if (user != null) {
                return Mono.just("u:" + user);
            }
            var addr = exchange.getRequest().getRemoteAddress();
            return Mono.just("ip:" + (addr == null ? "unknown"
                : addr.getAddress().getHostAddress()));
        };
    }
}
```

```

    }
}

@RestController
public class FallbackController {

    /** Served by the gateway itself when a breaker is open.
     * 503 + Retry-After tells clients (and browsers) to back off. */
    @GetMapping("/fallback/course")
    public ResponseEntity<Map<String, String>> course() {
        return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE)
            .header(HttpHeaders.RETRY_AFTER, "10")
            .body(Map.of("error", "course-service unavailable",
                "retryAfterSeconds", "10"));
    }
}

```

DECISION — WHAT TO KEY THE RATE LIMIT ON

Per *user id* is the right default for an authenticated API: it matches the thing you are protecting (a tenant's fair share) and cannot be dodged by rotating IPs. Per *IP* is the only option before authentication — login and refresh — and is the right one there, because the threat is credential stuffing from one source; it is wrong behind a corporate NAT, where one IP is a thousand users. Per *API key* is for machine clients and partners, where the key is the contract. Production edges usually stack two: a coarse per-IP limit on public paths and a per-user limit on everything else.

HANDS-ON — HIT THE BUCKET

With Redis added to `docker-compose.yml` and the gateway restarted, fire 50 requests as one user faster than the bucket refills:

```

$ for i in $(seq 1 50); do
    curl -s -o /dev/null -w "%{http_code} " \
        -H "Authorization: Bearer $TOKEN" \
        http://localhost:8080/api/courses/courses
done; echo
200 200 200 ... 200 429 429 429 429 429 429 429 429 429
$ curl -si -H "Authorization: Bearer $TOKEN" \
    http://localhost:8080/api/courses/courses | grep -i ratelimit
X-RateLimit-Remaining: 0
X-RateLimit-Burst-Capacity: 40
X-RateLimit-Replenish-Rate: 20

```

Forty succeed (the burst), then 429s until a second passes and twenty tokens

return. Stop course-service and repeat: after ten failures the same command returns the fallback JSON in under a millisecond, and `/actuator/circuitbreakers` shows `courseCb` as OPEN.

31.9 Outside the project

Kong, Envoy (and Istio's ingress built on it), nginx, and the cloud gateways (AWS API Gateway, Apigee) implement the same four ideas — timeouts, breakers, rate limits, header rewriting — as native configuration rather than Spring filters, with the token bucket typically in the proxy's own memory per node plus a shared store for global limits. Envoy's "outlier detection" is the circuit breaker; its "local rate limit" filter is the bucket. What you have learned here transfers directly: the questions an interviewer asks about any of them — where does state live, what happens when the store is down, what is the timeout hierarchy — have the same answers.

31.10 Questions from real interviews

Q: Traffic goes 10× on Monday morning. What breaks first in the gateway, and what do you tune?

Not the event loops. Four loops doing non-blocking I/O handle far more than ten times this project's traffic; CPU on the gateway is HMAC verification and header copying, which is microseconds. What breaks first is whatever is *unbounded*: the per-host connection pool (16 by default here) fills as soon as any downstream slows, and with no response timeout it never drains. So the tuning order is: response and connect timeouts so connections are reclaimed; the pool ceiling raised to something the downstream can actually serve (`max-connections`, and a short `acquire-timeout` so queued requests fail fast rather than pile up); a circuit breaker per route so a struggling service is shed instead of queued; then horizontal scaling — two or three replicas behind the Service with an HPA on CPU, which for a stateless gateway is trivial once rate-limit state is in Redis. Memory limits matter too: Netty keeps direct buffers off-heap, so a 512 Mi limit with `MaxRAMPercentage=75` leaves 128 Mi for them; under 10× buffering that is where an OOM kill would come from.

Q: One downstream is slow and the whole site hangs. Why, and what do you do?

Because the site shares the thing that is exhausted. If the gateway has one connection pool per host, only that host's route should degrade — so “the whole site” says something else is shared: an event loop blocked by a synchronous call in a filter, a global rate limiter keyed on the wrong thing, or the frontend firing all its calls in one `Promise.all` so one slow response holds the page. The fix is in that order: audit filters for blocking calls (BlockHound in tests), give the slow route a timeout and breaker so it fails in milliseconds with a fallback body, and make the frontend render partial pages. The senior point is that resilience is about *isolation*: the goal is not that course-service never slows down but that its slowness cannot reach the login page.

Q: How do you rate-limit per user across three gateway replicas?

State must be shared or the limit is per replica and off by a factor of three. Spring Cloud Gateway's `RequestRateLimiter` runs a token bucket as a Lua script in Redis, atomically, keyed by whatever the `KeyResolver` returns — here `u:<userId>` from the header the JWT filter set, falling back to the client IP on public paths. The numbers are the policy: `replenishRate` is sustained requests per second, `burstCapacity` what a page load may spend at once; set `burst` to roughly one screen's worth of calls. Redis becomes a dependency, so decide its failure mode deliberately: the filter's default is to *allow* when Redis is unreachable, which is right for an internal app and wrong for a public one facing abuse. For very high volume, a local per-replica bucket in front of the shared one absorbs most of the Redis round trips.

Q: Where do you put authentication, and where do you not put logic?

Authentication — “is this token valid and whose is it” — at the edge, once, because it is the same question for every request and because that is where the credential enters the system. Authorization — “may this user grade this submission” — in the service that owns submissions, because only it knows the data. The gateway may enforce coarse rules that are still request-shaped (“`/api/admin/**` requires role ADMIN”) but the moment a rule needs a database row it belongs downstream. Nothing business-shaped goes in the gateway: no aggregation, no transformation of bodies, no calls to data stores. The test I use in reviews: if a change to a domain rule requires a gateway deploy, the line has been crossed.

Q: How do you deploy the gateway with zero dropped requests?

Two replicas at minimum, a `PodDisruptionBudget` of `minAvailable: 1`, and a rolling update with `maxUnavailable: 0` so the new pod is `Ready` before an old one is removed. Readiness must be real: the probe should fail while Eureka's registry is still empty, or the new pod is routed to before it can route anywhere. On shutdown, the old pod must stop receiving *and* finish in-flight requests: `server.shutdown: graceful` with a `spring.lifecycle.timeout-per-shutdown-phase` longer than the response timeout, and a `preStop` sleep of a few seconds so the kube-proxy endpoint removal propagates before the process gets `SIGTERM`. That last gap — the endpoint list and the pod's lifetime are updated by different controllers — is what drops the requests people blame on "the rolling update".

Q: A partner integration needs one response built from three services. Gateway?

No. That is a backend-for-frontend: a small service that calls the three in parallel, tolerates one failing, and shapes the response for that client. It scales and deploys on its own schedule, owns its own timeouts, and keeps the gateway a proxy. Put it in the gateway and every partner change is an edge deploy, and the gateway's event loops start holding fan-out state for the slowest of three calls.

CHECK YOURSELF (answers: Appendix A)

1. The JWT filter returns `chain.filter(exchange)` without awaiting anything. Explain what has and has not happened at the moment that line returns, and why a servlet filter could not be written this way.
2. A colleague adds a synchronous Redis lookup to the JWT filter to check a deny-list. Under a 200 ms Redis stall, what happens to requests for *other* routes, and why is the impact on this machine 25 % per stalled call?
3. course-service hangs on 10 % of requests. With the project's current gateway configuration, how many concurrent hung requests block *all* traffic to that service, and which single setting changes that number?
4. The gateway's `CircuitBreaker` filter is added to a route without a `timelimiter` block. A healthy upload takes 1.5 s. What happens, and why?
5. Why is the rate limiter's state in Redis rather than in the gateway's memory, and what is the trade-off when Redis is down?
6. Every request from the frontend carries `Authorization`. What does that imply for CORS preflights in development, what single setting reduces them, and why does none of this apply in the cluster?

Deep Dive: Synchronous Calls — OpenFeign, Resilience4j, and Cascading Failure

Chapter 6 drew the line between the two ways services talk: HTTP when the caller needs an answer now, events when it needs a fact recorded. This chapter stays on the HTTP side and asks the question that every distributed system eventually asks at 3 a.m.: when the service I am calling is slow, what happens to *me*? The project has the right libraries on its classpath — OpenFeign for the call, Resilience4j for the protection — and, as you will see, has wired them in a way that protects nothing. That makes it a better teaching case than a finished one would be.

Three questions organize the chapter:

1. **What does a synchronous call cost** beyond its latency — and why is a slow dependency worse than a dead one?
2. **How does a circuit breaker decide**, with the project’s actual numbers, and how do timeouts, retries and breakers interact?
3. **What should a fallback return**, and when should the call not exist at all?

32.1 Why Feign, and what it solves

course-service occasionally needs to know who a user is — a name to print on a submission, a role to check. auth-service owns that data. The options are: call auth-service over HTTP, copy the data into course-service via events, or put both in one database. The project keeps the databases separate (Chapter 7) and reserves events for facts that happened; a lookup by id is a question, so it is a call.

OpenFeign turns that call into an interface:

```
@FeignClient(name = "auth-service",
             fallback = AuthServiceFallback.class) ①
public interface AuthServiceClient {

    @GetMapping("/users/{id}") ②
    UserDto getUserById(@PathVariable("id") Long id); ③
}
```

- ① `name` is the Eureka application name. Feign asks the discovery client for instances of `auth-service` and load-balances across them on the client side — no gateway, no hard-coded host. `fallback` names the class to call instead when the call fails. Hold that thought.
- ② The same Spring MVC annotations as a controller, read in reverse: this is what to *send*. The path is the service’s own (`/users/{id}`), not the gateway’s (`/api/auth/users`), because service-to-service calls bypass the gateway.
- ③ The return type is a record from `common`. That record is the *contract*: `auth-service` must produce JSON that deserializes into it.

What Feign solves is boilerplate and coupling to addresses. What it does not solve is the one property that matters under load: every call holds a Tomcat thread in `course-service` for as long as `auth-service` takes to answer.

DECISION — A CALL, NOT A COPY — FOR NOW

For a by-id lookup that is rare and whose answer changes rarely, a call is the simplest correct thing, and Feign makes it two lines. The decision flips when any of three things becomes true: the lookup sits on a hot path (every course listing needs a teacher’s name), the data is needed while `auth-service` is down (grading must not stop because login is broken), or the shape of `UserDto` starts changing with `auth-service`’s releases. Then `course-service` should keep its own copy of the fields it needs, fed by the events `auth-service` already publishes (`ts.user.registered`, `ts.user.activated`) — the pattern Chapter 7 already applies to students.

32.2 The contract in `common`, and its versioning risk

`UserDto` is a record with five fields: `id`, `email`, `firstName`, `lastName`, `role`. `auth-service`’s `UserController` does not return it — it returns its own `UserResponse`. The two classes agree today by coincidence of field names, and Jackson’s default of ignoring unknown properties hides the gap: `auth-service` can add fields freely, and `course-service` will not notice. It *will* notice a rename. Renaming `firstName` to `givenName` in `UserResponse` compiles on both sides, deploys on both sides, and leaves `course-service` reading `null` names — no exception, no log line. The interview version: “how do you version an internal API?” and the answer is: additive changes only, never rename or remove; the consumer’s DTO ignores unknowns; a consumer-driven contract test (Spring Cloud Contract, Pact) in `auth-service`’s build that fails when `UserResponse` stops satisfying `UserDto`. A URL version (`/v2/users`) is for breaking changes you cannot avoid, and it means

running both for as long as any consumer lags.

32.3 The circuit breaker, with the project's numbers

Resilience4j's breaker is a state machine over a *sliding window* of recent call outcomes. The project's configuration in `course-service.yml`:

```
resilience4j:
  circuitbreaker:
    instances:
      authService:
        sliding-window-size: 10           ①
        failure-rate-threshold: 50        ②
        wait-duration-in-open-state: 10s  ③
        permitted-number-of-calls-in-half-open-state: 3 ④
```

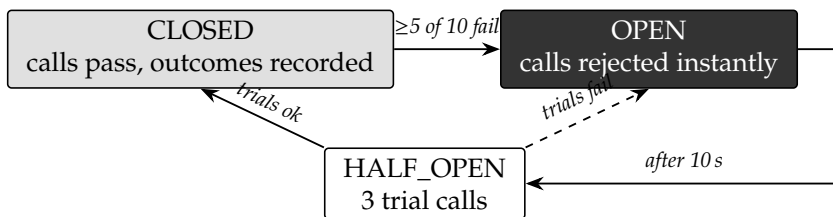


Figure 32.1: The breaker's three states with this project's thresholds. Rejected calls in OPEN go to the fallback without touching the network.

- ① The window holds the last 10 outcomes (count-based; the alternative is time-based, "the last 60 seconds"). The breaker does not evaluate until the window has minimum-number-of-calls outcomes — unset here, so it defaults to the window size. Translation: the first 9 calls after startup can *all* fail and the breaker stays CLOSED.
- ② At 10 outcomes, if 5 or more failed, the breaker opens. A "failure" is an exception the breaker does not ignore. Note what is *not* configured: `slow-call-duration-threshold`. Its default is 60 s, so a call that takes 30 s is a *success* to this breaker. Slow dependencies — the common case — never trip it.
- ③ OPEN for 10 s: every call is rejected with `CallNotPermittedException` in microseconds and routed to the fallback. `auth-service` gets a 10-second breather.
- ④ Then HALF_OPEN: 3 calls are let through as probes. If their failure rate is $\geq 50\%$ (2 of 3), back to OPEN for another 10 s; otherwise CLOSED with a fresh window.

Now the part that makes this chapter honest. That `authService` instance is bound to *nothing*. No method carries `@CircuitBreaker(name = "authService")`, and Feign's own integration — which would wrap `getUserById` in a breaker named `AuthServiceClient#getUserById(Long)` — is off, because the switch `openfeign.circuitbreaker.enabled` defaults to false, and the starter that provides the `CircuitBreakerFactory` it needs (the `circuitbreaker-resilience4j` starter) is not on the classpath. With that switch off, the fallback attribute in ① is silently ignored. And `AuthServiceClient` is not called from any service class today. The project has a breaker configured for a call that does not happen, protecting nothing, with a fallback that would not run. It is a perfect specimen of how resilience is usually misconfigured in production: every piece present, no piece connected.

32.4 Timeouts stack

When the call is wired properly, three timers surround it, and they must be ordered or the outer ones are decorative.

- **Feign's socket timeouts.** `connectTimeout` and `readTimeout` on the HTTP client. Unset in the project, so the defaults apply: 10 s to connect, 60 s to read. A hung auth-service pins a course-service thread for a minute.
- **Resilience4j's TimeLimiter.** A deadline on the whole decorated call, independent of the socket. It only applies to asynchronous return types (`CompletableFuture`), which is why synchronous Feign calls rely on the socket timeout instead.
- **The caller's deadline.** The gateway's response-timeout (Chapter 31), and behind it the browser's patience. If the gateway gives up at 5 s while Feign waits 60 s, the user sees an error at 5 s and course-service keeps a thread busy for 55 more seconds serving nobody.

The rule: every hop's timeout is *shorter* than the hop before it. Browser 10 s > gateway 5 s > course-service's Feign read 2 s. A timeout that fires inside an already-abandoned request is wasted work; one that fires later than its caller's is a leak.

32.5 Retry: only for calls you can repeat

Resilience4j's `@Retry` re-issues a failed call. On a GET `/users/{id}` that is harmless: reading twice is the same as reading once. On a POST `/enrollments` it is a duplicate enrollment, because a timeout does not tell you whether the first request was processed. Retry is therefore for *idempotent* operations only, or for op-

erations made idempotent with a client-supplied key the server deduplicates on. Two more rules from production: retry with *exponential backoff and jitter*, or every client retries at the same instant and the second wave is bigger than the first (the “retry storm”); and retry *inside* the breaker, so that an open breaker stops retries from being attempted at all. Resilience4j’s default aspect order — Retry outermost, then CircuitBreaker, then TimeLimiter, then Bulkhead — gets this right: a retry goes through the breaker each time, and a rejected call is not retried.

32.6 Bulkhead: bounding the blast

A bulkhead caps how many concurrent calls may be in flight to one dependency. Semaphore bulkhead: a counter; the 26th concurrent call is rejected immediately. Thread-pool bulkhead: calls run on a dedicated pool so the caller’s threads are never the ones waiting — necessary for a servlet app whose Tomcat pool is finite. course-service runs Tomcat with 200 threads. Without a bulkhead, 200 slow auth-service calls consume every one of them, and course-service cannot answer a health probe, let alone a course listing. With a semaphore bulkhead of 25, at most 25 threads are ever waiting on auth-service and 175 remain for everything else. That is the isolation the chapter’s title promises.

32.7 What the fallback returns

```
@Component
public class AuthServiceFallback implements AuthServiceClient {
    @Override
    public UserDto getUserById(Long id) {
        return null;
    }
}
```

①

- ① A null that the caller must remember to check. Every consumer of `getUserById` now has a hidden branch, and a forgotten check is a `NullPointerException` that fires only when auth-service is down — exactly when you least want a new bug.

A fallback has four honest options, in order of preference: a *cached* value from the last successful call (stale but real); a *degraded* value that the UI can show (“Unknown teacher”) and that is typed as such; an *empty* result where the domain allows one; or a *typed error* the caller can map to 503 with `Retry-After`. null is none of these. It is the absence of a decision, pushed onto every caller. The fallback

also needs the *cause*: a two-argument fallback method (`Throwable last`) can tell a breaker rejection from a 404, and a 404 — the user does not exist — must *not* be turned into “try later”.

32.8 Pros and cons, honestly

What the project has	What it is missing
Declarative client, Eureka-resolved	The client is never called
A breaker instance with sane thresholds	Nothing binds to it; Feign’s breaker switch is off
A fallback class registered	Ignored; and it returns null
Resilience4j on the classpath	No timeouts, no retry, no bulkhead, no slow-call threshold
Actuator exposed	Breaker endpoints not in the exposure list

And one more gap that is not about resilience at all: `auth-service`’s `SecurityConfig` permits only the public paths; `/users/{id}` requires an authenticated context, which its `GatewayHeaderAuthFilter` builds from `X-User-Id`. Feign sends no such header. The call, once wired, would return 401 on every attempt — and a 401 is an exception, so ten of them would open the breaker against a perfectly healthy service.

32.9 What breaks

PITFALL — THE FALLBACK THAT NEVER RUNS

Symptom: `auth-service` is stopped, and instead of the fallback’s null course-service throws `FeignException` with a stack trace through the load balancer, and the request returns 500. Cause: `@FeignClient(fallback = ...)` is only honored when `spring.cloud.openfeign.circuitbreaker.enabled=true` *and* a `CircuitBreakerFactory` bean exists, which requires the `spring-cloud-starter-circuitbreaker-resilience4j` dependency. With either missing, Spring logs nothing and the attribute is inert. Fix in Section 32.11.

PITFALL — HEALTHY SERVICE, OPEN BREAKER

Symptom: after a deploy of `auth-service`, `course-service`’s breaker opens and stays open, cycling `OPEN` → `HALF_OPEN` → `OPEN`, though `auth-service` answers `curl fine`. Cause: every call gets 401 (missing identity headers), or

404 for a deleted user, and both count as failures. Fix: propagate identity on service-to-service calls (a `RequestInterceptor`, below), and tell the breaker which exceptions are *not* dependency failures with `ignore-exceptions` — client errors mean the request was wrong, not that the service is down.

PITFALL — THE 60-SECOND THREAD

Symptom: auth-service hangs (deadlock, stuck GC); course-service's CPU is idle, yet it stops answering *all* requests, and its readiness probe fails so Kubernetes stops routing to it. Cause: Feign's default 60 s read timeout multiplied by Tomcat's 200 threads. Each hung lookup holds a thread for a minute; with a modest request rate every thread is waiting within seconds. The breaker cannot help — with the default slow-call threshold, hung calls are not failures until they time out. Fix: a 2 s read timeout, a slow-call threshold, and a bulkhead of 25.

32.10 What breaks at 3 a.m.

auth-service's Postgres begins a long vacuum; `/users/{id}` goes from 20 ms to 2 s. Assume the Feign call has been wired into the course-listing endpoint, as the project's design invites, and nothing else is changed.

1. Each course listing now takes 2 s instead of 20 ms. Tomcat threads in course-service are busy 100× longer per request.
2. At 50 listings per second, 100 threads are waiting on auth-service at any moment. At 100 per second, all 200 are, and requests queue in Tomcat's accept backlog.
3. course-service's readiness probe (`/actuator/health` is not even exposed here — Chapter 17 uses a TCP probe) keeps passing, because the socket accepts. The gateway keeps sending traffic.
4. The gateway's connection pool to course-service fills; its `/api/courses` route degrades for everyone, including requests that never needed auth-service. Chapter 31's cascade begins one hop up.
5. auth-service, meanwhile, is receiving 100 requests per second from a caller that will discard the answers after the gateway's timeout — load that makes the vacuum's contention worse.
6. The breaker never opens: 2 s is under the 60 s slow-call default, and none of the calls fail. The one mechanism that could shed load is watching for the wrong signal.

With the configuration below: calls are cut at 2 s (Feign read timeout) and counted as slow above 1 s; after ten outcomes with $\geq 80\%$ slow, the breaker opens; listings return with the cached or degraded teacher name in under a millisecond; at most 25 threads were ever waiting; auth-service's load drops to three probe calls per ten seconds; and the breaker's state change is on a dashboard.

32.11 Enhancing the resolution

Dependency first: add `spring-cloud-starter-circuitbreaker-resilience4j` to course-service's POM (it brings the `CircuitBreakerFactory` Feign needs; `resilience4j-spring-boot3` stays for the annotations and metrics). Then `course-service.yml`:

```
spring:
  cloud:
    openfeign:
      circuitbreaker:
        enabled: true                ①
      client:
        config:
          auth-service:             ②
            connect-timeout: 1000
            read-timeout: 2000
            logger-level: basic

resilience4j:
  circuitbreaker:
    instances:
      authService:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 10
        minimum-number-of-calls: 10
        failure-rate-threshold: 50
        slow-call-duration-threshold: 1s    ③
        slow-call-rate-threshold: 80
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
        automatic-transition-from-open-to-half-open-enabled: true
        register-health-indicator: true    ④
        ignore-exceptions:             ⑤
          - feign.FeignException.NotFound
          - feign.FeignException.BadRequest

retry:
  instances:
    authService:                    ⑥
      max-attempts: 3
      wait-duration: 200ms
```

```

enable-exponential-backoff: true
exponential-backoff-multiplier: 2
enable-randomized-wait: true
randomized-wait-factor: 0.5
retry-exceptions:
  - java.net.SocketTimeoutException
  - feign.RetryableException
ignore-exceptions:
  - feign.FeignException.FeignClientException
bulkhead:
  instances:
    authService:
      max-concurrent-calls: 25      ⑦
      max-wait-duration: 0
  timelimiter:
    instances:
      authService:
        timeout-duration: 3s      ⑧
        cancel-running-future: true
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus,circuitbreakers,circuitbreakerevents
  health:
    circuitbreakers:
      enabled: true

```

- ① The switch that makes `fallback = ... real`. Feign now wraps each method in a breaker named `AuthServiceClient#getUserById(Long)`; the explicit annotations below use the `authService` instance for the caller-side policy.
- ② Per-client timeouts, keyed by the Feign name. Two seconds is generous for a by-id lookup that normally takes 20 ms, and strictly shorter than the gateway's 5 s.
- ③ Slow now counts. Above 1 s is "slower than designed"; when 80 % of the window is slow, the breaker opens even if every call eventually succeeds.
- ④ The breaker's state appears in `/actuator/health` and as a Micrometer gauge (`circuitbreaker.state`) — the thing you alert on.
- ⑤ A missing user is an answer, not an outage. 4xx responses neither count as failures nor trigger the fallback.
- ⑥ Three attempts with 200 ms, then 400 ms, each jittered by $\pm 50\%$. Only transport failures are retried; any HTTP 4xx is final. Total worst case: $3 \times 2\text{ s} + 0.6\text{ s}$, still bounded and still under a minute — but longer than the gateway's 5 s,

which is the argument for at most 2 attempts on a request path and 3 only in background work.

- ⑦ Twenty-five threads may wait on auth-service; the 26th call fails immediately with BulkheadFullException and takes the fallback. max-wait-duration: 0 means no queueing — queueing is what we are trying to avoid.
- ⑧ Only used by the CompletableFuture variant below; harmless otherwise.

The service-side wrapper, with identity propagation and a fallback that says what it is:

```
/** Forwards the caller's identity on service-to-service calls so
 * auth-service's GatewayHeaderAuthFilter accepts them. */
@Configuration
public class FeignIdentityConfig {
    @Bean
    public RequestInterceptor identityPropagation() {
        return template -> {
            var auth = SecurityContextHolder.getContext()
                .getAuthentication();
            if (auth != null && auth.getPrincipal() instanceof Long id) {
                template.header(JwtConstants.HEADER_USER_ID, id.toString());
                auth.getAuthorities().stream().findFirst().ifPresent(a ->
                    template.header(JwtConstants.HEADER_USER_ROLE,
                        a.getAuthority().replace("ROLE_", "")));
            }
        };
    }
}

/** The one place course-service asks who a user is. */
@Service
@RequiredArgsConstructor
public class UserLookupService {

    private final AuthServiceClient authServiceClient;
    private final Map<Long, UserDto> lastKnown =
        new ConcurrentHashMap<>();

    @Retry(name = "authService")
    @CircuitBreaker(name = "authService", fallbackMethod = "degraded")
    @Bulkhead(name = "authService")
    public UserDto findUser(Long id) {
        UserDto user = authServiceClient.getUserById(id);
        lastKnown.put(id, user);
        return user;
    }

    /** Same signature plus the cause; called for breaker rejections,
```

```

* bulkhead rejections, timeouts and 5xx -- never for 4xx. */
private UserDto degraded(Long id, Throwable cause) { ②
    UserDto cached = lastKnown.get(id);
    if (cached != null) {
        return cached; // stale, real
    }
    return new UserDto(id, null, "Unknown", "user", null);
}
}

```

- ① A last-known-good cache. In one replica a map is enough; with several, the same idea belongs in Redis with a TTL, or — the real fix — in a local table fed by user events.
- ② The fallback is typed and visible: callers get a `UserDto` whose name says “Unknown”, never null. The `Throwable` lets you log *why* (breaker open vs. timeout) at WARN once per state change rather than per call.

The metrics to watch, all exported to Prometheus by the existing Actuator setup: `resilience4j_circuitbreaker_state` (a gauge per state per instance; alert when `state="open"` is 1 for more than a minute), `resilience4j_circuitbreaker_calls_seconds` by kind (successful, failed, ignored) for rates, `resilience4j_bulkhead_available_concurrent_calls` approaching zero as the leading indicator, and Feign’s own `http_client_requests_seconds` for the raw latency that started it all.

DECISION — WHEN THE CALL SHOULD NOT EXIST

Keep the synchronous call while it is off the hot path, rare, and tolerates a stale answer. Replace it with a local copy fed by events when a listing page needs it (latency and coupling), when grading must work while auth-service is down (availability), or when the two teams release on different cadences (contract drift). Replace it with a short-TTL cache in front of the call when the data is hot but must not be more than a minute old and eventual consistency via events is not worth a new consumer. The senior instinct is to count synchronous dependencies per request and treat each one as a multiplication of failure probability: three 99.9% dependencies in series are a 99.7% endpoint.

HANDS-ON — WATCH THE BREAKER OPEN

With the enhancement applied and the stack up, stop auth-service and call an endpoint that uses `UserLookupService` twelve times:

```

$ docker compose stop auth-service
$ for i in $(seq 1 12); do

```

```
curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" \
-H "Authorization: Bearer $TOKEN" \
http://localhost:8080/api/courses/courses/1
done
200 1.04s ... (10 slow lines: retries then fallback)
200 0.01s <- breaker open: fallback without a network call
200 0.01s
$ curl -s http://localhost:8082/actuator/circuitbreakers | jq
{ "circuitBreakers": { "authService": {
  "failureRate": "100.0%", "slowCallRate": "0.0%",
  "bufferedCalls": 10, "failedCalls": 10, "state": "OPEN" } } }
```

Every response is 200 with “Unknown user” as the teacher — the page degrades instead of failing. Start auth-service again, wait 10s, and `/actuator/circuitbreakerevents` shows the three HALF_OPEN probes succeed and the state return to CLOSED.

32.12 Outside the project

The same machinery under other names: Hystrix was Netflix’s original and is retired; Resilience4j is its successor in the Java world; Envoy and Istio implement timeouts, retries with budgets, and outlier ejection in the sidecar so that application code contains none of it, which is the direction large fleets have gone. gRPC carries deadlines in the protocol — a caller’s deadline propagates to every hop, solving the “timeouts stack” problem by construction. Cloud SDKs (AWS, GCP) ship with exponential backoff and jitter built in for the same reason this chapter recommends them. The vocabulary is universal; an interviewer who says “bulkhead” or “retry budget” means exactly what this chapter means.

32.13 Questions from real interviews

Q: auth-service latency goes from 20 ms to 2s. Walk me through what happens in course-service, and how you would stop it.

Each request that touches auth-service now holds a Tomcat thread a hundred times longer. At a modest rate the pool of 200 saturates within seconds, the accept queue fills, and course-service stops answering everything — including endpoints that never call auth-service and the readiness probe. Upstream, the gateway’s per-host pool to course-service fills and its route degrades for all users. Nothing in the default configuration stops this: Feign waits 60s, and a 2s call

is not slow to a breaker whose slow-call threshold is 60 s. To stop it, in order of leverage: a read timeout of 2 s so no thread waits longer than the gateway's deadline; a bulkhead of about 25 so the other 175 threads keep serving; a slow-call threshold of 1 s at 80 % so the breaker opens on latency, not just on errors; and a fallback that returns the last known name rather than `null`. Then, if the lookup is on a hot path, remove the call altogether with a local copy fed by user events. The senior framing: the goal is that auth-service's latency shows up as a stale name on a page, not as an outage of courses.

Q: How do you size timeouts across three hops?

From the outside in, each shorter than the one before. The browser or mobile client is the outermost budget — say 10 s before the user gives up. The gateway's response timeout must be inside that, 5 s, so the client gets a clean 503 instead of a hang. course-service's Feign read timeout must be inside *that* and leave room for retries: 2 s with at most one retry keeps the worst case around 4 s. Any timeout longer than its caller's is a leak — work done for a request that has already been abandoned — and any retry policy must be included in the arithmetic, because three attempts at 2 s is 6 s plus backoff. In production you also set a *connect* timeout separately and much shorter (1 s), because failure to connect is diagnosed in one round trip. The best systems propagate a deadline header from the edge so each hop computes its own remaining budget; gRPC does this natively.

Q: Retry storms — how do you prevent them?

Three controls, all needed. First, jitter: exponential backoff alone lets every client that failed at the same moment retry at the same moment, so the second wave is synchronized and larger; randomizing the wait by $\pm 50\%$ spreads it. Second, a retry budget: retries as a percentage of traffic (Envoy's default is 20 %), so that when a dependency is truly down, retries cannot triple its load. Resilience4j approximates this by putting Retry outside the breaker — an open breaker rejects attempts before they become retries. Third, idempotency: retry only GET and operations that carry an idempotency key the server deduplicates on; otherwise a timeout on a POST becomes a duplicate enrollment. And retry only transport failures and 503 — a 400 or 404 is deterministic and will fail again. The tell-tale sign of a storm in metrics is a dependency whose request rate rises as its success rate falls.

Q: The breaker is open, but users need the data. What does your fallback return?

The most truthful thing available, typed so the caller cannot mistake it for the real value. Order of preference: the last successful value from a short-lived cache (stale but real — a teacher’s name from ten minutes ago is fine); a clearly degraded placeholder the UI can render (“Unknown teacher”) that never masquerades as data; an empty collection where the domain permits; or a typed exception that becomes a 503 with Retry-After. Never null, which turns a resilience event into a `NullPointerException` somewhere else. The fallback must also see the cause: a 404 means the user does not exist and should surface as such, not be swallowed by “try later”. And fallbacks should be cheap and local — a fallback that calls another service is a second dependency on the failure path.

Q: How do you know the breaker is open at 3 a.m.?

Because it is a metric with an alert, not a log line. Resilience4j exports `resilience4j_circuitbreaker_state` as a gauge per state; a rule like “state=“open” equals 1 for more than 60 seconds” pages someone with the dependency name in the alert. Pair it with the leading indicators: `slow_call_rate` rising before the breaker trips, bulkhead available slots trending to zero, and the dependency’s p99 from `http_client_requests_seconds`. The breaker also feeds `/actuator/health` when `register-health-indicator` is on, so a readiness probe can be made to reflect it — though usually you do *not* want an open breaker to take a service out of rotation, since it is serving fallbacks precisely so it can stay in. And log the *transition* at WARN once, not every rejected call, or the log becomes the next outage.

Q: Would you put the breaker in the gateway or in the service?

Both, for different failures. The gateway’s breaker (Chapter 31) protects users from a whole service being slow; it opens per route and serves a fallback body. The service’s breaker protects the service’s own thread pool from one of its dependencies and is where the domain-aware fallback lives — only course-service knows that a missing teacher name is survivable. A gateway cannot know that. The mistake is having only the edge one and assuming services behind it are therefore safe; the cascade this chapter walks through happens entirely behind the gateway.

CHECK YOURSELF (answers: Appendix A)

1. The project configures a breaker named `authService`. List the three separate reasons it currently protects no call, and the one-line change that fixes each.
2. With `sliding-window-size: 10` and the other defaults, how many consecutive failures can occur after startup before the breaker opens, and why does a call that takes 30 s not count?
3. Order these and justify: gateway response timeout, browser timeout, Feign read timeout, retry count.
4. `AuthServiceFallback.getUserById` returns null. Name two concrete defects that follow from this and the two-argument signature that fixes the more subtle one.
5. `auth-service` returns 401 to every Feign call. Explain the cause in the project's security configuration, the effect on the breaker, and the two changes that resolve it.
6. Three synchronous dependencies each at 99.9 % availability sit in series on one endpoint. What is its availability, and which two design changes raise it without touching the dependencies?

Deep Dive: Data Across Services — Database-per-Service, Duplication, and the Outbox

Chapter 7 established the rule: each service owns its database, and the only road to another service’s data is its API or its events. The rule is easy to state and expensive to live with. This deep dive follows the one piece of data the project copies between services — a student’s name and e-mail — from the moment it is written in auth-service’s Postgres to the moment it appears in course-service’s, and asks what can go wrong on the way. The answer is the dual-write problem, and the repair is the transactional outbox.

Three questions organize the chapter:

1. **What does database-per-service buy**, and what does it forbid — joins, foreign keys, transactions — that a monolith takes for granted?
2. **How does duplicated data stay correct**, and what is the exact window in which the project is wrong today?
3. **Can “save the row and publish the event” be made atomic** — and if not, what is the closest thing to it?

33.1 Why this, and what it solves

Open `docker-compose.yml`: two Postgres 17 containers, `postgres-auth` holding `ts_auth` and `postgres-course` holding `ts_course`, on separate volumes and ports. Nothing in `course-service` knows the first one’s address. The isolation buys four things:

- **Schema ownership.** Auth-service can add a `totp_secret` column, rename `provider_id`, or run a destructive Flyway migration without coordinating with anyone. Chapter 7’s V3 and V4 migrations happened exactly that way.
- **Independent deploys.** A migration that locks users for a minute stalls logins, not course listings.
- **Independent scaling and tuning.** The course database will grow with submissions; the auth database will not. They can sit on different disks, get different connection pools, different backup schedules.

- **A hard stop on accidental coupling.** In a shared database the first JOIN users ON student.user_id = users.id written under deadline pressure welds two teams' schemas together forever. Here the join is impossible, so it is never written.

It forbids exactly as much:

- **No cross-service joins.** “List students in course 7 with their e-mails” cannot be one query if e-mails live elsewhere.
- **No cross-service foreign keys.** student.user_id in V1__create_course_tables.sql is BIGINT NOT NULL UNIQUE and nothing more — Postgres cannot check that user 42 exists in a database it cannot see.
- **No cross-service transactions.** A business operation that must change both databases cannot be rolled back as a unit.

DECISION — COPY THE STUDENT, DO NOT FETCH THE USER

Course-service needs a student's name and e-mail on every course roster and in every assignment notification. Two designs were possible: call auth-service each time (there is even a Feign client for it, AuthServiceClient.getUserById, with a fallback that returns null), or keep a local copy in the student table. The project chose the copy, and the Feign client is dead code — no class in course-service calls it. The reasoning: a roster is read far more often than a name changes; a synchronous call per row would make course-service's availability a product of auth-service's (Chapter 6); and the local copy can be indexed, joined, and paged with plain SQL. The cost is that the copy can be *stale*, and the rest of this chapter is about how stale, and how to bound it. The decision flips when the data changes often or must be authoritative at read time — an account balance, a permission — where a call, or a cache with explicit invalidation, is the honest design.

33.2 How a student reaches course-service

Trace the real path. It starts in auth-service's AuthService, which is annotated @Transactional at class level:

```
public void activate(String code) {
    User user = userRepository.findByActivationCode(code)
        .orElseThrow(() ->
            new IllegalArgumentException("Invalid activation code"));

    user.setActivated(true);
}
```

```
user.setActivationCode(null);  
userRepository.save(user); ①  
  
userActivatedPublisher.publish(user); ②  
}
```

UserActivatedPublisher builds a UserActivatedEvent record — user id, e-mail, first and last name, role — and calls `kafkaTemplate.send("ts.user.activated", event.userId(), event)`. On the other side, course-service's UserActivatedListener:

```
@KafkaListener(topics = "ts.user.activated", groupId = "course-group")  
public void onUserActivated(UserActivatedEvent event) {  
    if (!ROLE_STUDENT.equalsIgnoreCase(event.role())) {  
        return;  
    }  
    Long userId = Long.parseLong(event.userId());  
    if (studentRepository.findById(userId).isPresent()) { ③  
        return;  
    }  
    Student student = Student.builder()  
        .userId(userId)  
        .firstName(event.firstName())  
        .lastName(event.lastName())  
        .email(event.email())  
        .build();  
    studentRepository.save(student); ④  
}
```

- ① The row is dirty inside a transaction that has not yet committed. `save()` on a managed entity is a no-op until flush.
- ② The event is handed to the Kafka producer *inside* the same transaction, before the commit at method exit. Keep this line in mind; it is the whole problem.
- ③ The listener is idempotent by natural key: a second delivery of the same event finds the row and returns. This is why the project survives at-least-once delivery (Chapter 34) without a dedup table — here.
- ④ The copy. From now on, course-service answers every roster query from this row and never asks auth-service again.

The consistency window is therefore: from the commit in auth-service to the commit in course-service. On a quiet stack it is milliseconds. With notification-service and course-service both in the consumer path, and the broker on the same two-core box (Chapter 25), it can be seconds. A teacher who enrolls a student “immediately” after the student activates may get 404 Student not found from EnrollmentService.enroll — correct behaviour, surprising UX.

33.2.1 What is not propagated at all

Now look for the second event. There is none. `ts.user.activated` fires once, at activation. A user who later changes their name or e-mail in auth-service is never reflected in student; the DATABASE-DESIGN.md note “future: Kafka event propagates e-mail changes” is the acknowledgement. Assignment notifications will keep going to the old address. This is the honest state of eventual consistency in the project: *eventual* for creation, *never* for updates. Section 33.5 adds the missing event.

33.3 Pros and cons, candidly

	Shared database	Database-per-service (here)
Cross-entity read	one JOIN	local copy, or two calls
Integrity	foreign keys	application code + events
Multi-entity write	one transaction	saga or outbox
Schema change	coordinate all owners	owner decides
Failure blast radius	one database, all services	one service
Staleness	none	seconds (creation), unbounded (updates) today

The project’s real weak points, named:

- Update events do not exist; copies drift permanently.
- Save-and-publish is not atomic (next section).
- The Feign fallback returns null, so if anyone ever wires `AuthServiceClient` in, a circuit-open state will surface as a `NullPointerException` rather than a clear degraded response.
- No reporting path: “how many activated students per course” needs data from both databases and there is no read model for it.

33.4 The dual-write problem

Line ② above writes to two systems — Postgres and Kafka — that share no transaction manager. Whatever order you put them in, one of two failures is possible:

- **Commit, then lose the event.** `send()` is asynchronous: it appends to a client-side buffer and returns a future. The method exits, the transaction commits, and the JVM is killed (OOMKilled, a rolling update, `kubectl delete pod`) before the buffer flushes. The user is activated; course-service never hears of it; the student cannot be enrolled, ever, and nothing logs an error.
- **Publish, then roll back.** The buffer flushes within milliseconds; the broker acks; then the commit fails — a unique constraint, a connection dropped, a deadlock. Course-service creates a student for a user that does not exist. A *phantom* row.

Both are rare per request and certain over a year. The interviewer’s version: “you save an order and publish `OrderCreated`; how do you guarantee both happen?” The junior answer is “put it in a `try/catch`”. The senior answer starts with “you cannot, with two resources”, then names the outbox.

PITFALL — REGISTRATION HANGS FOR SIXTY SECONDS, THEN FAILS

Symptom: with Kafka down, `POST /api/auth/register` does not fail fast — it hangs for a minute and returns 500. Cause: the producer must fetch topic metadata before its first send; with no broker reachable it blocks for `max.block.ms` (default 60 000 ms) and then throws. The exception propagates out of `register()`, the transaction rolls back, and the user is *not* registered — because the mail-notification broker was down. A dependency that should have been asynchronous has become a synchronous point of failure with a one-minute timeout. The outbox below turns this into “registered; e-mail delayed”.

PITFALL — THE COPY THAT OUTLIVES THE ORIGINAL

Symptom: an assignment e-mail bounces from an address the user changed months ago. Cause: `student.email` was written once by `UserActivatedListener` and there is no `ts.user.updated` event. Fix: emit one, and let the listener upsert. Section 33.5 shows the emitter; the listener change is a `findById().map(update).orElseGet(create)`.

33.5 Enhancing the resolution: the transactional outbox

The idea is to write the event into the *same database*, in the *same transaction*, as the business row — and let a separate process copy it to Kafka afterwards. Postgres guarantees the row and the event commit together or not at all; the relay guarantees the event eventually reaches Kafka; the consumer’s idempotency guarantees

a retried relay does no harm. Here is the full implementation for auth-service.

The table, as Flyway migration V5__create_outbox.sql:

```
CREATE TABLE outbox (  
  id          UUID PRIMARY KEY,  
  aggregate_type VARCHAR(64) NOT NULL, -- 'user'  
  aggregate_id VARCHAR(64) NOT NULL, -- also the Kafka key  
  event_type   VARCHAR(64) NOT NULL, -- 'UserActivated'  
  topic        VARCHAR(128) NOT NULL,  
  payload      JSONB      NOT NULL,  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),  
  published_at TIMESTAMPTZ,  
  attempts     INT        NOT NULL DEFAULT 0  
);  
-- the relay's only query: oldest unpublished first  
CREATE INDEX idx_outbox_unpublished  
  ON outbox (created_at) WHERE published_at IS NULL;
```

The entity is deliberately dumb — a row, not a domain object:

```
@Entity @Table(name = "outbox")  
@Getter @Setter @NoArgsConstructor  
public class OutboxEvent {  
  @Id private UUID id;  
  private String aggregateType;  
  private String aggregateId;  
  private String eventType;  
  private String topic;  
  @JdbcTypeCode(SqlTypes.JSON)  
  private String payload;           // serialized event  
  private Instant createdAt;  
  private Instant publishedAt;  
  private int attempts;  
}  
  
public interface OutboxRepository extends JpaRepository<OutboxEvent, UUID> {  
  // SKIP LOCKED: two relay instances never pick the same row  
  @Query(value = ""  
    SELECT * FROM outbox WHERE published_at IS NULL  
    ORDER BY created_at LIMIT :n FOR UPDATE SKIP LOCKED""",  
    nativeQuery = true)  
  List<OutboxEvent> lockUnpublished(@Param("n") int n);  
}
```

The publisher no longer touches Kafka. It writes a row, and because it is called from inside AuthService's transaction, the row commits with the user:

```
@Component
```

```
@RequiredArgsConstructor
public class UserActivatedPublisher {

    private final OutboxRepository outbox;
    private final ObjectMapper mapper;

    public void publish(User user) {
        UserActivatedEvent event = new UserActivatedEvent(
            String.valueOf(user.getId()), user.getEmail(),
            user.getFirstName(), user.getLastName(),
            user.getRole().name());
        OutboxEvent row = new OutboxEvent();
        row.setId(UUID.randomUUID());           ①
        row.setAggregateType("user");
        row.setAggregateId(event.userId());       ②
        row.setEventType("UserActivated");
        row.setTopic("ts.user.activated");
        row.setPayload(toJson(event));
        row.setCreatedAt(Instant.now());
        outbox.save(row);                       ③
    }

    private String toJson(Object o) {
        try { return mapper.writeValueAsString(o); }
        catch (JsonProcessingException e) {
            throw new IllegalStateException(e);
        }
    }
}
```

The relay runs every second, takes a locked batch, sends each row *synchronously*, and marks it:

```
@Component
@RequiredArgsConstructor
@Slf4j
public class OutboxRelay {

    private final OutboxRepository outbox;
    private final KafkaTemplate<String, String> kafka;

    @Scheduled(fixedDelay = 1000)
    @Transactional                               ④
    public void relay() {
        for (OutboxEvent row : outbox.lockUnpublished(100)) {
            ProducerRecord<String, String> rec = new ProducerRecord<>(
                row.getTopic(), row.getAggregateId(), row.getPayload());
            rec.headers().add("eventId",
```



```

        row.getId().toString().getBytes(UTF_8)); ⑤
rec.headers().add("eventType",
    row.getEventType().getBytes(UTF_8));
try {
    kafka.send(rec).get(10, TimeUnit.SECONDS); ⑥
    row.setPublishedAt(Instant.now());
} catch (Exception e) {
    row.setAttempts(row.getAttempts() + 1);
    log.warn("outbox {} attempt {} failed: {}",
        row.getId(), row.getAttempts(), e.toString());
    break; ⑦
}
}
}
}

```

- ① The event id is minted *here*, once, and survives every retry. It is the consumer's deduplication key (Chapter 34).
- ② The aggregate id becomes the Kafka key, so all events about one user land on one partition, in order.
- ③ A plain INSERT, in the caller's transaction. If `activate()` rolls back, so does this row: no phantom events. If it commits, the row is durable before any Kafka involvement: no lost events.
- ④ The relay's transaction holds the FOR UPDATE SKIP LOCKED locks, so a second auth-service replica running the same scheduler takes different rows.
- ⑤ Headers carry identity and type without polluting the JSON payload the DTO classes deserialize.
- ⑥ The *only* place in auth-service that waits on Kafka, and it is a background thread. HTTP requests never block on the broker again.
- ⑦ On failure stop the batch, keep ordering (row 2 must not be published before row 1 for the same key), and let the next tick retry. `attempts` is what you alert on.

Add `@EnableScheduling` to `AuthServiceApplication` and change the `KafkaTemplate` to `<String, String>`, since the payload is already JSON. The consumer sees exactly what it saw before.

Two rows of bookkeeping complete it: a nightly `DELETE FROM outbox WHERE published_at < now() - interval '7 days'` keeps the table small, and the same `UserActivatedPublisher` shape gives you `ts.user.updated` for free — call it from wherever profile changes are saved, with `eventType = "UserUpdated"`, and have `UserActivatedListener` `upsert` instead of `insert`.

HANDS-ON — WATCH THE OUTBOX ABSORB A BROKER OUTAGE

With the outbox in place, stop Kafka, activate a user, and look:

```
$ docker compose stop kafka
$ curl -s -X POST localhost:8080/api/auth/activate?code=$CODE
{"message":"Account activated"} # instant, no 60s hang
$ docker compose exec postgres-auth psql -U postgres ts_auth -c \
  "select event_type, attempts, published_at from outbox"
event_type | attempts | published_at
UserActivated |      3 |
$ docker compose start kafka
# ...a few seconds later
UserActivated |      3 | 2026-09-07 03:04:11.20+00
```

Then check course-service's log for Provisioned student row. The activation was never at risk, and the student appeared as soon as the broker did.

33.5.1 Change data capture: the industrial relay

A polling relay is simple and adds up to one second of latency plus one query per second per replica. At scale the relay is replaced by *change data capture*: Debezium tails Postgres's write-ahead log, sees the INSERT INTO outbox, and publishes it to Kafka itself, with no application scheduler and sub-100 ms latency. The application code is identical — write the row — which is why the outbox is worth adopting early: the relay is the only part that changes.

33.6 Sagas: when two services must both change

Enrollment today is local — course, student, and enrollment are all in ts_course, so EnrollmentService.enroll is one ordinary transaction. The first truly cross-service write the project will meet is *account deletion*: auth-service must remove the user; course-service must remove the student, their enrollments, their submissions, and the submission files in MinIO (Chapter 9). There is no transaction spanning that.

A *saga* is a sequence of local transactions where each step publishes an event that triggers the next, and each step has a *compensating* action if a later step fails:

1. auth-service marks the user DELETING (not deleted) and writes UserDeletionRequested to the outbox.
2. course-service deletes the student's data and files, then publishes StudentDataDeleted — or, on failure, StudentDataDeletionFailed.
3. auth-service, on the success event, hard-deletes the user; on the failure event, compensates by restoring ACTIVE and notifying an operator.

This is *choreography*: no coordinator, each service reacts to events. It suits two or three steps. Past that — deletion across five services, with ordering constraints — the event graph becomes unreadable, and teams move to *orchestration*: one component (a saga orchestrator, or a workflow engine such as Temporal) holds the state machine and sends *commands* to each service. The trade is one more service to run against a flow you can actually read.

Two properties every saga must have, and interviewers check for: *semantic locks* — the `DELETING` state stops a user logging in during the seconds the saga runs — and *idempotent steps*, so the at-least-once delivery of Chapter 34 cannot delete twice or compensate twice.

33.7 Reads across services

The remaining gap is reporting. “Activated students per course, with last login” needs auth data and course data, and no service has both. Three answers, in ascending cost:

- **API composition.** The gateway or a small backend-for- frontend calls both services and joins in memory. Fine for one page, wrong for a nightly report over every course.
- **A read model.** A dedicated reporting database subscribes to *every* domain event and maintains denormalized tables shaped for the queries. This is CQRS: writes go to the owning services, reads go to a model built from their events. The outbox above is the prerequisite — without reliable events the read model drifts silently.
- **A warehouse.** CDC from every service database into an analytical store. The read model for people who are not you.

33.8 What breaks at 3 a.m.

- **postgres-course is down; users keep activating.** Without the outbox nothing is lost on the auth side, but `UserActivatedListener` throws on `save()`. Spring’s default error handler retries ten times in quick succession, then *logs and skips* the record (Chapter 34). The event is consumed, the student is never created, and the log line is the only trace. With the outbox the auth side is safe, but the consumer side still needs a retry topic — the outbox fixes the producer, not the consumer.
- **An event is replayed.** Someone resets `course-group`’s offsets to recover from the outage above. Every `ts.user.activated` since retention start is redelivered.

`findByUserId().isPresent()` makes each one a no-op. Idempotency by natural key is what makes replay a routine operation instead of an incident.

- **Ordering.** `UserActivated` and a future `UserUpdated` for the same user are keyed by `userId` and so share a partition; the update cannot overtake the activation. Events for *different* users may interleave freely, which is fine — there is no cross-user invariant. Change the key, or send from two different producers, and this guarantee is gone.
- **The relay stalls.** A poison row — a payload the broker rejects for size — makes ⑦ break on every tick, and rows behind it queue forever. Alert on `max(attempts)` and on `count(*)` where `published_at` is null; after N attempts, move the row to a `outbox_dead` table and let the queue drain.

DECISION — WHEN ONE DATABASE IS THE BETTER CALL

`docs/PLAN-A` describes the same product as a modular monolith: one Spring Boot application, one `ts` database, and `student.user_id` as a real foreign key. Every problem in this chapter — the dual write, the drift, the saga, the read model — disappears there, replaced by a JOIN and a transaction. That is not a small saving. The microservice layout (PLAN-B) earns its cost only when the forbidding things are the point: teams that must deploy without each other, data that must scale separately, or an organization large enough that a shared schema *would* be joined across in anger. For a three-person team this project is deliberately over-built, and Chapter 27's road ahead is honest about it. The senior answer in an interview is not “microservices” or “monolith”; it is “a modular monolith until a boundary has a reason to be a network boundary — and when it does, the outbox from day one”.

33.9 Outside the project

The outbox is the pattern behind Debezium's “outbox event router”, Axon's and Eventuate's transactional messaging, and the `TransactionalEventPublisher` shapes in every serious event-driven codebase. Sagas as orchestration are productized by Temporal, Camunda, and AWS Step Functions; as choreography they are simply “events with compensations” and a diagram someone must keep current. Netflix, Uber, and every bank you can name run some combination of the three — outbox for reliability, sagas for multi-step writes, CQRS read models for queries that cross ownership lines. The one thing none of them do is what the project does today: call `send()` inside a transaction and hope.

33.10 Questions from real interviews

Q: You need a report joining users and courses, and they live in two databases. How?

First I ask how fresh and how big. For one screen, API composition: a backend-for-frontend calls `/users?ids=` and `/courses`, joins in memory, and that is fine up to a few hundred rows. For a real report I build a read model: a reporting database that subscribes to `ts.user.activated`, `ts.user.updated`, and the course events, and maintains a denormalized `course_roster` table shaped for the query. That is CQRS — the owning services keep their writes, the read model answers cross-service reads. The prerequisite is reliable events, which is why I put the outbox in first; a read model fed by best-effort `send()` calls drifts and nobody notices until finance does. For analytics at company scale the same idea is CDC into a warehouse. What I refuse to do is give the reporting job a read replica of both databases and a cross-database join — that is the shared-database coupling the architecture exists to prevent, back in through the side door.

Q: Two services must both succeed or neither. How, without two-phase commit?

A saga. Each service does a local transaction and publishes an event; the next step reacts; every step has a compensating action. In the project, account deletion: auth-service sets `DELETING` and emits `UserDeletionRequested`; course-service deletes data and emits `StudentDataDeleted` or `...Failed`; auth-service either hard-deletes or restores `ACTIVE`. Two things make it correct rather than hopeful: a semantic lock — the `DELETING` state blocks logins for the seconds the saga runs — and idempotent steps, because Kafka will deliver at least once. I choose choreography for two or three steps and orchestration (Temporal, Camunda, or a hand-written state machine) once the event graph needs a diagram to follow. 2PC is off the table not because it is wrong but because Kafka and Postgres do not share a transaction coordinator, and holding locks across a network call is how you get the 3 a.m. page.

Q: An event was lost between the database commit and the Kafka send. How do you make that impossible?

By never sending from the request thread. The transactional outbox: the event is a row in an outbox table written in the same transaction as the business row, so Postgres guarantees both or neither. A relay — a `@Scheduled` poller with `FOR UPDATE SKIP LOCKED`, or Debezium tailing the WAL — publishes it afterwards

and marks it. The relay is at-least-once, so the row carries an event id that the consumer deduplicates on. In this project that also removes a latent outage: today `kafkaTemplate.send()` inside `AuthService.register` blocks for `max.block.ms` when the broker is down, so a mail-broker outage makes registration fail after sixty seconds. With the outbox, registration commits in milliseconds and the e-mail is late instead of the signup being lost.

Q: How do you migrate a shared database into database-per-service safely?

Strangler, not big bang. Step one: stop the bleeding — find every cross-schema join and foreign key and route them through the owning service’s API or a local copy, while the tables are still in one database. That is the hard part and it can take months; the physical split afterwards is a weekend. Step two: give each service its own credentials and schema, so an accidental join fails at compile time. Step three: dual-write or CDC the tables into the new database, verify counts and checksums, flip reads, then writes, keep the old tables read-only for a release, then drop. Throughout, the outbox goes in *before* the split, because the moment the join is gone you are relying on events to keep copies correct. Chapter 7’s “two Postgres servers, not two schemas” is the end state; you reach it by schema-per-service first.

Q: You duplicated Student data into course-service. How do you keep it correct?

I accept that it will be stale and I bound the staleness. Creation arrives on `ts.user.activated`; updates need a `ts.user.updated` the project does not emit yet — that is the first fix. Both are keyed by `userId`, so they share a partition and an update cannot overtake the creation. The listener upserts by natural key, which makes replay safe. The events come from an outbox, so none are lost; the consumer records processed event ids, so none are applied twice. Then I measure: a nightly job compares a sample of `student.email` against `auth-service` and alerts on drift, because every one of the mechanisms above has a bug I have not found yet. What I do not do is put the e-mail in the JWT — that freezes it for the token’s lifetime and couples the schemas through the token contract (Chapter 28).

CHECK YOURSELF (answers: Appendix A)

1. Read `AuthService.activate`. Describe the two distinct failure orderings that leave `auth-service` and `course-service` disagreeing, and say which one produces a row that should not exist.

2. Kafka is unreachable. Explain why `POST /register` today takes about a minute to fail and does not create the user, naming the producer setting responsible. What does the same request do after the outbox is in place?
3. In the relay, why is `FOR UPDATE SKIP LOCKED` necessary, and what would go wrong with two auth-service replicas if it were replaced by a plain `SELECT`?
4. Why does `break` (not `continue`) follow a failed send in the relay loop? Give the scenario the alternative corrupts.
5. A student's e-mail changes in auth-service. State what course-service knows about it today, and list the three code changes that close the gap.
6. Design account deletion across auth-service, course-service, and MinIO as a choreographed saga: name the states, the events, and the compensating action, and explain what the `DELETING` state protects against.

Deep Dive: Kafka Delivery Guarantees — At-Least-Once, Idempotent Consumers, and Dead Letters

Chapter 10 gave the rule of thumb — Kafka is at-least-once, so make consumers idempotent — and Chapter 11 showed the poison-pill defense. This deep dive reads the producer and consumer settings the project actually runs, most of which it never wrote down, and follows one message through every place it can be duplicated or lost. Then it adds the two things a production consumer needs and the project lacks: a deduplication record and a retry path that ends in a dead-letter topic instead of a log line.

Three questions organize the chapter:

1. **Where does the project sit** on the at-most-once / at-least-once / exactly-once scale — not by intent, but by the defaults it inherited?
2. **Exactly when is a message delivered twice**, and what must a consumer do to make that harmless?
3. **What happens to a message that cannot be processed**, and where should it go instead of nowhere?

34.1 Why this, and what it solves

The project uses Kafka for one job: decoupling a business write from its side effects. A user registers; a mail must go out; a student row must appear in another database. Doing those inline would make registration as slow and as fragile as SMTP and a second Postgres. Doing them through a log makes them *eventually* happen, survive consumer restarts, and be replayable. Chapter 6 covered the sync-vs-async split; what it did not say is that “eventually” has a precise meaning in Kafka, made of five or six settings, and the defaults of those settings are what the project is running today.

DECISION — EVENTS, NOT COMMANDS — AND WHEN A CALL IS RIGHT AFTER ALL

`ts.user.registered` says “this happened”; it does not say “send an e-mail”. Notification-service decides what a registration means for it, and course-

service ignores the topic entirely. That is the event style, and it is why a fourth consumer can be added without touching auth-service. The command style — a `SendEmail` message addressed to one handler — couples the sender to the receiver’s existence and is usually a sign that a synchronous call was wanted. And sometimes it was: when the caller needs the *result* now (does this user exist? is this file scanned?), a Feign call with a circuit breaker is the honest design, and dressing it as request-reply over Kafka only adds two hops and a correlation id. The project’s rule: events for facts other services react to; calls for questions the caller must have answered before it can continue.

34.2 The settings the project never wrote

34.2.1 Producer

Course-service builds its producer by hand in `KafkaProducerConfig`:

```
Map<String, Object> configProps = new HashMap<>();
configProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
configProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
    StringSerializer.class);
configProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
    JsonSerializer.class);
return new DefaultKafkaProducerFactory<>(configProps);
```

Three lines — and everything else is the Kafka 3.9 client default. Since Kafka 3.0 those defaults are the safe ones:

```
acks=all
enable.idempotence=true
retries=2147483647
max.in.flight.requests.per.connection=5
delivery.timeout.ms=120000
max.block.ms=60000
linger.ms=0
```

Auth-service and notification-service use Spring Boot’s auto-configured factory from the YAML, which sets the same two serializers and nothing else, so all three producers behave alike.

- ① The key serializer matters more than it looks. Every publisher in the project passes a key: `event.userId()` in `UserRegisteredPublisher` and `UserActivatedPublisher`, `assignmentId` in `AssignmentCreatedPublisher`. Kafka hashes the key to

choose a partition; equal keys, equal partition, and the broker preserves order within a partition. That is the entire per-user ordering guarantee, and it is one string.

What the defaults mean, one at a time:

- `acks=all`: the leader waits for every in-sync replica before acknowledging. With `replicas: 1` in `kafka-topics.yaml` and a single broker in Compose, “all” is one — durability equals one disk. In production this pairs with `replication.factor=3` and `min.insync.replicas=2`: a write survives one broker dying, and if two die the producer gets `NotEnoughReplicas` rather than a silent single-copy write.
- **Idempotent producer**: each producer gets an id and numbers its messages per partition; a retry after a lost ack is recognized by the broker and not appended twice. This closes the *producer-side* duplicate from Chapter 10. It does nothing for the consumer side.
- retries **unbounded, bounded by** `delivery.timeout.ms`: the producer retries transient errors for up to two minutes, then fails the future. `send()` returned long ago; nobody in the project reads that future.

PITFALL — THE YAML THAT DOES NOTHING

Symptom: you add `spring.kafka.producer.acks: 1` to `course-service.yml` to speed up a load test, and nothing changes. Cause: `KafkaProducerConfig` declares its own `ProducerFactory` bean, which switches off Boot’s auto-configuration for that factory — the YAML producer block is read by nobody in `course-service`. `Auth-service`, with no such bean, *would* honour it. Two services, one YAML convention, two behaviours. Fix: delete the hand-written bean (Boot’s factory does the same thing) or move every property into it; never both.

34.2.2 Consumer

The consumer YAML in `notification-service.yml` and `course-service.yml` names a group, `auto-offset-reset: earliest`, and the error-handling deserializers. Everything about *committing* is again a default, this time Spring Kafka’s:

- `enable.auto.commit=false`. Spring switches the client’s own timer-based commit off and commits itself.
- `AckMode.BATCH`. After the listener has returned for *every* record of one `poll()`, Spring commits the highest offset of the batch. Processing and commit are separated by the batch’s processing time.

- `DefaultErrorHandler` with `FixedBackOff(0, 9)`: a listener that throws is retried nine more times with no delay, then the record is *logged and skipped*. Not sent anywhere. Not counted anywhere the project reads.
- `max.poll.records=500`, `max.poll.interval.ms=300000`: a batch of up to 500 records must be fully processed within five minutes or the consumer is presumed dead.

Put those four together and the project's position is: at-least-once, with duplicates possible on any crash or rebalance, and with silent loss after ten failed attempts. Neither the ordering nor the loss is written in any file you can grep.

34.3 Exactly when a duplicate happens

The timeline that produces one, with notification-group running two instances A and B on a topic with two partitions:

```
t0  A polls partition 0, gets offsets 40..49  (committed: 40)
t1  A processes 40..47: eight e-mails sent
t2  A is killed (rolling update, OOMKilled, SIGKILL)
    -- offset 40 is still the committed position --
t3  group rebalances; partition 0 is assigned to B
t4  B polls from the committed offset: 40
t5  B processes 40..49: ten e-mails sent
    -- users 40..47 receive their activation mail twice --
```

Nothing here is a bug. It is the contract: Kafka promises the offset will not advance past a record you did not finish; the price is that a record you *did* finish, but did not commit, comes back. Every consumer that does anything non-reversible — send mail, insert a row, charge a card — must be written for t5.

Two variants matter for this project. `AckMode.RECORD` commits after each record instead of each batch: fewer duplicates on a crash (one instead of up to 500), more commit round-trips. And the *slow-consumer* variant needs no crash at all: if processing a batch takes longer than `max.poll.interval.ms`, the broker removes the consumer from the group, another instance takes the partition from the last commit, and *both* end up processing the same records — the first one has not noticed it was evicted yet.

PITFALL — ONE ASSIGNMENT, A STORM OF DUPLICATE E-MAILS

Symptom: a teacher creates an assignment in a course with 300 students; some students receive the notification three times; the consumer log shows Member ... has left the group and a rebalance. Cause: `AssignmentCreatedLis-`

tener loops over `enrolledStudentEmails` and calls `javaMailSender.send()` for each, synchronously, on the poll thread. At one second per SMTP round-trip, 300 e-mails is five minutes — exactly `max.poll.interval.ms`. The consumer is evicted mid-loop, the partition is reassigned, and the new owner starts the loop again from the first student. Fix, in order of cost: raise `max.poll.interval.ms` (a patch), publish one `ts.notification.email` record per recipient so each mail is its own unit of work (the right shape), and make the send idempotent per recipient (Section 34.5).

34.4 Pros and cons, candidly

	The project today	Production baseline
Producer durability	<code>acks=all</code> , 1 replica	3 replicas, <code>min.insync=2</code>
Producer duplicates	none (idempotent)	none (idempotent)
Consumer duplicates	possible, unhandled	possible, deduplicated
Failed record	log and skip after 10	retry topics, then DLT
Ordering	per key, 1 partition	per key, N partitions
Slow work	on the poll thread	fanned out, or async
Observability	none on lag	lag alerts, DLT alerts

The project’s honest weak points: silent loss after ten retries; e-mail fan-out on the poll thread; no dedup where the side effect is not naturally idempotent (mail); no lag metric anyone looks at; and single-partition topics, which make ordering trivial and parallelism impossible.

34.5 Enhancing the resolution

34.5.1 An idempotent consumer with a record of what it has seen

`UserActivatedListener` is already idempotent by natural key — a second delivery finds the student and returns. Where there is no natural key, record the event id. The outbox of Chapter 33 puts a unique `eventId` header on every message; the consumer keeps a table of ids it has finished, and writes that row *in the same transaction* as the business change. Flyway V3 `__create__processed__events.sql` in course-service:

```
CREATE TABLE processed_events (
  event_id    UUID PRIMARY KEY,
```

```

consumer    VARCHAR(64) NOT NULL,    -- listener name
processed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- retention: rows older than the topic's retention are useless
CREATE INDEX idx_processed_events_at ON processed_events (processed_at);

```

The listener, rewritten to read the header and to claim the id first:

```

@KafkaListener(topics = "ts.user.activated", groupId = "course-group")
@Transactional                                     ①
public void onUserActivated(
    UserActivatedEvent event,
    @Header("eventId") String eventId) {

    int claimed = jdbc.update("""
        INSERT INTO processed_events (event_id, consumer)
        VALUES (?,::uuid, 'UserActivatedListener')
        ON CONFLICT (event_id) DO NOTHING""", eventId);    ②
    if (claimed == 0) {
        log.info("duplicate event {} ignored", eventId);
        return;                                           ③
    }
    if (!ROLE_STUDENT.equalsIgnoreCase(event.role())) {
        return;
    }
    Long userId = Long.parseLong(event.userId());
    Student student = studentRepository.findById(userId)
        .orElseGet(() -> Student.builder().userId(userId).build());
    student.setFirstName(event.firstName());              ④
    student.setLastName(event.lastName());
    student.setEmail(event.email());
    studentRepository.save(student);
}

```

- ① One database transaction around claim and change. If `save()` fails, the claim rolls back too, and the retry can claim again. If the JVM dies after commit but before the Kafka offset commit, the redelivery hits ③.
- ② `ON CONFLICT DO NOTHING` is the whole dedup algorithm: an atomic insert-if-absent. No read-then-write race between two consumer instances that got the same record after a rebalance.
- ③ Return normally, not by throwing: the offset must advance past a duplicate.
- ④ While here, upsert instead of insert-if-missing, so the same listener also serves the `ts.user.updated` event Chapter 33 adds.

Notification-service has no database, and mail is the one side effect you cannot un-send. Options, honest about cost: accept an occasional duplicate mail (what

the project implicitly does); or give the service a tiny store — Redis SET eventId 1 NX EX 604800 before sending, which is the same claim in one command; or make the mail itself idempotent by including the event id in a Message-ID header, so a mailbox de-duplicates what the consumer could not.

34.5.2 Retry topics and a dead-letter topic

Replace “retry ten times in a millisecond, then forget” with retries that wait, and a destination for what still fails. Spring Kafka’s @RetryableTopic does this with topics instead of threads, so a stuck record does not block the partition behind it:

```
@RetryableTopic(
    attempts = "4",                                ①
    backoff = @Backoff(delay = 2_000, multiplier = 3), ②
    include = { MailSendException.class,
                MailAuthenticationException.class }, ③
    dltStrategy = DltStrategy.FAIL_ON_ERROR)
@KafkaListener(topics = "ts.user.registered",
               groupId = "notification-group")
public void onUserRegistered(UserRegisteredEvent event) {
    emailService.sendActivationEmail(
        event.email(), event.firstName(), event.activationCode());
}

@DltHandler                                       ④
public void onDeadLetter(UserRegisteredEvent event,
    @Header(KafkaHeaders.EXCEPTION_MESSAGE) String reason,
    @Header(KafkaHeaders.ORIGINAL_OFFSET) long offset) {
    log.error("DLT ts.user.registered offset={ } user={ } reason={ }",
        offset, event.userId(), reason);
    meter.counter("kafka.dlt", "topic", "ts.user.registered")
        .increment();                             ⑤
}
```

- ① One try on the main topic, three on retry topics.
- ② Delays of 2, 6, and 18 seconds — about half a minute for MailDev or an SMTP relay to come back. Spring creates the topics `ts.user.registered-retry-2000`, `-retry-6000`, `-retry-18000` and `ts.user.registered-dlt`; under Strimzi (Chapter 25) declare them as `KafkaTopic` objects or leave auto-creation on.
- ③ Retry only what a wait can fix. A deserialization failure or a `NullPointerException` goes straight to the DLT; retrying it three times is theatre.
- ④ The dead-letter handler is a normal listener on the `-dlt` topic. It *must not* throw.
- ⑤ A counter, so that “DLT non-empty” is an alert (Chapter 27’s observability) instead of a discovery.

What to do with a dead letter is an operational decision, not code: an alert fires; someone reads the `EXCEPTION_MESSAGE` header; the fix ships; and the records are *replayed* — copied from the DLT back to the main topic with a small admin tool or Kafka UI — where the idempotent consumer guarantees the ones that did succeed are ignored.

HANDS-ON — SEE THE RETRY LADDER AND THE DEAD LETTER

Break SMTP, register a user, and read the topics:

```
$ docker compose stop maildev
$ curl -s -X POST localhost:8080/api/auth/register -H \
  'Content-Type: application/json' -d '{"email":"dlt@test.io", \
  "password":"Passw0rd!", "firstName":"D", "lastName":"L", \
  "activationMethod":"EMAIL"}'
$ docker compose exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:19092 --list | grep registered
ts.user.registered
ts.user.registered-dlt
ts.user.registered-retry-18000
ts.user.registered-retry-2000
ts.user.registered-retry-6000
```

About 26 seconds after the request, `notification-service` logs `DLT ts.user.registered offset=0 user=... reason=Mail server connection failed`. Start MailDev again, copy the record from the DLT to `ts.user.registered` in Kafka UI (localhost:9090), and the activation mail arrives.

34.5.3 Exactly-once, and what it does not cover

Kafka’s transactions give exactly-once for one specific shape: consume from topic A, produce to topic B, commit both offsets and output atomically (isolation.level=read_committed on the reader). Stream processors need it; the project has no such stage. Transactions do *not* span Kafka and Postgres, or Kafka and SMTP — the moment the side effect leaves the broker, you are back to at-least-once plus idempotency. An interviewer who asks “can you get exactly-once?” wants exactly that distinction: yes inside Kafka, no across systems, and the outbox plus a dedup table is how you get the effect across systems anyway.

34.6 What breaks at 3 a.m.

- **notification-service down for four hours.** Nothing is lost: records accumulate on the topic, retention is 168 hours in Compose and Strimzi’s default

seven days. Consumer lag — the gap between the partition’s last offset and the group’s committed offset — grows to a few thousand. On restart the service drains them at SMTP speed; every activation mail is late, none is missing. What *would* lose them: an outage longer than retention, or a re-deploy under a *new* group id, which starts from earliest and resends every mail in seven days of history.

- **The broker is down.** Producers block for `max.block.ms` on their first meta-data fetch, then throw — the sixty-second registration failure of Chapter 33. Producers that were already connected buffer up to `buffer.memory` (32 MB) and retry within `delivery.timeout.ms`; after two minutes the future fails and, since nobody reads it, the event is gone. Consumers simply poll nothing. The outbox converts all of this into a growing `attempts` column.
- **A replay from yesterday.** An operator resets `course-group` to a timestamp with `kafka-consumer-groups.sh --reset-offsets --to-datetime`. Every event since then is redelivered; with the `processed_events` claim each one costs one failed insert. Without it, mail goes out twice and student rows are attempted twice (saved only by the `UNIQUE(user_id)` constraint).
- **A hot key.** Every event for one very large course lands on one partition; adding partitions does not help because the key chooses the partition. Re-key — by recipient instead of by assignment — or accept that one key’s throughput is one consumer’s throughput.

34.7 Outside the project

Every managed Kafka — Confluent Cloud, AWS MSK, Aiven — ships `acks=all`, three replicas, and `min.insync.replicas=2` as the starting point, and sells lag dashboards as the first thing you look at. Retry topics and DLTs are so standard that Confluent’s Parallel Consumer, Uber’s consumer proxy, and Spring’s `@RetryableTopic` all implement the same ladder; the differences are in how they preserve per-key ordering across retries (Spring does not, by default — a retried record can be processed after a later one for the same key, which is fine for mail and not for ledger entries). RabbitMQ and SQS give you per-message acknowledgement and a dead-letter queue as a broker feature rather than a library pattern; what they cannot give you is replay, which is the reason a log was chosen in the first place.

34.8 Questions from real interviews

Q: How do you increase the number of messages Kafka can handle per unit of time?

I start with the consumer, because that is where the project's limit is. Partitions are the unit of parallelism: a group cannot have more active consumers than the topic has partitions, and every topic in `kafka-topics.yaml` has one. So the first change is partition count — 6 or 12, chosen with headroom because partitions can be added but never removed, and adding them later changes the key-to-partition hash, which breaks per-key ordering for in-flight data. Then match consumers to partitions: `spring.kafka.listener.concurrency=3` per pod times two replicas gives six threads over six partitions. Next, get slow work off the poll thread: one `AssignmentCreated` that sends 300 e-mails synchronously is a five-minute batch that trips `max.poll.interval.ms`; publish one record per recipient instead, so throughput scales with partitions. On the producer, batching: `linger.ms=5-20` and `batch.size=64-256 KB` let the client send hundreds of records per request, and `compression.type=lz4` or `zstd` cuts bytes on the wire by three to five times for JSON. `acks=1` roughly doubles producer throughput at the cost of losing a write when the leader dies before replicating — I keep `acks=all` for anything I cannot regenerate. On the consumer, `max.poll.records` and `fetch.min.bytes` trade latency for fewer round-trips, bounded by what fits in `max.poll.interval.ms`. And I watch for a hot key: if one partition carries 80 % of the load because of a bad key choice, none of the above helps and the fix is a different key.

Q: Consumer lag is growing. How do you diagnose it?

First I establish which of three things it is: the consumer is slow, the consumer is dead, or the producer got faster. `kafka-consumer-groups.sh -describe -group notification-group` shows lag per partition and which member owns each; lag on *all* partitions with members present means slow processing; lag on *some* with missing members means an instance is down or stuck in rebalance; lag that appeared at a clear time with steady consumption means a producer burst. For slow processing I look at the listener's per-record time — in this project SMTP latency — and at rebalance churn in the logs (`Member ... has left`), which means `max.poll.interval.ms` is being hit and the group is thrashing. Fixes follow the diagnosis: more partitions and concurrency, move the slow call off the thread, or, if the burst is legitimate, let it drain and alert only when lag is older than the SLO in seconds, not larger than a number of records. The metric to graph is lag in *time*, not count.

Q: A message was processed twice. Why, and how do you make that safe?

Because Kafka commits offsets after processing, and the process died, or was evicted for being slow, between the two. That is at-least-once working as designed. Making it safe means every consumer either has a natural idempotency key — `UserActivatedListener`’s `findById` — or claims a unique event id in the same transaction as its side effect: `INSERT ... ON CONFLICT DO NOTHING` into `processed_events`, `return` if zero rows. The id has to come from the producer, minted once and carried through retries, which is what the outbox row gives you. For side effects with no transaction — e-mail — I put the id in the message’s `Message-ID` or claim it in Redis with `SET NX`. What I do not do is switch to `Ack-Mode.RECORD` and call it fixed; that narrows the window from a batch to one record and leaves the bug in place.

Q: How do you guarantee ordering for one user?

Key every event about that user by the user id, so the hash puts them on one partition, and keep one consumer thread per partition. That is the whole guarantee, and it has three ways to break: an idempotent producer disabled with `max.in.flight` above one, which lets a retried batch land after a later one; a partition count change, which re-hashes keys; and retry topics, which re-inject a failed record behind its successors. For the project, `userId` keys on `ts.user.*` already do this. If the business needs a stronger statement — “the update is applied after the create, always” — I add a version number to the event and have the consumer reject anything older than what it has, which is what turns ordering from a broker property into a consumer invariant.

Q: A broker dies. What does the producer do?

With three brokers and `replication.factor=3`, the controller elects a new leader from the in-sync replicas for each affected partition within seconds; the producer gets `NotLeaderOrFollower`, refreshes metadata, and retries — invisible to the application because `retries` is unbounded and `delivery.timeout.ms` is two minutes. If the dead broker took the ISR below `min.insync.replicas`, the producer gets `NotEnoughReplicas` and keeps retrying; with `acks=all` that is the correct refusal to write a single copy. On the project’s single-broker Compose stack there is no failover: sends buffer, the future fails after two minutes, and the outbox row is what saves the event. The one thing I check in a review is whether anyone reads the future — in this project nobody does, so a producer-side failure is silent.

Q: How do you replay a topic from yesterday?

Stop the consumer group, reset its offsets, start it again: `kafka-consumer-groups.sh --reset-offsets --group course-group --topic ts.user.activated --to-datetime 2026-09-06T00:00:00.000 --execute`. The group must be inactive or the command refuses. Before running it I answer two questions: is the consumer idempotent for everything since that time — because every record comes back — and is the data still there, which depends on retention (168h here) and on whether the topic is compacted. For a single record I replay from the DLT instead of resetting the whole group. And I never replay into a group that sends e-mail without first confirming the dedup path, because “we resent 40 000 activation mails” is the incident that follows a confident reset.

CHECK YOURSELF (answers: Appendix A)

1. Name the producer settings that make the project’s producers idempotent and durable, and explain why `acks=all` on the Compose stack is weaker than it sounds.
2. Draw (in words) the timeline in which `notification-group` sends the same activation mail twice without any crash, and name the setting whose value defines the trigger.
3. Why must the `processed_events` insert and the business write share one transaction? Describe the interleaving that goes wrong if the insert commits first on its own.
4. In the `@RetryableTopic` configuration, explain why `include` lists only mail exceptions and what happens to a `NullPointerException`.
5. `Course-service` declares its own `ProducerFactory` bean. State the consequence for the `spring.kafka.producer` block in its YAML, and the one-line fix.
6. `Notification-service` has been down for nine days. State what happens on restart, which setting decides it, and what would be different if it restarted with `group-id: notification-v2`.

Deep Dive: Files at Scale — Presigned URLs, Endpoints, and Why Bytes Don't Belong in Postgres

Chapter 9 introduced MinIO as “S3 at home” and explained the two-endpoints bug. That was enough to make submissions work. This chapter asks what the design costs once the files are many, large, and important — the moment a student’s graded essay is the only copy anywhere. The project’s own code is the specimen: one `S3Config`, one `S3StorageService`, one column called `file_object_key`.

Three questions organize the chapter:

1. **Where should bytes live** — in the database, on a disk, or in an object store — and what exactly does each choice cost?
2. **Who should move the bytes** — the browser through the service, or the browser straight to storage with a signed permission slip?
3. **What happens when the two stores disagree** — a row that names an object that does not exist, or an object no row names?

35.1 Why object storage, and what it solves

A submission has two halves: facts about it (who, when, which attempt, what score) and the file itself. The facts are relational and tiny; the file is opaque and potentially 20 MB. Three homes are possible for the file.

In Postgres, as bytea. One store, one transaction, one backup. It is also where the trouble starts. Every row read pulls the blob through shared buffers; `pg_dump` grows by the size of every essay ever submitted; each update rewrites the whole value and leaves a dead tuple for autovacuum; the write-ahead log carries every byte twice (WAL, then the page); replication streams it to every standby; and a download must be materialized in the JVM before it reaches the socket. Postgres will do it, and up to a few gigabytes nobody notices. Past that, every operational task — backup, restore, replication lag, vacuum — is dominated by data the database adds no value to.

On a volume mounted into the service. Fast and simple until the service has two replicas on two nodes, which is the first thing Kubernetes wants to do. A

ReadWriteOnce claim (Chapter 19) binds to one node; ReadWriteMany needs NFS or a storage driver, and the service now owns file layout, permissions, cleanup, and its own backup. The disk is also inside the failure domain of the pod.

In an object store. Flat namespace, HTTP API, content-addressed durability, and — decisively — the ability to hand a client a URL and step out of the data path. Backups become a bucket replication rule. Serving becomes a CDN's job. The database keeps a 500-character key.

DECISION — A KEY IN POSTGRES, BYTES IN MINIO

The project stores metadata in the `submission` table and the file under a key in the `ts-submissions` bucket. The row references the object by name; nothing references the row from the object. The alternatives lose on the axis that matters for a school: submissions are written once, read a few times, and kept for years — the profile object storage is built for and databases are worst at. The decision would reverse only if files were tiny (a few KB), needed transactional atomicity with the row, and were read on every row access — an avatar thumbnail, say. Even then the honest answer is usually “still S3, plus a cache”.

35.2 Reading the project's code

35.2.1 Two clients, two endpoints

The gateway to everything is `S3Config`. Read it with the `compose` environment in mind (`APP_S3_ENDPOINT=http://minio:9000`, `APP_S3_PUBLIC_ENDPOINT=http://localhost:9000`):

```
@Bean
public S3Client s3Client() {
    return S3Client.builder()
        .endpointOverride(URI.create(endpoint))           ①
        .region(Region.of(region))
        .credentialsProvider(credentialsProvider())
        .serviceConfiguration(S3Configuration.builder()
            .pathStyleAccessEnabled(true).build())        ②
        .build();
}

@Bean
public S3Presigner s3Presigner() {
    return S3Presigner.builder()
        .endpointOverride(URI.create(publicEndpoint))    ③
}
```

```
.region(Region.of(region))
.credentialsProvider(credentialsProvider())
.serviceConfiguration(S3Configuration.builder()
    .pathStyleAccessEnabled(true).build())
.build();
}
```

- ① The client that moves bytes talks to the *internal* name. Inside Compose or the cluster, minio is a DNS name on the private network (Chapter 18); from a browser it is nothing.
- ② Path-style addressing: the bucket goes in the URL path, so the request is `/ts-submissions/key` on host `minio:9000`. AWS's default is virtual-host style, where the bucket is a DNS subdomain — host `ts-submissions.s3.amazonaws.com`, path `/key`. MinIO can do that too, but only when it knows its own domain (`MINIO_DOMAIN`) and wildcard DNS exists for it; on a home network neither is true, so the path form is the only one that resolves.
- ③ The presigner never sends a request. It only computes signatures — and it must compute them for the host the *browser* will contact. This is a separate bean for that reason alone. Section 35.3 explains why the host cannot be swapped afterwards.

35.2.2 The key, the upload, the download

`S3StorageService` is short enough to read whole; the important lines are these:

```
@PostConstruct
void ensureBucketExists() {
    try {
        s3Client.headBucket(HeadBucketRequest.builder()
            .bucket(bucket).build());
    } catch (NoSuchBucketException e) { ①
        s3Client.createBucket(CreateBucketRequest.builder()
            .bucket(bucket).build());
    }
}

public String upload(MultipartFile file, Long assignmentId,
    Long studentId) {
    String key = "submissions/%d/%d/%s-%s".formatted(
        assignmentId, studentId, UUID.randomUUID(), original); ②
    s3Client.putObject(PutObjectRequest.builder()
        .bucket(bucket).key(key)
        .contentType(file.getContentType())
        .contentLength(file.getSize()).build(),
        RequestBody.fromInputStream(file.getInputStream()),
```

```

        file.getSize());
    }
    return key;
}

public String presignedDownloadUrl(String key) {
    GetObjectPresignRequest presignRequest = GetObjectPresignRequest
        .builder()
        .signatureDuration(Duration.ofMinutes(presignedExpiryMinutes))
        .getObjectRequest(GetObjectRequest.builder()
            .bucket(bucket).key(key).build())
        .build();
    return s3Presigner.presignGetObject(presignRequest).url().toString();
}

```

- ① Only `NoSuchBucketException` is caught. A connection refused — MinIO not up yet — is an `SdkClientException`, which escapes, fails bean creation, and takes the whole service down. The Kubernetes manifest for MinIO says so in its first comment: `course-service` will `CrashLoopBackOff` until MinIO answers. Compose hides this with `depends_on: service_healthy`.
- ② The key is the whole data model of an object store. S3 has no directories; the slashes are characters in a string. But *prefixes* are what you can list by, so `submissions/{assignmentId}/{studentId}/` means “every attempt by this student for this assignment” is one `ListObjectsV2` call with a prefix. The UUID guarantees a fresh key per attempt, so nothing is ever overwritten and versioning is not needed for safety. The original filename is appended for human readability in the console; the database keeps it separately in `file_name` because a key is not a good place for spaces, Unicode, or ...
- ③ The bytes flow *through* this JVM: Spring’s multipart support has already parsed them (spooled to a temp file above a small threshold), and now they are streamed to MinIO. The request thread is occupied for the whole transfer, twice.

And the seam with the relational half, in `SubmissionService`:

```

@Transactional
public SubmissionResponse submit(...) {
    ...
    if (hasFile) {
        String key = s3StorageService.upload(file, assignmentId,
            student.getId());
        submission.setFileObjectKey(key);
        submission.setFileName(file.getOriginalFilename());
    }
    submission = submissionRepository.save(submission);
    return toResponse(submission);
}

```

```
}
```

- ① An HTTP call to another system, inside a database transaction. It is not, and cannot be, part of that transaction.
- ② If this insert fails — the unique constraint on (assignment_id, student_id, attempt_number) loses a race between two tabs, or Postgres is briefly unreachable — the transaction rolls back and the object uploaded at ① stays in the bucket, named by a key no row will ever record. Chapter 11 called this the dual-write problem when the second system was Kafka; it is the same problem with a different second system.

35.3 Signature Version 4, and why the host is welded in

A presigned URL is an ordinary S3 request with the authentication moved from headers into the query string. The project's download link looks like this (wrapped):

```
http://localhost:9000/ts-submissions/submissions/7/3/2f9c...-essay.pdf
?X-Amz-Algorithm=AWS4-HMAC-SHA256
&X-Amz-Credential=minioadmin/20260907/us-east-1/s3/aws4_request
&X-Amz-Date=20260907T091500Z
&X-Amz-Expires=900
&X-Amz-SignedHeaders=host
&X-Amz-Signature=3c1f...e9a0
```

The signature is computed in two stages. First a *signing key* is derived from the secret through a chain of HMACs over the date, region, service name and the literal `aws4_request`, so the secret itself never touches a request. Then that key signs a *string to sign* that contains the SHA-256 of a *canonical request*: the HTTP method, the canonical URI (bucket and key, path-style), the sorted query string, the signed headers, and the payload hash. Look at `X-Amz-SignedHeaders=host`: the Host header is always signed. Change the host — minio to localhost, or localhost to 127.0.0.1 — and the canonical request changes, the hash changes, the signature no longer matches, and MinIO answers 403 `SignatureDoesNotMatch`. There is no way to sign against the internal name and rewrite the link for the outside world. Whoever signs must know the exact origin the browser will use.

HANDS-ON — BREAK A SIGNATURE ON PURPOSE

Log in, submit an assignment with a file, and copy the `downloadUrl` the API returns. It works as issued:


```
$ curl -s -o /dev/null -w "%{http_code}\n" "$URL"
200
```

Now change nothing but the host, to an address that reaches the very same MinIO container:

```
$ curl -s "${URL}/localhost/127.0.0.1" | head -c 160
<?xml version="1.0" encoding="UTF-8"?><Error>
<Code>SignatureDoesNotMatch</Code><Message>The request signature we
calculated does not match the signature you provided.
```

Same server, same object, same credentials — and a different Host header is enough. That is the whole reason public-endpoint exists.

PITFALL — DOWNLOAD LINKS WORK IN COMPOSE AND FAIL IN THE CLUSTER

Symptom: on the cluster, clicking a submission's file opens `http://localhost:9000/...` and the browser shows connection refused. **Cause:** the course-service Deployment sets `APP_S3_ENDPOINT` to `http://minio:9000` but never sets `APP_S3_PUBLIC_ENDPOINT`, so the config-server default `http://localhost:9000` is signed into every link — an address that meant “the developer's laptop” and now means “the user's laptop”. **Fix** in two parts: expose MinIO's S3 port through an Ingress rule on its own host (path-style signatures include the path, so the Ingress must not add a prefix), and set the public endpoint to that origin:

```
- { name: APP_S3_PUBLIC_ENDPOINT,
  value: "https://files.ts-server.tailbfe002.ts.net" }
```

Until a second hostname exists, the honest interim is to stream downloads through course-service (GetObject into the response body) so the browser never needs to reach MinIO at all — slower, but correct.

35.4 Who moves the bytes

Today's upload path is: browser → Traefik → api-gateway → course-service → MinIO. Every hop sees every byte. The service enforces `max-file-size: 20MB` and `max-request-size: 25MB` in its multipart settings, so the worst case is bounded; but each upload holds a Tomcat thread and a temp file for its whole duration, the reactive gateway (Chapter 4) forwards the body chunk by chunk but still spends its event loop on it, and any ingress-level body limit or idle timeout applies in

front of all of them. Under a deadline — 200 students uploading at 23:59 — the service's thread pool is the bottleneck for work it does not need to do.

The alternative is the pattern Chapter 9 mentioned for downloads, applied to uploads: the service signs a *PUT* for a key it chose, the browser uploads directly to MinIO, and only then does the service record the row. The service spends microseconds per file and never sees the bytes. What it gives up: it cannot inspect the content before it lands (virus scan, type sniffing) and it must learn, from somewhere, that the upload actually completed. Section 35.8 builds this properly.

Beyond about 100 MB a single *PUT* is the wrong tool on any network: *multipart upload* splits the object into parts of at least 5 MB, uploads them in parallel, resumes after a failure by re-sending one part, and assembles them server-side in a final call. The SDK's `S3TransferManager` does this automatically above a threshold. The operational tail: abandoned uploads leave invisible parts that are billed and counted — every bucket should carry a lifecycle rule that aborts incomplete multipart uploads after a few days.

35.5 Pros and cons, candidly

Strength	Where the project pays for it
Database stays small; backup is metadata only	Two stores to back up, and only one of them transactionally
Standard API: MinIO today, S3 tomorrow with two env vars	Two endpoints to keep straight, and the cluster gets one wrong
Presigned downloads take the service out of the data path	Uploads still go through the service, thread-per-file
Keys give free “directories” by prefix	Key design is a one-way door; renaming means copying
Durability is storage's problem	On this hardware it is one container and one 5 GB claim
Credentials are two strings	They are the MinIO <i>root</i> credentials, in every course-service pod

The last row deserves emphasis. `minioadmin` can create buckets, delete buckets, and read every object in every bucket. The service needs `PutObject`, `GetObject` and `HeadBucket` on one bucket. Least privilege here is a MinIO user with a policy scoped to `arn:aws:s3:::ts-submissions/*`; on AWS it is an IAM role attached to the

pod's ServiceAccount (IRSA) so that no static key exists at all. An interviewer who asks "how does your service authenticate to S3?" is listening for whether you say "an IAM role" or "a key in an env var" — and, if the latter, whether you know it is the weaker answer.

35.6 What breaks

PITFALL — THE SERVICE THAT WILL NOT START WITHOUT ITS BUCKET

Symptom: course-service in CrashLoopBackOff on a fresh cluster; logs end in Error creating bean with name 's3StorageService' with a Connection refused several causes down. Cause: the @PostConstruct bucket check treats any failure other than "bucket missing" as fatal, and MinIO's pod is still pulling its image. The kubelet's restart back-off eventually lands on a moment when MinIO is ready, so it self-heals — slowly. Fix: catch SdkClientException too, log a warning, and let the readiness probe (Chapter 17) keep traffic away until the first successful headBucket; a storage dependency should degrade the file feature, not the service.

PITFALL — ORPHANS

Symptom: the bucket holds more objects than the submission table has non-null keys; nobody notices for a year; the 5 GB claim fills. Cause: the dual write in submit. Upload first, insert second: a failed insert orphans an object. Insert first, upload second: a failed upload leaves a row pointing at nothing, and the student sees "file attached" with a dead link. Neither order is safe on its own. The standard resolution is to make one side idempotent and the other side reconcilable: insert the row with status PENDING_FILE and the chosen key, upload, then flip the status to SUBMITTED in a second transaction; a nightly job deletes objects under keys whose row is still PENDING_FILE after an hour, and lists the prefix for keys no row names at all. Section 35.8 folds this into the new flow, where it comes almost free.

35.7 What breaks at 3 a.m.

MinIO's pod is gone — node reboot, claim not yet reattached. Walk the endpoints:

- **Text and link submissions** still work: upload is only called when a file is present, and save touches only Postgres.

- **File submissions** fail with a 500 after the SDK's retry budget — several seconds of a held thread each. Under load these threads accumulate and starve the text submissions that would have worked. This is the Chapter 17 argument for a bulkhead: a separate, bounded executor for storage calls.
- **Listing submissions** works and even returns download links. Presigning is pure computation; no request is made. The links fail only when clicked, which is the correct failure — the metadata is true, the bytes are temporarily unavailable.
- **A course-service restart** during the outage does not come back up, for the reason in the pitfall above. An outage of the file store has become an outage of grading, enrollment, and everything else the service does.

Now the other way: Postgres is down and MinIO is fine. Everything fails, including uploads that would have succeeded at the storage layer — and each one that reached ① in `submit` before failing at ② left an orphan behind. A 3 a.m. database blip produces a small pile of unowned objects, which is the concrete reason the reconciliation job exists.

35.8 Enhancing the resolution: presigned PUT

The upgrade moves the bytes off the service and closes the orphan window at the same time. Three calls replace one:

1. `POST /assignments/{id}/submissions/upload-url` — the service checks the student may submit, chooses the key, signs a PUT, and inserts a `PENDING_FILE` row.
2. The browser PUTs the file to MinIO with that URL.
3. `POST /assignments/{id}/submissions` with the key — the service confirms the object exists with a HEAD, then completes the row.

The storage service gains two methods. Both are complete:

```
/** Sign a PUT for a key we chose; the browser must send exactly this
 * content type, because it is part of the signature. */
public String presignedUploadUrl(String key, String contentType,
                                long contentLength) {
    PutObjectRequest put = PutObjectRequest.builder()
        .bucket(bucket)
        .key(key)
        .contentType(contentType)
        .contentLength(contentLength)           ①
        .build();
    PutObjectPresignRequest req = PutObjectPresignRequest.builder()
        .signatureDuration(Duration.ofMinutes(10))
}
```

```

        .putObjectRequest(put)
        .build();
    return s3Presigner.presignPutObject(req).url().toString();
}

/** Confirm an object exists and report its size; empty if missing. */
public Optional<Long> objectSize(String key) {
    try {
        HeadObjectResponse head = s3Client.headObject(
            HeadObjectRequest.builder().bucket(bucket).key(key).build());
        return Optional.of(head.contentLength());
    } catch (NoSuchKeyException e) {
        return Optional.empty();
    }
}

```

- ① **Signing Content-Length and Content-Type turns them into a contract:** a browser that sends a different size or type gets 403. That is how the 20 MB limit survives the move out of the service.
- ② **HEAD costs one round trip and no bytes.** It is the cheapest possible proof that step 2 happened.

The submission service splits its old method in two. The key prefix is built from server-side facts, so a client cannot claim someone else's upload:

```

public UploadTicket prepareUpload(Long assignmentId, Long studentUserId,
    String fileName, String contentType,
    long size) {
    Assignment assignment = requireOpenAssignment(assignmentId);
    Student student = requireEnrolled(assignment, studentUserId);
    if (size > MAX_FILE_BYTES) {
        throw new ApiException(413, "File exceeds 20 MB");
    }
    String key = "submissions/%d/%d/%s-%s".formatted(
        assignmentId, student.getId(), UUID.randomUUID(),
        safeName(fileName));
    Submission pending = Submission.builder()
        .assignment(assignment).student(student)
        .attemptNumber(nextAttempt(assignmentId, student.getId()))
        .fileObjectKey(key).fileName(fileName)
        .status("PENDING_FILE")
        .build();
    submissionRepository.save(pending);
    String url = s3StorageService.presignedUploadUrl(key, contentType,
        size);
    return new UploadTicket(pending.getId(), key, url);
}

```

```
public SubmissionResponse completeUpload(Long submissionId,
                                         Long studentUserId) {
    Submission s = findOrThrow(submissionId);
    if (!s.getStudent().getUserId().equals(studentUserId)
        || !"PENDING_FILE".equals(s.getStatus())) {
        throw new ApiException(409, "Nothing to complete");
    }
    long size = s3StorageService.objectSize(s.getFileObjectKey())
        .orElseThrow(() -> new ApiException(400,
            "Upload not found; request a new upload URL")); ③
    if (size > MAX_FILE_BYTES) {
        throw new ApiException(413, "File exceeds 20 MB");
    }
    s.setStatus("SUBMITTED");
    return toResponse(submissionRepository.save(s));
}
```

- ① `safeName` strips path separators and non-ASCII; the display name stays in `file_name`.
- ② The row exists *before* the bytes. A crash anywhere after this point leaves a `PENDING_FILE` row that the cleanup job can find, instead of an object nobody can find.
- ③ The only way to reach `SUBMITTED` is through a successful `HEAD`. The row never claims a file that is not there.

The cleanup job is now a query, not a bucket scan:

```
@Scheduled(cron = "0 15 * * *") // hourly, at :15
public void reapAbandonedUploads() {
    List<Submission> stale = submissionRepository
        .findByStatusAndSubmittedAtBefore("PENDING_FILE",
            LocalDateTime.now().minusHours(1));
    for (Submission s : stale) {
        s3StorageService.deleteQuietly(s.getFileObjectKey()); // idempotent
        submissionRepository.delete(s);
    }
}
```

MinIO must allow the browser's cross-origin `PUT`. The bucket's CORS rule is set once with the MinIO client:

```
$ mc cors set local/ts-submissions cors.json
# cors.json: allow PUT from the frontend origin, expose ETag
{ "CORSRules": [ { "AllowedOrigins": ["http://localhost:3000"],
    "AllowedMethods": ["PUT", "GET"],
    "AllowedHeaders": ["Content-Type", "Content-Length"],
```

```
"ExposeHeaders": ["ETag"], "MaxAgeSeconds": 3600 } ] }
```

And the frontend side is a plain fetch; no axios instance, no gateway, no JWT — the URL *is* the credential:

```
const ticket = await api.post(
  `/assignments/${id}/submissions/upload-url`,
  { fileName: file.name, contentType: file.type, size: file.size });

const put = await fetch(ticket.url, {
  method: 'PUT',
  headers: { 'Content-Type': file.type }, // must match the signature
  body: file,
});
if (!put.ok) throw new Error(`upload failed: ${put.status}`);

await api.post(`/submissions/${ticket.submissionId}/complete`);
```

DECISION — PRESIGNED PUT VERSUS PROXIED UPLOAD

Proxy when the service must see the bytes before they are stored — virus scanning, image re-encoding, content-type sniffing — or when the storage endpoint cannot be reached from the client at all. Presign when files are large or many and the service adds nothing to the transfer: the thread pool, the ingress body limit, and the double network hop all disappear. The project's case is the second, with one caveat that decides the order of work: presigned uploads require a browser-reachable MinIO origin, which the cluster does not have yet. The pitfall above and this enhancement share a prerequisite — a public files hostname — and should be scheduled together.

35.9 Outside the project

On AWS the same code runs against S3 with the endpoint override removed and static keys replaced by an IAM role; CloudFront signed URLs play the presigner's role with a CDN in front, and S3 *POST policies* are the older, browser-form-friendly cousin of presigned PUT, with a content-length-range condition built in. Google Cloud Storage and Azure Blob Storage have the same three ideas under other names — signed URLs, SAS tokens, resumable uploads. Production MinIO runs as a distributed set with erasure coding across drives and nodes; a single container with one claim is a development shape, and the SDK cannot tell the difference, which is the point of the standard. The interview version of this chapter is one question — “a user uploads a 2GB video; walk me through it” —

and the senior answer names presigned or multipart upload, a size and type contract in the signature, a confirmation step, an orphan reaper, and a lifecycle rule for incomplete parts, in about that order.

35.10 Questions from real interviews

Q: Users started uploading 1 GB videos and the service runs out of memory. Fix it.

First establish that the bytes are the problem, not the metadata: a heap dump during an upload shows multipart buffers or a `ByteArrayOutputStream`, and the pod's memory limit (Chapter 17) is what kills it. The short-term fix is to make sure Spring spools to disk (`file-size-threshold` small, a temp volume with room) and stream to storage with a known `Content-Length`, as `S3StorageService.upload` already does — that gets you to hundreds of MB without heap growth, but still one thread and one temp file per upload. The real fix is to take the service out of the path: presigned PUT for anything under about 100 MB, and multipart upload with `S3TransferManager` or browser-side parts above it, with a lifecycle rule aborting incomplete uploads after seven days. Then the service's memory is independent of file size by construction, and `max-file-size` becomes a signed `Content-Length` instead of a servlet setting.

Q: We need to serve ten thousand downloads per second. How, without touching the service?

Presigned GET already means the service does no I/O per download, only a signature — microseconds, no network. Ten thousand per second is therefore a storage and edge question. Object stores scale reads horizontally by key prefix distribution; a CDN in front (CloudFront with signed URLs, or any CDN with origin-pull to the bucket) serves repeated objects from cache and absorbs the fan-out. Keep the signing cheap by caching nothing — signatures are stateless — and set the expiry to match the CDN's cache window so a cached object does not outlive its authorization by much. The trap is signing on every list call: `toResponse` in the project presigns for every row of every listing, which is fine at 15 minutes' expiry but means a listing of a thousand submissions computes a thousand HMAC chains. Sign lazily, on a dedicated "get link" endpoint, once volume justifies it.

Q: An object exists in the bucket but the row is missing, or the row exists and the object does not. How do you reconcile?

Say first that both are expected outcomes of a dual write and that the fix is design, not a script. Design: write the row in a PENDING state with the key before the upload, confirm with HEAD before marking it complete, and never let a row claim a file it has not verified. Reconciliation then has two cheap halves. Rows stuck in PENDING older than the presign expiry are deleted along with their objects — a query, not a scan. Objects with no row are found by listing the prefix and anti-joining against the table, which you can afford nightly because the prefix is per assignment; on AWS an S3 Inventory report replaces the listing. Whatever you do, make the object delete idempotent so the job can be rerun.

Q: How do you secure a presigned URL?

Four dials, and a senior answer names all of them. *Scope*: sign one method on one key; a GET URL cannot be used to PUT, and a PUT URL cannot be used on a different key because the canonical URI is signed. *Expiry*: minutes, not days; the project uses 15 for downloads and 10 would do for uploads — long enough for a slow connection, short enough that a leaked link in a chat log is dead by the time it is noticed. *Content contract*: sign Content-Type and Content-Length on uploads so the client cannot change what it declared. *Identity of the signer*: the credentials behind the signature should be a least-privilege user on that one bucket, so even a compromised signer cannot list or delete. And say the thing people forget: a presigned URL is a bearer credential — log it nowhere, return it over HTTPS only, and never put it in a redirect that a proxy might cache.

Q: Migrate from MinIO to AWS S3 with zero downtime.

Because the code speaks the S3 API, the migration is a data move plus a config change, and the trick is ordering. Set up bucket replication or run `mc mirror` — watch from MinIO to the S3 bucket until the lag is seconds. Switch *writes* first by changing `APP_S3_ENDPOINT` and credentials (an IAM role, now) on a rolling restart — new objects land in S3, mirroring keeps copying anything still landing in MinIO from pods not yet restarted. Switch the *public endpoint* last, after a final mirror pass, so download links are only ever signed for a host that has the object. Keys do not change, so the database needs no migration. Keep MinIO read-only for a week as a fallback, then decommission. The thing that breaks is addressing: virtual-host style becomes available, path-style keeps working, and any code that concatenated URLs by hand instead of asking the SDK is where you will find the bug.

Q: Storage cost is growing every month. What do you do?

Measure by prefix first — which assignments, which years. Then use the store's own tools rather than application logic: a lifecycle rule that transitions objects older than a term to an infrequent-access tier, and older than the retention policy to archive or deletion; an abort rule for incomplete multipart uploads, which are invisible and billed; and versioning off, or with expiry, unless a compliance need says otherwise. In the application, stop storing what you can regenerate (thumbnails, transcodes) and deduplicate by content hash if students upload the same template. On this project the numbers are small — 250 uploads of 20 MB fill the 5 GB claim — but the same rule set is what keeps an S3 bill flat at a thousand times the size.

CHECK YOURSELF (answers: Appendix A)

1. Two beans, `S3Client` and `S3Presigner`, share credentials and region and differ in one line. Name the line, and explain what goes wrong if the presigner used the internal endpoint.
2. A colleague suggests keeping one endpoint and having the gateway rewrite `minio:9000` to the public host in the JSON response. Using the structure of a Version 4 signature, explain why the rewritten link returns 403.
3. In `SubmissionService.submit`, the upload happens inside a `@Transactional` method. State what the transaction does and does not cover, and describe the artifact left behind by a failed save.
4. The key layout is `submissions/{assignment}/{student}/{uuid}-{name}`. Give one operation the layout makes cheap and one it makes expensive, and say why the UUID makes bucket versioning unnecessary for safety.
5. MinIO goes down at 3 a.m. Which of these still succeed: submitting a text answer, listing submissions, opening a download link, restarting course-service? Explain each in one clause.
6. In the presigned-PUT flow, an attacker obtains a valid upload URL from their own session and tries to attach that object to another student's submission. Identify the two checks that stop them.

Deep Dive: The Dictionary on MongoDB — Document Modeling, Regex Search, and Honest Trade-offs

Chapter 8 made the case for the dictionary as the project’s document service: an entry is read as a unit, written as a unit, and naturally nested. That case is sound. This chapter tests it the way a skeptical senior engineer would — by reading the queries the service actually runs, asking what each one costs at ten thousand words per user, and admitting where a relational database would have done as well. The goal is not to reverse the decision but to let you defend either side of it in an interview, with the code in front of you.

Three questions organize the chapter:

1. **What does the document model buy here**, and what would Postgres with jsonb have cost instead?
2. **How does the search query actually execute** — which part touches an index, which part scans, and where does the page get built?
3. **What breaks when a dictionary gets big** — and which of those breaks are Mongo’s fault, and which are the code’s?

36.1 Why MongoDB here, and what it solves

Open WordDefinition. It is the whole schema of the service:

```
@Document(collection = "word_definitions")
public class WordDefinition {
    @Id private String id;
    private String word;
    private String createdByUserId;
    private String meaning;
    private String usage;
    private String notes;
    private List<String> examples;
    private List<String> tags;
    /** Image attachments stored inline as base64 data URLs. */
    private List<String> images;
    private Map<String, Object> extra;
```

①

②

③

④

```
@CreatedDate private Instant createdAt;  
@LastModifiedDate private Instant updatedAt;  
}
```

- ① Every query in the service is scoped by this field. It is the tenant fence and, as we will see, the only thing the index helps with.
- ② Two arrays embedded in the document. In Postgres these are two child tables or two `text[]` columns; here they are part of the entry and cost nothing to fetch with it.
- ③ Images as inline base64 strings. This is Chapter 35’s “bytes in the database” in document clothing, with a hard ceiling attached: a BSON document may not exceed 16 MB. A dozen phone photos and a word cannot be saved at all.
- ④ A free-form map. The document model’s escape hatch: fields the code does not know about yet. Also the reason “schemaless” needs discipline (Chapter 8).

What the model buys: one read returns everything the page needs, no joins, no N+1 across tables; adding a field is a code change, not a migration; and the shape on the wire is the shape on disk, which makes the service almost entirely glue. What it costs: relationships between entries (WordLink) are not part of the document and must be traversed by hand; every array field is un-indexable-by-prefix for substring search; and nothing stops a document from growing without bound.

DECISION — MONGODB, AND THE CASE FOR HAVING CHOSEN POSTGRES

The dictionary is a per-user notebook with nested, optional, evolving fields — the document model’s home turf — and it shares nothing with the relational services, so isolation costs nothing. That is a good reason. The honest counter: Postgres has `jsonb` for the nested parts, `text[]` with a GIN index for tags, `pg_trgm` for exactly the substring search this service runs, and recursive CTEs for the word graph, all in a database the team already operates twice. A senior answer to “why Mongo?” is therefore not “because documents”; it is “because the data is document-shaped *and* we accepted a second database to keep the service independently deployable and to learn polyglot persistence — and here is what it cost us”. What would flip it: a requirement to join words to courses or users, or an operations team unwilling to run a second engine.

36.2 Reading the search path

The endpoint everyone uses is `search`. It runs through three pieces of code. The repository query:

```
@Query("{ 'createdByUserId': ?0, '$or': [ "
  + " { 'word': { '$regex': ?1, '$options': 'i' } }, "
  + " { 'tags': { '$regex': ?1, '$options': 'i' } } ] }")
List<WordDefinition> searchByWordOrTag(String userId, String regex);
```

The index that exists, from `MongoIndexConfig`:

```
mongoTemplate.indexOps("word_definitions")
    .ensureIndex(new Index().on("createdByUserId", Sort.Direction.ASC));
```

And the service method that turns the list into a page:

```
List<WordDefinitionResponse> responses = wordDefinitionRepository
    .searchByWordOrTag(userId, escapeRegex(query.trim())) ①
    .stream()
    .map(w -> toResponse(w, userId)) ②
    .sorted(Comparator.comparing(WordDefinitionResponse::word,
        String.CASE_INSENSITIVE_ORDER)) ③
    .toList();
int start = (int) pageable.getOffset();
int end = Math.min(start + pageable.getPageSize(), responses.size());
List<WordDefinitionResponse> pageContent = start < responses.size()
    ? responses.subList(start, end) : List.of(); ④
return new PageImpl<>(pageContent, pageable, responses.size());
```

- ① The user's text is spliced into a regular expression. `escapeRegex` backslashes every metacharacter first, so `a.*` matches the three literal characters and (does not crash the server with a parse error. Skip this and you have regex injection: at best wrong results, at worst a catastrophic pattern that pins a CPU core.
- ② `toResponse` runs two more queries per word — parents and children from `word_links` — so a search that matches n words issues $2n + 1$ round trips. This is the N+1 problem, reborn in a database that was supposed to avoid it.
- ③ Sorting happens in the JVM, on *every* match, not on the page.
- ④ So does paging. Mongo returned all matches; the service keeps twenty and discards the rest. Page 50 costs exactly what page 1 costs, plus the copying.

36.2.1 How the server executes it

Ask Mongo itself. With the stack running:

HANDS-ON — READ THE PLAN

```
$ docker compose exec mongodb mongosh ts_dictionary --quiet --eval '
db.word_definitions.find({ createdByUserId: "1", $or: [
  { word: { $regex: "clou", $options: "i" } },
  { tags: { $regex: "clou", $options: "i" } } ] })
.explain("executionStats").executionStats'
```

Trimmed, the answer looks like this:

```
{ nReturned: 3,
  totalKeysExamined: 412,
  totalDocsExamined: 412,
  executionStages: {
    stage: "FETCH",
    filter: { $or: [ { word: { $regex: "clou", $options: "i" } },
                    { tags: { $regex: "clou", $options: "i" } } ] },
    inputStage: {
      stage: "IXSCAN",
      indexName: "createdByUserId_1",
      indexBounds: { createdByUserId: [ "\u00271\u0027", "\u00271\u0027" ] } } } }
```

Three lines tell the story: IXSCAN on createdByUserId narrows to this user's 412 documents; FETCH then loads every one of them from disk; the filter runs both regexes against each. The ratio totalDocsExamined / nReturned is 137 — for each word returned, 137 were read and rejected.

Why the regex cannot use an index: a B-tree orders strings by their leading characters. `^clou` (anchored, case-sensitive) maps to a key range and Mongo will use an index on `word` for it. `clou` unanchored can appear anywhere in the string, and `i` means the index's byte order does not even match the comparison; there is no range to seek to. Mongo's planner is explicit about this: a case-insensitive or unanchored `$regex` is a full scan of whatever the other predicates left. Here that is the user's slice, which is why the `createdByUserId` index is doing all the useful work and the compound index proposed later does not change the regex part — it changes what happens *after* it.

36.2.2 The array predicate

`tags` is an array. A scalar condition on an array field matches if *any* element matches — there is no `$elemMatch` needed for a single condition. That is a genuine

convenience over SQL, where the same predicate is an EXISTS against a child table. It is also why the regex is run once per tag per document: a document with eight tags costs eight pattern evaluations on top of the one for word.

36.3 The graph

WordLink is an adjacency list: one document per edge with parentWordId, childWordId, position. The indexes on both ends are right — “children of X” and “parents of X” are each one index lookup. The cost is in the traversal:

```
private WordGraphResponse buildGraphNode(String wordId, String userId,
                                         Set<String> visited) {
    if (visited.contains(wordId)) return null;
    visited.add(wordId);
    WordDefinition word = wordDefinitionRepository
        .findByIdAndCreatedByUserId(wordId, userId).orElse(null); ①
    if (word == null) return null;
    List<WordLink> childLinks = wordLinkRepository
        .findByParentWordIdOrderByPosition(wordId);                ②
    List<WordGraphResponse> children = childLinks.stream()
        .map(link -> buildGraphNode(link.getChildWordId(), userId,
                                     visited))
        .filter(Objects::nonNull).toList();
    return new WordGraphResponse(word.getId(), word.getWord(),
                                  word.getMeaning(), children);
}
```

- ① One query per node.
- ② One more per node. A graph of 2,000 words costs 4,000 round trips, sequentially, on the request thread. Each is sub-millisecond on the same host; on a network hop it is a multi-second page. Embedding childIds in the parent document would make it two queries total (find all words for the user, build in memory) — at the price of updating two documents per link, without a transaction to hold them together.

Mongo does have a graph operator, \$graphLookup, that walks word_links recursively server-side in one aggregation. It returns the flattened set of reachable nodes with a depth field, and the tree is rebuilt in memory. The project’s visited set is doing by hand what \$graphLookup does with maxDepth; the difference is $2n$ round trips versus one.

36.4 Transactions and consistency, honestly

Mongo supports multi-document ACID transactions since 4.0 — on a replica set. The project runs a single `mongod` (Compose and `StatefulSet` alike), and a standalone node cannot start a transaction at all. So `deleteWord` is two independent writes: `delete` the definition, then `deleteByParentWordIdOrChildWordId`. A crash between them leaves edges pointing to a missing word; the traversal tolerates it (`orElse(null)`, then filtered), which is the right defensive posture, but it is a hidden data-integrity contract. The fix that costs nothing in code and one line in operations is to run a single-node replica set (`-replSet rs0` plus an `initiate` call): transactions become available, and so does the change stream you will want for the outbox pattern of Chapter 11.

Write concern and read preference matter the moment there is a second node. The default `w: majority` means an acknowledged write is on a majority of nodes; reading from a secondary can return an older state. For a single-user notebook, read-your-own-writes on the primary is what users expect, and it is the default — state that in an interview, and that you would revisit it before adding read replicas.

36.5 Pros and cons, candidly

Strength	Where the project pays for it
One document, one read, no joins	Links are separate; the graph is $2n$ queries
Arrays are first-class (tags, examples)	Substring search over them is a scan per element
No migrations for new fields	extra and images grow unchecked toward 16 MB
Regex search works out of the box	It cannot use an index unless anchored and case-sensitive
Paging is one skip/limit away	The search path pages in Java after loading everything
Independent database, independent deploy	No auth on <code>mongod</code> in Compose <i>or</i> the cluster

The last row is not a Mongo trait but a project fact worth knowing before someone asks: the official image enables authentication only when root credentials are set, and neither manifest sets them. Inside a ClusterIP-only network that

is a considered risk, documented in `mongodb.yaml`; it is the first thing to change if the service ever shares a cluster with anything untrusted.

36.6 What breaks

PITFALL — SEARCH GETS SLOWER WITH EVERY WORD, THEN TIMES OUT

Symptom: search latency grows linearly with a user’s dictionary; at a few thousand words the frontend’s timeout fires and the page shows nothing. Cause: three multipliers stacked on the same request — the regex scans every document in the user’s slice; `toResponse` then issues two queries per match; and sorting and paging happen after both. The database is doing what it was asked. Fix in order of payoff: page in the database (below), batch the link lookups into one `$in` query per page, and anchor or replace the regex.

PITFALL — DOCUMENT TOO LARGE

Symptom: saving a word fails with `BsonMaximumSizeExceededException: ... exceeds 16793600 bytes`. Cause: `images` holds base64 data URLs, and base64 inflates by a third; five 3 MB photos and the document is over the limit. Fix: store images in MinIO under a `dictionary/{userId}/{wordId}/` prefix and keep the keys in the document — Chapter 35’s rule, applied here. Until then, cap the array in `WordDefinitionRequest` validation.

PITFALL — THE REGEX THAT NEVER RETURNS

Symptom: one request pins a core at 100% and the rest of the service’s queries queue behind it. Cause: a pattern like `(a+)+$` against a long string — catastrophic backtracking. `escapeRegex` prevents users from sending one; the risk returns the day someone adds a “power search” that passes raw patterns, or forgets the escape on a new endpoint. Defense in depth: a `maxTimeMS` on the query (Spring Data’s `@Meta(maxExecutionTimeMs)`) so a runaway pattern is killed server-side after a second.

36.7 What breaks at 3 a.m.

The MongoDB `StatefulSet`’s node reboots; the pod is pending for four minutes while the claim reattaches.

- **dictionary-service** returns 500 on every endpoint. Its readiness probe (Chapter 17) does not include a database check, so the gateway keeps routing to it. Users see errors instead of a “service unavailable” page.
- **Everything else** is untouched. No other service holds a Mongo connection, no other service calls dictionary-service, and no Kafka topic depends on it. This is database-per-service working as designed: the blast radius of the outage is exactly one feature.
- **When the pod returns**, the driver’s connection pool reconnects on its own; there is no restart needed and no cache to warm. Mongo’s create-on-write means a *wrong* claim would come up empty and silent (Chapter 8’s pitfall) — check `show dbs` before believing a “data loss” report.

The other 3 a.m. is slower: the 5 GB claim fills. Mongo does not fail a single write cleanly at that point; it stalls, journals fail, and the process may exit. Because documents can grow (images, extra), there is no cap in the schema to prevent it. The alarm belongs on the PVC’s usage, not on the application.

36.8 Enhancing the resolution

Four changes, each complete, in the order they pay off.

36.8.1 Page in the database

Replace the list-returning query with a page-returning one. Spring Data Mongo needs a separate count query for @Query pages:

```
@Query(value = "{ 'createdByUserId': ?0, '$or': [ "
    + "{ 'word': { '$regex': ?1, '$options': 'i' } }, "
    + "{ 'tags': { '$regex': ?1, '$options': 'i' } } ] }",
    count = true)
long countByWordOrTag(String userId, String regex);

@Query("{ 'createdByUserId': ?0, '$or': [ "
    + "{ 'word': { '$regex': ?1, '$options': 'i' } }, "
    + "{ 'tags': { '$regex': ?1, '$options': 'i' } } ] }")
Page<WordDefinition> searchByWordOrTag(String userId, String regex,
    Pageable pageable); ①
```

```
public Page<WordDefinitionResponse> searchWords(String userId,
    String query, Pageable pageable) {
    Pageable sorted = PageRequest.of(pageable.getPageNumber(),
        pageable.getPageSize(),
        Sort.by("word").ascending()); ②
```

```

Page<WordDefinition> page = (query == null || query.isBlank())
    ? wordDefinitionRepository.findByCreatedByUserId(userId, sorted)
    : wordDefinitionRepository.searchByWordOrTag(userId,
        escapeRegex(query.trim()), sorted);
return toResponses(page, userId);
}

```

- ① Spring Data appends skip, limit and sort to the query. The regex still scans the user's slice, but only twenty documents are fetched, serialized and mapped.
- ② Sorting moves to the server. Case-insensitive order needs a collation on the query or the collection (`{locale: "en", strength: 2}`); add it to the index below so the sort can use it.
- ③ The link lookups become one batch per page, next.

36.8.2 Batch the links

```

private Page<WordDefinitionResponse> toResponses(Page<WordDefinition> page,
    String userId) {
    List<String> ids = page.map(WordDefinition::getId).toList();
    Map<String, List<String>> parents = new HashMap<>();
    Map<String, List<String>> children = new HashMap<>();
    for (WordLink l : wordLinkRepository.findByChildWordIdIn(ids)) { ①
        parents.computeIfAbsent(l.getChildWordId(), k -> new ArrayList<>())
            .add(l.getParentWordId());
    }
    for (WordLink l : wordLinkRepository
        .findByParentWordIdInOrderByPosition(ids)) {
        children.computeIfAbsent(l.getParentWordId(), k -> new ArrayList<>())
            .add(l.getChildWordId());
    }
    return page.map(w -> toResponse(w,
        parents.getOrDefault(w.getId(), List.of()),
        children.getOrDefault(w.getId(), List.of())));
}

```

- ① Two \$in queries per page instead of two per word: $2n + 1$ becomes 3. Both use the existing indexes on `childWordId` and `parentWordId`.

36.8.3 Index for the sort, and anchored search

```

// MongoIndexConfig
mongoTemplate.indexOps("word_definitions").ensureIndex(
    new CompoundIndexDefinition(

```

```
new Document("createdByUserId", 1).append("word", 1))
.collation(Collation.of("en").strength(2))); ①
```

- ① With this index, “this user’s words in alphabetical order” is an index walk, so the unfiltered listing and every sorted page avoid an in-memory sort. And a *prefix* search — {word: {\$regex: "^\clou"}} with the same collation — becomes a range seek on the same index. For a dictionary, prefix is usually what users mean when they type.

36.8.4 A text index for word-anywhere search

When substring-anywhere is genuinely required, stop using regex:

```
// MongoIndexConfig: one text index per collection, weighted
mongoTemplate.indexOps("word_definitions").ensureIndex(
    new TextIndexDefinition.TextIndexDefinitionBuilder()
        .onField("word", 10F) ①
        .onField("tags", 5F)
        .onField("meaning", 1F)
        .build());
```

```
// DictionaryService
public Page<WordDefinitionResponse> textSearch(String userId,
    String terms, Pageable pageable) {
    TextCriteria text = TextCriteria.forDefaultLanguage()
        .matchingAny(terms.split("\\s+")); ②
    Query q = TextQuery.queryText(text).sortByScore()
        .addCriteria(Criteria.where("createdByUserId").is(userId))
        .with(pageable);
    List<WordDefinition> hits = mongoTemplate.find(q, WordDefinition.class);
    long total = mongoTemplate.count(Query.of(q).skip(0).limit(0),
        WordDefinition.class);
    return toResponses(new PageImpl<>(hits, pageable, total), userId);
}
```

- ① A text index tokenizes and stems; cloud matches clouds and clouded, and a hit in word outranks the same hit in meaning. It does not match *inside* a token (lou will not find cloud) — that is the trade.
- ② No regex, no escaping, no backtracking. The index is consulted first; created-ByUserId filters the hits.

DECISION — ANCHORED PREFIX VERSUS FULL TEXT

Prefix-anchored regex on the compound index answers “words starting with...” in microseconds and needs no new index type; it is the right default for a type-ahead box. A text index answers “entries about ...” with ranking and stemming, at the cost of a larger index and no infix matching. Unanchored case-insensitive regex — what the project has — answers “anything containing...” and pays a scan for it. The recommendation is prefix for the search box, text search behind a separate “search everywhere” action, and regex retired. Atlas Search (Lucene under Mongo’s API) would give both at once, but only on Atlas; on a self-hosted `mongod`, these two are the tools.

36.9 Outside the project

Every document database has this chapter’s shape: DynamoDB with its partition-key-first access patterns and no substring search at all; Couchbase with N1QL and a full-text index; Elasticsearch as the sidecar that document stores grow when search becomes the product. Postgres’s `jsonb` plus `pg_trgm` is the answer most teams should consider before any of them — it turns exactly this service’s `ILIKE '%clou%'` into a GIN index seek. The senior framing of “SQL or NoSQL?” is never the engine; it is the access patterns written down first, then the engine whose indexes match them.

36.10 Questions from real interviews

Q: Dictionary search is slow. What do you check first?

The plan, before anything else: `explain("executionStats")` on the exact query the service sends, and two numbers from it — `totalDocsExamined` against `nReturned`, and which stage carries the filter. In this service that shows an `IXSCAN` on `createdByUserId` followed by a `FETCH` that examines hundreds of documents per hit, because an unanchored case-insensitive regex cannot use any index. Then look at what happens *after* the query: here, two link lookups per match and in-memory sorting and paging. I would fix in that order — page in the database, batch the links with `$in`, anchor the regex to a prefix on a compound `{createdByUserId, word}` index with a case-insensitive collation — and re-read the plan after each step. If “anywhere in the word” is a real requirement, that is a text index, not a faster regex.

Q: When would you not pick MongoDB?

When the access patterns are relational: data read in different groupings than it is written, invariants that span entities (a submission needs its assignment and its student to exist), or reporting that joins across the domain. When the team already runs Postgres well and the “document” is just a `jsonb` column away. When multi-document transactions are routine rather than exceptional — Mongo has them, but on a replica set, with retry semantics you have to code for. And when the data will be searched by substring at scale: `pg_trgm` is a one-line index; Mongo needs Atlas Search or an external engine. The project’s dictionary passes those tests because it is per-user, nested, and joins nothing; the course domain fails all of them, which is why it is in Postgres.

Q: Paginate a hundred thousand results without loading them.

Never with `skip` at depth — `skip(90000)` still walks ninety thousand index entries. Use *keyset* pagination: sort on an indexed, unique-ish key (word, tie-broken by `_id`) and ask for `{word: {$gt: lastWord}}` with `limit(20)`; each page is an index seek regardless of depth. Return the last key as an opaque cursor instead of a page number. Keep a count query only if the UI truly needs a total, and cache it, because counting a filtered regex is the same scan again. The project’s `subList` paging is the opposite of all this: it loads everything and returns twenty.

Q: One service uses Mongo, another Postgres. How do you keep their data consistent?

You do not, transactionally — there is no two-phase commit across them and you should not want one. You keep them *eventually* consistent through events (Chapter 11): the owning service commits its change and publishes a fact, the other consumes and updates its own copy, and an outbox table in the owner makes “committed” and “published” the same thing. Where the project needs this today is the user id: dictionary-service stores `createdByUserId` as a string that auth-service issued, and never needs to join on it, so no synchronization is required. If it ever needed the user’s display name, the answer is a `UserRegistered` consumer writing a tiny `users` collection — not a Feign call per word, and not a shared database.

Q: The document shape changed. Migrate without downtime.

Expand, migrate, contract. Deploy code that reads both shapes and writes the new one (a `@Field` alias or a converter handles the rename); run a background job that rewrites old documents in batches with `updateMany` on a bounded `_id` range,

throttled, idempotent, resumable; then, when a count of the old shape reaches zero, deploy code that reads only the new shape. Mongo makes the first step easy because nothing enforces the schema, and dangerous for the same reason — add a JSON Schema validator on the collection at the end so the old shape cannot come back. For the `images` field in this project the migration is also a data move: rewrite each data URL into a MinIO object and replace the array element with its key.

Q: Model the word graph — embed or reference, and what does traversal cost?

Reference, as the project does, when edges have their own attributes (position) and both directions are queried; embed a `childIds` array when the graph is shallow, mostly read, and parent-to-child is the only direction that matters. Traversal cost is the honest part: with references and naive recursion it is two round trips per node — 4,000 for a 2,000-word graph — which is what `buildGraphNode` does. The fixes are, in order, loading all edges for the user in one query and walking in memory; or a single `$graphLookup` with `maxDepth`; or embedding. Cycles need a visited set either way; the project has one. And say the trade-off out loud: embedding turns one link creation into two document writes with no transaction on a standalone node.

CHECK YOURSELF (answers: Appendix A)

1. From the explain output, `totalDocsExamined` is 412 and `nReturned` is 3. Name the stage doing the rejecting, say why the `createdByUserId` index cannot help it, and state the one change to the pattern that would let an index on `word` serve it.
2. `searchWords` with `page=50&size=20` on a user with 3,000 words: how many documents does Mongo return, how many `word_links` queries run, and where is the page built?
3. `deleteWord` issues two writes with no transaction. Explain why a transaction is unavailable on this deployment, what a crash between the writes leaves behind, and which line of `buildGraphNode` tolerates it.
4. A user attaches six photos to a word and the save fails. Give the exception, the arithmetic that produced it, and the storage rule from Chapter 35 that fixes it.
5. With a text index on `word`, `tags` and `meaning`, the query “lou” returns nothing although “cloud” exists. Explain, and say which index type answers it.
6. MongoDB is down for four minutes at 3 a.m. List what fails, what does

not, and the one probe change that would turn 500s into a clean “not ready”.

Deep Dive: Observability — Traces Across Feign and Kafka, Sampling, and Alerts That Matter

Chapter 24 taught you to debug the stack from the outside in — describe before logs — and Chapter 27 named the trio that comes next: metrics, traces, alerts. This deep dive opens the wiring that is already in every service and asks what it actually does. The answer is less than the configuration suggests: the tracing dependencies are on every classpath, the log pattern prints trace ids, the cluster samples at zero, and the Prometheus endpoint that Chapter 27 counts on does not exist yet. Reading why is the fastest route to understanding how distributed tracing works at all.

Three questions organize the chapter:

1. **Where does a trace start, and how does it survive** the hops browser → gateway → Feign → Kafka → consumer — and where in this project does it quietly die?
2. **What does sampling cost** at 1.0, at 0.0, and at the numbers production systems really use?
3. **Which signals deserve an alert**, and what has to be added before any alert can fire?

37.1 Why this, and what it solves

Logs answer “what did this process say?” Metrics answer “how is the system doing, in aggregate, over time?” Traces answer the question the other two cannot: “what happened to *this one request* as it crossed five services and a message broker?” In a monolith a stack trace is a trace. In this project a single “create assignment” click touches the gateway, course-service, auth-service (over Feign), Kafka, and notification-service; no single log file sees all of it.

The project chose Micrometer Tracing with the Brave bridge, reporting to Zipkin. Each service’s `pom.xml` carries the same pair:

```
<dependency>
<groupId>io.micrometer</groupId>
```

```
<artifactId>micrometer-tracing-bridge-brave</artifactId> ①
</dependency>
<dependency>
  <groupId>io.zipkin.reporter2</groupId>
  <artifactId>zipkin-reporter-brave</artifactId> ②
</dependency>
```

- ① Micrometer Tracing is the *facade* Spring Boot 3 instruments against; Brave (the Zipkin project’s tracer) is one of two implementations, OpenTelemetry being the other. The bridge is what turns Spring’s *observations* — the Micrometer Observation API that every Boot 3 starter emits — into spans.
- ② The reporter ships finished spans to Zipkin over HTTP. Swapping this one artifact for `opentelemetry-exporter-otlp` points the same spans at Tempo, Jaeger, or a vendor.

DECISION — BRAVE AND ZIPKIN, NOT OPENTELEMETRY

Brave plus Zipkin is the shortest path from zero to a working trace view: one container, no collector, no protocol negotiation, and Spring Boot’s auto-configuration knows both intimately. That is the right choice for learning the mechanics and for a homelab. What would flip it: a second language in the stack (OpenTelemetry’s SDKs cover all of them with one wire protocol), a vendor backend (Datadog, Honeycomb, Grafana Cloud all speak OTLP natively), or the need for tail-based sampling, which requires a collector between the apps and the store. The migration is mechanical — swap two dependencies, change the endpoint property — because the code never touched Brave directly. Interviewers ask “Zipkin or Jaeger or Tempo?” expecting exactly this answer: the backend is replaceable, the instrumentation standard is the decision that lasts.

37.2 How a trace travels through this project

37.2.1 The configuration, in every service

The five service documents in `config-server` all end the same way:

```
management:
  tracing:
    sampling:
      probability: 1.0 ①
  zipkin:
    tracing:
```

```

    endpoint: http://localhost:9411/api/v2/spans      ②
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus    ③
  logging:
    pattern:
      level: "%5p [${spring.application.name:},%X{traceId:-},%X{spanId:-}]" ④

```

- ① Every request is traced. Fine on a laptop; see Section 37.3 for what it costs elsewhere. In the cluster, every Deployment overrides this to 0.0 with the comment “Zipkin is not in the cluster yet”.
- ② Compose overrides the host to zipkin:9411; the reporter POSTs JSON batches here.
- ③ Exposure is a *list of names*. Naming prometheus does not create the endpoint; the endpoint exists only when micrometer-registry-prometheus is on the classpath, and no pom.xml in the repository has it. Today /actuator/prometheus answers 404 on every service — the Chapter 27 plan assumes an endpoint that still has to be added.
- ④ The log pattern reads two MDC keys that the tracing bridge populates. With no active span the fallbacks - print empty, which is itself diagnostic: an empty trace id in a log line means the code ran outside any traced context.

37.2.2 Hop by hop

Follow one POST `/api/courses/7/assignments` from the browser.

Browser → gateway. The browser sends no trace header; axios knows nothing of tracing. The gateway is a WebFlux application and Spring Cloud Gateway is instrumented through the Observation API, so the server-side observation of the incoming request *starts* a new trace: a random 128-bit trace id and a root span. When the gateway forwards to course-service, the reactive WebClient it uses is instrumented too, and the propagator writes the context into the outgoing request. Spring Boot 3 defaults matter here: Brave is configured to *produce* W3C Trace Context (traceparent: 00-<traceId>-<spanId>-01) and to *consume* W3C and both B3 formats. So this project speaks W3C on the wire and would still understand an older B3 caller.

Gateway → course-service. Course-service is servlet MVC. Its server observation reads traceparent, continues the same trace, and opens a child span. Everything logged inside the controller now carries that trace id — the pattern in ④ above prints it.

course-service → auth-service over Feign. AuthServiceClient is a Spring Cloud

OpenFeign client. OpenFeign ships a Micrometer capability, enabled by default, that wraps each call in an observation; the Brave propagator injects `traceparent` into the Feign request. Auth-service continues the trace. In Zipkin the Feign call appears as a client span in course-service nested under the controller span, with a matching server span in auth-service. If Resilience4j’s circuit breaker short-circuits the call (Chapter 6), no client span is created — the fallback ran, the wire was never touched, and the trace shows the gap.

course-service → Kafka. Here the trace dies. Read the project’s producer:

```
@Bean
public ProducerFactory<String, Object> producerFactory() {
    Map<String, Object> configProps = new HashMap<>();
    configProps.put(BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
    configProps.put(KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
    configProps.put(VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
    return new DefaultKafkaProducerFactory<>(configProps);
}

@Bean
public KafkaTemplate<String, Object> kafkaTemplate() {
    return new KafkaTemplate<>(producerFactory()); ①
}
```

① A hand-built `KafkaTemplate`. Spring Kafka *can* propagate trace context — it writes `traceparent` into the record’s headers and starts a producer span — but only when `setObservationEnabled(true)` is called on the template, or `spring.kafka.template.observation-enabled=true` is set for the auto-configured one. This bean does neither, and the property is absent from every config document. The record leaves without a header. On the consuming side, `@KafkaListener` in `notification-service` would need `spring.kafka.listener.observation-enabled=true` to read one. Neither half is on. The e-mail that results from the click is, to Zipkin, an unrelated event.

Auth-service’s `UserRegisteredPublisher` uses the auto-configured template and has the same gap for the same reason: the property, not the bean, is what is missing.

The scheduler boundary. `CourseLifecycleScheduler` runs `@Scheduled` at 01:00. Nothing instruments a cron trigger; its log line “Auto-archived 3 course(s)” prints with empty trace and span ids. The same is true of any `@Async` method unless the executor is wrapped with a context-propagating decorator. A trace is a property of a *request*; work that no request started has no trace unless you give it one.

37.2.3 Correlating logs

With the pattern from ④, a `grep` across all five services' logs for one trace id assembles the story Zipkin draws — useful precisely when Zipkin is down or sampled out:

```
$ docker compose logs --no-color api-gateway course-service \
  auth-service | grep 6f2c1a9e4b3d8e10
api-gateway   | INFO [api-gateway,6f2c1a9e4b3d8e10,9a1b...] ...
course-service | INFO [course-service,6f2c1a9e4b3d8e10,c04d...] ...
auth-service  | INFO [auth-service,6f2c1a9e4b3d8e10,e77f...] ...
```

Notice what is absent: no `notification-service` line, because no trace id crossed Kafka. The `grep` shows the hole exactly as Zipkin would.

37.3 Sampling

probability: 1.0 means every trace is recorded and every span shipped. The cost is linear in traffic: each span is a few hundred bytes of JSON, batched and POSTed by a background thread, then indexed and stored by Zipkin. At the project's traffic that is nothing. At 1,000 requests per second with six spans each it is 6,000 spans per second, roughly 2 MB/s of ingest and tens of gigabytes of storage a day — most of it recording requests that succeeded in 20 ms and that nobody will ever look at.

Head-based sampling — what probability configures — decides at the root span whether to keep the trace, and propagates the decision in the `traceparent` flags so every downstream service agrees. It is cheap and consistent, and it is blind: at 0.01 it keeps 1% of traces chosen at random, so the one request that failed at 3 a.m. was kept with probability 0.01. *Tail-based* sampling defers the decision until the whole trace is known and keeps every trace that errored or exceeded a latency budget, plus a random sample of the rest. It needs a component that buffers whole traces before the store — an OpenTelemetry Collector — which is why it is out of reach of the Brave-to-Zipkin direct wiring.

DECISION — THE SAMPLING RATE

Laptop and Compose: 1.0, because you are debugging and traffic is you. Cluster today: 0.0, honest about the absence of a Zipkin pod — sampling at zero also disables span creation cost, not only shipping. Cluster with Zipkin: start at 0.1; drop to 0.01 when the store grows faster than you read it. The senior answer to “what sample rate?” is “head-based 1–10% for the baseline, tail-based for errors and slow requests once there is a collector” — and the follow-up is

always “and error traces are what you actually want, so budget for the collector”.

37.4 Metrics: what exists and what does not

Actuator’s `/actuator/metrics` already exists on every service and lists a few hundred meters: JVM memory and GC, HTTP server requests tagged by uri, status and method, Kafka client metrics, HikariCP pool state, and Resilience4j circuit-breaker state in course-service. It answers one meter at a time, by name, over HTTP — a debugging tool, not a monitoring one. Monitoring means a system that *pulls* all of them every 15 seconds, stores them as time series, and evaluates rules. That system is Prometheus, and its endpoint is missing (see ③ above).

Two vocabularies to carry into interviews. **RED** is for request-driven services: Rate, Errors, Duration — the three the `http.server.requests` timer already provides. **USE** is for resources: Utilization, Saturation, Errors — CPU, heap, the Hikari pool, Kafka consumer lag. A dashboard per service should show RED at the top and USE below it; an alert should fire on a RED symptom the user feels, and USE panels explain why.

37.5 Pros and cons

Strengths	Weak points, today
Zero code changes: every hop is instrumented by starters, and the log pattern is one line.	Kafka propagation is off in both producer and consumer; the most interesting path in the system is the one that is invisible.
Vendor-neutral facade: Brave and Zipkin are swappable for OTLP without touching source.	No Prometheus registry on any class-path; prometheus in the exposure list is aspirational.
W3C traceparent on the wire, B3 accepted — interoperable with anything modern.	Head-based sampling only; an error at 1% sampling is kept 1% of the time.
Log correlation works even without Zipkin.	Zipkin runs <code>:latest</code> with no volume: traces vanish on every compose down.

37.6 What breaks

PITFALL — THE TRACE THAT ENDS AT KAFKA

Symptom: in Zipkin, the POST `/assignments` trace ends with the controller span; the notification that followed is a separate one-span trace, or absent. Cause: neither `spring.kafka.template.observation-enabled` nor `spring.kafka.listener.observation-enabled` is set, so no `traceparent` header is written to or read from the record. Fix: both properties, in both services' config documents (course-service and auth-service produce; notification-service consumes) — and the hand-built template in `KafkaProducerConfig` must call `setObservationEnabled(true)` because properties only configure the auto-configured bean. The Section 37.8 listing does all three.

PITFALL — HIGH-CARDINALITY TAGS

Symptom: a service's heap climbs for days and `/actuator/metrics` takes seconds to answer; eventually OOMKilled. Cause: a tag whose value set is unbounded. The classic is a handler mapped as `/courses/{id}` that someone rewrote to build the path by hand — the `uri` tag then carries `/courses/42`, `/courses/43`, ..., one time series each, forever. The same happens when a span is tagged with a user id or an e-mail. Fix: tag with the *template* (`/courses/{id}`), never the value; put identifiers in the span's log annotations or the log line, which are not indexed by value. Spring's default `uri` tag is the template precisely to avoid this.

PITFALL — A 401 THAT HAS NO TRACE

Symptom: a user reports "I got logged out" and there is no trace for their request. Cause: the gateway's `JwtAuthenticationFilter` rejects at order -1, before routing; the server observation still records the request, but with the sampling at 0.0 in the cluster nothing is exported anywhere. Fix is not tracing at all: this is a *metric* question. Count `http.server.requests` with `status=401` at the gateway and alert on a jump — one bad deploy of `JWT_SECRET` shows as a cliff on that counter within 15 seconds.

37.7 What breaks at 3 a.m.

Zipkin dies at 3 a.m.; the services keep running.

- **Request latency: unchanged.** The reporter is asynchronous. Spans finish into an in-memory queue; a background thread drains it in batches over HTTP. The request thread never waits on Zipkin. This is the property to state first in an interview, because the fear “tracing slows my service” is the common objection.
- **Memory: bounded.** The queue has a fixed capacity (`queuedMaxSpans`, 10,000 by default). When it is full, new spans are *dropped*, counted, and a warning is logged — the reporter does not grow the heap to keep them. You lose visibility, not the service.
- **Logs: still correlated.** Trace ids are generated locally; the MDC pattern keeps printing them. The grep in the correlation section works exactly as before. This is the argument for always having the log pattern, regardless of the trace backend.
- **Recovery: automatic.** The reporter retries each batch on the next drain; when Zipkin returns, new spans flow. The spans dropped during the outage are gone — there is no replay.

Now the other direction: *traffic* spikes at 3 a.m. with sampling at 1.0. The queue fills, spans drop, and the warning log itself becomes noise at hundreds of lines per second. That is the concrete form of the sampling trade-off — and the reason to set the rate before the spike, not during it.

37.8 Enhancing the resolution

Four additions turn the wiring into a monitoring system. Each is complete; add them in order.

1. The Prometheus endpoint. One dependency per service (the parent’s Boot BOM manages the version):

```
<dependency>
<groupId>io.micrometer</groupId>
<artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

With it on the classpath, the existing exposure list makes `/actuator/prometheus` real. Add the tracing switches that are missing, and a common tag so every series says which service it came from:

```
management:
  prometheus:
    metrics:
      export:
```



```

    enabled: true
metrics:
  tags:
    application: ${spring.application.name} # every series, one tag
tracing:
  sampling:
    probability: ${TRACING_SAMPLE:0.1} # env-overridable
spring:
  kafka:
    template:
      observation-enabled: true # producer span + traceparent header
    listener:
      observation-enabled: true # consumer span, continues the trace

```

And the hand-built template in `course-service`, which ignores the property:

```

@Bean
public KafkaTemplate<String, Object> kafkaTemplate() {
    KafkaTemplate<String, Object> template =
        new KafkaTemplate<>(producerFactory());
    // Properties configure only the auto-configured template; a
    // hand-built one must opt in explicitly or traces end here.
    template.setObservationEnabled(true);
    return template;
}

```

2. Scraping in the cluster. With `kube-prometheus-stack` installed (Chapter 27), a `ServiceMonitor` tells its operator which `Services` to scrape. One object covers all five, by label:

```

apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: ts-services
  namespace: ts
  labels:
    release: kube-prometheus-stack # the selector the chart installs
spec:
  selector:
    matchLabels:
      monitoring: actuator # add this label to each Service
  endpoints:
    - port: http # the Service port *name*
      path: /actuator/prometheus
      interval: 15s

```

The `auth-service` and `course-service` `Services` must permit `/actuator/prometheus`

in their `SecurityConfig` for this to work — the same permit that Chapter 17 needed for probes. Restrict it to the pod network with a `NetworkPolicy` rather than leaving it open to the Ingress.

3. Structured logs. A grep across containers works on a laptop; in the cluster, Loki or Elasticsearch parse JSON far more reliably than a bracketed pattern. Logback’s JSON encoder is a dependency plus a `logback-spring.xml`:

```
<dependency>
  <groupId>net.logstash.logback</groupId>
  <artifactId>logstash-logback-encoder</artifactId>
  <version>8.0</version>
</dependency>
```

```
<configuration>
  <springProperty name="app" source="spring.application.name"/>
  <appender name="JSON"
    class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <customFields>{"application": "${app}"}</customFields>
      <!-- traceId and spanId come from the MDC automatically -->
      <includeMdcKeyName>traceId</includeMdcKeyName>
      <includeMdcKeyName>spanId</includeMdcKeyName>
    </encoder>
  </appender>
  <springProfile name="!local">
    <root level="INFO"><appender-ref ref="JSON"/></root>
  </springProfile>
  <springProfile name="local">
    <include resource="org/springframework/boot/logging/logback/base.xml"/>
  </springProfile>
</configuration>
```

The profile split keeps the human-readable pattern on a laptop and emits one JSON object per line everywhere else. Every line now carries `traceId` as a field; a log query for a trace id is an index lookup, not a scan.

4. Two alerts tied to what users feel. Prometheus rules, in the operator’s `PrometheusRule` kind:

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: ts-slo
  namespace: ts
  labels: { release: kube-prometheus-stack }
spec:
  groups:
```

```

- name: ts.slo
  rules:
    - alert: GatewayErrorBudgetBurn
      expr: |
        sum(rate(http_server_requests_seconds_count{
          application="api-gateway", status=~"5.."}[5m]))
        /
        sum(rate(http_server_requests_seconds_count{
          application="api-gateway"}[5m])) > 0.02
      for: 10m
      labels: { severity: page }
      annotations:
        summary: "Gateway 5xx ratio above 2% for 10 minutes"
    - alert: NotificationConsumerLag
      expr: |
        sum by (topic) (
          kafka_consumer_fetch_manager_records_lag_max{
            application="notification-service"}) > 500
      for: 15m
      labels: { severity: ticket }
      annotations:
        summary: "notification-group is {{ $value }} records behind"

```

The first is a *symptom* alert on the RED “errors” ratio at the one place every request passes; 2% over ten minutes is a starting budget, not a law. The second is the Chapter 27 answer to “which Kafka metric first”: lag growing for 15 minutes means mail is not going out, whatever the pods’ health says. Note the meter name: Spring Kafka registers the client’s own `records-lag-max` under Micrometer’s naming, which the Prometheus registry renders with underscores.

HANDS-ON — PROVE THE TRACE CROSSES KAFKA

After adding the two observation-enabled properties and the template line, restart `course-service` and `notification-service` under Compose, create an assignment through the UI, then ask Zipkin for the trace by the id printed in `course-service`’s log:

```

$ curl -s "http://localhost:9411/api/v2/trace/6f2c1a9e4b3d8e10" \
  | jq -r '.[] | "\(.localEndpoint.serviceName) \(.name)'"
api-gateway      http post
course-service    http post /courses/{id}/assignments
course-service    http get      # Feign to auth
auth-service      http get /users/{id}
course-service    ts.assignment.created send
notification-service ts.assignment.created process

```

Before the change the last two lines do not exist. The consumer span's parent is the producer span; the e-mail is now attributable to the click that caused it.

37.9 Outside the project

Production stacks converge on the same shape with different names. Instrumentation: OpenTelemetry SDKs or the Java agent, which auto-instruments without even the starters. Collection: an OpenTelemetry Collector per node or per cluster, which is where tail-based sampling, redaction, and fan-out to several backends live. Storage: Tempo for traces, Loki for logs, Prometheus or Mimir for metrics — the Grafana trio — or Jaeger, Elasticsearch and Thanos; or a vendor that takes OTLP and bills by volume. The interview thread runs through all of them: “how do you find a slow request across services?” is answered by trace ids in logs and a trace store; “how do you know before users tell you?” by RED alerts on the edge; “what does tracing cost?” by the sampling discussion above — and the senior version of every answer names the component that stays (instrumentation and the trace id in every log line) and the ones that are replaceable (everything behind the exporter).

37.10 Questions from real interviews

Q: p99 latency doubled last night. Walk me through finding the cause.

Start with metrics, not logs: the `http_server_requests` histogram at the gateway, split by `uri`, tells you in one query whether *every* route slowed (infrastructure — node, network, GC) or one route did (a code path or a downstream). Then pull one slow trace from that window — with head-based sampling at 1% there will be a few hundred — and read the span waterfall: the slow span is either a child (a downstream call, say the Feign hop to auth-service at 800 ms instead of 8) or a gap between children (the service itself, usually a database call or a lock, which shows as time in the parent not covered by any child). If it is a gap, the trace id takes you to that service's logs for the exact minute; the JSON layout makes that an index lookup. The senior detail is the order — aggregate, then one exemplar, then its logs — and naming what you would have wished for: an exemplar link from the histogram bucket straight to a trace, which OpenTelemetry-native stacks provide and Zipkin does not.

Q: You cannot afford to trace 100%. What do you sample, and what do you never drop?

Head-based sampling at 1–10% for the steady state, decided at the gateway and propagated in the traceparent flags so downstream services never disagree. Never drop errors and never drop the slow tail — which head-based sampling cannot promise, because the root span is sampled before anyone knows the request will fail. So the real answer has two stages: a collector doing tail-based sampling that keeps every trace with a 5xx or above a latency budget, plus a 1% random baseline for the shape of normal traffic. Mention the cost honestly: tail-based sampling buffers whole traces in the collector’s memory for the decision window, typically 10–30s, so the collector is sized by traffic, not by what you keep. In this project that is the step from zipkin-reporter-brave to an OTLP exporter plus a collector; the application code does not change.

Q: A request disappears from the trace at Kafka. Why, and how do you fix it?

Because trace context crosses Kafka only as record headers, and nothing writes them unless the template is told to. In Spring Kafka two properties do it, both false by default in Boot 3: on the producer, `template.observation-enabled`; on the consumer, `listener.observation-enabled` (both under `spring.kafka`). The subtlety worth saying out loud: properties only configure the auto-configured template, so a hand-built `KafkaTemplate` bean — exactly what `course-service`’s `KafkaProducerConfig` does — needs `setObservationEnabled(true)` in code. After the fix the consumer span’s parent is the producer span and the whole e-mail path hangs off the original HTTP request. The follow-up an interviewer wants: the consumer span is a *child*, but it can run seconds or hours after the request finished, so a trace view sorted by time will show a long gap — that gap is queue latency, and it is itself the metric to watch.

Q: What would you page on for this stack?

Symptoms users feel, expressed as SLOs, never raw resources. Two pages: gateway 5xx ratio above the error budget for ten minutes — the one point every request passes — and p99 latency at the gateway above target for the same window. One ticket-level alert: notification consumer lag growing for fifteen minutes, because that is “mail is not going out” in the only form the system can express it. Everything else — CPU, heap, pod restarts, Hikari pool exhaustion — is a dashboard panel that explains a page, not a page itself; alerting on CPU at 80% wakes people for a system that is working. The senior framing is the error budget: at a

99.9% availability target you may burn 43 minutes of errors a month, and the alert fires on the *burn rate*, which is why the ratio is measured over a window rather than on a single failed request.

Q: One user’s request touched five services. How do you find those five log lines?

By the trace id, which every service prints because the log pattern includes `%X{traceId}` and the tracing bridge fills the MDC on every hop. On a laptop that is a `grep` across container logs; in the cluster it is a query on a log store that indexes the field, which is why logs are shipped as JSON rather than parsed with a regex. Two details separate a senior answer: first, the trace id must be *returned to the user* — a `X-Trace-Id` response header from the gateway — so that a support ticket can quote it instead of “around 3 p.m.”; second, anything that runs outside the request (`@Scheduled`, `@Async`, a Kafka consumer without propagation) logs an empty id, and you should know which of your code paths those are before the incident.

Q: Zipkin is down. Does production slow down?

No, and you should be able to say why from the reporter’s design: spans are handed to an in-memory queue and a background thread POSTs them in batches, so no request thread ever waits on Zipkin; the queue is bounded at 10,000 spans and drops on overflow, so memory is capped; the failed batches are logged and re-tried on the next drain, so recovery is automatic. What you lose is the spans from the outage window — there is no replay. The trap in the question is the opposite case: a *synchronous* exporter, or an unbounded queue, would turn a monitoring outage into an application outage, which is the property to check before adopting any agent or exporter. And the mitigation for the lost spans is the same as for everything else in this chapter: the trace id is still in every log line, so the logs carry the correlation even when the trace store cannot.

CHECK YOURSELF (answers: Appendix A)

1. Name the exact property or method call missing at each of the two Kafka-side halves of the project, and explain why setting the properties alone would fix auth-service but not course-service.
2. A request arrives at the gateway carrying `X-B3-TraceId` from an old upstream proxy. Does the project continue that trace or start a new one, and which Boot 3 default decides it? What header leaves the gateway toward course-service?

3. `/actuator/prometheus` is in every exposure list yet returns 404. Explain, and name the one artifact that changes the answer.
4. Zipkin is down for an hour under normal traffic. State what happens to request latency, to heap, and to the spans produced during the hour — and which of the three would change if the reporter were synchronous.
5. At probability: 0.01, what fraction of *failed* requests have a trace, and what architectural component is required to make that fraction 1.0 without tracing every success?
6. A colleague adds `span.tag("email", user.getEmail())` to the login span. Predict the failure over the following weeks and give the rule that prevents it.

Deep Dive: The Browser Meets the Gateway — Token Storage, Refresh Flow, and CORS

Chapter 26 put the React application on the same origin as the API and noted, almost in passing, that CORS “is simply never consulted”. Chapter 28 pushed the token design from the server’s side. This deep dive takes the other end of the wire: the browser, which is the one runtime in the system that you do not control, that runs code injected by anyone who can get a script tag onto your page, and that will happily send a cookie to an attacker’s form. Every decision about where a token lives is a decision about which of those threats you accept.

Three questions organize the chapter:

1. **Where does the token live** in the browser, and which attack does each home expose it to?
2. **What happens at expiry** — how the refresh flow runs, and why ten tabs must produce one refresh call?
3. **What is CORS protecting**, what does it cost per request, and why does one origin make it vanish?

38.1 Why this, and what it solves

The server issues two credentials (Chapter 5): a 15-minute JWT access token and a 7-day opaque refresh token. The browser must keep both across page reloads, attach the first to every API call, and exchange the second for a fresh first when it expires — without the user noticing. The project’s answer is a Zustand store with the persist middleware and an axios interceptor. Read the store first:

```
export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      accessToken: null,
      refreshToken: null,
      user: null,
      isAuthenticated: false,
      setTokens: (accessToken, refreshToken) =>
```



```

    set({ accessToken, refreshToken, isAuthenticated: true }),
    setUser: (user) => set({ user }),
    logout: () =>
      set({ accessToken: null, refreshToken: null,
        user: null, isAuthenticated: false }),
  }},
  { name: 'auth-storage' } ②
)
);

```

- ① persist writes the whole state to storage on every change and rehydrates it on page load. That is what survives a reload; without it every refresh of the page would be a logout.
- ② The storage key. The default backend is `localStorage`, so open DevTools, Application, Local Storage, and there they are: both tokens, in clear text, under `auth-storage`. Any script running on the page can read them with one line.

DECISION — LOCALSTORAGE, FOR A FIRST VERSION

The project stores both tokens in `localStorage` because it is the simplest thing that survives a reload, works identically on `localhost:3000` and `ts.local`, needs no cookie configuration on the server, and sidesteps CSRF entirely — a token that JavaScript attaches by hand is never sent by the browser on its own. The price is that the tokens are exactly as safe as every script on the page. That was acceptable while the page loads only its own bundle. It stops being acceptable the day a third-party script (an analytics tag, a chat widget, a compromised npm dependency) runs on the same origin: any of them can read `auth-storage` and ship a 7-day refresh token elsewhere. The upgrade in Section 38.7 moves the refresh token into an `HttpOnly` cookie and keeps the access token in memory — more moving parts, but a script can no longer steal the long-lived credential.

38.2 The refresh flow, as written

38.2.1 The interceptor

Every API call goes through one axios instance:

```

const apiClient = axios.create({ baseURL: '/api' }); ①

apiClient.interceptors.request.use((config) => {
  const token = useAuthStore.getState().accessToken;

```

```

    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
  });

  apiClient.interceptors.response.use(
    (response) => response,
    async (error) => {
      const originalRequest = error.config;
      if (error.response?.status === 401
        && !originalRequest._retry) {
        originalRequest._retry = true;
        const refreshToken = useAuthStore.getState().refreshToken;
        if (refreshToken) {
          try {
            const { data } = await axios.post(
              '/api/auth/refresh', { refreshToken });
            useAuthStore.getState()
              .setTokens(data.accessToken, data.refreshToken);
            originalRequest.headers.Authorization =
              `Bearer ${data.accessToken}`;
            return apiClient(originalRequest);
          } catch {
            useAuthStore.getState().logout();
            window.location.href = '/login';
          }
        }
      }
      return Promise.reject(error);
    }
  );

```

- ① A relative base URL. In development Vite’s proxy forwards `/api` to `localhost:8080`; in the cluster Traefik does (Chapter 26). The browser never sees a second origin.
- ② The bearer header is attached by code on every request. This is the CSRF immunity mentioned above: a form on `evil.example` cannot make the browser add this header.
- ③ A 401 from the gateway — the JSON body “Invalid or expired token” from `JwtAuthenticationFilter` — triggers one retry per request, flagged on the config object so a second 401 falls through to the caller.
- ④ The refresh call deliberately uses the *bare* `axios`, not `apiClient`: otherwise a 401 from `/refresh` itself would re-enter this interceptor and loop.
- ⑤ The original request is replayed with the new token. The caller’s `await` resolves as if nothing happened.

This is correct for one request. It is wrong for a page.

38.2.2 The single-flight problem

DashboardPage mounts and TanStack Query fires its queries in parallel: courses, assignments, the user profile, dictionary counts. If the access token expired while the laptop was closed, all of them return 401 within the same few milliseconds, and each one runs the interceptor independently. Four requests, four calls to `/api/auth/refresh`, four new access tokens, and — because `setTokens` runs four times — whichever response lands last wins. Today that is harmless because auth-service returns the same refresh token every time (Chapter 28 called this out as the missing rotation). Add rotation, which you should, and the race becomes fatal: the first refresh invalidates the token the other three are about to present, they fail, and the user is logged out by the very mechanism meant to keep them in.

The fix is a *single-flight* refresh: the first 401 starts one refresh and stores its promise; every concurrent 401 awaits the same promise instead of starting its own; when it resolves, all of them replay with the same new token. Section 38.7 has the complete code. Interviewers ask this as “ten tabs hit 401 at once — how many refresh calls?” and the senior answer is “one per tab without coordination, one in total with a lock across tabs”, followed by naming the tool for the cross-tab case: the Web Locks API or a BroadcastChannel, since each tab has its own JavaScript heap and cannot see another tab’s pending promise.

38.2.3 Logout and the OAuth callback

Header.handleLogout posts the refresh token to `/api/auth/logout`, which deletes its row, then clears the store and navigates to `/login`. That revokes the 7-day credential immediately. The 15-minute access token is not revoked — it cannot be, per Chapter 28 — so a copy taken before logout keeps working until it expires. “How do you log a user out everywhere?” has the same answer with one more step: delete *all* of the user’s refresh-token rows, not just the one this browser holds, and accept the 15-minute window or add the `jti deny-list`.

The Google login lands on OAuthCallbackPage with both tokens as query parameters — OAuth2LoginSuccessHandler builds the redirect that way. The page reads them, stores them, and navigates with `replace: true` so the URL with the tokens does not stay in history. That last detail matters more than it looks: a URL is copied into the browser history, into the Referer header of any resource the page loads next, and into server access logs. The `replace` shortens the exposure; it does

not remove it from the gateway's and auth-service's access logs, which now contain a valid refresh token in plain text.

38.3 CORS: the policy you are not using

Same-origin policy is the browser's rule that a script loaded from origin A may not *read* responses from origin B. CORS is the protocol by which B can relax that rule per origin. The project's gateway has a `CorsConfig` that lists `localhost:3000` and `localhost:5173`, allows all methods and headers, and sets `allowCredentials(true)`. Read it against the two ways the app is actually served:

- **Development.** Vite serves the page on `:3000` and *proxies* `/api` to `:8080`. The browser sends requests to `:3000`; Vite forwards them server-side. No cross-origin request exists, so no preflight, so `CorsConfig` is never exercised.
- **Cluster.** Traefik routes `/api` to the gateway and everything else to nginx on the same host (Chapter 26). Same origin again.

`CorsConfig` therefore serves exactly one scenario: someone running the built bundle from a different origin — say, opening `dist/index.html` through a static server on `:5173` without the proxy. It is a safety net, and a permissive one: `addAllowedHeader("*")` plus credentials is the widest policy Spring will accept without throwing.

What CORS would cost if it were active is worth knowing for the interview. A request with a JSON body and an `Authorization` header is not a “simple request”, so the browser first sends an `OPTIONS` preflight asking whether the method and headers are allowed. That is one extra round trip per distinct request, cacheable by `Access-Control-Max-Age`, which the project's config does not set — so every call would preflight. On a 40 ms link, every API call becomes 80 ms before any work happens. One origin removes that cost entirely, which is the performance argument for Chapter 26's design on top of the security one.

38.4 Pros and cons

	localStorage + bearer (today)	HttpOnly cookie
XSS steals the token	yes, both tokens, one line of JS	no — script cannot read the cookie
CSRF	immune: header is attached by code	exposed unless SameSite + CSRF token or custom-header check
Survives reload	yes, via persist	yes, the browser keeps it
Multi-tab	each tab reads storage; refresh races	same cookie in every tab; refresh races still
Works cross-origin	yes	needs Allow-Credentials and an exact origin list
Server changes	none	set-cookie on login, read-cookie on refresh

The honest summary a senior engineer gives: bearer-in-storage trades a CSRF problem you can solve for an XSS problem you cannot. Any injected script defeats it completely. Cookies trade the other way, and their problem has known, mechanical fixes.

38.5 What breaks

PITFALL — FOUR REFRESHES, ONE WINNER

Symptom: after leaving the app open overnight, the first click works, then the user is sent to /login even though nothing is wrong with their account. Cause: parallel 401s each ran the interceptor; with refresh-token rotation on the server, the second refresh presented a token the first had just invalidated, and the catch branch logged the user out. Fix: the single-flight interceptor in Section 38.7. Without rotation the symptom is only wasted calls — which is why the race is invisible today.

PITFALL — THE TOKEN IN THE URL

Symptom: a refresh token appears in the gateway's access log, in Traefik's log, and in the browser history of a shared computer. Cause: OAuth2LoginSuccessHandler passes both tokens as query parameters of the redirect to /oauth/callback. Fix: put the tokens in the URL *fragment* (#accessToken=...) — the fragment is never sent to any server — or better,

return only a one-time code that the page exchanges for tokens with a POST. The cookie design below removes the refresh token from the URL entirely.

PITFALL — A 401 THAT IS NOT ABOUT THE TOKEN

Symptom: a wrong password on the login page briefly flashes the app shell, then returns to /login. Cause: auth-service answers a bad login with 401; the interceptor treats *every* 401 as “token expired” and, if a stale refresh token exists in storage from an earlier session, tries to refresh, fails, and runs `logout()` plus a hard redirect. Fix: exempt the auth endpoints from the refresh path (`originalRequest.url` starting with /auth/), and clear the store on the login page’s mount.

38.6 What breaks at 3 a.m.

Three failures that only the client sees.

- **The laptop closed for eight days.** On return the access token is long dead and the refresh token expired yesterday. The first request gets 401, the refresh gets 401 (auth-service deletes the expired row and throws), the interceptor logs the user out. Correct behaviour — but every in-flight query on the page fails first, and TanStack Query may retry them three times each before the redirect lands. Set `retry: false` for 401s in the query client’s default options, or the user watches spinners for ten seconds before being sent to login.
- **Client clock skew.** The project never checks `exp` on the client, and that is right: the client’s clock is not trustworthy and the gateway decides. A “smart” client that pre-emptively refreshes when `exp` is near will, on a machine whose clock is 20 minutes fast, refresh on every request — or, 20 minutes slow, never refresh and get 401 anyway. Let the server’s 401 be the signal.
- **CORS misconfigured in production only.** The symptom is a console error, “No Access-Control-Allow-Origin header”, on a request that returns 200 in the network tab. The browser got the response and refused to hand it to the script. It happens when the page and API are on different origins in production (a CDN for the bundle, the gateway on another host) and the gateway’s allowed list still says `localhost`. The check is always the same: compare the `Origin` request header with the `allowedOrigins` list, character for character, scheme and port included. And note the shape of the error: it is

never visible to the server, whose logs show a successful request.

38.7 Enhancing the resolution

The target design: the access token lives only in JavaScript memory (lost on reload, restored by a silent refresh); the refresh token lives in an `HttpOnly` cookie that JavaScript cannot read and the browser sends only to the refresh endpoint.

1. Auth-service sets the cookie. On login, 2FA verification, and refresh, the controller returns the access token in the body and the refresh token in a cookie:

```
private ResponseCookie refreshCookie(String value, long maxAgeSec) {
    return ResponseCookie.from("refresh_token", value)
        .httpOnly(true) // no document.cookie access
        .secure(true) // HTTPS only; false on plain localhost
        .sameSite("Strict") // never sent from another site
        .path("/api/auth/refresh") // only this endpoint sees it
        .maxAge(maxAgeSec)
        .build();
}

@PostMapping("/login")
public ResponseEntity<LoginResponse> login(
    @Valid @RequestBody LoginRequest request) {
    LoginResponse r = authService.login(request);
    if (r.refreshToken() == null) { // 2FA or password change
        return ResponseEntity.ok(r);
    }
    return ResponseEntity.ok()
        .header(HttpHeaders.SET_COOKIE, refreshCookie(
            r.refreshToken(),
            jwtService.getRefreshTokenExpiryMs() / 1000).toString())
        .body(new LoginResponse(r.accessToken(), null,
            false, false, null)); // token not in the body
}

@PostMapping("/refresh")
public ResponseEntity<LoginResponse> refresh(
    @CookieValue(name = "refresh_token", required = false)
    String cookie) {
    if (cookie == null) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
    }
    LoginResponse r = authService.refreshToken(
        new TokenRefreshRequest(cookie));
    return ResponseEntity.ok()
        .header(HttpHeaders.SET_COOKIE, refreshCookie(
            r.refreshToken(),
```

```

        jwtService.getRefreshTokenExpiryMs() / 1000).toString())
        .body(new LoginResponse(r.accessToken(), null,
            false, false, null));
    }

    @PostMapping("/logout")
    public ResponseEntity<Void> logout(
        @CookieValue(name = "refresh_token", required = false)
        String cookie) {
        if (cookie != null) authService.logout(cookie);
        return ResponseEntity.noContent()
            .header(HttpHeaders.SET_COOKIE,
                refreshCookie("", 0).toString()) // delete it
            .build();
    }

```

The cookie's path is the strongest of the four attributes: even if some other endpoint were vulnerable, the browser never attaches this cookie to it. Note the path is the *gateway's* view (`/api/auth/refresh`), because the browser talks to the gateway; the gateway strips the prefix afterwards. `SameSite=Strict` is the CSRF defence: a cross-site form POST does not carry the cookie at all. The JSON body requirement on `/refresh` is a second layer — a plain HTML form cannot send `application/json` without a preflight, which the same-origin design would not grant to another site.

2. The client keeps the access token in memory. The store drops persist and the refresh token entirely:

```

export const useAuthStore = create<AuthState>()((set) => ({
  accessToken: null,           // memory only: gone on reload, by design
  user: null,
  isAuthenticated: false,
  setAccessToken: (accessToken) =>
    set({ accessToken, isAuthenticated: true }),
  setUser: (user) => set({ user }),
  logout: () =>
    set({ accessToken: null, user: null, isAuthenticated: false }),
}));

```

On page load nothing is in memory; the app performs one silent refresh before rendering protected routes. A reload therefore costs one round-trip to `/api/auth/refresh`, and if the cookie is gone the user is at `/login` — the same behaviour as an expired session.

3. Single-flight refresh with replay. The interceptor keeps one pending promise and a queue of requests waiting on it:


```

let refreshing: Promise<string> | null = null; // module-level lock

async function refreshAccessToken(): Promise<string> {
  if (!refreshing) { // first 401 starts it
    refreshing = axios
      .post('/api/auth/refresh', null, { withCredentials: true })
      .then(({ data }) => {
        useAuthStore.getState().setAccessToken(data.accessToken);
        return data.accessToken as string;
      })
      .finally(() => { refreshing = null; }); // release the lock
  }
  return refreshing; // others await it
}

apiClient.interceptors.response.use(
  (response) => response,
  async (error) => {
    const original = error.config;
    const isAuthCall = original.url?.startsWith('/auth/');
    if (error.response?.status !== 401 || original._retry
        || isAuthCall) {
      return Promise.reject(error);
    }
    original._retry = true;
    try {
      const token = await refreshAccessToken(); // shared promise
      original.headers.Authorization = `Bearer ${token}`;
      return apiClient(original); // replay
    } catch {
      useAuthStore.getState().logout();
      window.location.href = '/login';
      return Promise.reject(error);
    }
  }
);

```

Ten concurrent 401s now produce one POST; the other nine await the same promise and replay with the token it resolves to. `withCredentials: true` is what makes axios send the cookie — and what makes the request *credentialed* for CORS purposes. For cross-tab coordination, wrap the body of `refreshAccessToken` in `navigator.locks.request('refresh', ...)`: the Web Locks API is process-wide, so tabs take turns, and the second tab's refresh finds a rotated cookie already set by the first and simply succeeds.

4. The gateway, if a second origin ever appears. Credentialed requests forbid the wildcard; the origin must be listed exactly and the preflight cached:

```
spring:
  cloud:
    gateway:
      globalcors:
        cors-configurations:
          '[/**]':
            allowed-origins:
              - https://app.teachersupporter.example # exact, with scheme
            allowed-methods: [GET, POST, PUT, PATCH, DELETE, OPTIONS]
            allowed-headers: [Authorization, Content-Type]
            allow-credentials: true # cookies cross-origin
            max-age: 3600 # cache the preflight
```

With SameSite=Strict the cookie would still not travel to a different site, so a genuinely cross-site deployment needs SameSite=None; Secure and accepts the CSRF exposure that comes with it — the reason the same-origin design of Chapter 26 is the one to keep.

HANDS-ON — WATCH THE COOKIE DO ITS JOB

After deploying the cookie version, log in through the UI, then inspect what the browser holds and what it sends:

```
$ curl -si -c jar.txt -X POST http://localhost:8080/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"teacher@ts.local","password":"secret"}' | grep -i set-cookie
Set-Cookie: refresh_token=3f9c...; Path=/api/auth/refresh; Max-Age=604800;
HttpOnly; SameSite=Strict
$ curl -s -b jar.txt -X POST http://localhost:8080/api/auth/refresh \
  | jq .accessToken
"eyJhbGciOiJIUzI1NiJ9..."
$ curl -s -b jar.txt -o /dev/null -w '%{http_code}\n' \
  http://localhost:8080/api/courses
401
```

The last line is the path attribute at work: the cookie exists in the jar but curl, like a browser, does not send it to /api/courses. In DevTools the same cookie shows a padlock in the HttpOnly column and document.cookie returns an empty string.

38.8 Outside the project

The pattern above — access token in memory, refresh token in an HttpOnly cookie, silent refresh on load — is what Auth0’s SPA SDK, Okta’s, and most bank web

apps implement, usually with refresh rotation and reuse detection on the server. The alternative that removes the browser from token handling altogether is the *backend-for-frontend*: a small server (or the gateway itself) holds the tokens in a server-side session, and the browser carries only an ordinary session cookie; Spring Cloud Gateway supports this with its `TokenRelay` filter and an OIDC login. Interviewers ask “where do you store the JWT in the browser?” hoping for “nowhere that a script can read it, and the refresh token never in the URL” followed by the trade-off table above. “What about XSS?” has one answer that is not about storage at all: a Content Security Policy that stops foreign scripts from running in the first place, which is the defence every storage choice still depends on.

38.9 Questions from real interviews

Q: Where do you store the JWT in the browser, and why?

Start by separating the two tokens, because they have different lifetimes and different blast radii. The 15-minute access token can live in JavaScript memory: losing it on reload costs one silent refresh, and a script that steals it has fifteen minutes. The refresh token is the credential that matters — seven days here — and it belongs in an `HttpOnly`, `Secure`, `SameSite=Strict` cookie scoped by `Path` to the refresh endpoint, so no script can read it and the browser only ever sends it to one URL. `localStorage`, which this project uses today, is defensible for a first version and indefensible once any third-party script runs on the page: one `JSON.parse` away from a week of access. The senior close is that storage is the second line; the first is a Content Security Policy that keeps injected scripts from executing at all.

Q: Ten tabs get a 401 at the same moment. How many refresh calls happen?

With the interceptor as written, ten — each tab has its own memory and its own pending request, and within a single tab each parallel query runs the interceptor separately, so it can be more than ten. With refresh-token rotation on the server that is not waste, it is an outage: the first refresh rotates the token, the other nine present the old one and are rejected, and the reuse detection that good servers implement revokes the whole family. Inside one tab the fix is a shared promise — first 401 starts the refresh, later 401s await it, all replay with the result. Across tabs it is `navigator.locks`: tabs take the lock in turn, and the second finds the cookie already rotated and succeeds with a normal refresh. The follow-up worth volunteering is that the server should tolerate a short grace window on the previous

token, a few seconds, because network reordering means the “old” token can legitimately arrive after the rotation.

Q: XSS versus CSRF — which one does each storage choice expose you to?

Bearer tokens in storage: immune to CSRF, because the browser never attaches an `Authorization` header on its own, and fully exposed to XSS, because any script on the origin reads storage. Cookies: immune to XSS reading when `HttpOnly`, and exposed to CSRF because the browser attaches cookies to any request to that host, including one from an attacker’s page — unless `SameSite=Strict` or `Lax` stops it, and a custom-header or token check backs that up for older clients. So the real choice is between a problem with a mechanical fix (CSRF) and one without (XSS). A senior engineer adds that XSS on a page with an `HttpOnly` cookie can still *use* the session while the page is open — it just cannot exfiltrate the credential — which is why short access tokens and server-side revocation still matter.

Q: A user closes the laptop for eight days and comes back. What happens?

Both credentials are dead: the access token by fifteen minutes, the refresh token by a day. The first API call gets 401, the interceptor calls refresh, auth-service finds the row expired, deletes it and answers 401, and the client logs out and redirects to `/login`. That is the correct outcome; the quality question is what the user sees on the way. Every query that was in flight fails first, and if the query client retries failed requests three times with backoff — TanStack Query’s default — the redirect is preceded by several seconds of spinners and repeated 401s in the log. Configure the query client not to retry on 401, run one refresh on app mount before rendering protected routes, and the eight-day return is a clean bounce to login with the intended route preserved for after.

Q: How do you log a user out everywhere?

The refresh side is a database operation: delete every row in `refresh_tokens` for that user, not only the one the current browser presented. Every other device fails its next refresh and lands on login within fifteen minutes. The access side is the Chapter 28 problem — a signed token cannot be recalled — so state the window honestly and name the two ways to close it: a `jti deny-list` in Redis checked by the gateway, or shorter access tokens. In this project the concrete change is one repository method, `deleteAllByUser(user)`, called from a new `/auth/logout-all` endpoint and from the admin’s disable-user action. Mention the interview trap:

a client-side “logout” that only clears storage revokes nothing — the token is still valid in whoever copied it.

Q: A CORS error appears in production but never in development. What do you check?

First, whether the request is cross-origin at all in production and not in development — in this project Vite’s proxy and Traefik both keep the page and the API on one origin, so a CORS error in production means the deployment split them: a CDN for the bundle, a different host or port for the gateway. Then compare the Origin header the browser sent, byte for byte including scheme and port, against the gateway’s allowed list; `https://app.example` and `https://app.example:443` are different strings to Spring. Then the credentials rule: with cookies or with Credentials, the wildcard origin is rejected by the browser even if the server sends it, and `Access-Control-Allow-Credentials: true` must be present. Then the preflight: an OPTIONS that hits a route which strips the prefix and forwards to a service returns that service’s 401 instead of the CORS headers, so CORS must be answered at the gateway before routing — which CorsWebFilter does. Last, look at the server logs and expect to find nothing: the request succeeded there; the browser is the one refusing, and that is the shape of every CORS problem.

CHECK YOURSELF (answers: Appendix A)

1. Open DevTools on the running app and name the storage key that holds both tokens. Write the one-line JavaScript expression an injected script would use to read the refresh token, and say what it can do with it for the next seven days.
2. The interceptor calls bare `axios` for the refresh instead of `apiClient`. Explain the loop that would otherwise occur, step by step, starting from a 401 on `/refresh`.
3. With refresh-token rotation enabled on the server, trace four parallel 401s through the interceptor as written and state at which step the user is logged out. Then explain why the single-flight version avoids it.
4. The cookie design sets `Path=/api/auth/refresh`. Give two distinct attacks this attribute blunts, and explain why the path is written from the gateway’s point of view rather than `auth-service`’s.
5. `CorsConfig` lists two origins and allows credentials. In which of the project’s two deployment modes is it consulted, and what single change to the deployment would make it matter?
6. A user reports a CORS error; the gateway log shows the request re-

turned 200. Explain why both observations are true at once, and list the three header values you would compare first.

Quiz Answers

Chapter 1 — Spring Boot

1. The env var wins: env vars outrank the config-server document, which outranks the packaged YAML. Right for containers because the image must be immutable across environments; the runtime injects what differs.
2. @Configuration, @ComponentScan, @EnableAutoConfiguration. The last: auto-configuration is conditional on the classpath, so removing a dependency silently removes the beans it triggered.
3. It resolves to the literal string placeholder. The service can therefore *start* without Google credentials — OAuth fails at click time instead of boot time.
4. Exposure makes the endpoint exist; Spring Security still authorizes each request. If /actuator/** is not permitted, probes get 401, which the kubelet counts as failure.

Chapter 2 — Config Server

1. Env var > config-server > packaged YAML.
2. Native serves files from the classpath directory inside the jar; git adds audit, review, revert, and config changes without rebuilding the config-server.
3. optional: lets a service boot when the config-server is down — but then it boots on localhost defaults and fails later, far from the cause.
4. In its own application.yml, packaged in its jar. The bootstrapping principle: the source of configuration cannot configure itself; every such chain needs a self-contained root.

Chapter 3 — Eureka

1. course-service registers (POST) at startup; the gateway's discovery client refreshes its registry cache (default up to 30 s); routing then load-balances from cache. Worst case for a new instance to receive traffic: registration interval + cache refresh, tens of seconds.
2. Compose makes service *names* resolvable, and hostnames match them; in Kubernetes the pod hostname is the pod name, which no one can resolve

— so the IP must be registered instead.

3. Availability of registry data over accuracy. Helps: a switch reboot silences all heartbeats but instances still run. Hurts: mass real crashes leave dead instances registered.
4. The Service's endpoint list replaces the registry; readiness probes replace heartbeats; kube-proxy/DNS replace client-side lookup; the ReplicaSet replaces self-registration of new instances.

Chapter 4 — API Gateway

1. Browser → ts.local (hosts file) → Traefik (Ingress: /api → api-gateway Service) → gateway pod (predicate /api/auth/**, StripPrefix=2, JWT not required for /api/auth) → Eureka-resolved auth-service pod, arriving as /login → controller.
2. Google validates the redirect URI byte-for-byte; it was registered with the full /login/oauth2/... path, so it must arrive unmodified. API routes strip because controllers are written prefix-less.
3. Every request with a token signed by auth-service fails the gateway's validation — universal 401s on protected routes while login itself appears to work.
4. A proxy is I/O-bound with high concurrency — event loops excel. course-service does blocking JDBC work; reactive would complicate it without benefit until proven otherwise.

Chapter 5 — Security

1. CSRF rides an ambient credential (the session cookie sent automatically). A JWT in an Authorization header is attached explicitly by code; a forged cross-site request cannot add it.
2. Fifteen minutes (its expiry). Nothing — a signature cannot be revoked. /refresh: the refresh token is checked against the database, where revocation lives.
3. Invariant: every network path to a service passes the gateway's JWT filter. Breaks: an Ingress routing straight to a service (the deleted /courses route), or any in-cluster caller forging headers on the flat pod network.
4. The client-secret never leaves servers; the authorization code transits the browser. The code is single-use, short-lived, and worthless without the client-secret at exchange time.

-
5. HS256: one shared secret; anyone who can validate can also sign. RS256: private key signs, public key validates — so an IdP can let thousands of services validate without any of them being able to mint tokens.

Chapter 6 — Services and Common

1. (a) HTTP — the caller needs the data now. (b) Event — a fact others react to; grading must not wait on mail. (c) Neither: validation is local cryptography with the shared secret; no call.
2. auth-service only *produces* — sends fail per-call, boot succeeds. notification-service only *consumes*; without a broker it is pure idle cost, so it ships with Kafka.
3. Belongs: event records — both sides must agree on shape. Never: business logic/entities — shared behavior couples services' release cycles and hides the boundary.
4. Consumers deploy before producers, and changes are additive-only (new optional fields). Then old consumers ignore new fields and new consumers tolerate old events.

Chapter 7 — Postgres, JPA, Flyway

1. Two developers' schemas silently diverging (update applies per-database, no history); and destructive changes being impossible — update never drops/renames, so entity refactors quietly desync from the schema.
2. Each history row stores a checksum of its script; an edited script mismatches and Flyway refuses to start — the history is the audit trail, and edits would falsify it.
3. Release 1: add new column, write both (or trigger). Backfill migration. Release 2: read new column, stop writing old. Release 3: drop old column. Each step compatible with the version before it.
4. For: fault and blast-radius isolation — one server's crash, upgrade, or misconfiguration cannot touch the other's data. Against: double memory and operational surface for a homelab. Managed clouds usually take credential-isolation on one instance because servers are the billing unit.

Chapter 8 — MongoDB

1. Dictionary entries: self-contained, read/written as a unit, naturally nested, no cross-entry consistency. Enrollments: reference two other aggregates

- and require consistency between them — joins and transactions are the point.
2. Into the application code (and optional collection validators). Disciplines: backward-readable document classes (new fields optional), and explicit data-fix scripts for shape changes.
 3. Postgres refuses connections to a nonexistent database; Mongo creates databases on first write, so a wrong name yields empty results. Loud — failures you notice at deploy beat failures you notice at audit.
 4. ClusterIP-only + no Ingress means only in-cluster callers can reach it; the compose parity also keeps dev friction zero. Invalidated the moment it is exposed beyond the cluster or shares the cluster with untrusted workloads.

Chapter 9 — MinIO / S3

1. The service signs (microseconds) instead of streaming (seconds); storage serves bytes directly. New failure mode: the URL embeds an endpoint and expiry — wrong endpoint or clock skew breaks downloads in the browser only, invisible to the service.
2. The signing service reaches MinIO as `minio:9000` (cluster DNS); the browser cannot resolve that name. The signature covers the host, so rewriting after signing invalidates it — hence signing against a second, public endpoint.
3. Expiry (15 minutes) bounds the damage window; the URL grants one object, not the bucket; and it cannot be re-signed without the keys.
4. Changes: endpoint(s), credentials (ideally to an IAM role), durability guarantees. Unchanged: every SDK call, bucket/key/presign logic — the API is the standard.

Chapter 10 — Kafka Core

1. Key by `userId` — per-key order is per-partition order. Partitions $\geq$ max consumers you ever want in the group; parallelism is capped by partition count.
2. Groups are independent: each keeps its own offsets per partition, so both groups traverse the full log; consumption is non-destructive reading plus a per-group cursor.
3. `LISTENERS` = sockets bound; `ADVERTISED_LISTENERS` = addresses *returned to clients* in metadata. Bootstrap uses the address you supplied; all subsequent connections use the advertised ones — so the failure begins after a successful hello.

-
4. Retries and post-processing crashes both re-deliver; the broker cannot know if you acted. Notification: dedupe by event id (or accept a rare duplicate mail — idempotence by tolerance).

Chapter 11 — Spring Kafka

1. `send()` buffers client-side and returns a future; no broker needed until delivery. The outage surfaces per-send: futures fail after timeouts, and buffered events are lost if the process dies.
2. With no committed offsets, `earliest` starts from the oldest retained record — history is processed. Once the group commits offsets, the setting is never consulted again.
3. Without: deserialization throws inside poll, offset never advances, the partition stalls on one record forever. With: the error is materialized as a typed failure, the error handler skips (or retries then routes) — in full production, to `<topic>.DLT`.
4. JSON deserialization instantiates classes named in headers; an unrestricted trust list lets a message make the consumer construct arbitrary classpath types — a gadget-injection vector.
5. When “DB write + event publish” must not diverge. It makes the state change and the *recording of the intent to publish* atomic — one local transaction; a relay makes the publish eventually happen.

Chapter 12 — Docker

1. Namespaces (isolated views) and cgroups (resource limits). No guest kernel: millisecond starts and no per-instance OS cost — but also: the kernel is shared (weaker isolation than a VM) and the JVM sees host resources unless told otherwise.
2. Inside a container `localhost` is the container itself. Compose fixes it with service-name DNS on the bridge network; Kubernetes with Service DNS — both via env-var overrides of the localhost defaults.
3. Each container has its own network namespace, so both bind “their” 5432. Conflicts return at the shared host: published host ports (hence 5433/5434) — and with `-network host`, where there is no namespace separation at all.
4. Plain `depends_on` waits for *started*; `service_healthy` waits for the healthcheck to pass — here, config-server’s curl of its own `/actuator/health`, exposed by its management config.

Chapter 13 — Jib and the Registry

1. No daemon in CI, volatility-based layering (small re-pushes), non-root/OpenShift-ready images. The security one: no daemon means no docker socket in CI — the socket is root on the host.
2. The image name doubles as pull address for the cluster. They agree here because Jenkins shares the host's network namespace; if Jenkins were bridge-networked, its localhost:5000 would be itself and pushes would fail while pulls worked.
3. An import per node per deploy, minutes of serialization, and no immutable tag history; the pipeline's set image to a SHA-tagged name has nothing to point at — the registry is what makes tags addressable.
4. Same tag, new content: with IfNotPresent the node's cached image wins silently. Fixes: imagePullPolicy: Always (the manifests' choice), or deploy by immutable tags/digests (the pipeline's choice) — this project does both.

Chapter 14 — Cluster Anatomy

1. You write desired state; controllers reconcile actual toward it forever. A second apply produces zero diff, so zero action — idempotence by architecture.
2. Ordering is a procedural concept; convergence replaces it: everything starts, dependents crash and retry with backoff until dependencies exist. CrashLoop during convergence is the mechanism, not a bug.
3. kubectl sends a REST delete to the API server; state store updates; the ReplicaSet controller's loop sees replicas $0 < 1$ and creates a pod object; the scheduler assigns it; the kubelet on the node starts the container.
4. Apply merges desired state; it never deletes what is absent from input. kubectl delete ingress course-service -n ts, or an apply with `--prune` (or a GitOps controller) would.

Chapter 15 — Workloads

1. Each template change births a new ReplicaSet while old ones are retained at scale 0 — rolling update and one-command rollback fall out of keeping the counter separate from the versioned template.
2. Deployment: N interchangeable replicas, any can die, order irrelevant. StatefulSet: named replicas with per-identity storage and ordered replacement. Eureka: Deployment (registry rebuilt by re-registration). Redis

cache: Deployment if truly cache. Kafka broker: StatefulSet. Gateway: Deployment.

3. New pods must pass readiness before old pods terminate — Chapter 17's probes gate Chapter 15's rollout.
4. A Deployment update runs old and new pods concurrently; two postgres processes opening one data directory corrupt it. The StatefulSet replaces one-at-a-time and re-binds the claim to the successor, never two at once.

Chapter 16 — Config and Secrets

1. RBAC on who may read Secret objects; lifecycle separation from git; encryption at rest (optional). This project: RBAC yes (Jenkins' Role omits secret-read... it grants create/patch but the practical control is namespace scoping), git-separation yes (script), encryption at rest no.
2. Server image reads `POSTGRES_*`; Spring reads `SPRING_DATASOURCE_*`. Split Secrets mean each pod imports only names it understands — no foreign variables that mask typos or leak scope.
3. Update: `JWT_SECRET=new ./create-secrets.sh`; then immediately `kubectl -n ts rollout restart deployment/auth-service deployment/api-gateway`. Half-done: signer and validator hold different keys — every login 401s, or worse, only after the next unrelated restart.
4. The config-server's configurations/`*.yaml` tree, mounted as files — retiring the config-server itself.

Chapter 17 — Probes and Resources

1. Startup failing past budget: restart. Readiness failing: removed from endpoints, no restart — safe to be aggressive. Liveness failing: restart — the one to be conservative with.
2. `/v3/api-docs` answers only after the full context (Flyway included) is up and is permitAll'd; a bound port proves only Tomcat exists. `/actuator/health` is 401 where security does not permit it (course/auth) and honest where it does (dictionary, gateway).
3. The kernel's OOM killer terminated the cgroup for exceeding its memory limit — no JVM involvement, so no exception. Check: the limit vs. `MaxRAM-Percentage` arithmetic, and whether load or a leak grew the heap.
4. Scheduler reads requests (placement); kernel enforces limits. Memory is incompressible — exceeding it kills someone, so bound it predictably; CPU is compressible — limits would throttle startup exactly when the JVM needs

burst.

Chapter 18 — Networking

1. DNS resolves the Service name to a stable virtual IP; the endpoint controller keeps the Ready-pod list current; kube-proxy (or k3s' equivalent) rewrites virtual-IP traffic to a chosen pod IP; readiness decides membership.
2. Only Ready pods are endpoints, so a rolling update shifts traffic exactly when new pods pass probes and old ones terminate — zero-downtime is the probe-endpoint interaction.
3. Selector/label mismatch — the Service matches zero pods. Nothing errors because a Service with no endpoints is a legal state; the symptom is only visible in get endpoints.
4. ts.local is not real DNS; the hosts file pins it to the tailnet IP. Replacements: a real domain with actual DNS + cert-manager TLS, or a Cloudflare Tunnel publishing the hostname.

Chapter 19 — Storage

1. Pods mount PVCs; PVs (and StorageClasses) are cluster-scoped; the indirection lets the same claim get local-path here and EBS on EKS — manifests unchanged.
2. Postgres runs on one node at a time — RWO suffices. Shared uploads push people to RWX; object storage is usually the right answer instead.
3. StatefulSet deletion spares PVCs so data survives orchestration mistakes; PVC deletion (reclaim Delete) removes PV and data — claims, not workloads, are where data lives.
4. Persistence: survives restarts (yes). Durability: survives hardware failure (no — one SSD). Backup: survives your own mistakes (no — absent, the honest gap).

Chapter 20 — Kustomize

1. A definitive set (nothing forgotten, one command) and cross-cutting edits (namespace, labels) stamped consistently.
2. Namespace: cluster-scoped — Jenkins' namespaced Role cannot apply it. Secrets: values must not exist in git at all — shape in script, values in the operator's environment.

-
3. Kustomize: deploying your own app as readable YAML. Helm: distributing parameterized software to other people's clusters. Arriving as charts: Strimzi and kube-prometheus-stack.
 4. Nothing — apply does not prune; mongodb keeps running, now unmanaged. Correct: `kubectl delete -f mongodb.yaml` (then remove from the list), or prune/GitOps machinery that reconciles deletions.

Chapter 21 — CI/CD Concepts

1. CI: main is always verified buildable — the Build stage. CD: every verified change reaches production unaided — the Deploy stage plus its rollout gate.
2. Jenkins sits behind NAT with no public address — GitHub cannot call in, so Jenkins polls. Same constraint: CI inside a corporate network that the cloud git host cannot reach.
3. A rebuild is a different binary — different dependency resolutions, timestamps, toolchain — so tests certified something else. Chapter 1's env-var precedence lets one artifact take per-environment config at runtime.
4. Tests skipped (re-enable, or gate elsewhere and know it); deploy not verified (rollout status/smoke tests); success defined as exit-codes not behavior (probe-backed verification, alerts).

Chapter 22 — Jenkins

1. Jobs, credentials, plugin state, build history. Image = reproducible binary+plugins; volume = irreplaceable state; together the server is cattle with a pet directory.
2. Jenkins runs on the base image's JDK 21; builds export `JAVA_HOME=JDK25_HOME` in the Jenkinsfile's environment block — per-pipeline, not global.
3. apply -k needs get/list + create/patch/update on every applied kind; rollout status needs get/list/watch on deployments and replicasets. Drop-pable without set image: nothing new — set image's get+patch is already covered; the pods/log read rules exist only for the failure handler.
4. Jib pushes to localhost:5000, and that name must mean the same machine to pusher and puller; host networking makes Jenkins' localhost the host's, which is what the image name promised k3s.
5. ServiceAccount = identity; Role = capability list; RoleBinding = the join. Role-not-ClusterRole caps a compromised pipeline at one namespace — blast radius as a type choice.

Chapter 23 — The Pipeline

1. SNAPSHOT is mutable — a node could serve any historical build under the same name; the SHA pins the exact commit and makes “what is running?” answerable from `kubectl describe`.
2. Build only listed modules plus their dependencies. A forgotten MODULES entry: the new service never gets an image; the deploy loop then fails (or deploys a stale image if one exists) — the pipeline’s one manual coupling.
3. `kubectl rollout status --timeout=300s` — it converts probe results (Chapter 17) into pipeline exit codes.
4. All rollouts should proceed concurrently; waiting per-service serializes convergence that Kubernetes happily parallelizes — same total work, longer wall clock.
5. Changes: trigger (webhook), where builds run (ephemeral runners), secret mechanism (repo secrets). Unchanged: stage order, Maven commands, SHA tagging, the rollout verification.

Chapter 24 — Operations

1. `get pods` → `describe pod` → `logs` (–previous — essential for CrashLoop, where the current container has no history yet) → `exec` and probe from inside.
2. CrashLoop: the process exits — application layer. Running 0/1: the process lives but readiness fails — the probe’s target (often a dependency) is the layer to inspect.
3. Exec into the pod and try the connection from there — the pod’s DNS, network path, and injected env differ from your laptop’s; your laptop proves only that Postgres is alive, not that the pod can reach it.
4. k3s pod/service traffic traverses the host firewall; blocking 10.42.0.0/16 or 10.43.0.0/16 breaks DNS and pod-to-pod calls intermittently — symptoms indistinguishable from flaky applications.

Chapter 25 — Kafka in the Cluster

1. A controller whose reconciliation loop targets custom resource kinds defined by CRDs. Strimzi added `Kafka/KafkaNodePool` (the cluster) and `KafkaTopic` (the contracts) — plus `user`, `connect` and `bridge` kinds this project does not use.

-
2. The operator bundle contains cluster-scoped objects (CRDs, ClusterRoles) that a namespaced Role cannot grant; `kafka.yaml` contains only namespaced custom resources, which the Role now permits. Role vs. ClusterRole is the line.
 3. Advertised listeners, the controller quorum voters, the per-listener security protocol map (also the cluster ID and per-broker Services). The operator computes them from the listener declaration and the node pool.
 4. The broker pod's actual status (`get pods`) and the operator log's reconciliation lines. The condition may be a stale write from a reconciliation that timed out during a long image pull; the next periodic loop corrects it.
 5. Reviewed partition counts, no day-one producer/topic race, and an inventory of event contracts as objects. One partition: total order per topic, sufficient while every consumer group has one instance; partitions can be added later but never removed.

Chapter 26 — The Frontend and the Single Host

1. Stage one (Node) compiles and bundles; stage two (nginx) serves. Only `dist/` is copied with `COPY --from=build`; Node, `node_modules` and the compiler are discarded, leaving a ~15 MB image with a smaller attack surface.
2. Traefik matches `/courses/7` against both Ingresses; `/` is the longest matching prefix (no `/courses` rule exists), so nginx receives it. No such file exists; `try_files $uri $uri/ /index.html` falls back to `index.html`, the app boots and React Router renders the route from the URL.
3. Rules for the same host are merged into one routing table and the longest matching path prefix wins: `/api/courses` matches `/api` (longer) over `/`; `/assets/x.js` matches only `/`.
4. CORS applies only to cross-origin requests. In the cluster the page and the API share scheme, host and port (`http://ts.local`), so the browser never sends a preflight; on a laptop the page is `localhost:3000` and the API `localhost:8080` — a different port is a different origin.
5. The socket is owned by the host's docker group (GID 984 on `ts-server`); the jenkins user inside the container is not in a group with that numeric ID, so writes are denied. `--group-add` attaches the host's GID at run time; computing it keeps the script correct on any host without hard-coding a machine-specific number.

Chapter 27 — The Road Ahead

1. Consumer-group lag: the distance between the newest offset in a partition and the group's committed offset — growing lag means notification-service is falling behind or down.
2. Push: the credential lives in CI (Jenkins' kubeconfig). Pull: it never leaves the cluster; CI only writes git. Pruning resolves Chapter 20's apply-never-deletes pitfall.
3. Gateway routes change `lb://name` to `http://name:port`; Service endpoints replace registration, readiness probes replace heartbeats; delete discovery-server (and eureka client deps), free its resources.
4. They already run as non-root with group-0 permissions, so OpenShift's arbitrary-UID SCC admits them unmodified; only Ingress → Route remains.

Chapter 28 — Deep Dive: JWT at Scale

1. No. The config-server now holds only the public key, which verifies but cannot sign. Forging requires the private PEM, which lives solely in auth-service's Secret — that Secret (or the auth-service pod) is what would have to leak.
2. Publish the new public key in the JWKS under a new kid while keeping the old; wait for verifier caches to refresh; switch signing to the new private key; after the longest token lifetime, remove the old key. In step 3 both generations of token are in flight at once; kid tells each verifier *which* public key to apply, so old tokens keep verifying and the old key can later be retired with certainty rather than by trial.
3. Users already holding an access token are unaffected until it expires (15 minutes), because verifiers cached the public key. Then `/refresh` fails and they are logged out; new logins fail from the first minute. A verifier restarted during the outage boots with an empty JWKS cache and, failing closed, rejects every token — which is why the public key should also be available from configuration as a fallback.
4. Staleness: the claim is frozen at issue time, so a course created after login is invisible until the token expires — the token cannot know about data it does not own. Ownership: course membership is course-service's data; encoding it in an auth-service-issued token couples the two services' schemas through the token contract. (Size is the third reason, and the least important.)

-
5. The header is forwarded, but no public endpoint reads `X-User-Id`: `/refresh` authenticates by refresh token, `/actuator` by nothing. So it cannot be exploited today, yet the invariant “only the gateway writes these headers” is unenforced. `default-filters: RemoveRequestHeader=X-User-Id (and X-User-Role)` in `api-gateway.yml` enforces it.
 6. Up to 15 minutes — the access-token lifetime — since the gateway verifies locally and only `/refresh` consults the database. A `jti deny-list` adds Redis (one lookup per request, Redis down means fail-open or fail-closed by your choice); the `phantom-token` pattern adds Redis on the critical path of every request with no fail-open option — Redis down is a full outage.

Chapter 29 — Deep Dive: Configuration

1. The Secret wins: it arrives as the `env var SPRING_DATASOURCE_PASSWORD`, and OS environment outranks the `config-server` document. If the Secret object is missing, `envFrom` fails the pod; if only the key is missing, the pod starts, the served file’s literal `root` takes effect, and the service connects as `root`, silently.
2. Precedence chooses *which document* defines `app.s3.secret-key`; the `config-server`’s document does, and it sends the literal text `$APP_S3_SECRET_KEY:minioadmin` unresolved. The client resolves the placeholder against its own Environment afterwards, so the `env var` wins because the text asked for it, not because of source order.
3. Window one: between the two refreshes, `auth-service` signs with the new key while the gateway verifies with the old — every login yields a token the gateway rejects. Window two: after both refresh, every token issued before the change is rejected until it would have expired anyway. Hot rotation needs overlapping keys selected by `kid`, which a single shared HS256 secret cannot express.
4. Non-optional: the fetch fails, the JVM exits, `kubelet` restarts with growing back-off until the server returns. `optional::` boots from packaged defaults pointing at `localhost`, passes its readiness probe, takes traffic, fails every request. `fail-fast` with `retry`: retries in-process with back-off for the configured window, then exits — no wasted JVM starts, and no lying pod.
5. `Kustomize`’s generator appends a content hash to the `ConfigMap` name; a changed file is a new name, the Deployment referencing it is a new pod template, and Kubernetes rolls the pods. `Config`, `code` and `rollback` become one mechanism with history — the server offered none of that without `Bus` or restarts.

6. Remove it when every runtime is Kubernetes, no value must change without a pod restart, and secrets already live in the platform. Keep it when a non-Kubernetes runtime (VMs, Compose in production) must share the same configuration source.

Chapter 30 — Deep Dive: Service Discovery

1. Registration is immediate on web-server start (Spring forces it). Then the server's read-only response cache (30 s), the gateway's registry fetch interval (30 s), and the LoadBalancer cache TTL (35 s): about 95 s worst case, roughly half that typically.
2. Lease expiry 90 s, then the eviction sweep (up to 60 s), then server cache 30 s, client fetch 30 s, LoadBalancer cache 35 s — up to 245 s. A graceful shutdown sends DELETE and skips the first two (150 s), leaving only the caches.
3. Expected renewals per minute: $5 \times 2 = 10$; threshold $\lfloor 10 \times 0.85 \rfloor = 8$; the server stays healthy only while renewals exceed 8. One dead client gives exactly 8, so self-preservation engages and nothing is evicted. `eureka.server.enable-self-preservation: false` (or a lower `renewal-percent-threshold`) fixes it on a small fleet.
4. Eureka's self-reported status gates the gateway path, because `lb://` resolves to a registered pod IP and never touches the Kubernetes Service where readiness is enforced. Routing to `http://course-service:8082` puts kube-proxy, and therefore readiness, back on the path.
5. Compose activates no profile, so the base document applies: Eureka client enabled, `lb://` routes, and the Feign url resolves to an empty string, which Feign treats as "use load-balanced resolution by name". Under k8s the property is a Service URL and Feign bypasses the LoadBalancer entirely.
6. The registry knows an instance only through what the instance says about itself, and learns of silence only after a lease it does not sweep immediately; a Service's endpoints are maintained by the kubelet, which actively probes the pod every few seconds and removes it the moment a probe fails. Self-report with a grace period versus external observation with none.

Chapter 31 — Deep Dive: The Gateway Under Load

1. Nothing has been sent yet: `chain.filter` returns a Mono describing the rest of the chain; it runs only when the framework subscribes, and the calling thread is already free. A servlet filter's `doFilter` blocks until the downstream

call completes, so the thread is held for the whole request — which is the model the event loop exists to avoid.

2. Every request whose socket is owned by the stalled loop stops — including routes that never touch Redis — because loops are shared across routes, not per route. Reactor Netty runs $\max(\text{cores}, 4)$ loops; on a 2-core box that is four, so one blocked loop is a quarter of the gateway's capacity.
3. Sixteen: the default per-host pool is $\max(\text{cores}, 8) \times 2$. With no response timeout, 16 hung requests hold all 16 connections and the 17th waits 45 s in the acquire queue. Setting `spring.cloud.gateway.httpclient.response-timeout` (or raising `pool.max-connections`) changes the number; the timeout is the one that fixes the cause.
4. The filter wraps each call in a Resilience4j TimeLimiter whose default is 1 s. The 1.5 s upload is cut off, counted as a failure, and after enough of them the breaker opens against a healthy service. Set `resilience4j.timelimiter.instances.<name>.timeout-duration` at least as long as the response timeout.
5. With two or more replicas an in-memory bucket gives each replica its own limit, so a user gets $N \times$ the quota. Redis holds one bucket per key, updated atomically by a Lua script. Trade-off: Redis is a dependency on every request; the filter's default when Redis is unreachable is to allow the request (fail-open), which must be a deliberate choice.
6. Every request is non-simple, so in dev each API call is preceded by an OPTIONS preflight. `setMaxAge` lets the browser cache the preflight result (Chrome's default is 5 s). In the cluster the frontend and `/api` share one origin behind Traefik, so no request is cross-origin and the CORS filter is never consulted.

Chapter 32 — Deep Dive: Synchronous Calls

1. (a) No code calls `AuthServiceClient` — wire it into a service such as `UserLookupService`. (b) Feign's breaker integration is off — set `circuit-breaker.enabled: true` under `spring.cloud.openfeign`. (c) No `CircuitBreakerFactory` bean exists — add the `spring-cloud-starter-circuitbreaker-resilience4j` dependency. Separately, nothing references the `authService` instance — annotate the calling method with `@CircuitBreaker(name = "authService")`.
2. Nine: `minimum-number-of-calls` defaults to the window size, so the breaker does not evaluate until the tenth outcome. A 30 s call is a success because `slow-call-duration-threshold` defaults to 60 s; only calls slower than that count as slow, and only if the `slow-call rate threshold` (default 100 %) is met.

3. Browser (longest) > gateway response timeout > Feign read timeout $\times$ retry attempts (plus backoff). Each hop must give up before its caller does, or it works on abandoned requests; and the retry count multiplies the innermost timeout, so it must fit inside the gateway's budget — 2 s $\times$ 2 attempts under a 5 s gateway timeout, not 3.
4. A `NullPointerException` in every caller that forgets to check, surfacing only during an outage; and the inability to tell a 404 (user genuinely missing) from a breaker rejection, so both get the same treatment. `degraded(Long id, Throwable cause)` — the two-argument fallback — receives the cause and can return a cached or placeholder value for infrastructure failures while letting a 4xx propagate.
5. `auth-service`'s `SecurityConfig` permits only public paths; `/users/{id}` needs an authenticated context that its `GatewayHeaderAuthFilter` builds from `X-User-Id`, which Feign does not send. Each 401 is an exception, so ten of them open the breaker against a healthy service. Fix: a Feign `RequestInterceptor` that forwards the identity headers, and `ignore-exceptions` for 4xx so client errors never count as dependency failures.
6. $0.999^3 \approx 99.7\%$. Raise it by removing dependencies from the request path — a local copy fed by events, or a cache in front of the call — and by making the remaining ones optional with a typed fallback so their failure degrades the response instead of failing it.

Chapter 33 — Deep Dive: Data Across Services

1. Order A: the transaction commits, then the JVM dies before the producer buffer flushes — user activated, no event, no student ever. Order B: the buffer flushes and the broker acks, then the commit fails — course-service creates a student for a user that was rolled back. B produces the phantom row.
2. The producer cannot fetch metadata, blocks for `max.block.ms` (60 000 ms), then throws; the exception leaves `register()`, the transaction rolls back, and the user is not created. With the outbox the request writes a user row and an outbox row in one commit and returns in milliseconds; the relay retries the send in the background.
3. Two replicas run the same scheduler. Without the lock both select the same unpublished rows and both publish them — a duplicate for every event, every second. `FOR UPDATE` makes the second replica wait; `SKIP LOCKED` makes it take the next rows instead, so they share the work without overlap.
4. Ordering per key. If row 1 for user 42 fails and the loop continues to row 2 for

the same user, row 2 is published first; the next tick publishes row 1 behind it, and the consumer sees “updated” before “activated”. Stopping the batch keeps the partition’s order equal to the outbox’s order.

5. **Nothing:** `ts.user.activated` fires once, and no `ts.user.updated` exists, so `student.email` is frozen at activation. **Changes:** emit an outbox row with `eventType=UserUpdated` wherever the profile is saved; make `UserActivatedListener` **upsert** (find-or-create, then set fields); subscribe it to both topics or add a second listener.
6. **States:** `ACTIVE` → `DELETING` → deleted, or back to `ACTIVE`. **Events:** `UserDeletionRequested` (auth, via outbox), `StudentDataDeleted` / `StudentDataDeletionFailed` (course, after removing student, enrollments, submissions, and `MinIO` objects). **Compensation:** on failure auth restores `ACTIVE` and alerts. `DELETING` is the semantic lock: it refuses logins and new enrollments while the saga runs, so no step operates on data another step is deleting.

Chapter 34 — Deep Dive: Kafka Delivery Guarantees

1. `enable.idempotence=true` (broker de-duplicates producer retries), `acks=all`, unbounded retries within `delivery.timeout.ms` — all Kafka 3.x client defaults the project never set. On Compose there is one broker and `replicas: 1`, so “all in-sync replicas” is one disk; the setting only buys durability once `replication.factor=3` and `min.insync.replicas=2` exist.
2. A batch is polled; the listener sends mails one by one on the poll thread; processing exceeds `max.poll.interval.ms` (300 000 ms); the broker evicts the consumer and reassigns the partition from the last committed offset; the new owner processes the same records while the old one finishes its loop.
3. If the claim commits alone and the business write then fails, the retry finds the id already claimed, returns, and the offset advances — the change is lost forever while the table says it happened. In one transaction, failure rolls back the claim too, so the retry can process it.
4. A wait only helps transient faults: an SMTP relay coming back, a network blip. A `NullPointerException` or a deserialization error is deterministic; retrying it three times delays the same failure. It is excluded, so it goes straight to the DLT on the first attempt.
5. Boot’s `KafkaAutoConfiguration` backs off when a `ProducerFactory` bean exists, so `spring.kafka.producer.*` in `course-service.yml` is ignored by `course-service`. **Fix:** delete `KafkaProducerConfig` (Boot’s factory reads the same serializers from YAML) — or keep the bean and delete the YAML block.

6. Retention is 168 hours; nine days is longer. The group's committed offset now points before the oldest retained record, so auto-offset-reset: earliest applies and it resumes from the oldest surviving record — two days of events are gone. With a new group id there is no committed offset at all; earliest makes it process all seven retained days, re-sending every mail in them.

Chapter 35 — Deep Dive: Files at Scale

1. `endpointOverride`: the client uses `endpoint` (`minio:9000`), the presigner uses `publicEndpoint`. If the presigner signed against the internal name, every download link would embed a host the browser cannot resolve — and rewriting it afterwards breaks the signature.
2. `SigV4` signs a canonical request that includes the signed headers, and `Host` is always among them (`X-Amz-SignedHeaders=host`). A link signed for `minio:9000` and rewritten to another host presents a different canonical request; `MinIO` recomputes the signature, it does not match, and it returns `403 SignatureDoesNotMatch`.
3. The transaction covers the `Postgres` insert only; the `putObject` is an HTTP call outside it. A failed `save` rolls back the row while the object remains in the bucket under a key no row records — an orphan.
4. Cheap: listing every attempt by one student for one assignment (one `ListObjectsV2` with the prefix). Expensive: finding all files by one student across assignments (no prefix matches; full scan or database query). The `UUID` makes every key unique, so no upload ever overwrites another; versioning would only protect against overwrites that cannot happen.
5. Text submission succeeds (no storage call). Listing succeeds and returns links (presigning is local computation). Opening a link fails (`MinIO` must serve it). Restarting course-service fails: the `@PostConstruct` bucket check throws on connection refused.
6. The key is built server-side from `assignmentId` and the caller's own `studentId`, and stored on a `PENDING_FILE` row owned by that student; `completeUpload` checks that the row belongs to the caller and that a `HEAD` on *that* row's key succeeds. A URL for another student's key cannot be attached to a different row because rows are never given a client-supplied key.

Chapter 36 — Deep Dive: The Dictionary on MongoDB

1. The `FETCH` stage's filter rejects them. The `createdByUserId` index only narrows to the user's slice; an unanchored, case-insensitive `$regex` has no lead-

ing characters to seek by, so every fetched document is tested. Anchoring the pattern (`^clou`) with a matching collation lets an index on word serve it as a range.

2. All matches — potentially all 3,000 — come back from Mongo; `toResponse` then runs two `word_links` queries per match (about 6,000); the page is built in Java by `subList` after sorting the full list.
3. Multi-document transactions need a replica set; the project runs a standalone `mongod`. A crash after deleting the definition and before deleting its links leaves edges pointing at a missing word. `findByIdAndCreatedByUserId(...).orElse(null)` followed by the filter(`Objects::nonNull`) drops such nodes silently.
4. `BsonMaximumSizeExceededException` (16 MB limit). Six photos of about 2–3 MB each, inflated by a third as base64, exceed 16,793,600 bytes. Chapter 35’s rule: bytes in object storage, keys in the database — store the images in MinIO and keep their keys in images.
5. A text index tokenizes and stems whole words; “lou” is not a token of “cloud”, so there is no hit. Infix matching needs a regex scan or an n-gram/trigram index (Atlas Search, or `pg_trgm` in Postgres).
6. dictionary-service fails on every endpoint; every other service and topic is unaffected (no shared connection, no caller, no event). Adding a Mongo ping to the readiness probe would take the pod out of the gateway’s endpoints, turning 500s into a clean 503 at the edge.

Chapter 37 — Deep Dive: Observability

1. Producer side: `spring.kafka.template.observation-enabled` (absent in every config document); consumer side: `spring.kafka.listener.observation-enabled`. Properties configure only the *auto-configured* `KafkaTemplate`; `auth-service` uses that one, so the property fixes it. `Course-service` builds its own template in `KafkaProducerConfig`, which ignores the property and needs `setObservationEnabled(true)` in code.
2. It continues the trace: Boot 3’s default is to *consume* W3C, B3 and B3-multi and to *produce* W3C, decided by `management.tracing.propagation` defaults. The header leaving the gateway is `traceparent`, with the same trace id and a new parent span id.
3. Exposure is a name list; the endpoint exists only when a registry that renders that format is on the classpath. No service depends on `micrometer-registry-prometheus`, so `Actuator` has nothing to expose under that name. Adding that one artifact (Boot BOM-managed) makes the existing exposure

line real.

4. Latency: unchanged — spans are queued and shipped by a background thread. Heap: bounded — the queue holds 10,000 spans and drops on overflow with a counted warning. Spans: those produced during the hour are lost; there is no replay. With a synchronous reporter, *latency* would change: every request would block on a failing HTTP call to Zipkin until its timeout.
5. One percent — head-based sampling decides at the root before the outcome is known, so failures are kept at the same rate as successes. Raising it to 1.0 for errors only requires tail-based sampling, which needs an OpenTelemetry Collector buffering complete traces before the store; the direct Brave-to-Zipkin path cannot do it.
6. An e-mail is an unbounded tag value; each distinct value creates a new time series in the metrics registry and a new indexed value in the trace store. Memory grows with the user base until `/actuator/metrics` crawls and the pod is OOMKilled. Rule: tags carry *templates and enums* (route template, status, outcome); identifiers go into span annotations or log lines, which are not indexed by value.

Chapter 38 — Deep Dive: The Browser Meets the Gateway

1. Key auth-storage in `localStorage`, written by Zustand's `persist`. One line: `JSON.parse(localStorage.getItem('auth-storage')).state.refreshToken`. With it, the script (or whoever it sends it to) can call `/api/auth/refresh` for seven days and mint fresh access tokens at will, from any machine.
2. A 401 from `/refresh` through `apiClient` would enter the response interceptor; `_retry` is unset on that new request, so it would read the refresh token and POST `/refresh` again through `apiClient`; that 401 enters the interceptor again — an unbounded chain of refresh calls, each spawning the next, until the tab is closed. Bare `axios` has no interceptor, so its 401 reaches the `catch` and logs out.
3. All four get 401 and each sets its own `_retry`; four POSTs to `/refresh` race. The first succeeds and rotates the token; the second presents the now-invalid old token and gets 401 from auth-service; that lands in the `catch` — step 4, `logout()` and a hard redirect — while the first request's replay may even succeed. Single-flight: one promise, started by the first 401, awaited by the rest; one POST, one rotation, four replays with the same new token.
4. (a) A vulnerable or forgotten endpoint elsewhere under `/api` never receives the cookie, so a CSRF or log-leak bug there cannot touch it; (b) the cookie is not attached to ordinary API calls, so it never appears in gateway access

logs, proxies or browser traffic for `/api/courses`. The browser matches Path against the URL *it* requests, which is the gateway's `/api/auth/refresh`; auth-service's stripped `/refresh` is never a browser-visible path.

5. In neither: Vite's dev proxy and Traefik in the cluster both keep page and API on one origin, so no request is cross-origin and no preflight occurs. Serving the built bundle from a different origin — a CDN, or a static host on another port without the proxy — would make every API call cross-origin and the config decisive.
6. CORS is enforced by the browser after the response arrives: the gateway processed the request and returned 200, then the browser refused to hand the body to the script because the response lacked an acceptable `Access-Control-Allow-Origin`. Compare: the request's `Origin` header versus the allowed-origins list (exact string, scheme and port); `Access-Control-Allow-Credentials` versus whether the request was credentialed; and for the preflight, `Access-Control-Allow-Headers` versus the headers the request actually sends (`Authorization`, `Content-Type`).

Glossary

Actuator Spring Boot’s operational endpoints (`/actuator/health`, `/metrics`, `/prometheus`); exposure and authorization are separate decisions (Ch. 1).

Advertised listener The address a Kafka broker tells clients to use after bootstrap — the usual cause of connects-then-times-out (Ch. 10).

Algorithm confusion Attack that rewrites a token’s `alg` header so a verifier applies HMAC with the public key as secret; defeated by letting the key type, not the header, choose the algorithm (Ch. 28).

Artifact Anything a build produces worth keeping (jar, image, report); must be immutable and traceable (Ch. 21).

Auto-configuration Spring Boot conditionally applying configuration based on classpath, properties, and user beans (Ch. 1).

Backend for frontend A small service that composes several service calls into one client-shaped response; the right home for aggregation the gateway must not do (Ch. 31).

Bootstrap servers The initial Kafka addresses a client contacts to fetch metadata; all later traffic uses advertised listeners (Ch. 10).

Bulkhead Cap on concurrent calls to one dependency (semaphore or dedicated thread pool) so a slow dependency cannot consume every caller thread (Ch. 32).

Cipher value A `{cipher}`-prefixed config-server value encrypted with the server’s key so the repository holds only ciphertext; decrypted on serve (Ch. 29).

Circuit breaker State machine over a sliding window of call outcomes: CLOSED passes calls, OPEN rejects them instantly for a wait period, HALF_OPEN probes with a few calls (Ch. 32).

ClusterIP Default Service type: a stable virtual IP reachable only inside the cluster (Ch. 18).

Consumer group A named team of Kafka consumers splitting a topic’s partitions; each group keeps its own offsets (Ch. 10).

Consumer lag Distance between a partition’s newest offset and a group’s committed offset; measure it in seconds, alert on it (Ch. 34).

Controller (k8s) A reconciliation loop converging actual state to desired state for one kind of object (Ch. 14).

CORS / preflight Browser-enforced relaxation of same-origin policy; non-simple cross-origin requests cost an extra OPTIONS round trip unless cached — and never occur when page and API share one origin (Ch. 31, 38).

CQRS Command/query separation across services: writes go to the owning service, reads to a model built from its events (Ch. 33).

CrashLoopBackOff Pod status: the container keeps exiting; restart delay doubles each time (Ch. 24).

CRD / Operator Custom resource kinds plus a controller for them — how Strimzi manages Kafka declaratively (Ch. 25).

Dead-letter topic Where a consumer routes messages that failed processing repeatedly (Ch. 11).

Deployment Workload controller for interchangeable replicas; manages ReplicaSets; enables rolling updates and rollback (Ch. 15).

Digest The immutable content hash of an image (@sha256:...); tags move, digests do not (Ch. 13).

Dual write Writing to two systems (database and Kafka) with no shared transaction; one can succeed while the other fails (Ch. 33).

Endpoint (k8s) A Ready pod backing a Service; the live list is maintained by label selector and readiness (Ch. 18).

Eureka Netflix/Spring Cloud service registry: registration, heartbeats, client-side caching (Ch. 3).

Event loop Reactor Netty's small fixed pool of threads, each multiplexing many sockets with non-blocking I/O; one blocking call stalls every request on that loop (Ch. 31).

explain (Mongo) `find(...).explain("executionStats")`: the query plan with IXSCAN/FETCH stages and `totalDocsExamined` versus `nReturned` — the first thing to read when a query is slow (Ch. 36).

Fallback The value or action substituted when a protected call fails or is rejected; should be cached, degraded-but-typed, empty, or a typed error — never null (Ch. 32).

Flyway Versioned, immutable SQL migrations applied at startup and recorded in a history table (Ch. 7).

GitOps Deployment model where an in-cluster agent pulls desired state from git and converges the cluster to it (Ch. 26).

Head-based / tail-based sampling Deciding at the root span whether to keep a trace (cheap, blind to outcome) versus deciding after the whole trace is known (keeps errors and slow requests; needs a collector) (Ch. 37).

HS256 / RS256 JWT signature algorithms. HS256: one shared secret both signs and verifies. RS256: RSA private key signs, public key verifies — one issuer, any number of verifiers (Ch. 28).

HttpOnly cookie A cookie JavaScript cannot read; with Secure, SameSite and a narrow Path it is the safe home for a refresh token (Ch. 38).

Idempotent consumer A listener that claims each event id in the same transaction as its side effect, so redelivery is a no-op (Ch. 34).

Idempotent producer Kafka producer whose retries the broker de-duplicates by sequence number; default since Kafka 3.0 (Ch. 34).

Ingress L7 HTTP routing object (host/path → Service), executed by an Ingress controller such as Traefik (Ch. 18).

Jib Maven plugin building layered OCI images without a Docker daemon (Ch. 13).

JWKS JSON Web Key Set: the public keys an issuer publishes at a well-known URL so verifiers can fetch and cache them (Ch. 28).

JWT Signed, self-contained identity token: header.payload.signature; validated without a database (Ch. 5; at scale, Ch. 28).

Keyset pagination Paging by “rows after this key” on an indexed sort column instead of skip, so deep pages cost the same as the first (Ch. 36).

kid Key id in a JWT header naming the key that signed it; what makes rotation possible while old tokens are in flight (Ch. 28).

KRaft Kafka’s built-in Raft consensus replacing ZooKeeper (Ch. 10).

kubelet The per-node agent that runs pods and executes probes (Ch. 14).

Kustomize Template-free manifest composition: bases, overlays, patches; built into kubectl (Ch. 20).

Lease (Eureka) An instance’s registration, renewed by 30s heartbeats and expired after 90s of silence; swept by the server every 60s, so a dead instance can linger for minutes (Ch. 30).

Liveness probe “Restart me if this fails” — the probe to be most conservative with (Ch. 17).

Multipart upload S3 mechanism splitting a large object into parts of at least 5 MB uploaded in parallel and assembled server-side; abandoned parts need a lifecycle abort rule (Ch. 35).

Namespace Cluster partition; scope for names and RBAC (Ch. 14).

OOMKilled Exit 137: the kernel killed the container for exceeding its memory limit; no application trace exists (Ch. 17).

Outbox pattern Making DB-write + event-publish effectively atomic by writing

the event to a table in the same transaction (Ch. 11; Ch. 33).

Partition Kafka's unit of ordering and parallelism; an ordered immutable record sequence (Ch. 10).

Path-style addressing S3 URL form with the bucket in the path (host/bucket/key) rather than as a DNS subdomain; required by MinIO without wildcard DNS (Ch. 35).

Phantom token Opaque identifier outside, full JWT inside; the gateway swaps one for the other via a fast store, giving thin external tokens and instant revocation at the cost of a new dependency (Ch. 28).

Poison pill A message that fails deserialization forever, stalling its partition unless error handling skips it (Ch. 11).

Presigned URL A time-limited, signed link letting a browser access one object directly on S3/MinIO (Ch. 9).

Probe (startup/readiness/liveness) kubelet health checks gating restart, traffic, and boot grace respectively (Ch. 17).

Property source precedence Spring Boot's ordered list of configuration sources; env vars outrank the config-server document, which outranks the packaged YAML (Ch. 29).

PV / PVC / StorageClass Provisioned storage, a workload's claim on it, and the factory that makes PVs on demand (Ch. 19).

RBAC Role-based access control: ServiceAccount (identity) + Role (verbs on resources) + RoleBinding (the join) (Ch. 22).

Reconciliation The control loop: compare desired and actual state, act on the difference, forever (Ch. 14).

RED / USE Rate, Errors, Duration for request-driven services; Utilization, Saturation, Errors for resources — the two dashboard vocabularies (Ch. 37).

Refresh scope Beans rebuilt on /actuator/refresh so a config change applies without restart; per instance unless broadcast by Spring Cloud Bus (Ch. 29).

Registry Image storage speaking the Docker/OCI protocol; the interface between builder and cluster (Ch. 13).

Relaxed binding Spring's mapping of `SPRING_DATASOURCE_URL` to `spring.datasource.url` (Ch. 1).

ReplicaSet The counter keeping N pods of one template alive; managed by Deployments, kept for rollback (Ch. 15).

Retry storm Synchronized retries from many clients that multiply load on an already failing dependency; prevented by jitter, retry budgets, and retrying only idempotent calls (Ch. 32).

- Retry topic** Per-delay topic a failed record is re-published to (-retry-2000, ...), ending in a dead-letter topic; Spring's @RetryableTopic (Ch. 34).
- Rolling update** Replacing pods incrementally, gated on readiness, so service never fully stops (Ch. 15).
- Saga** A multi-service write as a chain of local transactions, each with a compensating action if a later step fails (Ch. 33).
- Secret (k8s)** Namespaced key/value object for credentials; base64-encoded, protected by RBAC not cryptography (Ch. 16).
- Self-preservation** Eureka mode entered when renewals fall below 85% of expected; eviction stops entirely — triggered by a single crash on a five-instance fleet (Ch. 30).
- Service (k8s)** Stable name + virtual IP over a label-selected, readiness-filtered set of pods (Ch. 18).
- ServiceAccount** A non-human identity for API access — what Jenkins deploys as (Ch. 22).
- SigV4** AWS Signature Version 4: an HMAC chain over date, region and service signs a canonical request that always includes the Host header — a pre-signed URL is bound to the host it was signed for (Ch. 35).
- Single-flight refresh** One shared pending promise so that concurrent 401s trigger exactly one token refresh and all replay with its result; mandatory once refresh tokens rotate (Ch. 38).
- StatefulSet** Workload controller for stateful replicas: stable names, per-replica PVCs, ordered replacement (Ch. 15).
- Text index (Mongo)** Index that tokenizes and stems string fields with weights, queried via \$text/TextCriteria; matches whole terms, not substrings (Ch. 36).
- Token bucket** Rate-limit algorithm: tokens refill at a steady rate up to a burst capacity and each request spends one; Spring Cloud Gateway keeps the bucket in Redis (Ch. 31).
- TOTP** Time-based one-time passwords: shared secret + clock, RFC 6238 (Ch. 5).
- traceparent** The W3C Trace Context header carrying trace id, parent span id and sampling flag between services; Boot 3 emits it and also accepts B3 (Ch. 37).
- Trusted edge** Gateway validates JWTs once and forwards identity headers; services trust them — valid only if no path bypasses the gateway (Ch. 5).

Command Cheat Sheet

kubectl — daily

```
kubectl -n ts get pods           # the first command, always
kubectl -n ts get pods -w       # ...and watch it change
kubectl -n ts describe pod <pod> # platform's view; read Events
kubectl -n ts logs <pod>        # app's view
kubectl -n ts logs <pod> --previous # the container that just died
kubectl -n ts exec -it <pod> -- sh # see the world from inside
kubectl -n ts get endpoints <svc> # is anyone behind the Service?
kubectl -n ts get events --sort-by=.lastTimestamp | tail
```

kubectl — deploy and rollback

```
kubectl apply -k deploy/k8s/base # converge on the manifests
kubectl kustomize deploy/k8s/base | less # render without applying
kubectl -n ts set image deployment/<s> <s>=localhost:5000/<s>:<tag>
kubectl -n ts rollout status deployment/<s> --timeout=300s
kubectl -n ts rollout undo deployment/<s> # back to previous ReplicaSet
kubectl -n ts rollout restart deployment/<s> # re-read Secrets, re-pull
kubectl -n ts get rs -l app=<s> # revision history, live
```

kubectl — capacity and RBAC

```
kubectl top pods -n ts # actual vs requested
kubectl -n ts get pvc; kubectl get pv # storage and its backing
kubectl auth can-i <verb> <resource> -n ts \
  --as system:serviceaccount:ts:jenkins # test permissions dry
```

docker — host services on ts-server

```
docker ps # jenkins, registry up?
```

```
docker logs -f jenkins
docker restart jenkins
curl -s localhost:5000/v2/_catalog      # what the registry holds
curl -s localhost:5000/v2/<name>/tags/list
docker system df; docker system prune  # disk hygiene (careful)
```

maven — local and CI

```
mvn -B -pl <modules> -am verify      # build listed + deps
mvn -B -pl course-service -am jib:build # image for one service
mvn -B jib:build -Djib.to.tags=<sha>   # extra tag, as CI does
mvn dependency:tree -pl course-service # who pulled that jar in
mvn spring-boot:run -pl course-service # run one service raw
```

compose — the dev stack

```
docker compose up -d postgres-course kafka minio # infra only
docker compose up --no-deps course-service        # one service, own terminal
docker compose logs -f --tail=100 <svc>
docker compose down                               # keep volumes
docker compose down -v                             # DESTROY volumes too
```

k3s — the cluster itself

```
sudo systemctl status k3s
sudo systemctl restart k3s      # rereads registries.yaml; pods bounce
sudo ls /var/lib/rancher/k3s/storage/ # PVs as directories
kubectl get nodes -o wide
```

kafka — the broker in the cluster (Ch. 25)

```
kubectl -n ts get kafka,kafkanodepools,kafkatopics # operator's view
kubectl -n ts logs ts-kafka-dual-role-0 --tail=50  # the broker itself
# a throwaway pod with the Kafka CLI, same network position as services:
kubectl -n ts run kcli --rm -it --restart=Never \
  --image=quay.io/stnimzi/kafka:1.2.0-kafka-4.3.1 -- sh
B=ts-kafka-kafka-bootstrap:9092
```

```
bin/kafka-topics.sh --bootstrap-server $B --list
bin/kafka-console-consumer.sh --bootstrap-server $B \
  --topic ts.user.registered --from-beginning
bin/kafka-consumer-groups.sh --bootstrap-server $B \
  --describe --group notification-group    # lag per partition
kubectl -n ts port-forward svc/maildev 1080:1080    # caught mail UI
```

The Configuration Files, Verbatim

The files as they stand in the repository at build time — included directly from source, so this appendix cannot drift from reality. Line-by-line commentary lives in the chapters; this is the reference copy for reading offline.

Jenkinsfile

```

pipeline {
  agent any

  options {
    timestamps()
    disableConcurrentBuilds()
    buildDiscarder(logRotator(numToKeepStr: '10'))
  }

  triggers { pollSCM('H/5 * * * *') }

  environment {
    JAVA_HOME = "${env.JDK25_HOME}"
    PATH      = "${env.JDK25_HOME}/bin:${env.PATH}"
    TAG       = "${env.GIT_COMMIT.take(7)}"
    NS        = 'ts'
    REGISTRY  = 'localhost:5000'
    // frontend is in SERVICES (kubectrl) but not MODULES (Maven): it is built by
    // the 'Frontend image' stage with docker, not by Jib.
    SERVICES  = 'config-server discovery-server course-service api-gateway auth-service dictionary-
    ↪ service notification-service frontend'
    MODULES   = 'common,config-server,discovery-server,course-service,api-gateway,auth-service,
    ↪ dictionary-service,notification-service'
  }

  stages {
    stage('Build') {
      steps {
        dir('Project/TeacherSupporter') {
          sh 'mvn -B -DskipTests -pl ${MODULES} -am verify'
        }
      }
    }

    stage('Push images') {
      steps {
        dir('Project/TeacherSupporter') {
          // `package` before jib:build is load-bearing: a bare goal runs no

```

```

    // lifecycle, so sibling modules (common) are never built in this
    // session and Jib cannot resolve com.ts:common from the reactor.
    sh 'mvn -B -DskipTests -pl ${MODULES} -am package jib:build -Djib.to.tags=${TAG}'
  }
}

stage('Frontend image') {
  steps {
    dir('Project/TeacherSupporter') {
      // Multi-stage Dockerfile: node builds dist/, nginx ships it. The
      // registry is localhost:5000 from the host's point of view and the
      // container shares the host network, so push needs no extra config.
      sh '''
        set -eu
        docker build -t ${REGISTRY}/frontend:${TAG} -t ${REGISTRY}/frontend:latest frontend
        docker push ${REGISTRY}/frontend:${TAG}
        docker push ${REGISTRY}/frontend:latest
      '''
    }
  }
}

stage('Deploy') {
  steps {
    dir('Project/TeacherSupporter') {
      withCredentials([file(credentialsId: 'kubeconfig-ts', variable: 'KUBECONFIG')]) {
        sh '''
          set -eu
          kubectl apply -k deploy/k8s/base
          for s in ${SERVICES}; do
            kubectl -n ${NS} set image deployment/${s} ${s}=${REGISTRY}/${s}:${TAG}
          done
          for s in ${SERVICES}; do
            kubectl -n ${NS} rollout status deployment/${s} --timeout=300s
          done
        '''
      }
    }
  }
}

post {
  success { echo "Deployed ${TAG} to namespace ${NS}." }
  failure {
    dir('Project/TeacherSupporter') {
      withCredentials([file(credentialsId: 'kubeconfig-ts', variable: 'KUBECONFIG')]) {
        sh 'kubectl -n ${NS} get pods || true'
      }
    }
  }
}
}

```

deploy/jenkins/Dockerfile

```
# Jenkins controller for ts-server, with the build toolchain baked in.
#
# Homelab compromise, stated openly: builds run ON the controller (2 executors),
# not on agent pods. On a 2-core box, agent pods would cost more CPU than they
# save. Migrating to the Kubernetes plugin is a later, self-contained exercise.
#
# docker build -t ts-jenkins:1 deploy/jenkins
# see run.sh for the docker run line
FROM jenkins/jenkins:lts-jdk21

USER root

# --- JDK 25 (Temurin) -- the project targets Java 25 -----
RUN apt-get update && apt-get install -y --no-install-recommends wget gpg apt-transport-https \
  && wget -qO- https://packages.adoptium.net/artifactory/api/gpg/key/public \
  | gpg --dearmor -o /usr/share/keyrings/adoptium.gpg \
  && echo "deb [signed-by=/usr/share/keyrings/adoptium.gpg] https://packages.adoptium.net/
    ↪ artifactory/deb $(. /etc/os-release && echo $VERSION_CODENAME) main" \
    > /etc/apt/sources.list.d/adoptium.list \
  && apt-get update && apt-get install -y --no-install-recommends temurin-25-jdk \
  && rm -rf /var/lib/apt/lists/*

# --- Maven 3.9 (Debian's apt maven is 3.8, too old for some plugins) -----
ARG MAVEN_VERSION=3.9.9
RUN wget -qO /tmp/mvn.tgz https://archive.apache.org/dist/maven/maven-3/${
  ↪ MAVEN_VERSION}/binaries/apache-maven-${MAVEN_VERSION}-bin.tar.gz \
  && tar -xzf /tmp/mvn.tgz -C /opt && ln -s /opt/apache-maven-${MAVEN_VERSION} /opt/
  ↪ maven \
  && rm /tmp/mvn.tgz

# --- kubect!l: the deploy stage talks to k3s with this -----
RUN wget -qO /usr/local/bin/kubect!l \
  "https://dl.k8s.io/release/$(wget -qO- https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/
  ↪ kubect!" \
  && chmod +x /usr/local/bin/kubect!l

# --- Docker CLI (client only): the frontend image is a classic Dockerfile ---
# build, which Jib cannot do. Only the CLI is installed; it talks to the
# HOST's daemon through /var/run/docker.sock mounted by run.sh. No daemon
# inside the container, no docker-in-docker.
RUN install -m 0755 -d /etc/apt/keyrings \
  && wget -qO- https://download.docker.com/linux/debian/gpg | gpg --dearmor -o /etc/apt/keyrings/
  ↪ docker.gpg \
  && echo "deb [signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian
    ↪ $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
    > /etc/apt/sources.list.d/docker.list \
  && apt-get update && apt-get install -y --no-install-recommends docker-ce-cli \
  && rm -rf /var/lib/apt/lists/*

# Jenkins itself keeps running on the bundled JDK 21; only *builds* see JDK 25.
# JAVA_HOME is therefore NOT changed here -- the Jenkinsfile sets it per stage.
ENV MAVEN_HOME=/opt/maven \
  JDK25_HOME=/usr/lib/jvm/temurin-25-jdk-amd64 \
```

```
PATH=/opt/maven/bin:${PATH}
```

```
USER jenkins
```

```
# Plugins pre-installed so the first boot needs no wizard clicks beyond unlock.
```

```
RUN jenkins-plugin-cli --plugins \  
    git workflow-aggregator pipeline-stage-view junit timestamper credentials-binding
```

deploy/k8s/jenkins-rbac.yaml

```
apiVersion: v1  
kind: ServiceAccount  
metadata:  
  name: jenkins  
  namespace: ts  
---  
apiVersion: rbac.authorization.k8s.io/v1  
kind: Role  
metadata:  
  name: jenkins-deployer  
  namespace: ts  
rules:  
  # How to read a rule: "may perform <verbs> on <resources> in <apiGroups>".  
  # Rules are additive; anything not listed is Forbidden. Namespace-scoped  
  # (this is a Role, not a ClusterRole), so nothing here reaches kube-system.  
  #  
  # Verb cheat-sheet for the pipeline's commands:  
  #   kubectl apply      -> get + create (new) or patch (existing)  
  #   kubectl set image  -> get + patch  
  #   kubectl rollout status -> get/list/watch on deployments AND replicaset  
  #   kubectl logs / get pods -> get/list on pods, get on pods/log  
  
  # -- What `kubectl apply -k deploy/k8s/base` creates, by API group -----  
  # Core group is spelled "" (empty string). Namespace is deliberately absent:  
  # it is cluster-scoped and cannot be granted by a Role, so the pipeline must  
  # not try to apply namespace.yaml (ts already exists).  
- apiGroups: [""]  
  resources: ["services", "persistentvolumeclaims", "secrets", "configmaps"]  
  verbs: ["get", "list", "create", "patch", "update"]  
  
- apiGroups: ["apps"]  
  resources: ["deployments", "statefulsets"]  
  verbs: ["get", "list", "watch", "create", "patch", "update"]  
  
- apiGroups: ["networking.k8s.io"]  
  resources: ["ingresses"]  
  verbs: ["get", "list", "create", "patch", "update"]  
  
# Strimzi custom resources (milestone 6). Namespaced like everything  
# else here -- the pipeline may declare Kafka clusters and topics in ts,  
# but installing the operator/CRDs stays an admin act (install-strimzi.sh).  
- apiGroups: ["kafka.strimzi.io"]
```

```
resources: ["kafkas", "kafkanodepools", "kafkatopics"]
verbs: ["get", "list", "create", "patch", "update"]

# -- Read-only extras for `rollout status` and post-failure debugging -----
# A Deployment rolls out via ReplicaSets it owns; rollout status watches them.
- apiGroups: ["apps"]
  resources: ["replicasets"]
  verbs: ["get", "list", "watch"]

- apiGroups: [""]
  resources: ["pods", "pods/log", "events"]
  verbs: ["get", "list", "watch"]

# NOT granted, on purpose:
# delete -- CI should roll forward, never wipe; a human deletes.
# nodes, namespaces -- cluster-scoped, out of a Role's reach anyway.
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: jenkins-deployer
  namespace: ts
subjects:
- kind: ServiceAccount
  name: jenkins
  namespace: ts
roleRef:
  kind: Role
  name: jenkins-deployer
  apiGroup: rbac.authorization.k8s.io
```

deploy/k8s/create-secrets.sh

```
#!/usr/bin/env bash
#
# Create the Secrets the ts namespace needs. Deliberately NOT a manifest:
# secrets do not belong in git. Re-runnable -- it replaces what is there.
#
# ./deploy/k8s/create-secrets.sh
#
# Values match docker-compose.yml so behaviour is identical to local dev.
# Override any of them from the environment before running.

set -euo pipefail

NS="${NS:-ts}"
PG_USER="${PG_USER:-postgres}"
PG_PASSWORD="${PG_PASSWORD:-root}"
MINIO_USER="${MINIO_USER:-minioadmin}"
MINIO_PASSWORD="${MINIO_PASSWORD:-minioadmin}"
# Default matches the fallback baked into config-server's api-gateway.yml /
# auth-service.yml so dev behaviour is unchanged. Override in anything public.
```

```

JWT_SECRET="${JWT_SECRET:-
    ↪ mySecretKeyThatIsAtLeast256BitsLongForHS256Algorithm123456}"

kubectl get namespace "$NS" >/dev/null 2>&1 || kubectl create namespace "$NS"

# Four secrets, not two: the same credential is spelled differently by the
# server that owns it and the client that consumes it. Postgres wants
# POSTGRES_USER; Spring wants SPRING_DATASOURCE_USERNAME. Keeping them as
# separate secrets means each pod's envFrom pulls in only names it understands,
# rather than a grab-bag it has to ignore half of.
#
# --dry-run=client | apply is the idempotent-create idiom: `kubectl create secret`
# alone fails if the secret exists, and there is no `kubectl create --force`.

# --- postgres, server side ---
kubectl -n "$NS" create secret generic postgres-course \
  --from-literal=POSTGRES_USER="$PG_USER" \
  --from-literal=POSTGRES_PASSWORD="$PG_PASSWORD" \
  --from-literal=POSTGRES_DB=ts_course \
  --dry-run=client -o yaml | kubectl apply -f -

# --- postgres, client side (course-service) ---
kubectl -n "$NS" create secret generic postgres-course-app \
  --from-literal=SPRING_DATASOURCE_USERNAME="$PG_USER" \
  --from-literal=SPRING_DATASOURCE_PASSWORD="$PG_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f -

# --- minio, server side ---
kubectl -n "$NS" create secret generic minio \
  --from-literal=MINIO_ROOT_USER="$MINIO_USER" \
  --from-literal=MINIO_ROOT_PASSWORD="$MINIO_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f -

# --- minio, client side (course-service) ---
kubectl -n "$NS" create secret generic minio-app \
  --from-literal=APP_S3_ACCESS_KEY="$MINIO_USER" \
  --from-literal=APP_S3_SECRET_KEY="$MINIO_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f -

# --- postgres-auth, server side ---
kubectl -n "$NS" create secret generic postgres-auth \
  --from-literal=POSTGRES_USER="$PG_USER" \
  --from-literal=POSTGRES_PASSWORD="$PG_PASSWORD" \
  --from-literal=POSTGRES_DB=ts_auth \
  --dry-run=client -o yaml | kubectl apply -f -

# --- postgres-auth, client side (auth-service) ---
kubectl -n "$NS" create secret generic postgres-auth-app \
  --from-literal=SPRING_DATASOURCE_USERNAME="$PG_USER" \
  --from-literal=SPRING_DATASOURCE_PASSWORD="$PG_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f -

# --- google oauth (auth-service) ---
# Real values come from .env / the environment; the placeholders keep the pod
# bootable when they are absent -- Google login then fails at click time, not
# the whole service at startup. Mirrors the ${GOOGLE_CLIENT_ID:placeholder}

```

```
# defaults in config-server's auth-service.yml.
kubectl -n "$NS" create secret generic google-oauth \
  --from-literal=GOOGLE_CLIENT_ID="${GOOGLE_CLIENT_ID:-placeholder}" \
  --from-literal=GOOGLE_CLIENT_SECRET="${GOOGLE_CLIENT_SECRET:-placeholder}" \
  --dry-run=client -o yaml | kubectl apply -f -

# --- jwt, shared by api-gateway (validates) and auth-service (signs) ---
kubectl -n "$NS" create secret generic jwt \
  --from-literal=JWT_SECRET="$JWT_SECRET" \
  --dry-run=client -o yaml | kubectl apply -f -

echo
kubectl -n "$NS" get secrets
```

deploy/k8s/install-strimzi.sh

```
#!/usr/bin/env bash
#
# One-time ADMIN step: install the Strimzi operator + CRDs into the ts
# namespace, watching that namespace only. Run before the first
# `kubectl apply -k deploy/k8s/base` that includes kafka.yaml.
#
# Same category as namespace creation: CRDs and the operator's RBAC are
# cluster-scoped, so the Jenkins ServiceAccount cannot (and should not)
# install them. Jenkins only applies the namespaced Kafka/KafkaTopic
# resources afterwards.
#
# ./deploy/k8s/install-strimzi.sh

set -euo pipefail

NS="${NS:-ts}"

kubectl get namespace "$NS" >/dev/null 2>&1 || kubectl create namespace "$NS"

# Server-side apply, because the Strimzi bundle's CRDs are too large for
# the client-side last-applied-configuration annotation. Idempotent: safe
# to re-run, also how you take operator upgrades later.
kubectl apply --server-side --force-conflicts \
  -f "https://strimzi.io/install/latest?namespace=${NS}" -n "$NS"

kubectl -n "$NS" rollout status deployment/strimzi-cluster-operator --timeout=300s

echo
echo "Operator ready. Kafka clusters in '${NS}' will now be reconciled."
```

deploy/k8s/base/kustomization.yaml

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: ts

# Applied in dependency order for readability only -- Kubernetes starts pods in
# parallel regardless. Ordering here is documentation, not orchestration.
resources:
  # namespace.yaml is intentionally NOT listed. Namespaces are cluster-scoped,
  # and the Jenkins ServiceAccount only holds a namespaced Role, so `apply`
  # would be Forbidden. The namespace is created once by create-secrets.sh
  # (or `kubectl apply -f namespace.yaml` by an admin).
  - postgres-course.yaml
  - postgres-auth.yaml
  - mongodb.yaml
  - minio.yaml
  # kafka.yaml requires the Strimzi operator (one-time admin step:
  # ../install-strimzi.sh) -- without its CRDs, apply fails on kind Kafka.
  - kafka.yaml
  - kafka-topics.yaml
  - maildev.yaml
  - config-server.yaml
  - discovery-server.yaml
  - course-service.yaml
  - auth-service.yaml
  - dictionary-service.yaml
  - notification-service.yaml
  - api-gateway.yaml
  # Non-JVM deployable: nginx + React build, claims / on ts.local.
  - frontend.yaml

# Stamped onto every object, so `kubectl delete -l part-of=teacher-supporter`
# cleans up the whole slice without touching kube-system.
commonLabels:
  app.kubernetes.io/part-of: teacher-supporter

# Secrets are NOT here on purpose -- see ../create-secrets.sh.

```

deploy/k8s/base/namespace.yaml

```

apiVersion: v1
kind: Namespace
metadata:
  name: ts

```

deploy/k8s/base/postgres-course.yaml

```

# StatefulSet rather than Deployment: a database needs a stable identity and a

```

```
# volume that is never handed to a second replica. volumeClaimTemplates gives
# each pod its own PVC, provisioned by k3s' local-path on the node's SSD.
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres-course
spec:
  serviceName: postgres-course
  replicas: 1
  selector:
    matchLabels: { app: postgres-course }
  template:
    metadata:
      labels: { app: postgres-course }
    spec:
      containers:
        - name: postgres
          image: postgres:17
          ports:
            - { name: postgres, containerPort: 5432 }
          envFrom:
            - secretRef: { name: postgres-course }
          env:
            # The official image refuses to initialise into a non-empty directory,
            # and local-path volumes arrive with a lost+found. A subdirectory
            # sidesteps it -- the standard fix, not a workaround.
            - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres", "-d", "ts_course"]
            initialDelaySeconds: 10
            periodSeconds: 5
            failureThreshold: 6
          resources:
            requests: { memory: 256Mi, cpu: 100m }
            limits: { memory: 512Mi }
      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            accessModes: [ReadWriteOnce]
            resources:
              requests:
                storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: postgres-course
spec:
  selector: { app: postgres-course }
  ports:
    - { name: postgres, port: 5432, targetPort: 5432 }
```

deploy/k8s/base/postgres-auth.yaml

```
# Second Postgres, same shape as postgres-course.yaml -- see the comments
# there for why StatefulSet, PGDATA subdirectory, and volumeClaimTemplates.
#
# Deliberately a SEPARATE server rather than a second database in the same
# one: database-per-service is the microservice contract. auth and course
# must not be able to join each other's tables, today or by accident later.
# On this box it costs ~256Mi -- cheap insurance.
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres-auth
spec:
  serviceName: postgres-auth
  replicas: 1
  selector:
    matchLabels: { app: postgres-auth }
  template:
    metadata:
      labels: { app: postgres-auth }
    spec:
      containers:
        - name: postgres
          image: postgres:17
          ports:
            - { name: postgres, containerPort: 5432 }
          envFrom:
            - secretRef: { name: postgres-auth }
          env:
            - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres", "-d", "ts_auth"]
              initialDelaySeconds: 10
              periodSeconds: 5
              failureThreshold: 6
          resources:
            requests: { memory: 256Mi, cpu: 100m }
            limits: { memory: 512Mi }
      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            accessModes: [ReadWriteOnce]
            resources:
              requests:
                storage: 5Gi
      ---
    apiVersion: v1
    kind: Service
    metadata:
      name: postgres-auth
```

```
spec:
  selector: { app: postgres-auth }
  ports:
    - { name: postgres, port: 5432, targetPort: 5432 }
```

deploy/k8s/base/mongodb.yaml

```
# Same reasoning as postgres-course.yaml: databases get a StatefulSet for the
# stable identity + a volumeClaimTemplate PVC that is never shared.
#
# No credentials, matching docker-compose.yml -- the official mongo image only
# enables auth when MONGO_INITDB_ROOT_USERNAME/PASSWORD are set. Acceptable
# here because the Service is ClusterIP: only pods inside the cluster can
# reach it, and there is no Ingress to it. If this ever went multi-tenant or
# network policies were the goal, auth would come first.
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongodb
spec:
  serviceName: mongodb
  replicas: 1
  selector:
    matchLabels: { app: mongodb }
  template:
    metadata:
      labels: { app: mongodb }
    spec:
      containers:
        - name: mongodb
          image: mongo:8
          ports:
            - { name: mongo, containerPort: 27017 }
      volumeMounts:
        - { name: data, mountPath: /data/db }
      readinessProbe:
        # mongosh ping: same idea as pg_isready -- "is the server
        # accepting commands", not just "is the port open".
        exec:
          command: ["mongosh", "--quiet", "--eval", "db.adminCommand('ping')"]
          initialDelaySeconds: 10
          periodSeconds: 5
          failureThreshold: 6
      resources:
        requests: { memory: 256Mi, cpu: 100m }
        limits: { memory: 512Mi }
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
```

```

    accessModes: [ReadWriteOnce]
    resources:
      requests:
        storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: mongodb
spec:
  selector: { app: mongodb }
  ports:
    - { name: mongo, port: 27017, targetPort: 27017 }

```

deploy/k8s/base/minio.yaml

```

# MinIO is a hard startup dependency for course-service: S3StorageService's
# @PostConstruct calls headBucket and only catches NoSuchBucketException, so an
# unreachable MinIO fails bean creation. Kubernetes has no depends_on -- course
# service will CrashLoopBackOff until this pod is serving, then recover.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: minio
spec:
  replicas: 1
  strategy: { type: Recreate } # RWO volume: never two pods holding it at once
  selector:
    matchLabels: { app: minio }
  template:
    metadata:
      labels: { app: minio }
    spec:
      containers:
        - name: minio
          image: minio/minio:latest
          args: ["server", "/data", "--console-address", ":9001"]
          ports:
            - { name: s3, containerPort: 9000 }
            - { name: console, containerPort: 9001 }
          envFrom:
            - secretRef: { name: minio }
          volumeMounts:
            - { name: data, mountPath: /data }
          readinessProbe:
            httpGet: { path: /minio/health/ready, port: s3 }
            initialDelaySeconds: 10
            periodSeconds: 5
          resources:
            requests: { memory: 256Mi, cpu: 100m }
            limits: { memory: 512Mi }
      volumes:

```

```
- name: data
  persistentVolumeClaim: { claimName: minio-data }
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: minio-data
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: minio
spec:
  selector: { app: minio }
  ports:
    - { name: s3, port: 9000, targetPort: 9000 }
    - { name: console, port: 9001, targetPort: 9001 }
```

deploy/k8s/base/kafka.yaml

```
# Kafka via the Strimzi operator (KRaft mode -- no ZooKeeper).
#
# PREREQUISITE (one-time, admin): the operator + CRDs must be installed
# first -- ../install-strimzi.sh. Without it, applying this file fails with
# "no matches for kind Kafka". Same class of admin step as namespace.yaml.
#
# We do not create StatefulSets or Services for Kafka ourselves. We declare
# a Kafka cluster as data; the operator (Chapter 14's reconciliation idea,
# extended to custom kinds) computes the pods, Services, listener config,
# and rolling restarts from it. What we hand-tuned in docker-compose.yml --
# the advertised-listeners maze -- the operator now derives.
#
# Client address inside the cluster: ts-kafka-kafka-bootstrap:9092
# (<cluster-name>-kafka-bootstrap -- Strimzi's naming convention)
---
# Node pool: WHERE Kafka runs and with what storage/resources.
# One node holding both KRaft roles -- the k8s equivalent of the compose
# broker's process.roles=broker,controller. RF=1 everywhere: a homelab
# accepts zero fault tolerance; production starts at 3 nodes / RF=3.
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool
metadata:
  name: dual-role
  labels:
    # Joins this pool to the Kafka resource below. The operator matches on
    # this label -- it must equal the Kafka resource's metadata.name.
    strimzi.io/cluster: ts-kafka
```



```

spec:
  replicas: 1
  roles: [controller, broker]
  storage:
    type: persistent-claim      # local-path PVC, like the databases
    size: 5Gi
    deleteClaim: false        # deleting the Kafka CR keeps the log data
  resources:
    requests: { memory: 768Mi, cpu: 200m }
    limits: { memory: 1Gi }
---
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: ts-kafka
  annotations:
    strimzi.io/kraft: enabled
    strimzi.io/node-pools: enabled
spec:
  kafka:
    # No version pinned: the operator's default (its newest supported
    # Kafka) applies, and operator upgrades roll the broker for us.
    listeners:
      # One internal plaintext listener. No TLS/auth for the same reason
      # Mongo has none: ClusterIP-only, nothing routes here from outside.
      - name: plain
        port: 9092
        type: internal
        tls: false
    config:
      # Single broker: every replication setting must be 1, or topic
      # creation fails asking for brokers that do not exist.
      offsets.topic.replication.factor: 1
      transaction.state.log.replication.factor: 1
      transaction.state.log.min.isr: 1
      default.replication.factor: 1
      min.insync.replicas: 1
      log.retention.hours: 168      # matches docker-compose.yml
    jvmOptions:
      # Kafka's default heap assumes a server; half a GiB serves a homelab
      # event trickle comfortably and leaves RAM for the Spring services.
      -Xms: 256m
      -Xmx: 512m
    entityOperator:
      # Reconciles KafkaTopic resources (kafka-topics.yaml) into real topics.
      # userOperator is omitted: no authentication, so no users to manage.
      topicOperator: {}

```

deploy/k8s/base/kafka-topics.yaml

```

# Topics as declared objects, reconciled by Strimzi's topic operator.
#

```

```
# Kafka would auto-create these on first use (auto.create.topics.enable
# defaults to true), but declared topics beat auto-created ones: partition
# count and retention are decisions, not defaults; they exist before the
# first producer runs (no race on day one); and `kubectl get kafkatopics`
# becomes the inventory of every event contract in the system.
#
# partitions: 1 -- total order per topic, which trivially preserves the
# per-user ordering the publishers key for. Raise partitions later only
# with a consumer that scales past one instance; note partitions can be
# added but never removed.
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.registered
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
---
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.activated
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
---
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.admin-provisioned
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
---
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.assignment.created
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
```

deploy/k8s/base/maildev.yaml

```

# Maildev: an SMTP server that delivers nothing -- it catches every mail
# and shows it in a web UI. The dev/homelab stand-in for a real relay;
# swapping to real mail later is only MAIL_HOST/credentials env changes
# on notification-service.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: maildev
spec:
  replicas: 1
  selector:
    matchLabels: { app: maildev }
  template:
    metadata:
      labels: { app: maildev }
    spec:
      containers:
        - name: maildev
          image: maildev/maildev:2.2.1
          ports:
            - { name: web, containerPort: 1080 }
            - { name: smtp, containerPort: 1025 }
          readinessProbe:
            tcpSocket: { port: smtp }
            periodSeconds: 5
          resources:
            requests: { memory: 64Mi, cpu: 25m }
            limits: { memory: 128Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: maildev
spec:
  selector: { app: maildev }
  ports:
    - { name: web, port: 1080, targetPort: 1080 }
    - { name: smtp, port: 1025, targetPort: 1025 }
# No Ingress on purpose: the UI is an operator tool, not part of the app.
# To read caught mail: kubectl -n ts port-forward svc/maildev 1080:1080
# then open http://localhost:1080 on Windows.

```

deploy/k8s/base/config-server.yaml

```

# Config Server holds no state: its YAML is baked into the jar under
# classpath:/configurations (spring.profiles.active=native). No PVC, no DB.
# Every other pod blocks on this one, so it starts first in practice by virtue
# of being the only thing with no dependencies of its own.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: config-server

```

```
spec:
  replicas: 1
  selector:
    matchLabels: { app: config-server }
  template:
    metadata:
      labels: { app: config-server }
    spec:
      containers:
        - name: config-server
          image: localhost:5000/config-server:1.0.0-SNAPSHOT
          # SNAPSHOT tags are overwritten on every push; the default IfNotPresent
          # would keep serving whatever the node cached. Always re-check the
          # registry (cheap: only the manifest is fetched if digests match).
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8888 }
          env:
            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
            # startupProbe buys a slow JVM time to boot without forcing a long
            # liveness initialDelay. Until it passes, liveness is not evaluated.
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 30    # up to 150s to start
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 10
          resources:
            requests: { memory: 384Mi, cpu: 100m }
            limits: { memory: 640Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: config-server
spec:
  selector: { app: config-server }
  ports:
    - { name: http, port: 8888, targetPort: 8888 }
```

deploy/k8s/base/discovery-server.yaml

```
# Eureka. Kept deliberately -- this project is the classic Spring Cloud
# showcase, and running it on Kubernetes alongside Service DNS is the point.
#
# Note the two service-discovery systems now coexist: Kubernetes resolves
# `discovery-server` to a ClusterIP, and Eureka resolves `course-service` to a
# pod IP that the clients registered themselves. Only the first survives a pod
# reschedule instantly; Eureka takes up to 90s to evict a dead instance.
apiVersion: apps/v1
kind: Deployment
```

```

metadata:
  name: discovery-server
spec:
  replicas: 1
  selector:
    matchLabels: { app: discovery-server }
  template:
    metadata:
      labels: { app: discovery-server }
    spec:
      containers:
        - name: discovery-server
          image: localhost:5000/discovery-server:1.0.0-SNAPSHOT
          # SNAPSHOT tags are overwritten on every push; the default IfNotPresent
          # would keep serving whatever the node cached. Always re-check the
          # registry (cheap: only the manifest is fetched if digests match).
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8761 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 30
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 10
          resources:
            requests: { memory: 384Mi, cpu: 100m }
            limits: { memory: 640Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: discovery-server
spec:
  selector: { app: discovery-server }
  ports:
    - { name: http, port: 8761, targetPort: 8761 }

```

deploy/k8s/base/course-service.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: course-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: course-service }

```

```
template:
  metadata:
    labels: { app: course-service }
  spec:
    containers:
      - name: course-service
        image: localhost:5000/course-service:1.0.0-SNAPSHOT
        # SNAPSHOT tags are overwritten on every push; the default IfNotPresent
        # would keep serving whatever the node cached. Always re-check the
        # registry (cheap: only the manifest is fetched if digests match).
        imagePullPolicy: Always
        ports:
          - { name: http, containerPort: 8082 }
        env:
          # Same values as docker-compose.yml, with compose service names
          # swapped for Kubernetes Service names. They happen to be identical,
          # because both resolve a name to an address on a private network.
          - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
          - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-
↪ server:8761/eureka/" }
          - { name: SPRING_DATASOURCE_URL, value: "jdbc:postgresql://postgres-course:5432/
↪ ts_course" }
          - { name: APP_S3_ENDPOINT, value: "http://minio:9000" }

          # Eureka on Kubernetes needs this or nothing can call anything.
          # By default a client registers its *hostname*, which inside a pod is
          # the pod name -- a string no other pod can resolve. Pod IPs are
          # routable cluster-wide, so register the IP instead.
          - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

          # Strimzi's bootstrap Service (milestone 6). This service both
          # produces (ts.assignment.created) and consumes (ts.user.activated).
          - { name: SPRING_KAFKA_BOOTSTRAP_SERVERS, value: "ts-kafka-kafka-bootstrap
↪ :9092" }

          # Zipkin is not in the cluster yet (observability is milestone 8).
          - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

          - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
        envFrom:
          - secretRef: { name: postgres-course-app }
          - secretRef: { name: minio-app }

        # --- Health probes -----
        # No actuator in this service, so /actuator/health is a 404. The one
        # unauthenticated HTTP path that exists is springdoc's /v3/api-docs
        # (permitAll in SecurityConfig). It only answers 200 once the whole
        # Spring context is up -- which is AFTER Flyway has migrated and the
        # MinIO bucket check has run. That makes it a true "ready" signal,
        # unlike a tcpSocket check that passes as soon as Tomcat binds.

        # startupProbe: "give the JVM time to boot, then hand over."
        # While this probe is failing, readiness/liveness are NOT evaluated.
        # 12 x 10s = up to 120s of grace for a cold start on two cores.
        # If it still isn't up after that, kubelet restarts the container.
        startupProbe:
```

```

    httpGet: { path: /v3/api-docs, port: http }
    periodSeconds: 10
    failureThreshold: 12

    # readinessProbe: "should the Service send traffic here right now?"
    # Failing readiness pulls the pod out of the Service endpoints; it
    # does NOT restart anything. Cheap and frequent.
    readinessProbe:
      httpGet: { path: /v3/api-docs, port: http }
      periodSeconds: 5
      failureThreshold: 3

    # No livenessProbe on purpose. Liveness restarts the container, and
    # the only thing it would catch beyond readiness is a wedged JVM --
    # rare, and a probe firing during a long GC pause on a 2-core box
    # would kill a healthy pod. Add one later if we ever see a real hang.
    # -----

    resources:
      requests: { memory: 512Mi, cpu: 200m }
      limits: { memory: 768Mi }
      # No CPU limit on purpose. A limit throttles the JVM exactly during
      # class loading, when it wants every core it can get -- turning a
      # 60s start into a 200s one and failing the startupProbe.
---
apiVersion: v1
kind: Service
metadata:
  name: course-service
spec:
  selector: { app: course-service }
  ports:
    - { name: http, port: 8082, targetPort: 8082 }
# NOTE: the direct Ingress that used to live here (ts.local/courses ->
# course-service) is gone. It was milestone-3 scaffolding to prove the chain
# end-to-end, but it bypassed the api-gateway's JWT filter. All external
# traffic now enters through the gateway's Ingress (api-gateway.yaml).
# In-cluster callers still reach this Service directly -- that is what the
# Service object above is for.

```

deploy/k8s/base/auth-service.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: auth-service }
  template:
    metadata:

```

```
labels: { app: auth-service }
spec:
  containers:
    - name: auth-service
      image: localhost:5000/auth-service:1.0.0-SNAPSHOT
      imagePullPolicy: Always
      ports:
        - { name: http, containerPort: 8081 }
      env:
        - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
        - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-
↪ server:8761/eureka/" }
        - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }
        - { name: SPRING_DATASOURCE_URL, value: "jdbc:postgresql://postgres-auth:5432/
↪ ts_auth" }

        # Strimzi's bootstrap Service (milestone 6). Producer-only here:
        # registration/activation/admin-provisioned events.
        - { name: SPRING_KAFKA_BOOTSTRAP_SERVERS, value: "ts-kafka-kafka-bootstrap
↪ :9092" }

        # Google's OAuth callback must round-trip through the browser, so
        # it uses the PUBLIC name (the Tailscale MagicDNS host, via Traefik
        # -> gateway -> here), not a cluster-internal one. This exact URL
        # must also be registered under "Authorized redirect URIs" in the
        # Google Cloud console, or Google refuses with redirect_uri_mismatch.
        # (ts.local, the LAN-only name, was the value up to build #9.)
        - { name:
↪ SPRING_SECURITY_OAUTH2_CLIENT_REGISTRATION_GOOGLE_REDIRECT_URI
↪ ,
          value: "http://ts-server.tailbfe002.ts.net/login/oauth2/code/google" }
        # After Google login, OAuth2LoginSuccessHandler redirects the
        # browser to the frontend with the tokens. config-server's default
        # says localhost:3000 (laptop Vite dev server); in the cluster the
        # frontend is the same origin as everything else (milestone 7).
        - { name: APP_OAUTH2_SUCCESS_REDIRECT_URI, value: "http://ts-server.tailbfe002.
↪ ts.net/oauth/callback" }

        # Zipkin arrives with observability (milestone 8).
        - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

        - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
      envFrom:
        - secretRef: { name: postgres-auth-app }
        # Signs JWTs with the same key the gateway validates with.
        - secretRef: { name: jwt }
        - secretRef: { name: google-oauth }

        # /actuator/health is NOT usable here: this service's SecurityConfig
        # permitAll list covers /register, /login, ... and /v3/api-docs, but
        # not /actuator/** -- so health answers 401 and the kubelet would
        # restart a perfectly healthy pod forever. Same fallback as
        # course-service: /v3/api-docs only turns 200 after the full context
        # (Flyway migration included) is up, which is an honest ready signal.
      startupProbe:
        httpGet: { path: /v3/api-docs, port: http }
```



```

    periodSeconds: 10
    failureThreshold: 12
  readinessProbe:
    httpGet: { path: /v3/api-docs, port: http }
    periodSeconds: 5
    failureThreshold: 3

  resources:
    requests: { memory: 512Mi, cpu: 200m }
    limits: { memory: 768Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: auth-service
spec:
  selector: { app: auth-service }
  ports:
    - { name: http, port: 8081, targetPort: 8081 }

```

deploy/k8s/base/dictionary-service.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: dictionary-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: dictionary-service }
  template:
    metadata:
      labels: { app: dictionary-service }
    spec:
      containers:
        - name: dictionary-service
          image: localhost:5000/dictionary-service:1.0.0-SNAPSHOT
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8083 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-
↪ server:8761/eureka/" }
            - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

            # Config-server's dictionary-service.yml says localhost:27017;
            # in the cluster mongo lives behind the `mongodb` Service.
            - { name: SPRING_DATA_MONGODB_URI, value: "mongodb://mongodb:27017/
↪ ts_dictionary" }

            # Zipkin arrives with observability (milestone 8).

```

```
- { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }
- { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }

# This service's SecurityConfig permits /actuator/** -- so unlike
# course-service (which had to use /v3/api-docs), the probe can hit
# the purpose-built health endpoint. It aggregates a Mongo ping, so
# "ready" here really means "can serve dictionary lookups".
startupProbe:
  httpGet: { path: /actuator/health, port: http }
  periodSeconds: 10
  failureThreshold: 12
readinessProbe:
  httpGet: { path: /actuator/health, port: http }
  periodSeconds: 5
  failureThreshold: 3

resources:
  requests: { memory: 512Mi, cpu: 200m }
  limits: { memory: 768Mi }
  # No CPU limit -- same class-loading argument as course-service.
---
apiVersion: v1
kind: Service
metadata:
  name: dictionary-service
spec:
  selector: { app: dictionary-service }
  ports:
    - { name: http, port: 8083, targetPort: 8083 }
```

deploy/k8s/base/notification-service.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: notification-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: notification-service }
  template:
    metadata:
      labels: { app: notification-service }
    spec:
      containers:
        - name: notification-service
          image: localhost:5000/notification-service:1.0.0-SNAPSHOT
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8084 }
          env:
```

```

- { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
- { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-
↪ server:8761/eureka/" }
- { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

# Strimzi's bootstrap Service: <cluster>-kafka-bootstrap.
- { name: SPRING_KAFKA_BOOTSTRAP_SERVERS, value: "ts-kafka-kafka-bootstrap
↪ :9092" }

# The config's ${MAIL_HOST:localhost} default, repointed at the
# maildev Service. Overriding the spring.mail.* keys directly
# (relaxed binding) rather than the custom placeholder names --
# same result, and consistent with how compose does it.
- { name: SPRING_MAIL_HOST, value: "maildev" }
- { name: SPRING_MAIL_PORT, value: "1025" }

# Links inside emails (activate / invite / login) must work from
# whatever device opens the mail, so they carry the PUBLIC origin
# -- the Tailscale MagicDNS name every tailnet device resolves --
# never a Service name. Overrides app.frontend.base-url from
# config-server (relaxed binding), same trick as SPRING_MAIL_HOST.
- { name: APP_FRONTEND_BASE_URL, value: "http://ts-server.tailbfe002.ts.net" }

# Zipkin arrives with observability (milestone 8).
- { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

- { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }

# This service has no Spring Security at all -- no SecurityConfig,
# no security starter -- so /actuator/health is open and the probes
# can use the real endpoint (compare auth-service's workaround).
startupProbe:
  httpGet: { path: /actuator/health, port: http }
  periodSeconds: 10
  failureThreshold: 12
readinessProbe:
  httpGet: { path: /actuator/health, port: http }
  periodSeconds: 5
  failureThreshold: 3

resources:
  requests: { memory: 512Mi, cpu: 100m }
  limits: { memory: 768Mi }
---
# Even a pure consumer gets a Service: Eureka registration aside, the
# actuator endpoints (and later Prometheus scraping) need an address.
apiVersion: v1
kind: Service
metadata:
  name: notification-service
spec:
  selector: { app: notification-service }
  ports:
    - { name: http, port: 8084, targetPort: 8084 }

```

deploy/k8s/base/api-gateway.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-gateway
spec:
  replicas: 1
  selector:
    matchLabels: { app: api-gateway }
  template:
    metadata:
      labels: { app: api-gateway }
    spec:
      containers:
        - name: api-gateway
          image: localhost:5000/api-gateway:1.0.0-SNAPSHOT
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8080 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-
↪ server:8761/eureka/" }
            - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

            # Zipkin is not in the cluster yet (observability is milestone 8).
            - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
          envFrom:
            # JWT_SECRET -- shared with auth-service, which signs the tokens the
            # gateway validates. One secret, two consumers, or logins break in a
            # way that looks like a gateway bug.
            - secretRef: { name: jwt }

            # Unlike course-service, the gateway exposes actuator, so probes can
            # use the real health endpoint. Reactive stack (Netty) boots faster
            # than the MVC services: 6 x 10s of startup grace instead of 12.
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 10
            failureThreshold: 6
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 3

          resources:
            requests: { memory: 384Mi, cpu: 100m }
            limits: { memory: 512Mi }
---
apiVersion: v1
kind: Service
metadata:
```

```

name: api-gateway
spec:
  selector: { app: api-gateway }
  ports:
    - { name: http, port: 8080, targetPort: 8080 }
---
# Edge routing decision: with the gateway in the cluster there are two routers.
# Traefik (this Ingress) is the front door on ts.local; Spring Cloud Gateway
# knows Eureka and validates JWTs. Decision made here:
#
# /api          -> gateway  (all REST traffic, JWT-checked)
# /oauth2       -> gateway  (Google login redirect start)
# /login/oauth2 -> gateway  (Google login callback)
# /             -> free     (reserved for the frontend, milestone 7)
#
# The old direct `/courses` Ingress on course-service is deleted with this
# change: it reached the service without passing the gateway's JWT filter,
# i.e. an unauthenticated side door. From now on the ONLY way in is through
# the gateway -- which is the entire point of having one.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-gateway
spec:
  # Two hosts, identical paths. ts.local is the LAN name (hosts-file entry on
  # the laptop). The second is the machine's Tailscale MagicDNS name: every
  # device on the tailnet resolves it with no hosts-file edit, so the app is
  # reachable from anywhere Tailscale is -- the same Ingress, one more rule.
  # Traefik routes on the Host header, so a name it does not list gets 404.
  rules:
    - host: ts.local
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
          - path: /oauth2
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
          - path: /login/oauth2
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
    - host: ts-server.tailbfe002.ts.net
      http:
        paths:
          - path: /api
            pathType: Prefix

```

```
    backend:
      service:
        name: api-gateway
        port: { number: 8080 }
- path: /oauth2
  pathType: Prefix
  backend:
    service:
      name: api-gateway
      port: { number: 8080 }
- path: /login/oauth2
  pathType: Prefix
  backend:
    service:
      name: api-gateway
      port: { number: 8080 }
```

deploy/k8s/base/frontend.yaml

```
# Frontend (milestone 7): nginx serving the built React app.
#
# Image comes from frontend/Dockerfile (docker build in Jenkins), not Jib --
# it is the one non-JVM deployable. Being static files it has no config
# server, no Eureka, no secrets: the app talks to "/api" on its own origin,
# and Traefik (Ingress below) splits that off to the gateway.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 1
  selector:
    matchLabels: { app: frontend }
  template:
    metadata:
      labels: { app: frontend }
    spec:
      containers:
        - name: frontend
          image: localhost:5000/frontend:latest
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 80 }

          # nginx is up in milliseconds; no startupProbe needed. /healthz is
          # a location in nginx.conf that answers without touching disk.
          readinessProbe:
            httpGet: { path: /healthz, port: http }
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /healthz, port: http }
            periodSeconds: 20
```

```

# Static file serving: the cheapest pod in the namespace by far.
resources:
  requests: { memory: 32Mi, cpu: 10m }
  limits: { memory: 64Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  selector: { app: frontend }
  ports:
    - { name: http, port: 80, targetPort: 80 }
---
# Claims "/" on ts.local -- the path api-gateway.yaml's Ingress left free.
# Traefik merges all Ingress rules for a host and picks the LONGEST matching
# prefix, so /api, /oauth2 and /login/oauth2 (declared on the gateway
# Ingress) still win over this catch-all. Net effect: one origin
# (http://ts.local) for pages and API alike -- no CORS preflights, and the
# Google-login redirect lands on the same host that started it.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend
spec:
  # Two hosts, identical paths. ts.local is the LAN name (hosts-file entry on
  # the laptop). The second is the machine's Tailscale MagicDNS name: every
  # device on the tailnet resolves it with no hosts-file edit, so the app is
  # reachable from anywhere Tailscale is -- the same Ingress, one more rule.
  # Traefik routes on the Host header, so a name it does not list gets 404.
  rules:
    - host: ts.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port: { number: 80 }
    - host: ts-server.tailbfe002.ts.net
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port: { number: 80 }

```

frontend/Dockerfile

```
# Frontend image: the ONLY Dockerfile the pipeline builds.
#
# The JVM services use Jib (no Dockerfile, no daemon). A React app is a pile
# of static files behind a web server, so it gets the classic two-stage
# build instead: stage 1 has Node and produces dist/, stage 2 is a plain
# nginx that ships only dist/ -- Node, node_modules (~300 MB) and the
# TypeScript compiler never reach the runtime image (~15 MB).
#
#   docker build -t localhost:5000/frontend:dev frontend
#   docker run --rm -p 3000:80 localhost:5000/frontend:dev -> http://localhost:3000

# --- stage 1: build -----
FROM node:24-alpine AS build
WORKDIR /app

# Copy the lockfile first and install BEFORE copying sources: this layer is
# cached until package*.json changes, so a source edit does not re-download
# every dependency. `npm ci` (not `install`) refuses to touch the lockfile,
# i.e. the build gets exactly the versions committed in git.
COPY package.json package-lock.json ./
RUN npm ci --no-audit --no-fund

COPY . .
# = `tsc -b && vite build` (package.json). tsc type-checks, Vite bundles and
# content-hashes the assets (index-<hash>.js) so they can be cached forever.
RUN npm run build

# --- stage 2: serve -----
FROM nginx:1.27-alpine
# The stock image ships a default server block; replace it with ours.
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

frontend/nginx.conf

```
# nginx server block for the React frontend inside the cluster.
#
# Division of labour on ts.local (Traefik decides by longest path prefix):
#   /api, /oauth2, /login/oauth2 -> api-gateway (never reach this nginx)
#   everything else             -> here        (static files + SPA fallback)
# So this file does NOT proxy anything: in production the frontend's
# "/api" calls are same-origin and Traefik, not nginx, routes them. The
# Vite dev-server proxy in vite.config.ts plays that role only on a laptop.
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    # SPA fallback. React Router (BrowserRouter) owns URLs like /courses/42
    # purely in the browser -- no such file exists on disk. On a deep link or
    # a page refresh nginx must answer index.html and let the router take
```



```

# over, otherwise the user gets a 404 for any URL except "/"
location / {
    try_files $uri $uri/ /index.html;
}

# Vite content-hashes everything under /assets (index-CqZING8N.js), so a
# changed file always has a new name: cache these forever. index.html is
# deliberately NOT covered -- it must be re-fetched so it can point at
# the new hashes after a deploy.
location /assets/ {
    add_header Cache-Control "public, max-age=31536000, immutable";
}

# Kubernetes probes hit this; keeps "is nginx up" separate from "/",
# which would also succeed but pollutes access logs less this way.
location = /healthz {
    access_log off;
    add_header Content-Type text/plain;
    return 200 "ok\n";
}

gzip on;
gzip_types text/css application/javascript application/json image/svg+xml;
}

```

docker-compose.yml

```

services:
  # ===== INFRASTRUCTURE =====
  postgres-auth:
    image: postgres:17
    environment:
      POSTGRES_DB: ts_auth
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: root
    ports:
      - "5433:5432"
    volumes:
      - postgres-auth-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      retries: 5

  postgres-course:
    image: postgres:17
    environment:
      POSTGRES_DB: ts_course
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: root
    ports:
      - "5434:5432"

```

```
volumes:
  - postgres-course-data:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 10s
  retries: 5
```

```
mongodb:
  image: mongo:8
  ports:
    - "27017:27017"
  volumes:
    - mongo-data:/data/db
```

```
kafka:
  image: apache/kafka:3.9.0
  hostname: kafka
  ports:
    - "9092:9092"
  environment:
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    KAFKA_LISTENERS: PLAINTEXT://:19092,CONTROLLER://:9093,EXTERNAL://:9092
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:19092,EXTERNAL://localhost
    ↪ :9092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,
    ↪ PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT
    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_LOG_RETENTION_HOURS: 168
    CLUSTER_ID: Mku3OEVBNTcwNTJENDM2Qk
```

```
kafka-ui:
  image: provectuslabs/kafka-ui:latest
  ports:
    - "9090:8080"
  environment:
    KAFKA_CLUSTERS_0_NAME: local
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:19092
  depends_on:
    - kafka
```

```
zipkin:
  image: openzipkin/zipkin:latest
  ports:
    - "9411:9411"
```

```
maildev:
  image: maildev/maildev:2.2.1
  ports:
    - "1080:1080"
    - "1025:1025"
```

```
minio:
```

```

image: minio/minio:latest
command: server /data --console-address ":9001"
ports:
  - "9000:9000" # S3 API
  - "9001:9001" # Web console
environment:
  MINIO_ROOT_USER: minioadmin
  MINIO_ROOT_PASSWORD: minioadmin
volumes:
  - minio-data:/data
healthcheck:
  test: ["CMD", "mc", "ready", "local"]
  interval: 10s
  retries: 5

# ===== SPRING CLOUD =====
config-server:
  build:
    context: ./config-server
    dockerfile: Dockerfile
  ports:
    - "8888:8888"
    - "5888:5888"
  environment:
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
↳ =*:5888"
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8888/actuator/health"]
    interval: 15s
    timeout: 5s
    retries: 10
    start_period: 30s

discovery-server:
  build:
    context: ./discovery-server
    dockerfile: Dockerfile
  ports:
    - "8761:8761"
    - "5761:5761"
  depends_on:
    config-server:
      condition: service_healthy
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
↳ =*:5761"

api-gateway:
  build:
    context: ./api-gateway
    dockerfile: Dockerfile
  ports:
    - "8080:8080"
    - "5080:5080"
  depends_on:

```

```
- discovery-server
environment:
  SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
  EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
  MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
  JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
↳ =*:5080"

# ===== APPLICATION SERVICES =====
auth-service:
  build:
    context: ./auth-service
    dockerfile: Dockerfile
  ports:
    - "8081:8081"
    - "5081:5081"
  depends_on:
    config-server:
      condition: service_healthy
    postgres-auth:
      condition: service_healthy
    kafka:
      condition: service_started
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-auth:5432/ts_auth
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
    MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
    GOOGLE_CLIENT_ID: ${GOOGLE_CLIENT_ID}
    GOOGLE_CLIENT_SECRET: ${GOOGLE_CLIENT_SECRET}
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
↳ =*:5081"

course-service:
  build:
    context: ./course-service
    dockerfile: Dockerfile
  ports:
    - "8082:8082"
    - "5082:5082"
  depends_on:
    config-server:
      condition: service_healthy
    postgres-course:
      condition: service_healthy
    kafka:
      condition: service_started
    minio:
      condition: service_started
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-course:5432/ts_course
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
    MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
```

```
APP_S3_ENDPOINT: http://minio:9000
APP_S3_PUBLIC_ENDPOINT: http://localhost:9000
APP_S3_ACCESS_KEY: minioadmin
APP_S3_SECRET_KEY: minioadmin
JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
↳ =*:5082"
```

dictionary-service:

```
build:
  context: ./dictionary-service
  dockerfile: Dockerfile
ports:
  - "8083:8083"
  - "5083:5083"
depends_on:
  config-server:
    condition: service_healthy
  mongodb:
    condition: service_started
environment:
  SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
  EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
  SPRING_DATA_MONGODB_URI: mongodb://mongodb:27017/ts_dictionary
  MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
  JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
↳ =*:5083"
```

notification-service:

```
build:
  context: ./notification-service
  dockerfile: Dockerfile
ports:
  - "8084:8084"
  - "5084:5084"
depends_on:
  config-server:
    condition: service_healthy
  kafka:
    condition: service_started
environment:
  SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
  EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
  SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
  SPRING_MAIL_HOST: maildev
  SPRING_MAIL_PORT: 1025
  MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
  JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
↳ =*:5084"
```

volumes:

```
postgres-auth-data:
postgres-course-data:
mongo-data:
minio-data:
```

config-server: configurations/api-gateway.yml

```
server:
  port: 8080
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service
          uri: lb://auth-service
          predicates:
            - Path=/api/auth/**
          filters:
            - StripPrefix=2
        - id: auth-oauth2
          uri: lb://auth-service
          predicates:
            - Path=/oauth2/**
        - id: auth-oauth2-callback
          uri: lb://auth-service
          predicates:
            - Path=/login/oauth2/**
        - id: course-service-courses
          uri: lb://course-service
          predicates:
            - Path=/api/courses/**
          filters:
            - StripPrefix=2
        - id: course-service-students
          uri: lb://course-service
          predicates:
            - Path=/api/students/**
          filters:
            - StripPrefix=2
        - id: course-service-assignments
          uri: lb://course-service
          predicates:
            - Path=/api/assignments/**
          filters:
            - StripPrefix=2
        - id: course-service-enrollments
          uri: lb://course-service
          predicates:
            - Path=/api/enrollments/**
          filters:
            - StripPrefix=2
        - id: course-service-submissions
          uri: lb://course-service
          predicates:
            - Path=/api/submissions/**
          filters:
            - StripPrefix=2
        - id: dictionary-service
          uri: lb://dictionary-service
          predicates:
```

```

    - Path=/api/dictionary/**
    filters:
    - StripPrefix=2
app:
  jwt:
    secret: ${JWT_SECRET:mySecretKeyThatIsAtLeast256BitsLongForHS256Algorithm123456}
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
endpoints:
  web:
    exposure:
      include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"

```

config-server: configurations/auth-service.yml

```

server:
  port: 8081
  error:
    include-message: always
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_auth
    username: postgres
    password: root
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: none
      show-sql: true
      properties:
        hibernate:
          format_sql: true
          dialect: org.hibernate.dialect.PostgreSQLDialect
  flyway:
    enabled: true
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
  security:
    oauth2:
      client:
        registration:

```

```
google:
  client-id: ${GOOGLE_CLIENT_ID:placeholder}
  client-secret: ${GOOGLE_CLIENT_SECRET:placeholder}
  scope: openid, profile, email
  redirect-uri: http://localhost:8080/login/oauth2/code/google
app:
  jwt:
    secret: ${JWT_SECRET:mySecretKeyThatIsAtLeast256BitsLongForHS256Algorithm123456}
    access-token-expiry-ms: 900000
    refresh-token-expiry-ms: 604800000
  totp:
    issuer: TeacherSupporter
  activation:
    base-url: http://localhost:3000/activate
  oauth2:
    success-redirect-uri: http://localhost:3000/oauth/callback
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
endpoints:
  web:
    exposure:
      include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```

config-server: configurations/course-service.yml

```
server:
  port: 8082
  error:
    include-message: always
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_course
    username: postgres
    password: root
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: none
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
  flyway:
```



```

    enabled: true
servlet:
  multipart:
    max-file-size: 20MB
    max-request-size: 25MB
kafka:
  bootstrap-servers: localhost:9092
  producer:
    key-serializer: org.apache.kafka.common.serialization.StringSerializer
    value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
  consumer:
    group-id: course-group
    auto-offset-reset: earliest
    key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
    value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
  properties:
    spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.StringDeserializer
    spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.
    ↪ JsonSerializer
    spring.json.trusted.packages: com.ts.common.dto
app:
  s3:
    endpoint: ${APP_S3_ENDPOINT:http://localhost:9000}
    # Browser-reachable endpoint used to sign download URLs. When the service runs
    # inside Docker the internal endpoint is minio:9000, which a browser cannot resolve.
    public-endpoint: ${APP_S3_PUBLIC_ENDPOINT:http://localhost:9000}
    region: ${APP_S3_REGION:us-east-1}
    access-key: ${APP_S3_ACCESS_KEY:minioadmin}
    secret-key: ${APP_S3_SECRET_KEY:minioadmin}
    bucket: ${APP_S3_BUCKET:ts-submissions}
    presigned-url-expiry-minutes: ${APP_S3_PRE_SIGNED_EXPIRY_MIN:15}
  resilience4j:
    circuitbreaker:
      instances:
        authService:
          sliding-window-size: 10
          failure-rate-threshold: 50
          wait-duration-in-open-state: 10s
          permitted-number-of-calls-in-half-open-state: 3
  management:
    tracing:
      sampling:
        probability: 1.0
    zipkin:
      tracing:
        endpoint: http://localhost:9411/api/v2/spans
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%{spring.application.name:},%X{traceId:-},%X{spanId:-}]"

```

config-server: configurations/dictionary-service.yml

```
server:
  port: 8083
  error:
    include-message: always
spring:
  data:
    mongodb:
      uri: mongodb://localhost:27017/ts_dictionary
management:
  tracing:
    sampling:
      probability: 1.0
zipkin:
  tracing:
    endpoint: http://localhost:9411/api/v2/spans
endpoints:
  web:
    exposure:
      include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```

config-server: configurations/notification-service.yml

```
server:
  port: 8084
  error:
    include-message: always
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: notification-group
      auto-offset-reset: earliest
      key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
    properties:
      spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.StringDeserializer
      spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.
      ↪ JsonDeserializer
      spring.json.trusted.packages: com.ts.common.dto
  mail:
    host: ${MAIL_HOST:localhost}
    port: ${MAIL_PORT:1025}
    username: ${MAIL_USERNAME:}
    password: ${MAIL_PASSWORD:}
    properties:
      mail.smtp.auth: false
```

```
    mail.smtp.starttls.enable: false
# Where the links inside outgoing emails point. This is the URL a BROWSER
# uses to reach the frontend, so it differs per environment: the Vite dev
# server here and under docker-compose, the public Ingress host in the
# cluster (overridden there with the APP_FRONTEND_BASE_URL env var).
app:
  frontend:
    base-url: http://localhost:3000
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
endpoints:
  web:
    exposure:
      include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```